BÁO CÁO XÂY DỰNG KHO DỮ LIỆU VÀ PHÂN TÍCH NYC TAXI

Thiết kế kho dữ liệu ETL SSAS Multidimensional và Power BI

Báo cáo đồ án

Thời điểm biên soạn ngày 01 tháng 11 năm 2026

Phạm vi bằng chứng kỹ thuật được trích dẫn theo từng lần chụp trong dự án, chủ yếu ngày 31 tháng 10 năm 2026. Các sơ đồ được dựng để giải thích thiết kế; trạng thái kiểm thử dữ liệu và kiểm thử giao diện được phân biệt trong nội dung.

LỜI MỞ ĐẦU

Dữ liệu chuyến đi NYC Taxi cung cấp cơ hội phân tích nhu cầu vận tải, đặc điểm phục vụ và các khoản tiền ghi nhận ở quy mô lớn. Tuy nhiên, dữ liệu phân tán theo nguồn, theo tháng và khác biệt cấu trúc đòi hỏi một quy trình tổ chức, chuẩn hóa và kiểm chứng trước khi khai thác. Đồ án xây dựng kho dữ liệu nhằm giải quyết nhu cầu đó và tạo nền tảng phân tích đa chiều nhất quán.

Báo cáo gồm năm chương. Chương 1 giới thiệu nghiệp vụ và từ điển dữ liệu. Chương 2 thiết kế Galaxy Schema, các dimension, fact và grain. Chương 3 giải thích quy trình ETL cùng cấu hình, chất lượng và phục hồi. Chương 4 trình bày mô hình SSAS, measure và đối soát. Chương 5 mô tả các dashboard, cách khai thác và giới hạn diễn giải. Ba phụ lục cung cấp đặc tả kỹ thuật, bằng chứng và hướng dẫn vận hành.

MỤC LỤC

CHƯƠNG 1 TỔNG QUAN NGHIỆP VỤ VÀ DỮ LIỆU NYC TAXI 18
1.1 Giới thiệu đề tài 18
1.1.1 Bối cảnh và lý do lựa chọn đề tài 18
1.1.2 Mục tiêu xây dựng hệ thống 18
1.1.3 Phạm vi dữ liệu và phạm vi thực hiện 19
1.2 Tổng quan nghiệp vụ vận tải hành khách tại New York 19
1.2.1 Vai trò của NYC Taxi and Limousine Commission 19
1.2.2 Taxi Yellow 20
1.2.3 Taxi Green 20
1.2.4 Dịch vụ For Hire Vehicle FHV 21
1.2.5 Dịch vụ High Volume For Hire Vehicle HVFHV 21
1.2.6 Quy trình yêu cầu điều phối và thực hiện chuyến đi 21
1.3 Giới thiệu nguồn dữ liệu 22
1.3.1 Nguồn cung cấp và phương thức công bố 22
1.3.2 Định dạng cách tổ chức file và phạm vi thời gian 22
1.3.3 Danh mục vùng Taxi Zone 23
1.3.4 Đặc điểm và sự khác biệt giữa các bộ dữ liệu 23
1.4 Từ điển dữ liệu nguồn 24
1.4.1 Các trường dữ liệu Yellow 24
1.4.2 Các trường dữ liệu Green 25
1.4.3 Các trường dữ liệu FHV 27
1.4.4 Các trường dữ liệu HVFHV 28
1.4.5 Các trường dữ liệu Taxi Zone Lookup 30
1.5 Đánh giá chất lượng và giới hạn dữ liệu 30

1.5.1 Dữ liệu thiếu và giá trị không xác định 30
1.5.2 Dữ liệu không hợp lệ và giá trị bất thường 30
1.5.3 Khác biệt cấu trúc dữ liệu giữa các thời kỳ 31
1.5.4 Giới hạn của dữ liệu dùng trong đồ án 31
1.6 Các yêu cầu và câu hỏi phân tích nghiệp vụ 32
CHƯƠNG 2 PHÂN TÍCH VÀ THIẾT KẾ KHO DỮ LIỆU 33
2.1 Kiến trúc tổng thể của hệ thống 33
2.1.1 Các thành phần và công nghệ sử dụng 33
2.1.2 Các lớp dữ liệu từ nguồn đến báo cáo 33
2.1.3 Luồng dữ liệu tổng thể 33
2.2 Xác định các quy trình nghiệp vụ cần phân tích 34
2.3 Xây dựng ma trận nghiệp vụ Data Warehouse Bus Matrix 34
2.4 Lựa chọn mô hình kho dữ liệu 35
2.4.1 Cơ sở lựa chọn Galaxy Schema 35
2.4.2 Dimension dùng chung và dimension đặc thù 36
2.4.3 Dimension đóng nhiều vai trò 36
2.5 Thiết kế các bảng chiều 36
2.5.1 Nhóm chiều ngày và thời gian 37
2.5.2 Nhóm chiều địa lý 39
2.5.3 Nhóm chiều dịch vụ và nhà cung cấp 40
2.5.4 Nhóm chiều base điều phối 42
2.5.5 Nhóm chiều phân loại chuyến đi và thanh toán 43
2.5.6 Nhóm chiều cờ nghiệp vụ 45
2.5.7 Chiều lô nạp dữ liệu 47
2.6 Thiết kế các bảng sự kiện 48

2.6.1 Fact_YellowGreen_Trips 48
2.6.2 Fact_FHV_Trips 52
2.6.3 Fact_HVFHV_Trips 54
2.6.4 Fact_Daily_Financial_Snapshot 58
2.6.5 Fact_Zone_Traffic_Snapshot 59
2.6.6 Fact_Base_Performance_Snapshot 60
2.7 Thiết kế phân cấp và quy tắc tổng hợp dữ liệu 63
2.8 Thiết kế vật lý và toàn vẹn dữ liệu 63
2.8.1 Kiểu dữ liệu khóa chính và khóa ngoại 63
2.8.2 Khóa thay thế và thành viên Unknown 68
2.8.3 Chỉ mục phục vụ nạp và truy vấn 68
2.8.4 Quản lý thay đổi thuộc tính dimension 68
2.8.5 Lưu thông tin truy vết nguồn dữ liệu 69
2.9 Sơ đồ mô hình kho dữ liệu hoàn chỉnh 69
CHƯƠNG 3 XÂY DỰNG QUY TRÌNH ETL 70
3.1 Mục tiêu và kiến trúc ETL 70
3.2 Chuẩn bị môi trường và cấu hình 71
3.2.1 Cơ sở dữ liệu và cấu trúc thư mục 71
3.2.2 Các kết nối sử dụng 71
3.2.3 Tham số biến và phạm vi xử lý 71
3.2.4 Kiểm tra điều kiện trước khi chạy 72
3.3 Xây dựng luồng điều phối chính 73
3.3.1 Cấu trúc package Master 73
3.3.2 Vai trò của SSIS PowerShell và Python 76
3.3.3 Thứ tự thực thi và điều kiện chuyển bước 76

3.3.4 Truyền tham số và quản lý mã lần chạy 76
3.4 Xây dựng luồng trích xuất và nạp Staging 77
3.4.1 Tải dữ liệu nguồn 77
3.4.2 Chuyển đổi Parquet sang CSV 77
3.4.3 Kiểm tra cấu trúc metadata và số dòng 77
3.4.4 Nạp dữ liệu vào vùng làm việc 78
3.4.5 Ghi dữ liệu vào Staging và cập nhật trạng thái 79
3.5 Xây dựng luồng biến đổi và làm sạch 79
3.5.1 Chuẩn hóa danh mục Taxi Zone 79
3.5.2 Xử lý dữ liệu Yellow 79
3.5.3 Xử lý dữ liệu Green 80
3.5.4 Xử lý dữ liệu FHV 80
3.5.5 Xử lý dữ liệu HVFHV 81
3.5.6 Phân loại bản ghi hợp lệ bị loại và có cảnh báo 81
3.5.7 Kiểm tra kết quả sau biến đổi 82
3.6 Xây dựng luồng nạp kho dữ liệu 83
3.6.1 Xác định file cần xử lý 83
3.6.2 Nạp và cập nhật dimension 83
3.6.3 Tra cứu khóa và xử lý Unknown 83
3.6.4 Nạp các fact giao dịch 84
3.6.5 Xử lý file hiệu chỉnh và bản ghi trùng 84
3.6.6 Quản lý transaction và checkpoint 85
3.6.7 Đối soát sau khi nạp 85
3.7 Xây dựng luồng tổng hợp dữ liệu 85
3.7.1 Tổng hợp tài chính theo ngày 85

3.7.2 Tổng hợp lưu lượng theo vùng 86
3.7.3 Tổng hợp hiệu suất điều phối 86
3.7.4 Kiểm tra tính nhất quán của các bảng tổng hợp 87
3.8 Xử lý lỗi và vận hành 87
3.8.1 Nhật ký và theo dõi tiến trình 87
3.8.2 Cách ly file lỗi 87
3.8.3 Retry chạy lại và phục hồi 88
3.8.4 Kiểm soát chạy đồng thời 88
3.8.5 Lập lịch thực thi 89
3.9 Kiểm thử và đánh giá kết quả ETL 89
CHƯƠNG 4 XÂY DỰNG MÔ HÌNH PHÂN TÍCH SSAS 91
4.1 Mục tiêu và tổ chức mô hình SSAS 91
4.2 Khởi tạo Data Source và Data Source View 91
4.3 Xây dựng các dimension phân tích 92
4.3.1 Cấu hình thuộc tính khóa và tên hiển thị 92
4.3.2 Xây dựng hierarchy và quan hệ thuộc tính 98
4.3.3 Cấu hình dimension đóng nhiều vai trò 98
4.3.4 Xử lý Unknown và lỗi khóa 98
4.4 Xây dựng các cube nghiệp vụ 99
4.4.1 Cube YellowGreen Trips 99
4.4.2 Cube FHV Trips 102
4.4.3 Cube HVFHV Trips 103
4.4.4 Cube Daily Financial 106
4.4.5 Cube Zone Traffic 107
4.4.6 Cube Base Performance 108

4.5 Thiết kế measure và calculated measure 109
4.5.1 Các độ đo cộng dồn 109
4.5.2 Các độ đo trung bình 110
4.5.3 Các tỷ lệ nghiệp vụ 110
4.5.4 Xử lý NULL mẫu số bằng không và phạm vi áp dụng 111
4.6 Cấu hình Dimension Usage 111
4.7 Thiết kế partition lưu trữ và aggregation 112
4.8 Triển khai và xử lý mô hình 120
4.9 Truy vấn MDX và đối soát với kho dữ liệu 120
CHƯƠNG 5 XÂY DỰNG DASHBOARD POWER BI 121
5.1 Mục tiêu và yêu cầu của bộ báo cáo 121
5.2 Kết nối Power BI với SSAS 121
5.3 Thiết kế bộ lọc bố cục và tương tác 122
5.4 Xây dựng các trang phân tích 123
5.4.1 Phân tích hoạt động taxi Yellow và Green 123
5.4.2 Phân tích thanh toán và tiền tip 125
5.4.3 Phân tích hoạt động xe công nghệ 127
5.4.4 Phân tích phí và thu nhập 130
5.4.5 Phân tích hoạt động FHV 132
5.4.6 Phân tích tài chính theo ngày 134
5.4.7 Phân tích nhu cầu theo khu vực và khung giờ 137
5.4.8 Phân tích hiệu suất điều phối 140
5.5 Khai thác dashboard để trả lời câu hỏi nghiệp vụ 142
5.6 Kiểm thử dữ liệu và tương tác trên dashboard 143
5.7 Đánh giá kết quả và giới hạn diễn giải 144

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 145
TÀI LIỆU THAM KHẢO 146
PHỤ LỤC A SQL VÀ CẤU HÌNH KỸ THUẬT CHI TIẾT 147
PHỤ LỤC B TRUY VẤN KIỂM TRA VÀ KẾT QUẢ ĐỐI SOÁT 250
PHỤ LỤC C HƯỚNG DẪN TRIỂN KHAI VÀ VẬN HÀNH 272
C.1 Chuẩn bị cấu hình môi trường 272
C.2 Chạy mới và theo dõi 272
C.3 Chạy lại sau lỗi 272
C.4 Mở báo cáo và thực hiện demo 273
C.5 Bảo trì đối soát và khôi phục 273

DANH MỤC BẢNG

Bảng 1.1 Metadata nguồn yellow 24
Bảng 1.2 Metadata nguồn green 25
Bảng 1.3 Metadata nguồn fhv 27
Bảng 1.4 Metadata nguồn fhvhv 28
Bảng 1.5 Metadata nguồn lookup 30
Bảng 1.6 Câu hỏi nghiệp vụ và nguồn trả lời 32
Bảng 2.1 Ma trận nghiệp vụ và dimension dùng chung 35
Bảng 2.2 Cấu trúc và ý nghĩa các cột Dim_Date 37
Bảng 2.3 Cấu trúc và ý nghĩa các cột Dim_Time 38
Bảng 2.4 Cấu trúc và ý nghĩa các cột Dim_Location 40
Bảng 2.5 Cấu trúc và ý nghĩa các cột Dim_ServiceType 41
Bảng 2.6 Cấu trúc và ý nghĩa các cột Dim_Vendor 41
Bảng 2.7 Cấu trúc và ý nghĩa các cột Dim_HVFHSProvider 41
Bảng 2.8 Cấu trúc và ý nghĩa các cột Dim_Base 42
Bảng 2.9 Cấu trúc và ý nghĩa các cột Dim_RateCode 43
Bảng 2.10 Cấu trúc và ý nghĩa các cột Dim_PaymentType 44
Bảng 2.11 Cấu trúc và ý nghĩa các cột Dim_TripType 44
Bảng 2.12 Cấu trúc và ý nghĩa các cột Dim_PassengerGroup 44
Bảng 2.13 Cấu trúc và ý nghĩa các cột Dim_TaxiFlags 45
Bảng 2.14 Cấu trúc và ý nghĩa các cột Dim_FHVFlags 46
Bảng 2.15 Cấu trúc và ý nghĩa các cột Dim_HVFHVFlags 46
Bảng 2.16 Cấu trúc và ý nghĩa các cột Dim_LoadBatch 47
Bảng 2.17 Cấu trúc và ý nghĩa các cột Fact_YellowGreen_Trips 49
Bảng 2.18 Cấu trúc và ý nghĩa các cột Fact_FHV_Trips 52
Bảng 2.19 Cấu trúc và ý nghĩa các cột Fact_HVFHV_Trips 55
Bảng 2.20 Cấu trúc và ý nghĩa các cột Fact_Daily_Financial_Snapshot 58
Bảng 2.21 Cấu trúc và ý nghĩa các cột Fact_Zone_Traffic_Snapshot 60
Bảng 2.22 Cấu trúc và ý nghĩa các cột Fact_Base_Performance_Snapshot 61
Bảng 2.23 Hierarchy và quy tắc khóa 63
Bảng 2.24 Danh mục khóa ngoại và vai trò 64
Bảng 3.1 Tham số Master lưu trong source 72
Bảng 3.2 Các tác vụ điều phối Master 73

Bảng 3.3 Các task cấp chính PKG_01_Extract_Stage 74
Bảng 3.4 Các task cấp chính PKG_02_Transform_Clean 74
Bảng 3.5 Các task cấp chính PKG_03_Load_DWH 75
Bảng 3.6 Các task cấp chính PKG_04_Load_Aggregates 75
Bảng 3.7 Điều kiện chất lượng và cách diễn giải 81
Bảng 3.8 Bằng chứng kiểm thử ETL đã lưu 89
Bảng 4.1 Attribute key và quan hệ Dim Base 92
Bảng 4.2 Attribute key và quan hệ Dim Date 93
Bảng 4.3 Attribute key và quan hệ Dim FHV Flags 94
Bảng 4.4 Attribute key và quan hệ Dim HVFHS Provider 94
Bảng 4.5 Attribute key và quan hệ Dim HVFHV Flags 94
Bảng 4.6 Attribute key và quan hệ Dim Load Batch 95
Bảng 4.7 Attribute key và quan hệ Dim Location 95
Bảng 4.8 Attribute key và quan hệ Dim Passenger Group 96
Bảng 4.9 Attribute key và quan hệ Dim Payment Type 96
Bảng 4.10 Attribute key và quan hệ Dim Rate Code 96
Bảng 4.11 Attribute key và quan hệ Dim Service Type 96
Bảng 4.12 Attribute key và quan hệ Dim Taxi Flags 97
Bảng 4.13 Attribute key và quan hệ Dim Time 97
Bảng 4.14 Attribute key và quan hệ Dim Trip Type 97
Bảng 4.15 Attribute key và quan hệ Dim Vendor 98
Bảng 4.16 Measure và nguồn dữ liệu cube YellowGreen Trips 99
Bảng 4.17 Dimension Usage cube YellowGreen Trips 101
Bảng 4.18 Measure và nguồn dữ liệu cube FHV Trips 102
Bảng 4.19 Dimension Usage cube FHV Trips 102
Bảng 4.20 Measure và nguồn dữ liệu cube HVFHV Trips 103
Bảng 4.21 Dimension Usage cube HVFHV Trips 105
Bảng 4.22 Measure và nguồn dữ liệu cube Daily Financial 106
Bảng 4.23 Dimension Usage cube Daily Financial 107
Bảng 4.24 Measure và nguồn dữ liệu cube Zone Traffic 107
Bảng 4.25 Dimension Usage cube Zone Traffic 107
Bảng 4.26 Measure và nguồn dữ liệu cube Base Performance 108
Bảng 4.27 Dimension Usage cube Base Performance 109

Bảng 4.28 Quy tắc calculated measure 110
Bảng 4.29 Partition và phạm vi nguồn Base_Performance 112
Bảng 4.30 Partition và phạm vi nguồn Daily_Financial 112
Bảng 4.31 Partition và phạm vi nguồn FHV_Trips 113
Bảng 4.32 Partition và phạm vi nguồn HVFHV_Trips 114
Bảng 4.33 Partition và phạm vi nguồn YellowGreen_Trips 116
Bảng 4.34 Partition và phạm vi nguồn Zone_Traffic 119
Bảng 5.1 Project Power BI và trang nghiệp vụ 121
Bảng 5.2 Visual và binding trang Taxi truyền thống 123
Bảng 5.3 Visual và binding trang Hành vi & tiền tip 126
Bảng 5.4 Visual và binding trang Xe công nghệ 128
Bảng 5.5 Visual và binding trang Phí & thu nhập 130
Bảng 5.6 Visual và binding trang FHV truyền thống 133
Bảng 5.7 Visual và binding trang Tài chính theo ngày 135
Bảng 5.8 Visual và binding trang Nhu cầu theo khu vực 137
Bảng 5.9 Visual và binding trang Chi tiết không gian & giờ 139
Bảng 5.10 Visual và binding trang Hiệu suất điều phối 140
Bảng 5.11 Phân biệt kiểm thử dữ liệu và giao diện 143
Bảng A.1 Cấu hình tác vụ EPT_00_Open_Monitor 147
Bảng A.2 Cấu hình tác vụ EPT_01_Extract 148
Bảng A.3 Cấu hình tác vụ EPT_02_Transform 148
Bảng A.4 Cấu hình tác vụ EPT_03_Dwh 149
Bảng A.5 Cấu hình tác vụ EPT_04_Aggregates 150
Bảng A.6 Cấu hình tác vụ EPT_05_Ssas 151
Bảng A.7 Cấu hình tác vụ EPT_99_Complete 152
Bảng A.8 Cấu hình tác vụ EST_90_Report_Original_Error 153
Bảng A.9 Cấu hình tác vụ EST_LogOriginalError 153
Bảng A.10 Precedence constraint PKG_00_Master 153
Bảng A.11 Cấu hình tác vụ EPT_01_Download_Parquet 154
Bảng A.12 Cấu hình tác vụ EPT_02_Convert_To_CSV 155
Bảng A.13 Cấu hình tác vụ EST_03_Prepare_Staging 167
Bảng A.14 Cấu hình tác vụ EPT_Inspect_Yellow_CSV 167
Bảng A.15 Cấu hình tác vụ EST_Begin_Yellow_File 168

Bảng A.16 Cấu hình component SRC_Yellow_CSV 169
Bảng A.17 Metadata cột đầu ra SRC_Yellow_CSV 169
Bảng A.18 Cấu hình component RC_Yellow 171
Bảng A.19 Cấu hình component DST_wrk_YellowTrip 171
Bảng A.20 Metadata cột đầu ra DST_wrk_YellowTrip 172
Bảng A.21 Cấu hình tác vụ EST_Commit_Yellow_File 172
Bảng A.22 Cấu hình tác vụ EST_Mark_Yellow_Failed 172
Bảng A.23 Cấu hình tác vụ EPT_Inspect_Green_CSV 173
Bảng A.24 Cấu hình tác vụ EST_Begin_Green_File 174
Bảng A.25 Cấu hình component SRC_Green_CSV 174
Bảng A.26 Metadata cột đầu ra SRC_Green_CSV 174
Bảng A.27 Cấu hình component RC_Green 176
Bảng A.28 Cấu hình component DST_wrk_GreenTrip 176
Bảng A.29 Metadata cột đầu ra DST_wrk_GreenTrip 177
Bảng A.30 Cấu hình tác vụ EST_Commit_Green_File 177
Bảng A.31 Cấu hình tác vụ EST_Mark_Green_Failed 178
Bảng A.32 Cấu hình tác vụ EPT_Inspect_FHV_CSV 178
Bảng A.33 Cấu hình tác vụ EST_Begin_FHV_File 179
Bảng A.34 Cấu hình component SRC_FHV_CSV 179
Bảng A.35 Metadata cột đầu ra SRC_FHV_CSV 179
Bảng A.36 Cấu hình component RC_FHV 180
Bảng A.37 Cấu hình component DST_wrk_FHVTrip 180
Bảng A.38 Metadata cột đầu ra DST_wrk_FHVTrip 181
Bảng A.39 Cấu hình tác vụ EST_Commit_FHV_File 181
Bảng A.40 Cấu hình tác vụ EST_Mark_FHV_Failed 182
Bảng A.41 Cấu hình tác vụ EPT_Inspect_FHVHV_CSV 182
Bảng A.42 Cấu hình tác vụ EST_Begin_FHVHV_File 183
Bảng A.43 Cấu hình component SRC_FHVHV_CSV 183
Bảng A.44 Metadata cột đầu ra SRC_FHVHV_CSV 183
Bảng A.45 Cấu hình component RC_FHVHV 186
Bảng A.46 Cấu hình component DST_wrk_FHVHVTrip 186
Bảng A.47 Metadata cột đầu ra DST_wrk_FHVHVTrip 187
Bảng A.48 Cấu hình tác vụ EST_Commit_FHVHV_File 187

Bảng A.49 Cấu hình tác vụ EST_Mark_FHVHV_Failed 187
Bảng A.50 Cấu hình tác vụ EPT_Inspect_TaxiZone_CSV 188
Bảng A.51 Cấu hình tác vụ EST_Begin_TaxiZone_File 189
Bảng A.52 Cấu hình component SRC_TaxiZone_CSV 189
Bảng A.53 Metadata cột đầu ra SRC_TaxiZone_CSV 189
Bảng A.54 Cấu hình component RC_TaxiZone 190
Bảng A.55 Cấu hình component DST_wrk_TaxiZone 190
Bảng A.56 Metadata cột đầu ra DST_wrk_TaxiZone 190
Bảng A.57 Cấu hình tác vụ EST_Commit_TaxiZone_File 191
Bảng A.58 Cấu hình tác vụ EST_Mark_TaxiZone_Failed 191
Bảng A.59 Cấu hình tác vụ EST_09_Validate_Staging 192
Bảng A.60 Cấu hình tác vụ EST_LogOriginalError 192
Bảng A.61 Precedence constraint PKG_01_Extract_Stage 192
Bảng A.62 Cấu hình tác vụ EST_01_Setup_Clean 198
Bảng A.63 Cấu hình tác vụ EST_02_Transform_TaxiZone 199
Bảng A.64 Cấu hình tác vụ EST_03_Transform_YELLOW 204
Bảng A.65 Cấu hình tác vụ EST_04_Transform_GREEN 208
Bảng A.66 Cấu hình tác vụ EST_05_Transform_FHV 211
Bảng A.67 Cấu hình tác vụ EST_06_Transform_FHVHV 216
Bảng A.68 Cấu hình tác vụ EST_07_Validate_Transform 217
Bảng A.69 Cấu hình tác vụ EST_LogOriginalError 217
Bảng A.70 Precedence constraint PKG_02_Transform_Clean 217
Bảng A.71 Cấu hình tác vụ EST_00_Precheck_DWH 219
Bảng A.72 Cấu hình tác vụ EST_01_Load_Dimensions 222
Bảng A.73 Cấu hình tác vụ EST_02_Load_FactYellowGreen 227
Bảng A.74 Cấu hình tác vụ EST_03_Load_FactFHV 232
Bảng A.75 Cấu hình tác vụ EST_04_Load_FactHVFHV 238
Bảng A.76 Cấu hình tác vụ EST_05_Validate_DWH 239
Bảng A.77 Cấu hình tác vụ EST_LogOriginalError 239
Bảng A.78 Precedence constraint PKG_03_Load_DWH 239
Bảng A.79 Cấu hình tác vụ EST_00_Atomic_Rebuild_And_Validate 247
Bảng A.80 Cấu hình tác vụ EST_LogOriginalError 247
Bảng B.1 Bằng chứng lưu trong row-counts.csv 250

Bảng B.2 Bằng chứng lưu trong foreign-key-audit.csv 251
Bảng B.3 Bằng chứng lưu trong browse-checks.csv 255
Bảng B.4 Bằng chứng lưu trong reconciliation-Full.csv 256

DANH MỤC HÌNH

Hình 2.1 Kiến trúc từ nguồn dữ liệu đến báo cáo 34
Hình 2.2 Dimension dùng chung và các nhóm fact 69
Hình 3.1 Luồng điều phối và các điểm kiểm tra ETL 70
Hình 4.1 Tổ chức tầng phân tích SSAS 92
Hình 5.1 Khai thác Power BI qua kết nối live 122

DANH MỤC TỪ VIẾT TẮT

TLC — Taxi and Limousine Commission

FHV — For Hire Vehicle

HVFHV — High Volume For Hire Vehicle

HVFHS — High Volume For Hire Services

DWH — Data Warehouse

ETL — Extract Transform Load

SSIS — SQL Server Integration Services

SSAS — SQL Server Analysis Services

DSV — Data Source View

MDX — Multidimensional Expressions

MOLAP — Multidimensional Online Analytical Processing

SCD — Slowly Changing Dimension

WAV — Wheelchair Accessible Vehicle

PBIP — Power BI Project

CHƯƠNG 1 TỔNG QUAN NGHIỆP VỤ VÀ DỮ LIỆU NYC TAXI

1.1 Giới thiệu đề tài

1.1.1 Bối cảnh và lý do lựa chọn đề tài

Hoạt động vận tải hành khách tại New York tạo ra nhiều bản ghi chuyến đi, phản ánh thời gian phục vụ, địa điểm đón trả, quãng đường và các khoản thu. Dữ liệu được công bố theo từng loại dịch vụ và từng tháng nên thuận lợi cho việc tải về nhưng chưa sẵn sàng để phân tích thống nhất. Tên cột, mức độ đầy đủ và ý nghĩa của một số trường khác nhau giữa taxi truyền thống, xe đặt trước qua base và xe công nghệ quy mô lớn. Nếu truy vấn trực tiếp từng file, người phân tích phải lặp lại công việc làm sạch, giải mã và tổng hợp cho mỗi câu hỏi nghiệp vụ.

Đề tài xây dựng kho dữ liệu NYC Taxi nhằm tổ chức các nguồn này thành một hệ thống có cấu trúc và có thể kiểm chứng. Kho giữ chi tiết chuyến đi để truy vết, đồng thời cung cấp các bảng tổng hợp cho phân tích tài chính, nhu cầu theo vùng và hiệu suất điều phối. SSAS tạo tầng ngữ nghĩa dùng chung; Power BI hỗ trợ người sử dụng lựa chọn thời gian, dịch vụ và địa bàn mà không phải trực tiếp viết các truy vấn xử lý dữ liệu nguồn.

1.1.2 Mục tiêu xây dựng hệ thống

Hệ thống hướng đến ba kết quả liên kết với nhau. Thứ nhất, dữ liệu được tiếp nhận theo hợp đồng rõ ràng, chuẩn hóa và phân loại chất lượng trước khi vào kho. Thứ hai, mô hình fact và dimension cho phép phân tích nhất quán theo thời gian, địa lý, loại dịch vụ, nhà cung cấp và base. Thứ ba, các chỉ tiêu trên báo cáo có công thức, đơn vị và phạm vi áp dụng cụ thể, có thể đối chiếu giữa SQL và SSAS.

Mục tiêu vận hành là cho phép nhận biết lần chạy nào đã thành công, file nào đã được xử lý và bước nào cần chạy lại khi có lỗi. Khả năng chạy lại phải dựa trên checkpoint và phiên bản đầu vào, tránh nạp trùng do thực hiện lại cùng một file.

Tính đúng được ưu tiên trước tối ưu tốc độ: bản ghi thiếu thuộc tính có thể được giữ với Unknown, còn lỗi cấu trúc hoặc điều kiện chất lượng cốt lõi phải được phát hiện và ghi nhận.

1.1.3 Phạm vi dữ liệu và phạm vi thực hiện

Phạm vi nguồn gồm Yellow, Green, FHV, HVFHV và danh mục Taxi Zone. Phạm vi sản phẩm gồm lớp raw và CSV, cơ sở dữ liệu Staging, lớp clean, kho NYCTaxi_DWH, mô hình SSAS Multidimensional và các project Power BI kết nối live. Báo cáo trình bày thiết kế đã có trong mã nguồn, giải thích cách xây dựng và đối chiếu với các hồ sơ kiểm thử được lưu trong dự án.
Các kết quả định lượng sử dụng ảnh chụp bằng chứng ngày 31 tháng 10 năm 2026. Việc biên soạn không đồng nghĩa đã chạy lại ETL, xử lý lại cube hoặc kiểm tra toàn bộ giao diện Power BI. Tập demo có giới hạn số dòng theo file; vì vậy báo cáo không suy rộng các thống kê mẫu thành toàn bộ thị trường vận tải New York. Thông tin thành viên và phân công thực hiện không được đưa vào vì chưa có dữ liệu xác nhận.

1.2 Tổng quan nghiệp vụ vận tải hành khách tại New York

1.2.1 Vai trò của NYC Taxi and Limousine Commission

NYC Taxi and Limousine Commission, viết tắt TLC, là cơ quan quản lý lĩnh vực taxi và các dịch vụ vận tải hành khách thuộc phạm vi cấp phép tại New York. Trong đề tài, TLC là đầu mối công bố dữ liệu, danh mục vùng và từ điển giải thích các trường. Bản ghi được hình thành từ các hệ thống công nghệ và hồ sơ doanh nghiệp cung cấp cho TLC; việc được công bố chính thức không bảo đảm từng dòng đã hoàn toàn chính xác hoặc đầy đủ.
Từ góc độ kho dữ liệu, TLC cung cấp ngữ nghĩa chung để giải thích nguồn, còn hệ thống ETL phải thực hiện kiểm tra kỹ thuật và chất lượng dữ liệu của chính tập đã tải. Do đó, báo cáo phân biệt rõ tài liệu định nghĩa nguồn với dữ liệu đã

quan sát trong kho. Mã dịch vụ, mã giấy phép và danh mục địa lý được dùng để nhận diện bối cảnh chuyến đi, không thay cho định danh hành khách hoặc tài xế.

1.2.2 Taxi Yellow

Yellow là nhóm taxi màu vàng thuộc hệ thống medallion, gắn với hình thức nhận khách vẫy xe và các phương thức nhận chuyến được TLC cho phép. Bản ghi Yellow chứa thời điểm bật và tắt đồng hồ, điểm đón trả, số hành khách, quãng đường, biểu giá, thanh toán và các khoản tiền. Prefix tpep trong tên trường liên quan hệ thống ghi nhận dữ liệu của taxi Yellow.
Đối tượng phân tích cơ bản là chuyến đi, không phải một lần quẹt thẻ hay một hành khách riêng lẻ. VendorID nhận diện nhà cung cấp công nghệ ghi nhận bản tin, không phải mã hãng vận tải hoặc tài xế. FareAmount là cước cơ bản; TotalAmount là khoản tổng nguồn ghi nhận. Khi phân tích tip cần nhớ dữ liệu taxi không bao gồm tip tiền mặt trong các trường được công bố tương ứng.

1.2.3 Taxi Green

Green còn được gọi là Boro Taxi hoặc Street Hail Livery. Dịch vụ này được tổ chức để phục vụ nhu cầu đón khách tại những khu vực được phép ngoài vùng hạn chế đối với Green. Màu xe không đủ để suy ra quyền đón ở mọi địa điểm; các giới hạn khai thác thuộc quy định TLC và không được hệ thống kho tự suy diễn từ LocationID.
Về dữ liệu, Green gần Yellow ở các nhóm thời gian, địa lý và tài chính, nhưng dùng các trường lpep và có trip_type để phân biệt cách nhận chuyến. Chính sự tương đồng này cho phép gộp Yellow và Green vào cùng transaction fact với ServiceTypeKey phân biệt nguồn. Các trường riêng của Green không được giả lập cho Yellow; trạng thái không áp dụng được giữ theo quy tắc dimension hoặc NULL của mô hình.

1.2.4 Dịch vụ For Hire Vehicle FHV

FHV là xe phục vụ chuyến được đặt trước và điều phối qua base, bao gồm các nhóm dịch vụ theo giấy phép như livery, black car và limousine. Base là đơn vị có vai trò tiếp nhận hoặc điều phối, không phải địa điểm đón khách và cũng không phải tài xế. Bộ dữ liệu FHV trong dự án chứa mã base, thời gian, mã vùng và cờ SR_Flag, nhưng không cung cấp nhóm cước và quãng đường đầy đủ như Yellow hoặc HVFHV.

Vì phạm vi thuộc tính khác nhau, fact FHV được thiết kế riêng để phân tích số chuyến, thời lượng, địa lý và hoạt động base. Không bổ sung cước bằng 0 để tạo một báo cáo tài chính giả cho FHV. SR_Flag có quy ước nguồn riêng; bản sửa loader chuẩn hóa giá trị 1 thành Y, 0 thành N và NULL thành U, đồng thời giữ giới hạn diễn giải của cờ theo phiên bản dữ liệu.

1.2.5 Dịch vụ High Volume For Hire Vehicle HVFHV

HVFHV là bộ dữ liệu chuyến của nhóm dịch vụ FHV quy mô điều phối lớn. HVFHS là loại giấy phép ở cấp nhà cung cấp, còn HVFHV là tên nhóm bản ghi chuyến thường dùng trong dự án. Nguồn này được công bố tách riêng và chi tiết hơn từ năm 2019; hợp đồng tải của chương trình bắt đầu nhận file từ tháng 2 năm 2019.

Nguồn có các mốc yêu cầu, xe đến điểm đón, pickup và dropoff; đồng thời chứa cước cơ bản, tiền tài xế, thuế phí, tip và các cờ đi chung hoặc tiếp cận xe lăn. Không phải mọi mốc thời gian đều có độ bao phủ như nhau: on_scene_datetime trong từ điển TLC gắn với xe tiếp cận, nên thiếu trường này không mặc nhiên có nghĩa xe không đến. Provider được xác định qua giấy phép HVFHS; không gọi mọi bản ghi là chuyến của một thương hiệu duy nhất.

1.2.6 Quy trình yêu cầu điều phối và thực hiện chuyến đi

Với chuyến có dữ liệu đặt trước, khách tạo yêu cầu, hệ thống hoặc base nhận yêu cầu và điều phối xe, xe đến điểm đón, hành khách lên xe và chuyến kết thúc

tại điểm trả. Trong nguồn HVFHV, originating_base_num và dispatching_base_num có thể đại diện hai vai trò khác nhau. Các sự kiện tương ứng được giữ trên cùng dòng fact chuyến đi để tính thời lượng phục vụ mà không tạo thêm dòng cho từng trạng thái.

Đối với Yellow và Green, hệ thống chủ yếu quan sát các mốc đồng hồ và kết quả tài chính. Không có mốc request không chứng minh không có đặt xe, mà chỉ phản ánh cấu trúc nguồn đang sử dụng. Khi tổng hợp nhu cầu, một chuyến tạo một sự kiện đón và một sự kiện trả; hai sự kiện có thể thuộc vùng, phút hoặc ngày khác nhau. Đây là cơ sở để tách các phép đo TotalPickups và TotalDropoffs.

1.3 Giới thiệu nguồn dữ liệu

1.3.1 Nguồn cung cấp và phương thức công bố

Dữ liệu được lấy từ trang TLC Trip Record Data và danh mục Taxi Zone đi kèm. TLC tổ chức các liên kết tải theo tháng và loại dịch vụ. Pipeline sử dụng downloader để lựa chọn phạm vi tháng, lưu file raw và chỉ chuyển sang bước tiếp theo sau khi xác nhận đủ đầu vào cần thiết. Thông tin nguồn chính thức được liệt kê tại phần tài liệu tham khảo [1] đến [6].

Việc nhận lại file tháng cũ có thể phản ánh hiệu chỉnh nguồn. Vì vậy tên file tháng không đủ để nhận biết phiên bản; hệ thống dùng hash và metadata để phân biệt dữ liệu đã thay đổi. Tình huống không còn file ở cache hoặc nguồn không còn cung cấp một liên kết không được tự coi là yêu cầu xóa lịch sử trong DWH.

1.3.2 Định dạng cách tổ chức file và phạm vi thời gian

Nguồn chuyến đi ở định dạng Parquet, thuận lợi cho lưu trữ dữ liệu cột. Pipeline chuyển thành CSV UTF 8 với kết thúc dòng LF và thứ tự trường ổn định để phục vụ SSIS và SqlBulkCopy. Tệp sidecar .meta.json ghi số dòng, giới hạn dòng, danh sách cột, hash, kích thước và thông tin phiên bản chuyển đổi.

Hợp đồng chương trình quy định tháng đầu tương ứng Yellow 2009 01, Green 2013 08, FHV 2015 01 và HVFHV 2019 02. Các mốc này dùng cho việc chọn file, không có nghĩa kho hiện chứa toàn bộ lịch sử từ các thời điểm đó. MaxRows bằng 0 đọc toàn file đã chọn; MaxRows lớn hơn 0 lấy N dòng đầu của từng file. Giới hạn được áp dụng riêng từng file và không phải lấy mẫu ngẫu nhiên.

1.3.3 Danh mục vùng Taxi Zone

Taxi Zone Lookup ánh xạ LocationID sang Borough, Zone và service_zone. Đây là danh mục dùng chung cho PULocationID và DOLocationID ở các bộ dữ liệu chuyến đi. Trong kho, LocationID là mã nghiệp vụ nguồn còn LocationKey là khóa nội bộ dùng trong quan hệ fact dimension.

Danh mục phải được chuẩn hóa trước khi biến đổi chuyến đi. Hash danh mục tham gia phiên bản Transform vì thay đổi vùng có thể tác động đến phép kiểm tra vị trí. Các mã không tra được đi vào Unknown theo quy tắc, thay vì bị gán tùy tiện về một quận cụ thể. ZoneName là tên hiển thị; phân tích phải dựa trên khóa vùng để tránh gộp hai tên giống nhau.

1.3.4 Đặc điểm và sự khác biệt giữa các bộ dữ liệu

Yellow và Green có các khoản cước taxi và phương thức thanh toán; HVFHV có tài chính theo nhà cung cấp cùng thông tin chờ; FHV có cấu trúc gọn hơn. Những khác biệt này được giữ xuyên suốt đến fact và cube. Chỉ các góc nhìn có ngữ nghĩa chung như ngày, vùng và loại dịch vụ mới phù hợp cho so sánh liên nguồn.

Chuỗi xử lý không ép mọi dịch vụ vào một tập measure tài chính chung. Daily Financial chỉ lấy nguồn có tài chính, còn Zone Traffic có thể thống nhất các nguồn theo sự kiện đón trả. Khi một trường xuất hiện từ thời kỳ mới, file lịch sử được xử lý theo danh sách optional của hợp đồng; thiếu cột optional khác với thiếu một cột thiết yếu.

1.4 Từ điển dữ liệu nguồn

Các bảng trong mục này liệt kê toàn bộ cột của hợp đồng extract đang sử dụng. Kiểu tiếp nhận CSV là chuỗi để giữ nguyên giá trị nguồn trước chuẩn hóa; kiểu phân tích và đơn vị được giải thích ở cột mô tả. Cột thiết yếu phải có trong header, nhưng điều đó không đồng nghĩa mọi giá trị trong cột đều khác NULL. Một giá trị thiếu còn phải qua kiểm tra chất lượng tại Transform.

1.4.1 Các trường dữ liệu Yellow

Bảng 1.1 Metadata nguồn yellow

Tên trường	**Mô tả và đơn vị**	**Hợp đồng CSV**
VendorID	Mã nhà cung cấp hệ thống ghi nhận chuyến đi; không phải mã tài xế hay hãng taxi.	Chuỗi 50; thiết yếu
tpep_pickup_datetime	Thời điểm đón khách theo bản ghi nguồn; chuyển DATETIME2 tại Transform.	Chuỗi 50; thiết yếu
tpep_dropoff_datetime	Thời điểm trả khách theo bản ghi nguồn; chuyển DATETIME2 tại Transform.	Chuỗi 50; thiết yếu
passenger_count	Số hành khách do nguồn ghi nhận; có thể thiếu và không dùng thay số chuyến.	Chuỗi 50; thiết yếu
trip_distance	Quãng đường chuyến đi theo mile.	Chuỗi 50; thiết yếu
RatecodeID	Mã biểu giá áp dụng cho chuyến đi; giải mã theo phiên bản từ điển TLC.	Chuỗi 50; thiết yếu
store_and_fwd_flag	Cờ lưu dữ liệu tại thiết bị rồi truyền về máy chủ; chuẩn hóa Y, N hoặc chưa biết.	Chuỗi 50; thiết yếu
PULocationID	Mã Taxi Zone nơi đón khách; tra LocationID của danh mục vùng.	Chuỗi 50; thiết yếu
DOLocationID	Mã Taxi Zone nơi trả khách; dùng cùng danh mục với nơi đón.	Chuỗi 50; thiết yếu

Tên trường	Mô tả và đơn vị	Hợp đồng CSV
payment_type	Mã phương thức hoặc tình trạng thanh toán; không áp dụng chung cho FHV.	Chuỗi 50; thiết yếu
fare_amount	Tiền cước cơ bản của taxi, USD; khác tổng tiền khách trả.	Chuỗi 50; thiết yếu
extra	Khoản phụ thu bổ sung của taxi, USD.	Chuỗi 50; thiết yếu
mta_tax	Thuế MTA được ghi riêng, USD.	Chuỗi 50; thiết yếu
tip_amount	Tiền tip được hệ thống ghi nhận, USD; tip tiền mặt có thể không nằm trong dữ liệu.	Chuỗi 50; thiết yếu
tolls_amount	Phí cầu đường taxi, USD.	Chuỗi 50; thiết yếu
improvement_surcharge	Phụ thu cải thiện dịch vụ taxi, USD.	Chuỗi 50; thiết yếu
total_amount	Tổng tiền taxi được nguồn ghi nhận, USD; không diễn giải thành lợi nhuận.	Chuỗi 50; thiết yếu
congestion_surcharge	Phụ thu ùn tắc, USD; tách với phí CBD.	Chuỗi 50; optional
Airport_fee	Phí sân bay taxi, USD.	Chuỗi 50; optional
cbd_congestion_fee	Phí ùn tắc khu trung tâm CBD; cột bổ sung cho dữ liệu từ năm 2025, USD.	Chuỗi 50; optional

1.4.2 Các trường dữ liệu Green

Bảng 1.2 Metadata nguồn green

Tên trường	Mô tả và đơn vị	Hợp đồng CSV
VendorID	Mã nhà cung cấp hệ thống ghi nhận chuyến đi; không phải mã tài xế hay hãng taxi.	Chuỗi 50; thiết yếu

Tên trường	Mô tả và đơn vị	Hợp đồng CSV
lpep_pickup_datetime	Thời điểm đón khách theo bản ghi nguồn; chuyển DATETIME2 tại Transform.	Chuỗi 50; thiết yếu
lpep_dropoff_datetime	Thời điểm trả khách theo bản ghi nguồn; chuyển DATETIME2 tại Transform.	Chuỗi 50; thiết yếu
store_and_fwd_flag	Cờ lưu dữ liệu tại thiết bị rồi truyền về máy chủ; chuẩn hóa Y, N hoặc chưa biết.	Chuỗi 50; thiết yếu
RatecodeID	Mã biểu giá áp dụng cho chuyến đi; giải mã theo phiên bản từ điển TLC.	Chuỗi 50; thiết yếu
PULocationID	Mã Taxi Zone nơi đón khách; tra LocationID của danh mục vùng.	Chuỗi 50; thiết yếu
DOLocationID	Mã Taxi Zone nơi trả khách; dùng cùng danh mục với nơi đón.	Chuỗi 50; thiết yếu
passenger_count	Số hành khách do nguồn ghi nhận; có thể thiếu và không dùng thay số chuyến.	Chuỗi 50; thiết yếu
trip_distance	Quãng đường chuyến đi theo mile.	Chuỗi 50; thiết yếu
fare_amount	Tiền cước cơ bản của taxi, USD; khác tổng tiền khách trả.	Chuỗi 50; thiết yếu
extra	Khoản phụ thu bổ sung của taxi, USD.	Chuỗi 50; thiết yếu
mta_tax	Thuế MTA được ghi riêng, USD.	Chuỗi 50; thiết yếu
tip_amount	Tiền tip được hệ thống ghi nhận, USD; tip tiền mặt có thể không nằm trong dữ liệu.	Chuỗi 50; thiết yếu
tolls_amount	Phí cầu đường taxi, USD.	Chuỗi 50; thiết yếu
ehail_fee	Phí đặt xe điện tử của Green, USD; có thể thiếu theo phiên bản.	Chuỗi 50; optional

Tên trường	Mô tả và đơn vị	Hợp đồng CSV
improvement_surcharge	Phụ thu cải thiện dịch vụ taxi, USD.	Chuỗi 50; thiết yếu
total_amount	Tổng tiền taxi được nguồn ghi nhận, USD; không diễn giải thành lợi nhuận.	Chuỗi 50; thiết yếu
payment_type	Mã phương thức hoặc tình trạng thanh toán; không áp dụng chung cho FHV.	Chuỗi 50; thiết yếu
trip_type	Loại chuyến Green, dùng phân biệt cách nhận chuyến theo mã nguồn.	Chuỗi 50; thiết yếu
congestion_surcharge	Phụ thu ùn tắc, USD; tách với phí CBD.	Chuỗi 50; optional
cbd_congestion_fee	Phí ùn tắc khu trung tâm CBD; cột bổ sung cho dữ liệu từ năm 2025, USD.	Chuỗi 50; optional

1.4.3 Các trường dữ liệu FHV

Bảng 1.3 Metadata nguồn fhv

Tên trường	Mô tả và đơn vị	Hợp đồng CSV
dispatching_base_num	Số giấy phép TLC của base điều phối chuyến.	Chuỗi 50; thiết yếu
pickup_datetime	Thời điểm đón khách theo bản ghi nguồn; chuyển DATETIME2 tại Transform.	Chuỗi 50; thiết yếu
dropOff_datetime	Thời điểm trả khách theo bản ghi nguồn; chuyển DATETIME2 tại Transform.	Chuỗi 50; thiết yếu
PULocationID	Mã Taxi Zone nơi đón khách; tra LocationID của danh mục vùng.	Chuỗi 50; thiết yếu
DOLocationID	Mã Taxi Zone nơi trả khách; dùng cùng danh mục với nơi đón.	Chuỗi 50; thiết yếu
SR_Flag	Cờ chuỗi chuyến đi chung theo quy ước FHV; không đồng nhất trực tiếp với shared_request_flag.	Chuỗi 50; optional

Tên trường	Mô tả và đơn vị	Hợp đồng CSV
Affiliated_base_number	Base mà xe liên kết theo hồ sơ nguồn FHV.	Chuỗi 50; optional

1.4.4 Các trường dữ liệu HVFHV

Bảng 1.4 Metadata nguồn fhvhv

Tên trường	Mô tả và đơn vị	Hợp đồng CSV
hvfhs_license_num	Số giấy phép nhà cung cấp dịch vụ HVFHS; ánh xạ Dim_HVFHSProvider.	Chuỗi 50; thiết yếu
dispatching_base_num	Số giấy phép TLC của base điều phối chuyến.	Chuỗi 50; thiết yếu
originating_base_num	Base nhận yêu cầu ban đầu của chuyến HVFHV.	Chuỗi 50; thiết yếu
request_datetime	Thời điểm yêu cầu xe theo bản ghi nguồn; chuyển DATETIME2 tại Transform.	Chuỗi 50; thiết yếu
on_scene_datetime	Mốc xe đến nơi đón do nguồn cung cấp; từ điển TLC giới hạn độ bao phủ ở xe tiếp cận. Dùng tính OnSceneToPickup khi hợp lệ.	Chuỗi 50; thiết yếu
pickup_datetime	Thời điểm đón khách theo bản ghi nguồn; chuyển DATETIME2 tại Transform.	Chuỗi 50; thiết yếu
dropoff_datetime	Thời điểm trả khách theo bản ghi nguồn; chuyển DATETIME2 tại Transform.	Chuỗi 50; thiết yếu
PULocationID	Mã Taxi Zone nơi đón khách; tra LocationID của danh mục vùng.	Chuỗi 50; thiết yếu
DOLocationID	Mã Taxi Zone nơi trả khách; dùng cùng danh mục với nơi đón.	Chuỗi 50; thiết yếu
trip_miles	Quãng đường chuyến HVFHV theo mile.	Chuỗi 50; thiết yếu
trip_time	Thời lượng chuyến HVFHV do nguồn cung cấp, đơn vị giây.	Chuỗi 50; thiết yếu

Tên trường	Mô tả và đơn vị	Hợp đồng CSV
base_passenger_fare	Cước cơ bản hành khách HVFHV, USD, trước các khoản thuế và phí được tách riêng.	Chuỗi 50; thiết yếu
tolls	Phí cầu đường HVFHV, USD.	Chuỗi 50; thiết yếu
bcf	Khoản đóng góp Black Car Fund trong bản ghi HVFHV, USD.	Chuỗi 50; thiết yếu
sales_tax	Thuế bán hàng HVFHV, USD.	Chuỗi 50; thiết yếu
congestion_surcharge	Phụ thu ùn tắc, USD; tách với phí CBD.	Chuỗi 50; thiết yếu
airport_fee	Phí sân bay HVFHV, USD.	Chuỗi 50; thiết yếu
tips	Tiền tip HVFHV, USD.	Chuỗi 50; thiết yếu
driver_pay	Khoản tiền trả tài xế HVFHV do nguồn công bố, USD; không phải lợi nhuận ròng.	Chuỗi 50; thiết yếu
shared_request_flag	Khách yêu cầu chuyến đi chung, cờ Y/N.	Chuỗi 50; thiết yếu
shared_match_flag	Chuyến được ghép đi chung, cờ Y/N.	Chuỗi 50; thiết yếu
access_a_ride_flag	Chuyến theo chương trình Access A Ride, cờ Y/N.	Chuỗi 50; thiết yếu
wav_request_flag	Khách yêu cầu xe hỗ trợ xe lăn WAV.	Chuỗi 50; thiết yếu
wav_match_flag	Chuyến thực hiện bằng xe WAV; không mặc nhiên là đáp ứng một yêu cầu WAV trong cùng tập.	Chuỗi 50; thiết yếu
cbd_congestion_fee	Phí ùn tắc khu trung tâm CBD; cột bổ sung cho dữ liệu từ năm 2025, USD.	Chuỗi 50; optional

1.4.5 Các trường dữ liệu Taxi Zone Lookup

Bảng 1.5 Metadata nguồn lookup

Tên trường	**Mô tả và đơn vị**	**Hợp đồng CSV**
LocationID	Mã vùng taxi trong danh mục TLC.	Chuỗi 50; thiết yếu
Borough	Quận hoặc nhóm địa lý do danh mục TLC cung cấp.	Chuỗi 255; thiết yếu
Zone	Tên Taxi Zone.	Chuỗi 255; thiết yếu
service_zone	Phân vùng phục vụ theo danh mục TLC.	Chuỗi 255; thiết yếu

1.5 Đánh giá chất lượng và giới hạn dữ liệu

1.5.1 Dữ liệu thiếu và giá trị không xác định

NULL biểu thị giá trị không được nguồn cung cấp hoặc không chuyển được theo quy tắc, còn 0 có thể là giá trị được ghi nhận thật. Vì vậy phép tính trung bình chỉ sử dụng số dòng có giá trị tương ứng; không dùng tổng số chuyến làm mẫu số cho mọi measure. Đối với thuộc tính phân loại, Unknown giữ chuyến trong tổng thể nhưng chỉ ra rằng bối cảnh chưa được xác định đầy đủ.
Hệ thống phân biệt mã nguồn NULL với mã có giá trị nhưng chưa tra được dimension. Trường hợp thứ hai có khả năng được sửa khi bổ sung mapping và có thể làm file được chọn nạp lại. Phân biệt này giúp tránh đọc lại mọi file chỉ vì nguồn vốn không cung cấp một thuộc tính.

1.5.2 Dữ liệu không hợp lệ và giá trị bất thường

Nhóm điều kiện cốt lõi gồm thời gian pickup hoặc dropoff không hợp lệ, thời lượng không dương, thời lượng vượt ngưỡng 48 giờ và quãng đường thiếu hoặc âm ở nguồn yêu cầu quãng đường. Các điều kiện này xác định IsCoreValid. Những vấn đề như vị trí chưa xác định, số hành khách không hợp lý, khoảng

cách bằng 0 hoặc thiếu một khoản tiền có thể được ghi vào QualityIssue theo nhánh quy tắc cụ thể.

Một dòng có cảnh báo chưa chắc là dòng bị loại. Ví dụ, thời gian hợp lệ nhưng vị trí thiếu có thể vẫn đóng góp vào số chuyến và thành viên Unknown; nếu loại bỏ hoàn toàn sẽ làm giảm nhu cầu quan sát được. Ngược lại, không nên giữ một dòng có dropoff trước pickup rồi cộng thời lượng âm vào báo cáo. Các quy tắc và dấu vết lỗi được lưu để giải thích chênh lệch từ nguồn đến kho.

1.5.3 Khác biệt cấu trúc dữ liệu giữa các thời kỳ

Converter chấp nhận khác biệt hoa thường trong tên cột nhưng từ chối tên trùng không phân biệt hoa thường, cột ngoài hợp đồng và cột thiết yếu bị thiếu. Cột optional có thể được bổ sung NULL khi không tồn tại trong phiên bản nguồn. Đây là kiểm soát schema drift có chủ đích: dừng để xem xét thay đổi sẽ an toàn hơn việc nạp lệch thứ tự cột.

Từ năm 2025, các nguồn liên quan có thêm cbd_congestion_fee theo tài liệu TLC. Hợp đồng dự án đã chứa trường này. Đối với file cũ, giá trị thiếu không được giải thích như một khoản phí đã thu bằng 0 trong mọi phép so sánh; cần đọc cùng phạm vi thời gian và quy tắc tài chính của nguồn.

1.5.4 Giới hạn của dữ liệu dùng trong đồ án

Lần rà soát lưu trong dự án ghi nhận 56 file chuyến, mỗi file giới hạn 10.000 dòng đầu, tương ứng 560.000 dòng clean. Tập này có 558.183 dòng hợp lệ và 1.817 dòng bị loại theo điều kiện cốt lõi. Các con số là ảnh chụp bằng chứng, không phải số liệu hiện tại của nguồn TLC và không phải kết quả chạy mới khi viết báo cáo.

Lấy các dòng đầu có thể làm dữ liệu chỉ bao phủ một phần đầu tháng. Bởi vậy biểu đồ theo giờ, ngày hoặc dịch vụ chỉ mô tả tập đã nạp, chưa cho phép kết luận giờ cao điểm toàn tháng hoặc thị phần toàn thành phố. Kho không có đầy đủ chi phí doanh nghiệp, danh tính tài xế hay dữ liệu đối chứng để suy luận lợi nhuận ròng và tác động nhân quả của chính sách.

1.6 Các yêu cầu và câu hỏi phân tích nghiệp vụ

Yêu cầu phân tích được chia thành nhu cầu chuyến đi, đặc điểm phục vụ, tài chính và điều phối. Mỗi yêu cầu phải xác định nguồn có dữ liệu phù hợp, dimension dùng lọc và chỉ tiêu cần đo. Ma trận dưới đây đồng thời là cơ sở kiểm tra tính đầy đủ của các dashboard ở Chương 5.

Bảng 1.6 Câu hỏi nghiệp vụ và nguồn trả lời

Nhóm	**Câu hỏi**	**Nguồn phân tích**
Nhu cầu	Số chuyến biến động ra sao theo ngày tháng giờ và dịch vụ?	Transaction cube theo nguồn; Zone Traffic cho sự kiện liên nguồn.
Địa lý	Vùng và Borough nào có nhiều lượt đón hoặc lượt trả?	Zone Traffic, tách TotalPickups và TotalDropoffs.
Taxi	Cước và tip khác nhau theo thanh toán quận và giờ như thế nào?	YellowGreen Trips.
HVFHV	Thời gian chờ và tiền tài xế khác nhau theo provider ra sao?	HVFHV Trips và Base Performance.
Tài chính	Tổng tiền khách tip thuế phí thay đổi theo ngày dịch vụ thế nào?	Daily Financial; không bổ sung FHV giả lập.
Điều phối	Base nào có nhiều chuyến, chờ và yêu cầu đi chung?	Base Performance theo dispatch base và provider.

CHƯƠNG 2 PHÂN TÍCH VÀ THIẾT KẾ KHO DỮ LIỆU

2.1 Kiến trúc tổng thể của hệ thống

2.1.1 Các thành phần và công nghệ sử dụng

SQL Server lưu dữ liệu Staging, clean và DWH; SSIS thể hiện luồng điều phối và các package xử lý; Python thực hiện tải, chuyển đổi và kiểm tra file; PowerShell điều phối tiến trình, thực thi SQL và giao tiếp với SSAS. SSAS Multidimensional tổ chức dimension, measure group và cube; Power BI Desktop trình bày kết quả qua kết nối live.

Sự phối hợp này phản ánh đường chạy hiện có của dự án. Không mô tả SSIS như công cụ trực tiếp chạy toàn bộ Data Flow nếu Master thực tế đang gọi runner. Tương tự, không gọi các project Power BI là mô hình import khi chúng tham chiếu cube trên SSAS. Mỗi công nghệ có trách nhiệm cụ thể và được đối chiếu ở các chương triển khai.

2.1.2 Các lớp dữ liệu từ nguồn đến báo cáo

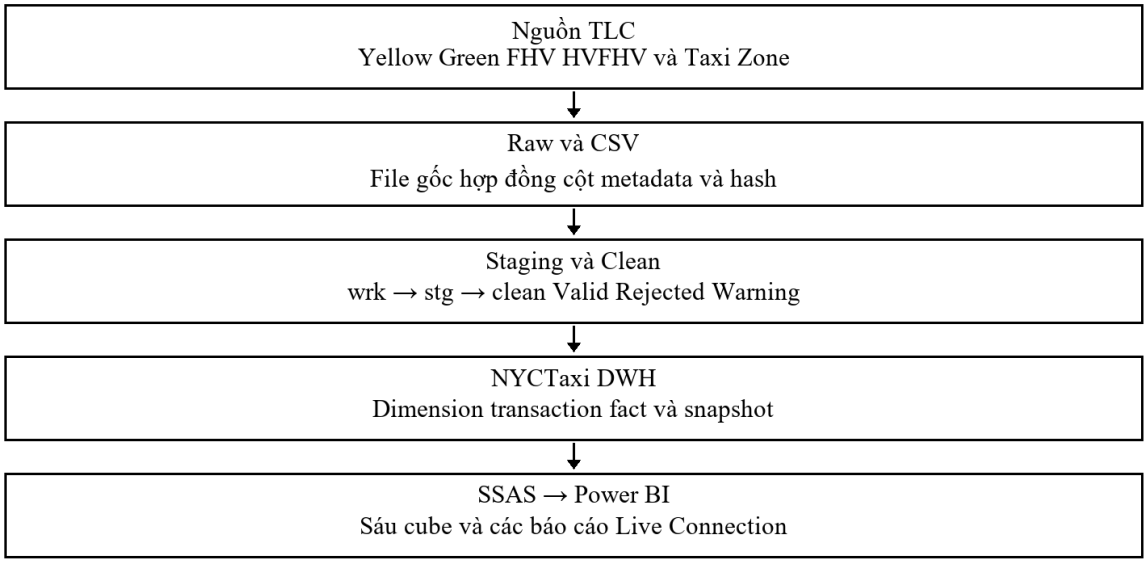
Raw giữ file Parquet gốc; CSV tạo hợp đồng tiếp nhận ổn định; wrk là vùng đệm nạp; stg giữ nội dung nguồn đã nhận cùng thông tin file; clean lưu dữ liệu được chuẩn hóa và kết quả kiểm tra chất lượng. DWH chứa các dimension và fact phục vụ phân tích. SSAS đọc DWH và xây cấu trúc đa chiều; Power BI truy vấn các cube đã được xử lý.

Việc tách các lớp cho phép truy tìm nơi phát sinh chênh lệch. Khi số dòng fact thấp hơn nguồn, có thể đối chiếu số dòng CSV, staging, clean hợp lệ và checkpoint của file. Không phải mọi lớp đều là bản sao có thể xóa tùy ý: lineage, log và trạng thái hoàn tất là điều kiện để chạy lại đúng phạm vi.

2.1.3 Luồng dữ liệu tổng thể

Luồng xử lý đi từ lựa chọn tháng và nguồn, tiếp nhận file, chuẩn hóa, nạp dimension, nạp transaction fact, dựng snapshot, process SSAS rồi khai thác báo

cáo. Điều kiện chuyển bước dựa trên kiểm tra đầu ra của bước trước. DWH có dữ liệu mới nhưng SSAS chưa process có thể khiến Power BI vẫn nhìn thấy dữ liệu của lần xử lý trước.



Hình 2.1 Kiến trúc từ nguồn dữ liệu đến báo cáo

2.2 Xác định các quy trình nghiệp vụ cần phân tích

Ba quy trình mức giao dịch tương ứng nhóm chuyến YellowGreen, FHV và HVFHV. Chúng có cùng khái niệm chuyến đi nhưng khác tập thuộc tính và phép đo, do đó được giữ ở ba fact. Ba quy trình mức tổng hợp là tài chính theo ngày, lưu lượng vùng theo thời điểm sự kiện và hiệu suất base theo ngày.
Mỗi snapshot được dựng từ các transaction fact phù hợp chứ không phải từ việc nối các fact theo ngày. Nếu nối hai fact giao dịch chỉ bằng ngày hoặc vùng, một chuyến của nguồn này có thể khớp nhiều chuyến của nguồn khác và làm nhân số lượng. Thiết kế tổng hợp trước theo grain chung tránh lỗi này và tạo phạm vi phân tích rõ ràng cho từng cube.

2.3 Xây dựng ma trận nghiệp vụ Data Warehouse Bus Matrix

Bus Matrix xác định những chiều được chia sẻ giữa các quy trình, cùng vai trò của chúng. Ngày và địa lý có nhiều vai trò nên ký hiệu có mặt trong ma trận chỉ

thể hiện sự tham gia, không đủ để xác định cột nối. Danh mục quan hệ chi tiết ở mục 2.8 bổ sung khóa ngoại và vai trò cụ thể.

Bảng 2.1 Ma trận nghiệp vụ và dimension dùng chung

Fact	**Date**	**Time**	**Location**	**Service**	**Chiều đặc thù**
Fact_Base_Performance_Snapshot	X			X	Dim_Base; Dim_HVFHSProvider
Fact_Daily_Financial_Snapshot	X			X	
Fact_FHV_Trips	X	X	X	X	Dim_Base; Dim_FHVFlags; Dim_LoadBatch
Fact_HVFHV_Trips	X	X	X	X	Dim_Base; Dim_HVFHSProvider; Dim_HVFHVFlags; Dim_LoadBatch
Fact_YellowGreen_Trips	X	X	X	X	Dim_LoadBatch; Dim_PassengerGroup; Dim_PaymentType; Dim_RateCode; Dim_TaxiFlags; Dim_TripType; Dim_Vendor
Fact_Zone_Traffic_Snapshot	X	X	X	X	

2.4 Lựa chọn mô hình kho dữ liệu

2.4.1 Cơ sở lựa chọn Galaxy Schema

Galaxy Schema phù hợp vì hệ thống có nhiều fact cùng sử dụng các dimension ngày, giờ, vùng và loại dịch vụ. So với việc tạo các star schema hoàn toàn độc lập, dimension dùng chung giúp cùng một vùng hoặc ngày được diễn giải thống nhất. So với một fact rất rộng chứa mọi cột, ba fact giao dịch riêng hạn chế các measure không có nghĩa với từng nguồn.

Trong thiết kế này, sự dùng chung nằm ở dimension và quy tắc nghiệp vụ; không tạo một phép cộng tự động qua tất cả cube. Tổng liên nguồn chỉ hợp lệ khi cùng đơn vị sự kiện, cùng phạm vi dữ liệu và cùng bộ lọc. Các snapshot bổ

sung tốc độ và sự thuận tiện cho những câu hỏi tổng hợp đã được xác định trước.

2.4.2 Dimension dùng chung và dimension đặc thù

Dim_Date, Dim_Time, Dim_Location và Dim_ServiceType là những chiều dùng chung chủ yếu. Dim_Base và Dim_HVFHSProvider phù hợp với nghiệp vụ điều phối và nguồn HVFHV; Dim_Vendor, Dim_PaymentType, Dim_RateCode và Dim_PassengerGroup mô tả taxi truyền thống. Các chiều cờ được tách theo nhóm nguồn để không gộp những cờ có ý nghĩa khác nhau. Không áp dụng một dimension cho fact chỉ vì tên thuộc tính có vẻ tương tự. VendorID của YellowGreen không thay HVFHS license; base không thay provider; payment type không mô tả thanh toán FHV khi nguồn không có trường tương ứng. Việc xác định đúng dimension đặc thù cũng quyết định Dimension Usage của các cube.

2.4.3 Dimension đóng nhiều vai trò

Dim_Date được dùng ở các vai trò Pickup, Dropoff, Request, On Scene hoặc Snapshot; Dim_Time giữ các vai trò giờ tương ứng. Dim_Location được dùng làm Pickup Location và Dropoff Location trong transaction fact, còn snapshot Zone Traffic sử dụng vai trò vị trí của sự kiện. Một dimension vật lý được tái sử dụng qua nhiều khóa ngoại thay vì nhân bản toàn bộ danh mục.
Dim_Base cũng xuất hiện ở DispatchBaseKey, OriginBaseKey và AffiliatedBaseKey tùy nguồn. Ví dụ lọc Dispatch Base trả lời câu hỏi về đơn vị điều phối, trong khi lọc Origin Base hỏi nơi nhận yêu cầu ban đầu. Gộp các vai trò thành một khóa base duy nhất sẽ làm mất khả năng phân biệt hai trách nhiệm này.

2.5 Thiết kế các bảng chiều

Mỗi bảng chiều dưới đây trình bày ý nghĩa, khóa nghiệp vụ, khóa thay thế và toàn bộ thuộc tính trong metadata đã lưu. Dòng khóa 0 là thành viên Unknown.

Kiểu chuỗi được hiển thị theo số ký tự; metadata nvarchar gốc lưu độ dài byte nên phải chia hai. Bản chụp columns.json không có precision và scale của decimal, do đó cấu hình số thập phân chính xác phải đối chiếu DDL và biểu thức chuyển kiểu trong đặc tả SQL, không suy ra từ kích thước byte.

2.5.1 Nhóm chiều ngày và thời gian

Dim_Date tách lịch khỏi sự kiện chuyến đi, cho phép cùng một cấu trúc năm quý tháng ngày được dùng ở mọi fact. Khóa ngày phục vụ join, FullDate phục vụ hiển thị và sắp xếp. IsWeekend được dùng theo lịch seed; IsHoliday chưa có lịch ngày lễ đã được xác nhận nên chưa đủ cơ sở phân tích nhu cầu ngày lễ. Dim_Time lưu đến phút và cho phép roll up lên giờ hoặc TimeBand. Khóa phút hợp lệ từ 1 đến 1440, khóa 0 dành Unknown để phân biệt thiếu giờ với đúng 00 giờ. Khi phân cấp lên giờ, khóa cần giữ sự phân biệt này; chỉ dùng HourNumber có thể gộp Unknown với nửa đêm.

Dim_Date

Bảng 2.2 Cấu trúc và ý nghĩa các cột Dim_Date

Cột	**Kiểu SQL**	**NULL và khóa**	**Ý nghĩa**
DateKey	int	Không; PK	Khóa thay thế của Dim_Date; thành viên 0 là Unknown theo thiết kế.
FullDate	date	Có; Thuộc tính	Ngày dương lịch đầy đủ để hiển thị và sắp xếp.
DayOfWeekNumber	tinyint	Có; Thuộc tính	Số thứ tự ngày trong tuần theo seed lịch của dự án.
DayOfWeekName	nvarchar(20)	Có; Thuộc tính	Tên ngày trong tuần.

Cột	Kiểu SQL	NULL và khóa	Ý nghĩa
DayOfMonth	tinyint	Có; Thuộc tính	Ngày trong tháng từ 1 đến 31.
MonthNumber	tinyint	Có; Thuộc tính	Số tháng 1 đến 12; khóa hierarchy ghép cùng năm.
MonthName	nvarchar(20)	Có; Thuộc tính	Tên tháng hiển thị, sắp theo MonthNumber.
QuarterNumber	tinyint	Có; Thuộc tính	Số quý 1 đến 4; khóa hierarchy ghép cùng năm.
YearNumber	smallint	Có; Thuộc tính	Năm dương lịch.
IsWeekend	bit	Có; Thuộc tính	Cờ cuối tuần theo seed lịch.
IsHoliday	bit	Có; Thuộc tính	Cờ ngày lễ; lịch ngày lễ chưa được xác nhận trong dữ liệu đang rà soát.

Dim_Time

Bảng 2.3 Cấu trúc và ý nghĩa các cột Dim_Time

Cột	Kiểu SQL	NULL và khóa	Ý nghĩa
TimeKey	smallint	Không; PK	Khóa thay thế của Dim_Time; thành viên 0 là Unknown theo thiết kế.
MinuteOfDay	smallint	Có; Thuộc tính	Phút trong ngày; khóa TimeKey hợp lệ 1 đến 1440, 0 dành Unknown.

Cột	Kiểu SQL	NULL và khóa	Ý nghĩa
HourNumber	tinyint	Có; Thuộc tính	Giờ 0 đến 23 với dữ liệu hợp lệ; phân biệt Unknown trong key.
MinuteNumber	tinyint	Có; Thuộc tính	Phút trong giờ 0 đến 59.
TimeLabel	char(5)	Có; Thuộc tính	Nhãn giờ phút của thành viên.
TimeBand	varchar(20)	Có; Thuộc tính	Nhóm khung giờ theo seed để phân cấp.
IsPeakHour	bit	Có; Thuộc tính	Cờ khung giờ cao điểm theo quy tắc seed, không phải kết luận từ toàn bộ nguồn NYC.

2.5.2 Nhóm chiều địa lý

Dim_Location sử dụng LocationID làm mã nguồn và LocationKey làm khóa nội bộ. Borough, ZoneName và ServiceZone mô tả vùng ở các mức khác nhau. Hierarchy Borough đến Zone cho phép từ phân tích quận drill xuống từng vùng, nhưng không tự cung cấp tọa độ hoặc thông tin tuyến đường chi tiết.

Cập nhật tên vùng hiện áp dụng SCD Type 1, nghĩa là ghi đè thuộc tính hiện hành trên cùng khóa. Nếu cần tái hiện tên vùng đúng theo từng giai đoạn lịch sử thì phải thiết kế thêm hiệu lực và phiên bản; báo cáo không coi cơ chế đó đã tồn tại. Một mã vùng nguồn chưa được tra cứu được ánh xạ Unknown có kiểm soát và có thể được xử lý lại khi mapping bổ sung.

Dim_Location

Bảng 2.4 Cấu trúc và ý nghĩa các cột Dim_Location

Cột	**Kiểu SQL**	**NULL và khóa**	**Ý nghĩa**
LocationKey	int	Không; PK	Khóa thay thế của Dim_Location; thành viên 0 là Unknown theo thiết kế.
LocationID	int	Có; Thuộc tính	Mã Taxi Zone nghiệp vụ của TLC.
Borough	nvarchar(100)	Có; Thuộc tính	Quận hoặc nhóm địa lý của Taxi Zone.
ZoneName	nvarchar(200)	Có; Thuộc tính	Tên vùng hiển thị; LocationKey xác định tính duy nhất.
ServiceZone	nvarchar(100)	Có; Thuộc tính	Nhóm vùng phục vụ theo danh mục TLC.

2.5.3 Nhóm chiều dịch vụ và nhà cung cấp

Dim_ServiceType phân biệt Yellow, Green, FHV và HVFHV, là chiều quan trọng khi tổng hợp liên nguồn. Dim_Vendor giải mã nhà cung cấp công nghệ taxi. Dim_HVFHSProvider giải mã giấy phép nhà cung cấp dịch vụ quy mô lớn. Ba chiều có nhiệm vụ khác nhau nên không được thay thế bằng một cột tên doanh nghiệp chung.

Trong YellowGreen fact, ServiceTypeKey xác định nguồn dù các trường tiền được đặt chung tên. Trong HVFHV fact, provider cho phép phân tích theo giấy phép. Các mã doanh nghiệp trong danh mục là thông tin gắn với kỳ dữ liệu; không dùng danh sách lịch sử để khẳng định doanh nghiệp vẫn hoạt động ở hiện tại.

Dim_ServiceType

Bảng 2.5 Cấu trúc và ý nghĩa các cột Dim_ServiceType

Cột	Kiểu SQL	NULL và khóa	Ý nghĩa
ServiceTypeKey	int	Không; PK	Khóa thay thế của Dim_ServiceType; thành viên 0 là Unknown theo thiết kế.
ServiceCode	varchar(10)	Có; Thuộc tính	Mã Yellow Green FHV hoặc HVFHV.
ServiceName	nvarchar(100)	Có; Thuộc tính	Tên dịch vụ hiển thị.

Dim_Vendor

Bảng 2.6 Cấu trúc và ý nghĩa các cột Dim_Vendor

Cột	Kiểu SQL	NULL và khóa	Ý nghĩa
VendorKey	int	Không; PK	Khóa thay thế của Dim_Vendor; thành viên 0 là Unknown theo thiết kế.
VendorID	int	Có; Thuộc tính	Mã nhà cung cấp công nghệ TPEP hoặc LPEP.
VendorName	nvarchar(200)	Có; Thuộc tính	Tên nhà cung cấp công nghệ tương ứng mã nguồn.

Dim_HVFHSProvider

Bảng 2.7 Cấu trúc và ý nghĩa các cột Dim_HVFHSProvider

Cột	Kiểu SQL	NULL và khóa	Ý nghĩa
HVFHSProviderKey	int	Không; PK	Khóa thay thế của Dim_HVFHSProvider; thành viên 0 là Unknown theo thiết kế.

Cột	Kiểu SQL	NULL và khóa	Ý nghĩa
LicenseNumber	varchar(30)	Có; Thuộc tính	Số giấy phép HVFHS dùng nhận diện nhà cung cấp.
ProviderName	nvarchar(200)	Có; Thuộc tính	Tên nhà cung cấp tương ứng giấy phép nguồn.

2.5.4 Nhóm chiều base điều phối

Dim_Base lưu số giấy phép, tên và nhóm base nếu có. BaseNumber là mã nghiệp vụ; BaseKey là khóa thay thế được fact tham chiếu. Loader bổ sung mã base phát hiện từ tập file cần nạp, tránh tạo một dòng fact với mã nghiệp vụ nằm trực tiếp trong các cột khóa dimension.

Thiết kế hiện chủ yếu thêm mã base, chưa có SCD Type 2 và lookup theo khoảng hiệu lực. Các vai trò dispatch, origin và affiliated dùng cùng bảng chiều nhưng không có nghĩa ba khóa luôn bằng nhau. Việc phân tích hiệu suất phải chỉ rõ vai trò base đã chọn.

Dim_Base

Bảng 2.8 Cấu trúc và ý nghĩa các cột Dim_Base

Cột	Kiểu SQL	NULL và khóa	Ý nghĩa
BaseKey	int	Không; PK	Khóa thay thế của Dim_Base; thành viên 0 là Unknown theo thiết kế.
BaseNumber	varchar(30)	Có; Thuộc tính	Số giấy phép base của TLC, là mã nghiệp vụ tra cứu cho các vai trò base.
BaseName	nvarchar(200)	Có; Thuộc tính	Tên base theo danh mục, có thể thiếu khi chưa có thông tin mô tả.

Cột	Kiểu SQL	NULL và khóa	Ý nghĩa
BaseCategory	varchar(50)	Có; Thuộc tính	Nhóm phân loại base theo danh mục.

2.5.5 Nhóm chiều phân loại chuyến đi và thanh toán

Dim_RateCode lưu loại biểu giá; Dim_PaymentType lưu phương thức hoặc tình trạng thanh toán; Dim_TripType lưu cách nhận chuyến Green; Dim_PassengerGroup tạo nhóm số hành khách để phân tích. Những bảng này đưa thuộc tính phân loại ra khỏi fact và cung cấp tên hiển thị thống nhất cho báo cáo.

Mã thanh toán là mã danh mục, không phải giá trị tiền. PassengerCount trong fact vẫn là số hành khách của dòng; PassengerGroup chỉ bổ sung cách phân loại. Khi nguồn không cung cấp một phân loại, loader sử dụng quy tắc Unknown hoặc không áp dụng của mô hình, không tự suy ra từ cước hoặc loại xe.

Dim_RateCode

Bảng 2.9 Cấu trúc và ý nghĩa các cột Dim_RateCode

Cột	Kiểu SQL	NULL và khóa	Ý nghĩa
RateCodeKey	int	Không; PK	Khóa thay thế của Dim_RateCode; thành viên 0 là Unknown theo thiết kế.
RateCodeID	int	Có; Thuộc tính	Mã biểu giá taxi cuối chuyến theo nguồn.
RateCodeDescription	nvarchar(200)	Có; Thuộc tính	Tên diễn giải biểu giá.

Dim_PaymentType

Bảng 2.10 Cấu trúc và ý nghĩa các cột Dim_PaymentType

Cột	Kiểu SQL	NULL và khóa	Ý nghĩa
PaymentTypeKey	int	Không; PK	Khóa thay thế của Dim_PaymentType; thành viên 0 là Unknown theo thiết kế.
PaymentTypeID	int	Có; Thuộc tính	Mã phương thức hoặc tình trạng thanh toán, không có đơn vị tiền.
PaymentTypeDescription	nvarchar(200)	Có; Thuộc tính	Tên diễn giải mã thanh toán.

Dim_TripType

Bảng 2.11 Cấu trúc và ý nghĩa các cột Dim_TripType

Cột	Kiểu SQL	NULL và khóa	Ý nghĩa
TripTypeKey	int	Không; PK	Khóa thay thế của Dim_TripType; thành viên 0 là Unknown theo thiết kế.
TripTypeID	int	Có; Thuộc tính	Mã cách nhận chuyến Green theo nguồn.
TripTypeDescription	nvarchar(200)	Có; Thuộc tính	Tên diễn giải loại chuyến.

Dim_PassengerGroup

Bảng 2.12 Cấu trúc và ý nghĩa các cột Dim_PassengerGroup

Cột	Kiểu SQL	NULL và khóa	Ý nghĩa
PassengerGroupKey	int	Không; PK	Khóa thay thế của Dim_PassengerGroup; thành viên 0 là Unknown theo thiết kế.

Cột	Kiểu SQL	NULL và khóa	Ý nghĩa
PassengerGroupCode	varchar(20)	Có; Thuộc tính	Mã nhóm số hành khách theo seed.
MinCount	int	Có; Thuộc tính	Cận dưới số hành khách của nhóm.
MaxCount	int	Có; Thuộc tính	Cận trên số hành khách của nhóm.
GroupName	nvarchar(100)	Có; Thuộc tính	Tên nhóm số hành khách.

2.5.6 Nhóm chiều cờ nghiệp vụ

Dim_TaxiFlags giữ cờ store and forward. Dim_FHVFlags giữ trạng thái đi chung được chuẩn hóa từ SR_Flag. Dim_HVFHVFlags giữ tổ hợp cờ yêu cầu đi chung, đã ghép, Access A Ride, yêu cầu WAV và chuyến thực hiện bằng WAV. Các bảng này là junk dimension, gom tổ hợp cờ để fact tham chiếu bằng một khóa.

Trạng thái U được giữ khi chưa biết; không tự chuyển NULL thành N. Cờ yêu cầu đi chung khác với cờ đã chia sẻ xe; tương tự cờ yêu cầu WAV khác với cờ chuyến xảy ra bằng xe WAV. Sự khác biệt này quyết định cách xây tỷ lệ nghiệp vụ và là lý do không dùng mọi phép chia giữa các cờ như một KPI tỷ lệ đáp ứng.

Dim_TaxiFlags

Bảng 2.13 Cấu trúc và ý nghĩa các cột Dim_TaxiFlags

Cột	Kiểu SQL	NULL và khóa	Ý nghĩa
TaxiFlagKey	int	Không; PK	Khóa thay thế của Dim_TaxiFlags; thành viên 0 là Unknown theo thiết kế.

Cột	Kiểu SQL	NULL và khóa	Ý nghĩa
StoreAndFwd	varchar(5)	Có; Thuộc tính	Cờ dữ liệu được lưu trên xe rồi truyền về, Y N U.

Dim_FHVFlags

Bảng 2.14 Cấu trúc và ý nghĩa các cột Dim_FHVFlags

Cột	Kiểu SQL	NULL và khóa	Ý nghĩa
FHVFlagKey	int	Không; PK	Khóa thay thế của Dim_FHVFlags; thành viên 0 là Unknown theo thiết kế.
IsSharedRide	varchar(5)	Có; Thuộc tính	Cờ đi chung FHV chuẩn hóa từ SourceSRFlag, Y N U.

Dim_HVFHVFlags

Bảng 2.15 Cấu trúc và ý nghĩa các cột Dim_HVFHVFlags

Cột	Kiểu SQL	NULL và khóa	Ý nghĩa
HVFHVFlagKey	int	Không; PK	Khóa thay thế của Dim_HVFHVFlags; thành viên 0 là Unknown theo thiết kế.
IsSharedRequest	varchar(5)	Có; Thuộc tính	Cờ khách đồng ý hoặc yêu cầu chuyến đi chung, Y N U.
IsSharedMatch	varchar(5)	Có; Thuộc tính	Cờ chuyến có hành khách đặt riêng được ghép cùng xe, Y N U.
IsAccessARide	varchar(5)	Có; Thuộc tính	Cờ chuyến thuộc chương trình Access A Ride, Y N U.
IsWAVRequest	varchar(5)	Có; Thuộc tính	Cờ hành khách yêu cầu xe tiếp cận xe lăn, Y N U.

Cột	Kiểu SQL	NULL và khóa	Ý nghĩa
IsWAVMatch	varchar(5)	Có; Thuộc tính	Cờ chuyến thực hiện bằng xe tiếp cận xe lăn; không mặc nhiên là đáp ứng yêu cầu WAV cùng tập.

2.5.7 Chiều lô nạp dữ liệu

Dim_LoadBatch lưu ngữ cảnh nhóm xử lý như dataset, năm tháng và trạng thái lô. Trong bằng chứng hiện có, đây là nhóm nạp suy từ tháng pickup và SourceFileName có thể NULL. Vì vậy không coi LoadBatchKey là định danh file nguồn đầy đủ.

Lineage theo file phải dùng SourceDataset và SourceKey trong fact, kết hợp log Extract, Transform và DWH. Một tháng pickup có thể chứa chuyến từ nhiều file nguồn và một file tháng có thể chứa thời điểm ngoài tháng danh nghĩa. Sự phân biệt này giúp tránh truy vết sai khi nguồn được hiệu chỉnh.

Dim_LoadBatch

Bảng 2.16 Cấu trúc và ý nghĩa các cột Dim_LoadBatch

Cột	Kiểu SQL	NULL và khóa	Ý nghĩa
LoadBatchKey	int	Không; PK	Khóa thay thế của Dim_LoadBatch; thành viên 0 là Unknown theo thiết kế.
BatchID	varchar(50)	Có; Thuộc tính	Mã nhận diện nhóm lô xử lý trong bảng chiều, không phải TripID.
DatasetName	varchar(30)	Có; Thuộc tính	Tên dataset của nhóm lô.
SourceFileName	nvarchar(500)	Có; Thuộc tính	Tên file nếu được cung cấp; bằng chứng hiện có có giá trị NULL, không dùng thay lineage file fact.

Cột	Kiểu SQL	NULL và khóa	Ý nghĩa
SourceYear	smallint	Có; Thuộc tính	Năm của nhóm nạp suy theo thiết kế lô.
SourceMonth	tinyint	Có; Thuộc tính	Tháng của nhóm nạp suy theo thiết kế lô.
LoadedAt	datetime2	Có; Thuộc tính	Thời điểm dữ liệu được ghi vào đối tượng trong kho, không phải thời điểm chuyến đi.
BatchStatus	varchar(20)	Có; Thuộc tính	Trạng thái của nhóm lô theo ETL.

2.6 Thiết kế các bảng sự kiện

2.6.1 Fact_YellowGreen_Trips

Loại fact là Transaction Fact. Grain là một lượt xuất hiện chuyến Yellow hoặc Green hợp lệ trong một file nguồn. Khóa TripKey là khóa thay thế nội bộ; nội dung bản ghi, nguồn và thứ tự xuất hiện của dòng trùng được dùng khi đồng bộ. Không xem TripKey là mã chuyến kinh doanh được TLC bảo đảm bền vững.
Lý do gộp Yellow và Green là hai nguồn có nhiều measure đồng nhất về thời gian, quãng đường và cước taxi. ServiceTypeKey bảo toàn sự phân biệt dịch vụ. Fact nối đến ngày, giờ, vùng ở hai vai trò đón trả cùng các chiều vendor, biểu giá, thanh toán, loại chuyến, cờ taxi, nhóm hành khách và lô nạp.
TripCount cộng để tính số chuyến; PassengerCount cộng để tính số hành khách được ghi nhận; các khoản tiền và khoảng cách cộng trong phạm vi phù hợp. AverageSpeedMPH ở mức dòng không có tính cộng dồn. Tốc độ toàn nhóm phải tính từ tổng quãng đường và thời lượng của cùng tập dòng hợp lệ, thay vì cộng hoặc trung bình đơn giản tốc độ từng chuyến.

Fact_YellowGreen_Trips

Bảng 2.17 Cấu trúc và ý nghĩa các cột Fact_YellowGreen_Trips

Cột	Kiểu SQL	NULL và khóa	Ý nghĩa
TripKey	bigint	Không; PK	Khóa thay thế dòng fact; có thể đổi khi nguồn hiệu chỉnh, không phải TripID TLC.
PickupDateKey	int	Không; FK	Khóa ngoại Dim_Date.DateKey; vai trò PickupDate. Khóa 0 giữ Unknown.
DropoffDateKey	int	Không; FK	Khóa ngoại Dim_Date.DateKey; vai trò DropoffDate. Khóa 0 giữ Unknown.
PickupTimeKey	smallint	Không; FK	Khóa ngoại Dim_Time.TimeKey; vai trò PickupTime. Khóa 0 giữ Unknown.
DropoffTimeKey	smallint	Không; FK	Khóa ngoại Dim_Time.TimeKey; vai trò DropoffTime. Khóa 0 giữ Unknown.
PickupLocationKey	int	Không; FK	Khóa ngoại Dim_Location.LocationKey; vai trò PickupLocation. Khóa 0 giữ Unknown.
DropoffLocationKey	int	Không; FK	Khóa ngoại Dim_Location.LocationKey; vai trò DropoffLocation. Khóa 0 giữ Unknown.
ServiceTypeKey	int	Không; FK	Khóa ngoại Dim_ServiceType.ServiceTypeKey; vai trò ServiceType. Khóa 0 giữ Unknown.
VendorKey	int	Không; FK	Khóa ngoại Dim_Vendor.VendorKey; vai trò Vendor. Khóa 0 giữ Unknown.
RateCodeKey	int	Không; FK	Khóa ngoại Dim_RateCode.RateCodeKey; vai trò RateCode. Khóa 0 giữ Unknown.
PaymentTypeKey	int	Không; FK	Khóa ngoại Dim_PaymentType.PaymentTypeKey; vai trò PaymentType. Khóa 0 giữ Unknown.

Cột	Kiểu SQL	NULL và khóa	Ý nghĩa
TripTypeKey	int	Không; FK	Khóa ngoại Dim_TripType.TripTypeKey; vai trò TripType. Khóa 0 giữ Unknown.
TaxiFlagKey	int	Không; FK	Khóa ngoại Dim_TaxiFlags.TaxiFlagKey; vai trò TaxiFlag. Khóa 0 giữ Unknown.
PassengerGroupKey	int	Không; FK	Khóa ngoại Dim_PassengerGroup.PassengerGroupKey; vai trò PassengerGroup. Khóa 0 giữ Unknown.
LoadBatchKey	int	Không; FK	Khóa ngoại Dim_LoadBatch.LoadBatchKey; vai trò LoadBatch. Khóa 0 giữ Unknown.
TripCount	int	Không; Thuộc tính	Số chuyến của dòng giao dịch, dùng cộng tổng.
PassengerCount	int	Có; Thuộc tính	Số hành khách nguồn ghi nhận; thiếu dữ liệu được giữ theo quy tắc.
TripDistanceMiles	decimal	Có; Thuộc tính	Quãng đường chuyến, đơn vị mile.
TripDurationSeconds	bigint	Có; Thuộc tính	Thời lượng pickup đến dropoff, giây.
AverageSpeedMPH	decimal	Có; Thuộc tính	Tốc độ riêng dòng tính theo mile mỗi giờ; không cộng trực tiếp.
FareAmount	decimal	Có; Thuộc tính	Cước cơ bản taxi, USD; khác tổng tiền.
ExtraAmount	decimal	Có; Thuộc tính	Phụ thu bổ sung taxi, USD.

Cột	Kiểu SQL	NULL và khóa	Ý nghĩa
MTATaxAmount	decimal	Có; Thuộc tính	Thuế MTA nguồn ghi riêng, USD.
TipAmount	decimal	Có; Thuộc tính	Tip ghi nhận, USD; taxi không bao gồm tip tiền mặt.
TollsAmount	decimal	Có; Thuộc tính	Phí cầu đường, USD.
ImprovementSurcharge	decimal	Có; Thuộc tính	Phụ thu cải thiện taxi, USD.
CongestionSurcharge	decimal	Có; Thuộc tính	Phụ thu ùn tắc, USD; tách phí CBD.
AirportFee	decimal	Có; Thuộc tính	Phí sân bay theo nguồn, USD.
CBDCongestionFee	decimal	Có; Thuộc tính	Phí vùng trung tâm CBD theo nguồn và thời kỳ, USD.
TotalAmount	decimal	Có; Thuộc tính	Tổng tiền khách taxi nguồn ghi nhận, USD.
RecordHash	binary	Có; Thuộc tính	Hash payload cùng ngữ cảnh file và ordinal trùng, phục vụ đồng bộ; không là mã chuyến TLC.
LoadedAt	datetime2	Không; Thuộc tính	Thời điểm dữ liệu được ghi vào đối tượng trong kho, không phải thời điểm chuyến đi.

Cột	Kiểu SQL	NULL và khóa	Ý nghĩa
SourceDataset	varchar(20)	Có; Thuộc tính	Mã dataset nguồn dùng cùng SourceKey để truy vết và đồng bộ.
SourceKey	nvarchar(260)	Có; Thuộc tính	Khóa tên file nguồn, không phải mã chuyến.

2.6.2 Fact_FHV_Trips

Loại fact là Transaction Fact; grain là một lượt xuất hiện chuyến FHV hợp lệ từ file nguồn. Các khóa ngày, giờ và vùng ghi hai vai trò pickup, dropoff; base có vai trò dispatch và affiliated. FHVFlagKey giữ cờ đi chung theo quy tắc nguồn đã chuẩn hóa.

Nguồn không cung cấp đầy đủ quãng đường và các khoản thu nên fact chỉ tập trung số chuyến và thời lượng cùng ngữ cảnh điều phối. Đây là lựa chọn bảo toàn ý nghĩa dữ liệu, không phải thiếu một measure cần điền 0. Thời lượng được tính từ pickup đến dropoff và kiểm tra điều kiện cốt lõi trước khi nạp.

Mọi tổng hợp theo địa lý cần giữ Unknown vì tập FHV có thể thiếu nhiều vị trí. Thống kê trên các vùng đã biết phải nêu bộ lọc; nếu âm thầm bỏ Unknown, tổng trên bản đồ có thể thấp hơn card tổng chuyến mà không có giải thích.

Fact_FHV_Trips

Bảng 2.18 Cấu trúc và ý nghĩa các cột Fact_FHV_Trips

Cột	Kiểu SQL	NULL và khóa	Ý nghĩa
TripKey	bigint	Không; PK	Khóa thay thế dòng fact; có thể đổi khi nguồn hiệu chỉnh, không phải TripID TLC.
PickupDateKey	int	Không; FK	Khóa ngoại Dim_Date.DateKey; vai trò PickupDate. Khóa 0 giữ Unknown.

Cột	Kiểu SQL	NULL và khóa	Ý nghĩa
DropoffDateKey	int	Không; FK	Khóa ngoại Dim_Date.DateKey; vai trò DropoffDate. Khóa 0 giữ Unknown.
PickupTimeKey	smallint	Không; FK	Khóa ngoại Dim_Time.TimeKey; vai trò PickupTime. Khóa 0 giữ Unknown.
DropoffTimeKey	smallint	Không; FK	Khóa ngoại Dim_Time.TimeKey; vai trò DropoffTime. Khóa 0 giữ Unknown.
PickupLocationKey	int	Không; FK	Khóa ngoại Dim_Location.LocationKey; vai trò PickupLocation. Khóa 0 giữ Unknown.
DropoffLocationKey	int	Không; FK	Khóa ngoại Dim_Location.LocationKey; vai trò DropoffLocation. Khóa 0 giữ Unknown.
ServiceTypeKey	int	Không; FK	Khóa ngoại Dim_ServiceType.ServiceTypeKey; vai trò ServiceType. Khóa 0 giữ Unknown.
DispatchBaseKey	int	Không; FK	Khóa ngoại Dim_Base.BaseKey; vai trò DispatchBase. Khóa 0 giữ Unknown.
AffiliatedBaseKey	int	Không; FK	Khóa ngoại Dim_Base.BaseKey; vai trò AffiliatedBase. Khóa 0 giữ Unknown.
FHVFlagKey	int	Không; FK	Khóa ngoại Dim_FHVFlags.FHVFlagKey; vai trò FHVFlag. Khóa 0 giữ Unknown.
LoadBatchKey	int	Không; FK	Khóa ngoại Dim_LoadBatch.LoadBatchKey; vai trò LoadBatch. Khóa 0 giữ Unknown.
TripCount	int	Không; Thuộc tính	Số chuyến của dòng giao dịch, dùng cộng tổng.
TripDurationSeconds	bigint	Có; Thuộc tính	Thời lượng pickup đến dropoff, giây.

Cột	Kiểu SQL	NULL và khóa	Ý nghĩa
RecordHash	binary	Có; Thuộc tính	Hash payload cùng ngữ cảnh file và ordinal trùng, phục vụ đồng bộ; không là mã chuyến TLC.
LoadedAt	datetime2	Không; Thuộc tính	Thời điểm dữ liệu được ghi vào đối tượng trong kho, không phải thời điểm chuyến đi.
SourceDataset	varchar(20)	Có; Thuộc tính	Mã dataset nguồn dùng cùng SourceKey để truy vết và đồng bộ.
SourceKey	nvarchar(260)	Có; Thuộc tính	Khóa tên file nguồn, không phải mã chuyến.

2.6.3 Fact_HVFHV_Trips

Loại fact là Transaction Fact; grain là một lượt xuất hiện chuyến HVFHV hợp lệ. Các mốc Request, On Scene, Pickup và Dropoff cùng nằm trên một dòng để phân tích chờ và phục vụ. Tuy có nhiều mốc, fact không được coi là accumulating snapshot cập nhật tiến trình trực tiếp vì đầu vào đang là bản ghi chuyến được công bố theo file.

Fact tham chiếu provider, base điều phối, base nhận yêu cầu và dimension cờ chuyên biệt. Các measure gồm quãng đường, thời lượng, thời gian chờ, thời gian từ on scene đến pickup, cước cơ bản, các khoản thuế phí, tip, tiền tài xế và tổng tiền hành khách theo công thức nguồn trong loader. WaitingTimeSeconds sử dụng Request đến Pickup khi thứ tự thời gian hợp lệ; đây không phải luôn là thời gian xe đến điểm đón.

GrossPassengerAmount được xây từ các thành phần tài chính theo SQL, tách khỏi DriverPayAmount. Chênh lệch hai đại lượng không được đặt tên là lợi nhuận vì chưa có đầy đủ cấu phần chi phí và quy tắc kế toán. Các chỉ tiêu trung bình giữ số dòng hợp lệ làm mẫu số riêng cho từng measure.

Fact_HVFHV_Trips

Bảng 2.19 Cấu trúc và ý nghĩa các cột Fact_HVFHV_Trips

Cột	Kiểu SQL	NULL và khóa	Ý nghĩa
TripKey	bigint	Không; PK	Khóa thay thế dòng fact; có thể đổi khi nguồn hiệu chỉnh, không phải TripID TLC.
RequestDateKey	int	Không; FK	Khóa ngoại Dim_Date.DateKey; vai trò RequestDate. Khóa 0 giữ Unknown.
OnSceneDateKey	int	Không; FK	Khóa ngoại Dim_Date.DateKey; vai trò OnSceneDate. Khóa 0 giữ Unknown.
PickupDateKey	int	Không; FK	Khóa ngoại Dim_Date.DateKey; vai trò PickupDate. Khóa 0 giữ Unknown.
DropoffDateKey	int	Không; FK	Khóa ngoại Dim_Date.DateKey; vai trò DropoffDate. Khóa 0 giữ Unknown.
RequestTimeKey	smallint	Không; FK	Khóa ngoại Dim_Time.TimeKey; vai trò RequestTime. Khóa 0 giữ Unknown.
OnSceneTimeKey	smallint	Không; FK	Khóa ngoại Dim_Time.TimeKey; vai trò OnSceneTime. Khóa 0 giữ Unknown.
PickupTimeKey	smallint	Không; FK	Khóa ngoại Dim_Time.TimeKey; vai trò PickupTime. Khóa 0 giữ Unknown.
DropoffTimeKey	smallint	Không; FK	Khóa ngoại Dim_Time.TimeKey; vai trò DropoffTime. Khóa 0 giữ Unknown.
PickupLocationKey	int	Không; FK	Khóa ngoại Dim_Location.LocationKey; vai trò PickupLocation. Khóa 0 giữ Unknown.
DropoffLocationKey	int	Không; FK	Khóa ngoại Dim_Location.LocationKey; vai trò DropoffLocation. Khóa 0 giữ Unknown.

Cột	Kiểu SQL	NULL và khóa	Ý nghĩa
ServiceTypeKey	int	Không; FK	Khóa ngoại Dim_ServiceType.ServiceTypeKey; vai trò ServiceType. Khóa 0 giữ Unknown.
HVFHSProviderKey	int	Không; FK	Khóa ngoại Dim_HVFHSProvider.HVFHSProviderKey; vai trò HVFHSProvider. Khóa 0 giữ Unknown.
DispatchBaseKey	int	Không; FK	Khóa ngoại Dim_Base.BaseKey; vai trò DispatchBase. Khóa 0 giữ Unknown.
OriginBaseKey	int	Không; FK	Khóa ngoại Dim_Base.BaseKey; vai trò OriginBase. Khóa 0 giữ Unknown.
HVFHVFlagKey	int	Không; FK	Khóa ngoại Dim_HVFHVFlags.HVFHVFlagKey; vai trò HVFHVFlag. Khóa 0 giữ Unknown.
LoadBatchKey	int	Không; FK	Khóa ngoại Dim_LoadBatch.LoadBatchKey; vai trò LoadBatch. Khóa 0 giữ Unknown.
TripCount	int	Không; Thuộc tính	Số chuyến của dòng giao dịch, dùng cộng tổng.
TripDistanceMiles	decimal	Có; Thuộc tính	Quãng đường chuyến, đơn vị mile.
TripDurationSeconds	bigint	Có; Thuộc tính	Thời lượng pickup đến dropoff, giây.
WaitingTimeSeconds	bigint	Có; Thuộc tính	Thời gian request đến pickup hợp lệ, giây.
OnSceneToPickupSeconds	bigint	Có; Thuộc tính	Chênh lệch on scene đến pickup hợp lệ, giây; độ bao phủ phụ thuộc nguồn.

Cột	Kiểu SQL	NULL và khóa	Ý nghĩa
BasePassengerFare	decimal	Có; Thuộc tính	Cước cơ bản HVFHV trước các khoản tách riêng, USD.
TollsAmount	decimal	Có; Thuộc tính	Phí cầu đường, USD.
BCFAmount	decimal	Có; Thuộc tính	Khoản Black Car Fund, USD.
SalesTaxAmount	decimal	Có; Thuộc tính	Thuế bán hàng HVFHV, USD.
CongestionSurcharge	decimal	Có; Thuộc tính	Phụ thu ùn tắc, USD; tách phí CBD.
AirportFee	decimal	Có; Thuộc tính	Phí sân bay theo nguồn, USD.
TipAmount	decimal	Có; Thuộc tính	Tip ghi nhận, USD; taxi không bao gồm tip tiền mặt.
DriverPayAmount	decimal	Có; Thuộc tính	Tiền tài xế HVFHV được nguồn ghi nhận, USD, giữ riêng tip và phí theo định nghĩa nguồn.
CBDCongestionFee	decimal	Có; Thuộc tính	Phí vùng trung tâm CBD theo nguồn và thời kỳ, USD.
GrossPassengerAmount	decimal	Có; Thuộc tính	Tổng tiền khách HVFHV theo công thức loader, USD.

Cột	Kiểu SQL	NULL và khóa	Ý nghĩa
RecordHash	binary	Có; Thuộc tính	Hash payload cùng ngữ cảnh file và ordinal trùng, phục vụ đồng bộ; không là mã chuyến TLC.
LoadedAt	datetime2	Không; Thuộc tính	Thời điểm dữ liệu được ghi vào đối tượng trong kho, không phải thời điểm chuyến đi.
SourceDataset	varchar(20)	Có; Thuộc tính	Mã dataset nguồn dùng cùng SourceKey để truy vết và đồng bộ.
SourceKey	nvarchar(260)	Có; Thuộc tính	Khóa tên file nguồn, không phải mã chuyến.

2.6.4 Fact_Daily_Financial_Snapshot

Loại fact là Periodic Snapshot ở grain DateKey và ServiceTypeKey. Một dòng mô tả tổng tài chính của một ngày và một dịch vụ có dữ liệu tài chính. Khóa chính ghép bảo đảm không có hai dòng cùng tổ hợp này. FHV không được đưa vào tổng tiền bằng các giá trị giả định.

Các measure tổng gồm số chuyến, quãng đường, tiền khách trả, tip, thuế phí và tiền tài xế. NULL giữ ý nghĩa thiếu hoặc không áp dụng ở những cấu phần nguồn không cung cấp. Snapshot phục vụ so sánh tài chính theo ngày và dịch vụ, nhưng không được cộng cùng fact giao dịch để tạo tổng mới vì đó là hai mức biểu diễn của cùng dữ liệu.

Fact_Daily_Financial_Snapshot

Bảng 2.20 Cấu trúc và ý nghĩa các cột Fact_Daily_Financial_Snapshot

Cột	Kiểu SQL	NULL và khóa	Ý nghĩa
DateKey	int	Không; PK	Khóa ngoại Dim_Date.DateKey; vai trò Date. Khóa 0 giữ Unknown.

Cột	Kiểu SQL	NULL và khóa	Ý nghĩa
ServiceTypeKey	int	Không; PK	Khóa ngoại Dim_ServiceType.ServiceTypeKey; vai trò ServiceType. Khóa 0 giữ Unknown.
TotalTrips	bigint	Không; Thuộc tính	Tổng số chuyến ở grain snapshot.
TotalDistanceMiles	decimal	Có; Thuộc tính	Tổng quãng đường của tập được tổng hợp, mile.
TotalPassengerCharges	decimal	Có; Thuộc tính	Tổng tiền khách theo nguồn tài chính, USD.
TotalTips	decimal	Có; Thuộc tính	Tổng tip được ghi nhận, USD.
TotalTaxesAndFees	decimal	Có; Thuộc tính	Tổng thuế và phí theo công thức aggregate, USD.
TotalDriverPay	decimal	Có; Thuộc tính	Tổng tiền tài xế ghi nhận theo nguồn có dữ liệu, USD.
SnapshotUpdatedAt	datetime2	Không; Thuộc tính	Thời điểm cập nhật snapshot, khác ngày nghiệp vụ.

2.6.5 Fact_Zone_Traffic_Snapshot

Loại fact là Periodic Snapshot; grain vật lý là DateKey, TimeKey, LocationKey và ServiceTypeKey. TimeKey có mức phút nên việc trình bày theo giờ là roll up qua Dim_Time, không thay đổi grain của bảng. Khóa chính ghép bốn cột ngăn trùng nhóm tổng hợp.

Pickup được nhóm theo ngày, phút và vùng đón; dropoff được nhóm theo ngày, phút và vùng trả. Hai tập sự kiện được kết hợp để tính TotalPickups và TotalDropoffs ở cùng grain. Một chuyến có thể đón hôm trước và trả hôm sau nên không yêu cầu hai measure bằng nhau ở từng ngày hoặc vùng. Chỉ ở phạm vi đối soát phù hợp mới so tổng số sự kiện với số chuyến nguồn.

Fact_Zone_Traffic_Snapshot

Bảng 2.21 Cấu trúc và ý nghĩa các cột Fact_Zone_Traffic_Snapshot

Cột	**Kiểu SQL**	**NULL và khóa**	**Ý nghĩa**
DateKey	int	Không; PK	Khóa ngoại Dim_Date.DateKey; vai trò Date. Khóa 0 giữ Unknown.
TimeKey	smallint	Không; PK	Khóa ngoại Dim_Time.TimeKey; vai trò Time. Khóa 0 giữ Unknown.
LocationKey	int	Không; PK	Khóa ngoại Dim_Location.LocationKey; vai trò Location. Khóa 0 giữ Unknown.
ServiceTypeKey	int	Không; PK	Khóa ngoại Dim_ServiceType.ServiceTypeKey; vai trò ServiceType. Khóa 0 giữ Unknown.
TotalPickups	bigint	Không; Thuộc tính	Tổng sự kiện đón ở ngày phút vùng dịch vụ của dòng.
TotalDropoffs	bigint	Không; Thuộc tính	Tổng sự kiện trả ở ngày phút vùng dịch vụ của dòng.
SnapshotUpdatedAt	datetime2	Không; Thuộc tính	Thời điểm cập nhật snapshot, khác ngày nghiệp vụ.

2.6.6 Fact_Base_Performance_Snapshot

Loại fact là Periodic Snapshot; grain là ngày, base điều phối, dịch vụ và provider. Provider có thể là Unknown hoặc không áp dụng ở nguồn không có

giấy phép HVFHS. Khóa ghép ngăn việc trộn các nhà cung cấp hoặc dịch vụ chỉ vì dùng cùng base.

Bảng lưu tổng thời lượng và DurationTripCount, tổng chờ và WaitingTripCount để tính lại trung bình khi lọc hoặc roll up. Các cột trung bình lưu sẵn không được cộng trực tiếp và không phải mẫu số để lấy trung bình các nhóm. Cờ đi chung, WAV và tiền tài xế được tổng hợp theo phạm vi nguồn có trường tương ứng.

Tổng WAV matches chia tổng WAV requests không mặc nhiên là tỷ lệ phục vụ yêu cầu: hai cờ có định nghĩa khác nhau và tử số chưa được bảo đảm là tập con của mẫu số. Chỉ số đã được ẩn trong bản hardening của SSAS; số đếm gốc vẫn được giữ để không sửa dữ liệu nhằm ép một tỷ lệ về giới hạn mong muốn.

Fact_Base_Performance_Snapshot

Bảng 2.22 Cấu trúc và ý nghĩa các cột Fact_Base_Performance_Snapshot

Cột	**Kiểu SQL**	**NULL và khóa**	**Ý nghĩa**
DateKey	int	Không; PK	Khóa ngoại Dim_Date.DateKey; vai trò Date. Khóa 0 giữ Unknown.
DispatchBaseKey	int	Không; PK	Khóa ngoại Dim_Base.BaseKey; vai trò DispatchBase. Khóa 0 giữ Unknown.
ServiceTypeKey	int	Không; PK	Khóa ngoại Dim_ServiceType.ServiceTypeKey; vai trò ServiceType. Khóa 0 giữ Unknown.
HVFHSProviderKey	int	Không; PK	Khóa ngoại Dim_HVFHSProvider.HVFHSProviderKey; vai trò HVFHSProvider. Khóa 0 giữ Unknown.
TotalTrips	bigint	Không; Thuộc tính	Tổng số chuyến ở grain snapshot.
TotalTripDurationSeconds	bigint	Có; Thuộc tính	Tổng thời lượng hợp lệ, giây.

Cột	Kiểu SQL	NULL và khóa	Ý nghĩa
DurationTripCount	bigint	Không; Thuộc tính	Số chuyến có thời lượng hợp lệ, mẫu số trung bình.
AverageTripDurationSeconds	decimal	Có; Thuộc tính	Trung bình thời lượng tại grain dòng, giây; roll up cần tính lại từ tổng và count.
TotalWaitingTimeSeconds	bigint	Có; Thuộc tính	Tổng thời gian chờ hợp lệ, giây.
WaitingTripCount	bigint	Không; Thuộc tính	Số chuyến có thời gian chờ hợp lệ.
AverageWaitingTimeSeconds	decimal	Có; Thuộc tính	Trung bình chờ tại grain dòng, giây; không cộng trực tiếp.
TotalSharedRequests	bigint	Có; Thuộc tính	Tổng chuyến có cờ yêu cầu đi chung phù hợp nguồn.
TotalSharedMatches	bigint	Có; Thuộc tính	Tổng chuyến có cờ đã ghép đi chung.
TotalWAVRequests	bigint	Có; Thuộc tính	Tổng chuyến có cờ yêu cầu WAV.
TotalWAVMatches	bigint	Có; Thuộc tính	Tổng chuyến thực hiện bằng WAV; không bảo đảm là tập con các yêu cầu.
TotalDriverPay	decimal	Có; Thuộc tính	Tổng tiền tài xế ghi nhận theo nguồn có dữ liệu, USD.

Cột	Kiểu SQL	NULL và khóa	Ý nghĩa
SnapshotUpdatedAt	datetime2	Không; Thuộc tính	Thời điểm cập nhật snapshot, khác ngày nghiệp vụ.

2.7 Thiết kế phân cấp và quy tắc tổng hợp dữ liệu

Calendar đi theo Year → Quarter → Month → Date. Key của Quarter phải chứa năm và quý; key của Month phải chứa năm và tháng. Nếu chỉ dùng MonthNumber, tháng 1 của các năm sẽ bị gộp thành một member trong hierarchy. Dim_Time tổ chức TimeBand → Hour → Time, còn Dim_Location dùng Borough → Zone.

Measure cộng dồn như TripCount, khoảng cách và tiền được cộng theo đúng grain. Measure tỷ lệ và trung bình được tính từ tổng tử và tổng mẫu hợp lệ ở phạm vi lọc mới. Mẫu số bằng 0 hoặc thiếu trả NULL. Các phép cộng khác đơn vị sự kiện, chẳng hạn cộng lượt đón với lượt trả rồi gọi là số chuyến, bị loại khỏi định nghĩa chỉ tiêu.

Bảng 2.23 Hierarchy và quy tắc khóa

Chiều	Hierarchy	Quy tắc
Date	Year → Quarter → Month → Date	Quý và tháng có năm trong khóa.
Time	TimeBand → Hour → Time	Unknown khác giờ 00; TimeKey 1..1440 cho phút hợp lệ.
Location	Borough → Zone	Zone dùng LocationKey; caption ZoneName.

2.8 Thiết kế vật lý và toàn vẹn dữ liệu

2.8.1 Kiểu dữ liệu khóa chính và khóa ngoại

Dimension dùng các khóa nội bộ để fact join ổn định; thuộc tính mã nguồn được giữ riêng để tra cứu. Các fact giao dịch dùng TripKey làm khóa nội bộ của dòng,

còn snapshot dùng khóa ghép đúng grain. Khóa ngoại nối đến các bảng chiều giúp phát hiện nạp sai tham chiếu trước khi dữ liệu đi vào SSAS.
Kiểu số được chọn theo ngữ nghĩa: số đếm dùng kiểu nguyên, quãng đường dùng decimal có phần thập phân, tiền dùng decimal và thời điểm dùng datetime2 trong lớp chuẩn hóa. Không dùng kiểu chuỗi của CSV làm kiểu phân tích trong DWH. Các kiểu chính xác và khả năng NULL được trình bày ở từ điển từng bảng; DDL nguyên văn nằm tại Phụ lục A.
Bảng 2.24 Danh mục khóa ngoại và vai trò

Fact và cột	**Dimension và cột**	**Ràng buộc**
Fact_FHV_Trips.PickupDateKey	Dim_Date.DateKey	FK_FHV_PickupDate
Fact_FHV_Trips.DropoffDateKey	Dim_Date.DateKey	FK_FHV_DropoffDate
Fact_HVFHV_Trips.RequestDateKey	Dim_Date.DateKey	FK_HVFHV_RequestDate
Fact_HVFHV_Trips.OnSceneDateKey	Dim_Date.DateKey	FK_HVFHV_OnSceneDate
Fact_HVFHV_Trips.PickupDateKey	Dim_Date.DateKey	FK_HVFHV_PickupDate
Fact_HVFHV_Trips.DropoffDateKey	Dim_Date.DateKey	FK_HVFHV_DropoffDate
Fact_Daily_Financial_Snapshot.DateKey	Dim_Date.DateKey	FK_DFS_Date
Fact_Zone_Traffic_Snapshot.DateKey	Dim_Date.DateKey	FK_ZTS_Date
Fact_Base_Performance_Snapshot.DateKey	Dim_Date.DateKey	FK_BPS_Date
Fact_YellowGreen_Trips.PickupDateKey	Dim_Date.DateKey	FK_FYG_PickupDate

Fact và cột	Dimension và cột	Ràng buộc
Fact_YellowGreen_Trips.DropoffDateKey	Dim_Date.DateKey	FK_FYG_DropoffDate
Fact_FHV_Trips.PickupTimeKey	Dim_Time.TimeKey	FK_FHV_PickupTime
Fact_FHV_Trips.DropoffTimeKey	Dim_Time.TimeKey	FK_FHV_DropoffTime
Fact_HVFHV_Trips.RequestTimeKey	Dim_Time.TimeKey	FK_HVFHV_RequestTime
Fact_HVFHV_Trips.OnSceneTimeKey	Dim_Time.TimeKey	FK_HVFHV_OnSceneTime
Fact_HVFHV_Trips.PickupTimeKey	Dim_Time.TimeKey	FK_HVFHV_PickupTime
Fact_HVFHV_Trips.DropoffTimeKey	Dim_Time.TimeKey	FK_HVFHV_DropoffTime
Fact_Zone_Traffic_Snapshot.TimeKey	Dim_Time.TimeKey	FK_ZTS_Time
Fact_YellowGreen_Trips.PickupTimeKey	Dim_Time.TimeKey	FK_FYG_PickupTime
Fact_YellowGreen_Trips.DropoffTimeKey	Dim_Time.TimeKey	FK_FYG_DropoffTime
Fact_FHV_Trips.PickupLocationKey	Dim_Location.LocationKey	FK_FHV_PickupLocation
Fact_FHV_Trips.DropoffLocationKey	Dim_Location.LocationKey	FK_FHV_DropoffLocation
Fact_HVFHV_Trips.PickupLocationKey	Dim_Location.LocationKey	FK_HVFHV_PickupLocation
Fact_HVFHV_Trips.DropoffLocationKey	Dim_Location.LocationKey	FK_HVFHV_DropoffLocation

Fact và cột	Dimension và cột	Ràng buộc
Fact_Zone_Traffic_Snapshot.LocationKey	Dim_Location.LocationKey	FK_ZTS_Location
Fact_YellowGreen_Trips.PickupLocationKey	Dim_Location.LocationKey	FK_FYG_PickupLocation
Fact_YellowGreen_Trips.DropoffLocationKey	Dim_Location.LocationKey	FK_FYG_DropoffLocation
Fact_FHV_Trips.ServiceTypeKey	Dim_ServiceType.ServiceTypeKey	FK_FHV_ServiceType
Fact_HVFHV_Trips.ServiceTypeKey	Dim_ServiceType.ServiceTypeKey	FK_HVFHV_ServiceType
Fact_Daily_Financial_Snapshot.ServiceTypeKey	Dim_ServiceType.ServiceTypeKey	FK_DFS_Service
Fact_Zone_Traffic_Snapshot.ServiceTypeKey	Dim_ServiceType.ServiceTypeKey	FK_ZTS_Service
Fact_Base_Performance_Snapshot.ServiceTypeKey	Dim_ServiceType.ServiceTypeKey	FK_BPS_Service
Fact_YellowGreen_Trips.ServiceTypeKey	Dim_ServiceType.ServiceTypeKey	FK_FYG_ServiceType
Fact_YellowGreen_Trips.VendorKey	Dim_Vendor.VendorKey	FK_FYG_Vendor
Fact_HVFHV_Trips.HVFHSProviderKey	Dim_HVFHSProvider.HVFHSProviderKey	FK_HVFHV_Provider
Fact_Base_Performance_Snapshot.HVFHSProviderKey	Dim_HVFHSProvider.HVFHSProviderKey	FK_BPS_Provider
Fact_FHV_Trips.DispatchBaseKey	Dim_Base.BaseKey	FK_FHV_DispatchBase
Fact_FHV_Trips.AffiliatedBaseKey	Dim_Base.BaseKey	FK_FHV_AffiliatedBase

Fact và cột	Dimension và cột	Ràng buộc
Fact_HVFHV_Trips.DispatchBaseKey	Dim_Base.BaseKey	FK_HVFHV_DispatchBase
Fact_HVFHV_Trips.OriginBaseKey	Dim_Base.BaseKey	FK_HVFHV_OriginBase
Fact_Base_Performance_Snapshot.DispatchBaseKey	Dim_Base.BaseKey	FK_BPS_Base
Fact_YellowGreen_Trips.RateCodeKey	Dim_RateCode.RateCodeKey	FK_FYG_RateCode
Fact_YellowGreen_Trips.PaymentTypeKey	Dim_PaymentType.PaymentTypeKey	FK_FYG_PaymentType
Fact_YellowGreen_Trips.TripTypeKey	Dim_TripType.TripTypeKey	FK_FYG_TripType
Fact_YellowGreen_Trips.TaxiFlagKey	Dim_TaxiFlags.TaxiFlagKey	FK_FYG_TaxiFlag
Fact_FHV_Trips.FHVFlagKey	Dim_FHVFlags.FHVFlagKey	FK_FHV_Flag
Fact_HVFHV_Trips.HVFHVFlagKey	Dim_HVFHVFlags.HVFHVFlagKey	FK_HVFHV_Flag
Fact_YellowGreen_Trips.PassengerGroupKey	Dim_PassengerGroup.PassengerGroupKey	FK_FYG_PassengerGroup
Fact_FHV_Trips.LoadBatchKey	Dim_LoadBatch.LoadBatchKey	FK_FHV_LoadBatch
Fact_HVFHV_Trips.LoadBatchKey	Dim_LoadBatch.LoadBatchKey	FK_HVFHV_LoadBatch
Fact_YellowGreen_Trips.LoadBatchKey	Dim_LoadBatch.LoadBatchKey	FK_FYG_LoadBatch

2.8.2 Khóa thay thế và thành viên Unknown

Surrogate key tách định danh nội bộ khỏi mã nghiệp vụ, hỗ trợ dùng chung dimension giữa các nguồn. Dòng key bằng 0 được seed ở mỗi dimension để biểu diễn chưa xác định. Unknown giữ sự kiện trong tổng nhưng không khẳng định giá trị nghiệp vụ đã biết; cần cho phép người sử dụng nhìn thấy nhóm này trên báo cáo.

Unknown trong bảng dimension khác cơ chế UnknownMember bổ sung của SSAS. Nếu loader ánh xạ đúng về 0, fact tham chiếu một thành viên dữ liệu bình thường. Nếu có khóa không phân giải ngoài quy ước, SSAS vẫn phải báo lỗi theo ErrorConfiguration thay vì bỏ qua hoặc tăng giới hạn lỗi để process cho xong.

2.8.3 Chỉ mục phục vụ nạp và truy vấn

Chỉ mục theo SourceDataset và SourceKey, có RecordHash được INCLUDE, hỗ trợ lấy đúng tập fact của một file khi đồng bộ. Clean có chỉ mục theo TaxiType và ETLSourceKey để kiểm tra lineage và chọn phạm vi Transform. Các khóa chính dimension và snapshot cũng hỗ trợ join, lookup và bảo vệ tính duy nhất. Chỉ mục được thiết kế theo đường truy cập thực tế chứ không có nghĩa mọi truy vấn đã tối ưu. Dữ liệu demo nhỏ chưa đủ để kết luận hiệu năng toàn tập; cần đo chi phí tải, ghi log và process khi tăng dữ liệu. Snapshot rebuild toàn bảng và SSAS ProcessFull vẫn là những phần có chi phí phụ thuộc kích thước toàn kho.

2.8.4 Quản lý thay đổi thuộc tính dimension

Dim_Location áp dụng cập nhật Type 1 với các thuộc tính hiện hành. Các mã base mới được bổ sung nhưng chưa có cơ chế phiên bản Type 2 gắn khoảng hiệu lực. Vì vậy phân tích lịch sử sử dụng thuộc tính chiều theo thiết kế hiện có, không tái dựng được mọi thay đổi tên hoặc phân loại theo thời gian.

Khi mapping một nhóm dimension thay đổi, DWH loader có thể chọn lại file còn Unknown có khả năng sửa trong nhóm đó. Đổi tên vùng trên cùng khóa

không nhất thiết yêu cầu nạp lại fact. Thay quy tắc ánh xạ hoặc seed ngày giờ cần nâng LoaderVersion hoặc ForceReload phạm vi liên quan; không kỳ vọng checkpoint tự phát hiện mọi sửa đổi thủ công ngoài pipeline.

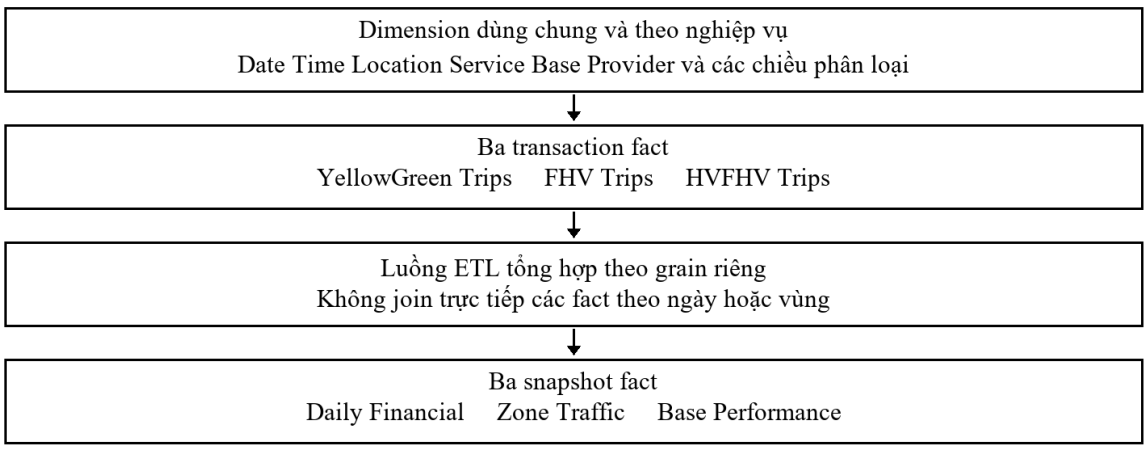
2.8.5 Lưu thông tin truy vết nguồn dữ liệu

Lineage của fact dựa trên SourceDataset, SourceKey và RecordHash, đối chiếu với checkpoint Extract, Transform và DWH. FileHash xác định nội dung CSV đã nhận; RuleVersion và ZoneHash mô tả bối cảnh Transform; LoaderVersion xác định logic đồng bộ fact. Một record hash không thay thế lịch sử trạng thái của cả file.

FactChangeAudit giữ bản ghi cũ trước các thao tác thay thế theo thiết kế loader. Khi nguồn hiệu chỉnh, TripKey có thể đổi do delete và insert; người vận hành cần truy vết bằng nguồn, payload và audit, không dựa duy nhất vào surrogate key. Xóa file local không phải quy trình tombstone và không tự yêu cầu xóa fact lịch sử.

2.9 Sơ đồ mô hình kho dữ liệu hoàn chỉnh

Mô hình gồm ba transaction fact cùng ba snapshot, chia sẻ các bảng chiều theo Bus Matrix. Sơ đồ sau phân biệt quan hệ tham chiếu chiều với luồng tạo snapshot; danh mục khóa ngoại là đặc tả chi tiết của từng quan hệ. Thiết kế không tạo quan hệ join nhiều nhiều trực tiếp giữa các transaction fact.



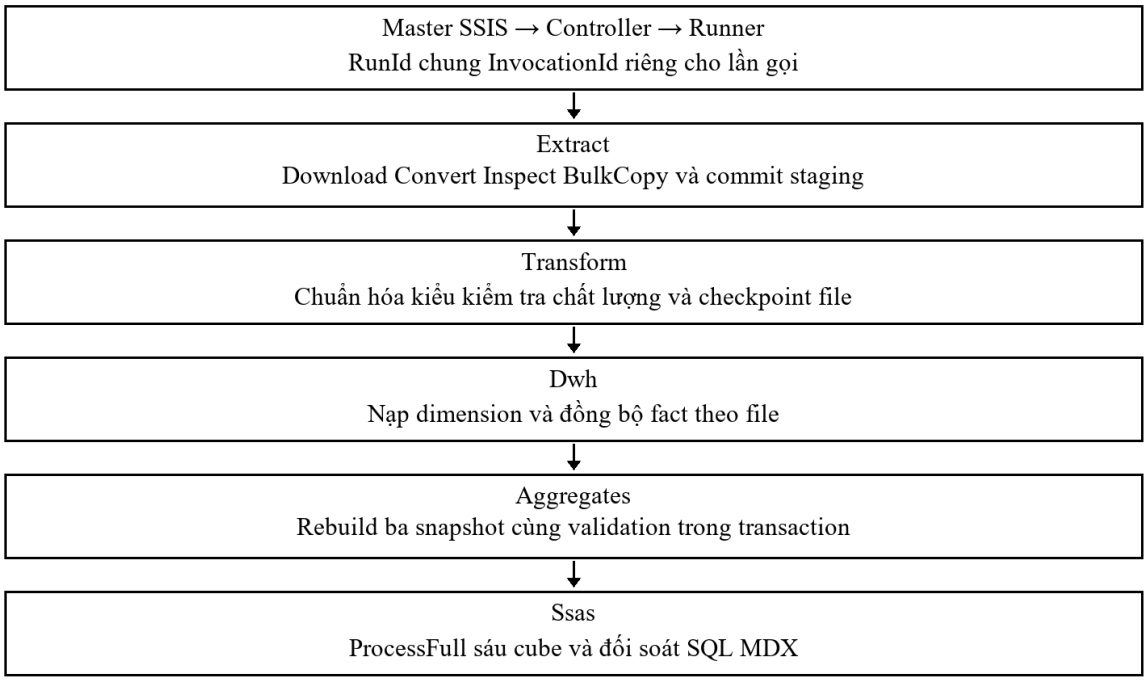
Hình 2.2 Dimension dùng chung và các nhóm fact

CHƯƠNG 3 XÂY DỰNG QUY TRÌNH ETL

3.1 Mục tiêu và kiến trúc ETL

ETL chuyển dữ liệu file thành các bảng có kiểu và ý nghĩa thống nhất, đồng thời duy trì bằng chứng để kiểm tra việc nạp. Điểm vào gồm Master SSIS hoặc runner theo chế độ vận hành đã cấu hình. Luồng chính giữ thứ tự Extract → Transform → Dwh → Aggregates → Ssas; mỗi stage chỉ được coi hoàn tất khi kết quả và log StageComplete đáp ứng điều kiện.

Master dùng Execute Process Task gọi controller, controller gọi Invoke-NYCTaxiPipeline.ps1. Runner dùng Python cho file, SqlBulkCopy cho wrk và SQL nhúng trong package cho các tác vụ nghiệp vụ. Các package con vẫn chứa mô tả luồng native, nhưng chạy trực tiếp chúng là một đường khác; không chạy song song package con với Master vì chưa phải mọi đường đều dùng chung khóa điều phối.



Hình 3.1 Luồng điều phối và các điểm kiểm tra ETL

3.2 Chuẩn bị môi trường và cấu hình

3.2.1 Cơ sở dữ liệu và cấu trúc thư mục

NYCTaxi_Staging chứa vùng làm việc, staging, clean và các đối tượng kiểm soát liên quan; NYCTaxi_DWH chứa các dimension, fact và log điều phối cấp kho. Các thư mục raw, csv và quarantine nằm dưới DataRoot. Log của runner nằm theo RunId trong etl/logs, tách với dữ liệu nguồn để vẫn đọc được khi một file lỗi.

Bản source trong workspace dùng etl/packages để lưu package, trong khi PackageRoot mặc định của runner trỏ bản package ở thư mục dự án ETL triển khai trên máy. Khi chuyển môi trường cần đặt lại PackageRoot, ScriptsRoot, DataRoot, PythonExe và server đồng nhất. Có file source tại workspace không chứng minh runner đang đọc đúng file nếu tham số trỏ nơi khác.

3.2.2 Các kết nối sử dụng

Kết nối SQL của runner dùng Windows Integrated Security, Connect Timeout 15 giây và Application Name riêng cho pipeline. Helper SQL đặt CommandTimeout 1800 giây. SqlBulkCopy cũng có timeout riêng; SSAS được truy cập bằng AMO và XMLA với quyền xử lý database. Tài khoản dịch vụ SSAS cần đọc DWH qua Data Source của mô hình.

Connection Manager trong package native mô tả OLE DB hoặc Flat File theo đường chạy đó. Không nhầm thuộc tính của Connection Manager với kết nối .NET thực tế mà runner mở. Khi mô tả hoặc kiểm tra lỗi, phải xác định lỗi thuộc SQL connection, trình đọc CSV, tiến trình Python hay kết nối SSAS để không thay cấu hình sai tầng.

3.2.3 Tham số biến và phạm vi xử lý

Các tham số chính gồm khoảng tháng, MaxRows, server SQL, server SSAS, các thư mục, đường dẫn Python, RunId, InvocationId và Stage. Offline bỏ download nhưng vẫn kiểm tra file có sẵn; RefreshSource nhận lại nguồn để dùng bản hiệu

chỉnh. InjectFailure là tham số hỗ trợ kiểm thử, không dùng trong vận hành bình thường.

Master có pStartMonth, pEndMonth, pMaxRows, pResumeRunId, pStartStage, pMaxRetries và pShowMonitor. Mặc định Master và runner có thể khác nhau: package demo được lưu với tháng 2021 02 và 10000 dòng, trong khi runner mặc định 2025 01 đến 2025 12 và 100000 dòng. Giá trị thực được truyền khi chạy mới là cấu hình quyết định phạm vi, không chỉ dựa trên mặc định của một file script.

Bảng 3.1 Tham số Master lưu trong source

Tham số	**Giá trị**
pControllerPath	E:\NYCTaxi\NYCTaxi_SSAS\etl\Invoke-SsisStage.ps1
pEndMonth	2021-02
pMaxRetries	1
pMaxRows	10000
pResumeRunId	
pRunnerPath	E:\NYCTaxi\NYCTaxi_SSAS\etl\Invoke-NYCTaxiPipeline.ps1
pShowMonitor	1
pStartMonth	2021-02
pStartStage	Extract

3.2.4 Kiểm tra điều kiện trước khi chạy

Preflight xác nhận các đường dẫn bắt buộc, kết nối NYCTaxi_Staging và NYCTaxi_DWH, cùng khả năng import pyarrow và lxml bằng Python đã chọn. Khoảng tháng phải hợp lệ, StartMonth không sau EndMonth và năm nằm trong phạm vi chương trình. Kiểm tra này phát hiện lỗi môi trường trước khi bắt đầu tải hoặc sửa dữ liệu.

Preflight đạt chưa đủ để chứng minh đủ quyền process SSAS hay dữ liệu nguồn đúng nghiệp vụ. Các điều kiện đó được kiểm tra tại stage tương ứng. Người vận hành cần giữ cấu hình đầu vào, phiên bản source và quyền truy cập ổn định trong cùng một RunId; sửa code giữa hai lần resume chưa được ConfigHash bảo vệ đầy đủ như một manifest phiên bản.

3.3 Xây dựng luồng điều phối chính

3.3.1 Cấu trúc package Master

PKG_00_Master bắt đầu bằng EPT_00_Open_Monitor, sau đó lần lượt EPT_01_Extract, EPT_02_Transform, EPT_03_Dwh, EPT_04_Aggregates, EPT_05_Ssas và EPT_99_Complete. Monitor mở cửa sổ theo dõi riêng, không chặn xử lý dữ liệu. Complete kiểm tra trạng thái SQL của run đã SUCCEEDED sau stage Ssas và đối soát.

Các đường thành công bảo đảm phụ thuộc tuần tự; nhánh lỗi từ các EPT đi về tác vụ báo lỗi gốc theo điều kiện OR. Một stage thất bại không được cho phép stage phụ thuộc chạy tiếp như đã thành công. Khi ghi lỗi SQL thất bại, log local vẫn giữ nguyên lỗi gốc và RunId để truy vết.

Bảng 3.2 Các tác vụ điều phối Master

Task	**Loại thực thi**	**Trách nhiệm**
EPT_00_Open_Monitor	Microsoft.ExecuteProcess	Mở monitor
EPT_01_Extract	Microsoft.ExecuteProcess	Gọi controller thực hiện stage tương ứng
EPT_02_Transform	Microsoft.ExecuteProcess	Gọi controller thực hiện stage tương ứng
EPT_03_Dwh	Microsoft.ExecuteProcess	Gọi controller thực hiện stage tương ứng
EPT_04_Aggregates	Microsoft.ExecuteProcess	Gọi controller thực hiện stage tương ứng

Task	Loại thực thi	Trách nhiệm
EPT_05_Ssas	Microsoft.ExecuteProcess	Gọi controller thực hiện stage tương ứng
EPT_99_Complete	Microsoft.ExecuteProcess	Xác nhận cuối run
EST_90_Report_Original_Error	Microsoft.ExecuteSQLTask	Báo lỗi gốc

PKG_01_Extract_Stage: Chuẩn bị staging, xử lý từng file, kiểm tra và cập nhật trạng thái tiếp nhận.

Bảng 3.3 Các task cấp chính PKG_01_Extract_Stage

Tên task theo thứ tự source	Kiểu thực thi
EPT_01_Download_Parquet	Microsoft.ExecuteProcess
EPT_02_Convert_To_CSV	Microsoft.ExecuteProcess
EST_03_Prepare_Staging	Microsoft.ExecuteSQLTask
FLC_04_Load_Yellow_Files	STOCK:FOREACHLOOP
FLC_05_Load_Green_Files	STOCK:FOREACHLOOP
FLC_06_Load_FHV_Files	STOCK:FOREACHLOOP
FLC_07_Load_FHVHV_Files	STOCK:FOREACHLOOP
FLC_08_Load_TaxiZone_Files	STOCK:FOREACHLOOP
EST_09_Validate_Staging	Microsoft.ExecuteSQLTask

PKG_02_Transform_Clean: Chuẩn bị clean, chuẩn hóa lookup và bốn nguồn chuyến, kiểm tra đầu ra.

Bảng 3.4 Các task cấp chính PKG_02_Transform_Clean

Tên task theo thứ tự source	Kiểu thực thi
EST_01_Setup_Clean	Microsoft.ExecuteSQLTask

Tên task theo thứ tự source	Kiểu thực thi
EST_02_Transform_TaxiZone	Microsoft.ExecuteSQLTask
EST_03_Transform_YELLOW	Microsoft.ExecuteSQLTask
EST_04_Transform_GREEN	Microsoft.ExecuteSQLTask
EST_05_Transform_FHV	Microsoft.ExecuteSQLTask
EST_06_Transform_FHVHV	Microsoft.ExecuteSQLTask
EST_07_Validate_Transform	Microsoft.ExecuteSQLTask

PKG_03_Load_DWH: Precheck, nạp dimension và ba fact, kiểm tra coverage cùng số dòng.

Bảng 3.5 Các task cấp chính PKG_03_Load_DWH

Tên task theo thứ tự source	Kiểu thực thi
EST_00_Precheck_DWH	Microsoft.ExecuteSQLTask
EST_01_Load_Dimensions	Microsoft.ExecuteSQLTask
EST_02_Load_FactYellowGreen	Microsoft.ExecuteSQLTask
EST_03_Load_FactFHV	Microsoft.ExecuteSQLTask
EST_04_Load_FactHVFHV	Microsoft.ExecuteSQLTask
EST_05_Validate_DWH	Microsoft.ExecuteSQLTask

PKG_04_Load_Aggregates: Rebuild và kiểm tra ba snapshot trong cùng transaction.

Bảng 3.6 Các task cấp chính PKG_04_Load_Aggregates

Tên task theo thứ tự source	Kiểu thực thi
EST_00_Atomic_Rebuild_And_Validate	Microsoft.ExecuteSQLTask

3.3.2 Vai trò của SSIS PowerShell và Python

SSIS cung cấp bề mặt package, tham số và luồng điều phối; controller quản lý việc gọi stage, retry, resume và kết quả tiến trình; runner thực hiện chuỗi thao tác cùng checkpoint. Python tải, chuyển đổi và inspect file; các biến đổi nghiệp vụ chính được thực thi bằng SQL đọc từ package thay vì bị viết lại riêng trong Python.

Kiến trúc này có ưu điểm là các quy tắc SQL có một nguồn định nghĩa trong package. Tuy nhiên, tiến độ native Data Flow không phải tiến độ của runner; việc mở tab package con không làm hiển thị các câu SQL của runner đang chạy. Báo cáo triển khai vì vậy mô tả cả đường điều phối và nơi thực thi thực tế.

3.3.3 Thứ tự thực thi và điều kiện chuyển bước

Extract phải hoàn tất file và số dòng trước Transform; Transform phải hoàn tất cùng phiên bản trước Dwh; Dwh phải hợp lệ trước Aggregate; snapshot phải nhất quán trước process SSAS. Khi chạy riêng một stage sau Extract, runner yêu cầu StageComplete SUCCEEDED của stage trước trong cùng RunId.

Khi stage trước được chạy lại, các StageComplete thành công phía sau bị đánh INVALIDATED. Quy tắc này ngăn việc dùng kết quả cũ của snapshot hoặc cube sau khi đầu vào thay đổi. Một run chỉ được ghi SUCCEEDED ở cuối đường All hoặc Ssas sau các bước bắt buộc, không phải ngay sau khi nạp fact.

3.3.4 Truyền tham số và quản lý mã lần chạy

RunId định danh lượt ETL xuyên các stage. Với lượt mới, Master dùng ExecutionInstanceGUID khi pResumeRunId trống; với resume, dùng RunId người vận hành chỉ định. InvocationId phân biệt từng lần gọi runner và từng lần thử lại, tránh đọc nhầm failure JSON của lần trước.

ConfigHash kiểm tra tháng, giới hạn dòng, server, đường dẫn và chế độ liên quan. Khi dùng lại RunId với cấu hình khác, runner từ chối. Để thay phạm vi

tháng hoặc chuyển môi trường phải tạo lượt mới theo quy trình, không sửa tham số của run cũ để làm checkpoint có vẻ hợp lệ.

3.4 Xây dựng luồng trích xuất và nạp Staging

3.4.1 Tải dữ liệu nguồn

Downloader lựa chọn file theo tháng và dataset, bỏ qua những nguồn chưa có ở mốc đầu của hợp đồng. Dữ liệu được lưu raw trước khi chuyển đổi.
RefreshSource yêu cầu nhận lại bản nguồn, còn chế độ Offline bỏ qua mạng nhưng không bỏ qua kiểm tra đầy đủ đầu vào. Thiếu file bắt buộc không được tự hiểu là dataset có 0 chuyến.
Thành công của bước download chỉ khẳng định có đầu vào cần thiết cho bước kế tiếp. Việc tái sử dụng cache còn phải phù hợp signature, cấu hình chuyển đổi và hash. File lỗi tải không được đưa sang parser như một Parquet hợp lệ hoặc được thay bằng file rỗng để pipeline tiếp tục.

3.4.2 Chuyển đổi Parquet sang CSV

Converter đọc theo batch nhằm tránh nạp toàn bộ file lớn vào bộ nhớ. Chế độ ssis xuất đúng thứ tự cột trong extract_contract.py, dùng UTF 8 và LF. Cột optional thiếu được xử lý theo hợp đồng; cột lạ và thiếu thiết yếu làm dừng để xem xét schema drift. Dữ liệu được ghi vào file tạm rồi đổi tên khi hoàn tất.
Metadata đi kèm ghi source_file, source_size, source_mtime_ns, dataset, columns, schema_mode, contract_version, encoding, line_ending, rows, source_rows, row_limit, csv_size, csv_mtime_ns và sha256. Điều kiện reuse so khớp thông tin nguồn, chế độ, số dòng, giới hạn và hash; đổi MaxRows từ mẫu sang toàn file làm thay phiên bản đầu vào thay vì sử dụng nhầm CSV cũ.

3.4.3 Kiểm tra cấu trúc metadata và số dòng

Inspector đọc file theo dataset và xác nhận header, thứ tự cột, số trường, bề rộng, số dòng, hash và giới hạn. File ngoài khoảng tháng được ghi trạng thái bỏ qua

ngoài phạm vi; file trong phạm vi phải đáp ứng hợp đồng. Các trường chuyến ở CSV có bề rộng tiếp nhận 50 ký tự, các trường mô tả lookup ngoài LocationID có bề rộng 255.

Lỗi cấu trúc như thiếu dấu phân cách làm sai số trường phải dừng file. Đây khác lỗi nghiệp vụ của một giá trị hợp lệ về cấu trúc nhưng không phù hợp điều kiện phân tích. Tách hai loại lỗi giúp quarantine đúng đầu vào và tránh quy tắc làm sạch vô tình che một CSV đã bị lệch cột.

3.4.4 Nạp dữ liệu vào vùng làm việc

Runner dùng TextFieldParser với dấu phẩy, cho phép trường đặt trong dấu nháy, UTF 8 và không trim toàn bộ chuỗi tại bước đọc. Header tạo các cột chuỗi của DataTable; cột được ánh xạ theo tên vào wrk của dataset. Chuỗi rỗng chuyển DBNull ở tầng nạp, còn các xử lý trim và đổi kiểu nghiệp vụ diễn ra ở Transform.

SqlBulkCopy đặt BatchSize 10000, BulkCopyTimeout 1800 và ghi từng bộ đệm rồi Clear để hạn chế bộ nhớ. Mỗi lần đọc dòng phải kiểm tra số trường bằng số cột. Tiến độ rows trên monitor phản ánh đã ghi wrk, chưa chứng minh dữ liệu đã được commit sang staging hoặc fact. Khi file lỗi, không bỏ qua riêng dòng lỗi CSV rồi báo nạp đầy đủ.

Ở đường chạy package native, Flat File Source, Row Count và OLE DB Destination được cấu hình theo từng dataset. Các cột và bề rộng theo hợp đồng CSV; lỗi chuyển đổi hoặc cắt ngắn giữ chế độ FailComponent theo metadata thay vì âm thầm redirect rồi bỏ dòng. Đường chạy Master hiện dùng parser và SqlBulkCopy của runner, nên cơ chế quarantine theo file thuộc runner, không phải một nhánh Data Flow đã thực thi trong package con. Các thuộc tính component và disposition cột được liệt kê ở Phụ lục A để phân biệt hai đường chạy.

3.4.5 Ghi dữ liệu vào Staging và cập nhật trạng thái

usp_BeginExtractFile nhận dataset, đường dẫn, hash, ExpectedRows và RowLimit để xác định file đã nạp đủ hay cần làm lại. File được bulk copy khi cần, sau đó usp_CommitExtractFile kiểm tra số dòng thực và đưa dữ liệu từ wrk vào stg theo đúng file. Các cột ETLSourceKey, ETLFileHash và ETLLoadedAt giữ lineage cho bước sau.

ExtractFileLog phản ánh trạng thái thực, không chỉ sự tồn tại của CSV. Nếu inspect hoặc nạp hay commit thất bại, CSV, sidecar và thông tin lỗi được sao chép vào quarantine theo RunId và dataset; log được cập nhật FAILED khi có thể. Cuối stage, Validate Staging nhận đúng tháng và giới hạn dòng để kiểm tra phạm vi đã chọn trước khi cho Transform chạy.

3.5 Xây dựng luồng biến đổi và làm sạch

3.5.1 Chuẩn hóa danh mục Taxi Zone

Task setup tạo hoặc kiểm tra các đối tượng clean cần thiết. Transform TaxiZone chuẩn hóa LocationID và các chuỗi Borough, Zone, ServiceZone. Các chuyến đi chỉ được chuẩn hóa sau khi danh mục đã sẵn sàng; task chuyến kiểm tra điều kiện tồn tại lookup trước khi xử lý dữ liệu.

ZoneHash được tính từ danh mục theo thứ tự xác định và tham gia phiên bản Transform. Khi danh mục đổi, những file chịu điều kiện chọn lại được biến đổi để phản ánh kiểm tra vùng mới. Chỉ lưu một lần tên lookup trong thư mục không đủ chứng minh fact đã sử dụng cùng phiên bản danh mục.

3.5.2 Xử lý dữ liệu Yellow

Task Yellow lấy các file Extract SUCCEEDED và đủ số dòng, kiểm tra clean hiện có theo SourceKey, rồi lập #Todo cho file mới, đổi hash, đổi rule, đổi ZoneHash hoặc thiếu dữ liệu. Trước khi sửa clean, staging của đúng các file cần xử lý phải khớp số dòng và hash. Phần clean thuộc #Todo được thay thế trong transaction, còn các file đã hoàn tất được giữ.

Các trường tpep được đổi DATETIME2(0); mã số được chuyển có kiểm soát qua số đủ chính xác để không âm thầm cắt phần thập phân; quãng đường và tiền chuyển decimal theo SQL. Store and forward được chuẩn hóa Y, N hoặc chưa biết. DurationSeconds lấy chênh pickup dropoff; TripDistanceKm nhân mile với 1,609344; tốc độ tính khi thời gian dương và quãng đường không âm. Các phép đo suy ra được giới hạn trong tập file vừa làm lại.

3.5.3 Xử lý dữ liệu Green

Task Green thực hiện cơ chế chọn file và lineage tương tự Yellow, nhưng đọc các mốc lpep và trường riêng như trip_type, ehail_fee. Kiểu dữ liệu tài chính được chuẩn hóa vào cấu trúc clean dùng chung, đồng thời giữ TaxiType để không mất nguồn dịch vụ. Những khoản phí không có ở một kỳ được giữ theo quy tắc NULL thay vì điền một hằng số tùy ý.

Green và Yellow cùng chịu kiểm tra thời lượng, khoảng cách và cảnh báo hành khách theo nhóm taxi. Việc đưa về một cấu trúc clean không làm trip_type trở thành thuộc tính có nghĩa cho mọi nguồn. Bước load DWH sẽ tra dimension chuyên biệt theo TaxiType và quy tắc seed.

3.5.4 Xử lý dữ liệu FHV

Task FHV chuyển pickup, dropoff, mã vùng, base điều phối, base liên kết và SourceSRFlag. Do nguồn không có quãng đường như các nguồn khác, thiếu TripDistanceMiles không được dùng để loại toàn bộ FHV bằng quy tắc áp cho taxi. Các điều kiện thời gian cốt lõi vẫn được kiểm tra để bảo vệ chỉ tiêu thời lượng.

Bản hardening sửa loader để đọc đúng SourceSRFlag, đưa trường này vào hash payload và nâng LoaderVersion riêng FHV. Fixture kiểm tra 1 thành Y, 0 thành N và NULL thành U. Việc sửa mã nguồn không đồng nghĩa toàn bộ fact production đã được nạp lại; lần DWH kế tiếp cần xử lý file FHV theo phiên bản mới và đối soát sau đó.

3.5.5 Xử lý dữ liệu HVFHV

Task HVFHV đọc các mốc request, on scene, pickup, dropoff; quãng đường và trip_time nguồn; các thành phần tài chính và cờ nghiệp vụ. Thời lượng thực phục vụ phân tích được tính từ pickup đến dropoff, còn trip_time nguồn được giữ để so sánh. Lệch quá 60 giây giữa hai đại lượng được ghi cảnh báo theo SQL.

RequestWaitSeconds được tính từ request đến pickup khi request không sau pickup. Nếu on scene sau pickup hoặc request thiếu, chất lượng được ghi nhận thay vì tự dựng thời gian. Tổng tiền khách được suy ra theo công thức loader từ các thành phần nguồn; tiền tài xế và tip được giữ riêng. Cờ shared request và shared match không được gộp thành một trường duy nhất.

3.5.6 Phân loại bản ghi hợp lệ bị loại và có cảnh báo

IsCoreValid xác định tập được phép nạp fact. QualityIssue ghép các mã lỗi hoặc cảnh báo để một dòng có thể phản ánh nhiều vấn đề. clean.ValidTrip cung cấp tập hợp lệ, clean.RejectedTrip hỗ trợ xem dòng bị loại, clean.WarningTrip giữ các dòng có cảnh báo theo định nghĩa. Không cộng số dòng cảnh báo vào số bị loại vì một dòng cảnh báo có thể vẫn hợp lệ.

Các trường hợp cốt lõi gồm pickup hoặc dropoff không chuyển được, dropoff không sau pickup, thời lượng hơn 48 giờ, và khoảng cách thiếu hoặc âm đối với nguồn không phải FHV. Các cảnh báo còn lại gồm khoảng cách 0, hành khách không hợp lý ở taxi, vị trí Unknown, tổng tiền âm hoặc thiếu, request/on scene sai thứ tự và lệch thời lượng nguồn. Phân loại cụ thể phải theo từng task, không dùng một danh sách diễn giải để thay SQL.

Bảng 3.7 Điều kiện chất lượng và cách diễn giải

Điều kiện	**Xử lý**	**Ảnh hưởng**
Pickup hoặc dropoff không hợp lệ	Không qua điều kiện core	Không nạp transaction fact.

Điều kiện	Xử lý	Ảnh hưởng
Dropoff không sau pickup; thời lượng quá 48 giờ	Không qua điều kiện core	Bảo vệ thời lượng và tập chuyến hợp lệ.
Khoảng cách thiếu hoặc âm ở nguồn khác FHV	Không qua điều kiện core	Không áp thiếu khoảng cách để loại mọi FHV.
Khoảng cách 0 hoặc hành khách không hợp lý ở taxi	Ghi QualityIssue theo SQL	Có thể còn hợp lệ, cần phân biệt warning với rejected.
Vùng không tra được	Ghi cảnh báo và lookup Unknown	Giữ chuyến trong tổng, hạn chế diễn giải địa lý.
Request thiếu hoặc sau pickup; on scene sau pickup	Ghi QualityIssue; chỉ tính thời gian khi phù hợp	Mẫu số chờ khác tổng chuyến.
Lệch trip_time nguồn và duration quá 60 giây	SOURCE_DURATION_MISMATCH	Giữ cả thời lượng tính và thông tin kiểm tra nguồn.

3.5.7 Kiểm tra kết quả sau biến đổi

Task tổng hợp số dòng source, clean, valid, rejected và warning cho các file vừa xử lý. Điều kiện clean bằng ExpectedRows và valid cộng rejected bằng ExpectedRows phải đúng trước khi ghi TransformFileLog. Log giữ FileHash, RuleVersion, ZoneHash, SourceRows, CleanRows, ValidRows, RejectedRows, WarningRows và thời điểm hoàn tất trong cùng transaction với dữ liệu.
Nếu phát hiện staging thiếu lineage hoặc số dòng sai, task rollback và trả lỗi gốc. Transform hiện còn kiểm tra clean lịch sử theo dataset trước khi chọn #Todo, nên chưa thể khẳng định toàn stage chỉ đọc file mới. Checkpoint làm giảm phần biến đổi nhưng không đồng nghĩa mọi truy vấn chuẩn bị đã độc lập với kích thước lịch sử.

3.6 Xây dựng luồng nạp kho dữ liệu

3.6.1 Xác định file cần xử lý

etl.DwhSourceState đối chiếu Extract và Transform, chỉ nhận bốn dataset chuyến đi đã đầy đủ và cùng phiên bản. Lookup Taxi Zone không phải file fact nên không yêu cầu DwhFileLog giả cho lookup. SourceVersion kết hợp nội dung file, rule, ZoneHash, số dòng và thông tin hoàn tất Transform để nhận biết đầu vào nạp kho.

File cần làm lại khi chưa có checkpoint thành công, SourceVersion hoặc LoaderVersion thay đổi, ForceReload được đánh dấu hoặc mapping khiến file có Unknown cần sửa thuộc phạm vi chọn lại. Bản không đổi có thể được bỏ qua. Lần đầu chuyển sang hash theo file phải khởi tạo checkpoint bằng xử lý lại file liên quan, không chỉ đánh dấu trạng thái bằng tay.

3.6.2 Nạp và cập nhật dimension

Loader dimension lấy #DeltaTrip từ các file cần xử lý để tìm mã nghiệp vụ mới. Các danh mục nhỏ như Taxi Zone được đọc riêng để cập nhật Type 1; các nhóm mã service, vendor, rate code, payment, trip type, provider và base được tra theo quy tắc tương ứng. Việc này tránh quét toàn bộ lịch sử chuyến để thêm một mã mới mỗi lượt nạp.

Dimension cần được nạp trước fact để các khóa ngoại đã có đích tham chiếu. Dòng Unknown được kiểm tra và giữ ổn định. Các fingerprint mapping theo nhóm Location, Base và nhóm mã khác được dùng để xác định khi nào file còn mã chưa giải quyết cần được chọn lại.

3.6.3 Tra cứu khóa và xử lý Unknown

Mỗi mã nguồn được lookup vào dimension đúng vai trò. Pickup và Dropoff cùng tra Location nhưng ghi vào hai khóa khác nhau; Request và On Scene cũng tra ngày giờ riêng. Null nguồn đi theo trạng thái chưa biết hoặc không áp dụng

đã định nghĩa; mã có giá trị nhưng chưa tra được được ghi nhận vào UnresolvedMask để hỗ trợ xử lý khi mapping thay đổi.
Không được dùng một mã như LocationID trực tiếp làm LocationKey nếu hai miền khóa chưa được bảo đảm đồng nhất. UnknownMask không phải lý do xóa chuyến khỏi tổng; đó là dấu vết chất lượng và phụ thuộc mapping. Sau khi nạp, khóa ngoại vẫn phải hợp lệ dù ngữ nghĩa một số thuộc tính còn chưa biết.

3.6.4 Nạp các fact giao dịch

Mỗi task fact lấy file pending của dataset tương ứng, dựng #Desired từ clean.ValidTrip của một file và lookup đầy đủ các khóa. Tập mong đợi được so với fact hiện có theo hash và lineage. Các thao tác audit, delete, update, insert, kiểm tra số dòng và checkpoint thành công nằm trong transaction theo file. YellowGreen ghi ServiceTypeKey để tách hai nguồn; FHV ghi các vai trò base và FHVFlagKey; HVFHV ghi provider, các mốc thời gian và cờ chuyên biệt. File không có dòng hợp lệ vẫn phải đồng bộ thành tập fact rỗng trong phạm vi file đó, thay vì bỏ qua và để lại các dòng hợp lệ của phiên bản cũ.

3.6.5 Xử lý file hiệu chỉnh và bản ghi trùng

Hash payload chứa ngữ cảnh nguồn và file; ordinal phân biệt các lượt xuất hiện có nội dung giống nhau trong cùng file. Cách này bảo toàn multiplicity: hai dòng giống nhau không mặc nhiên là lỗi trùng cần loại. Đồng thời việc một dòng giống nội dung xuất hiện ở file khác không làm checkpoint của hai file ảnh hưởng nhau.
Khi nguồn hiệu chỉnh, tập #Desired có thể khác tập fact hiện tại. Loader ghi audit trước khi xóa phần không còn phù hợp và bổ sung nội dung mới. Một correction có thể làm TripKey thay đổi; người vận hành phải theo lineage và audit để truy vết. Thiếu stable TripID nguồn là giới hạn thiết kế cần được công bố, không được che bằng cách gọi RecordHash là mã chuyến chính thức.

3.6.6 Quản lý transaction và checkpoint

Transaction theo file bảo đảm dữ liệu fact và checkpoint thành công cùng commit hoặc cùng rollback. Nếu file thứ ba lỗi, hai file trước đã commit vẫn được giữ; khi chạy lại, checkpoint cho phép chỉ chọn phần cần xử lý. Run tổng không phải một transaction bao phủ mọi file và mọi công nghệ.
DwhFileLog ghi dataset, SourceKey, phiên bản nguồn và loader, trạng thái, ForceReload, số dòng valid và fact, số insert update delete, UnknownMask và lỗi. Các session context hỗ trợ thử lỗi trước checkpoint trong fixture. FAILED được chọn lại theo quy trình; không sửa Status thành SUCCEEDED để vượt qua validation.

3.6.7 Đối soát sau khi nạp

Validation theo lượt kiểm tra coverage checkpoint và số dòng của các file cần xử lý. Full reconciliation là kiểm tra riêng trên toàn kho khi không có ETL chạy, hỗ trợ phát hiện sửa đổi ngoài pipeline hoặc checkpoint không còn phản ánh fact. Hai cấp kiểm tra bổ sung nhau và không nên coi đếm tổng bằng nhau là chứng minh mọi thuộc tính đúng.
Một file có đúng số dòng nhưng lookup sai provider hoặc công thức tiền sai vẫn có thể vượt đối soát count. Do đó fixture cần kiểm tra payload, mapping và correction; khóa ngoại kiểm tra tham chiếu; SQL MDX kiểm tra sự thống nhất tầng phân tích. Báo cáo giữ riêng từng loại bằng chứng để không phóng đại phạm vi kiểm thử.

3.7 Xây dựng luồng tổng hợp dữ liệu

3.7.1 Tổng hợp tài chính theo ngày

Daily Financial nhóm các transaction có tài chính theo DateKey và ServiceTypeKey. Đầu ra gồm TotalTrips, TotalDistanceMiles, TotalPassengerCharges, TotalTips, TotalTaxesAndFees và TotalDriverPay.

Cách xây từng khoản dựa trên khả năng nguồn: tiền tài xế chủ yếu có ở HVFHV, trong khi TotalAmount của taxi là giá trị nguồn riêng.
Không nối FHV vào với các cột tiền bằng 0 và không cộng lặp cước cơ bản vào tổng tiền đã bao gồm cước. Các giá trị thiếu cần được giữ để người đọc hiểu độ bao phủ tài chính. Kiểm tra tổng snapshot so với phép nhóm SQL nguồn là điều kiện trước commit.

3.7.2 Tổng hợp lưu lượng theo vùng

Zone Traffic tạo một tập pickup theo khóa ngày giờ vùng đón và một tập dropoff theo khóa ngày giờ vùng trả. Hai tập được kết hợp rồi group ở grain ngày, phút, vùng, dịch vụ. Unknown vẫn nằm trong tổng hợp để không làm mất lượng chuyến chưa có địa lý.
So sánh nguồn được thực hiện trên cùng sự kiện. Nếu hỏi nhu cầu đón tại quận, dùng TotalPickups và ngày pickup; nếu hỏi nơi kết thúc, dùng TotalDropoffs và ngày dropoff. Không dùng hai đại lượng cộng lại để báo cáo số chuyến, và không ép chúng bằng nhau ở từng ngày hoặc từng zone.

3.7.3 Tổng hợp hiệu suất điều phối

Base Performance nhóm theo ngày, dispatch base, dịch vụ và provider. Thời lượng và chờ được cộng cùng các số đếm hợp lệ tương ứng. Các tổng request, match, WAV và tiền tài xế giữ riêng để phục vụ các công thức đã định nghĩa ở tầng SSAS.
AverageTripDurationSeconds và AverageWaitingTimeSeconds ở một dòng snapshot chỉ diễn tả nhóm grain đó. Khi người dùng chọn nhiều base hoặc nhiều ngày, SSAS phải dùng TotalTripDurationSeconds chia DurationTripCount, hoặc TotalWaitingTimeSeconds chia WaitingTripCount, rồi đổi đơn vị. Lấy trung bình các giá trị trung bình sẽ sai khi số chuyến mỗi nhóm khác nhau.

3.7.4 Kiểm tra tính nhất quán của các bảng tổng hợp

Ba snapshot và watermark được rebuild trong cùng một transaction, gồm cả validation. Nếu một phép kiểm tra thất bại, các bảng tổng hợp và watermark cùng rollback. Điều này tránh trạng thái Daily Financial đã mới nhưng Zone Traffic vẫn cũ trong một lần aggregate được xem là thành công.
Thiết kế hiện rebuild toàn bộ, có lợi cho phản ánh correction và deletion lịch sử nhưng chưa tối ưu theo ngày bị ảnh hưởng. Watermark không được mô tả như bằng chứng aggregate đã incremental theo vùng thay đổi. Khi dữ liệu tăng, có thể thiết kế phạm vi tổng hợp lại dựa trên lineage, nhưng phải giữ tính nguyên tử và kiểm tra tổng.

3.8 Xử lý lỗi và vận hành

3.8.1 Nhật ký và theo dõi tiến trình

PipelineRun ghi RunId, ConfigHash, khoảng tháng, giới hạn dòng, trạng thái, thời điểm và lỗi cấp lượt. PipelineStep ghi stage, task, thời gian, trạng thái và thống kê. PackageError ghi lỗi native SSIS khi handler hoạt động; runner còn giữ events.jsonl, status.json, failure JSON, stdout và stderr để không phụ thuộc hoàn toàn vào SQL.
Monitor hiển thị stage, task hoặc file, số dòng wrk, thời gian và heartbeat. Các câu SQL không cung cấp phần trăm hoàn tất nên monitor không tự tạo phần trăm giả. Một khoảng thiếu heartbeat chưa đủ để khẳng định tiến trình chết vì một số thao tác đồng bộ có thể vẫn đang làm việc; cần kiểm tra cả process, khóa SQL và log trước khi phục hồi.

3.8.2 Cách ly file lỗi

Quarantine sao chép CSV, sidecar và error JSON vào thư mục theo RunId và dataset khi file không đáp ứng inspect, bulk hoặc commit. Lỗi gốc được giữ và luồng dừng. Đây là cách ly lỗi cấp file; bản ghi lỗi nghiệp vụ có cấu trúc hợp lệ

được phản ánh trong clean.RejectedTrip hoặc clean.WarningTrip, không nhất thiết nằm trong quarantine.
Alert JSON được tạo cho công cụ giám sát đọc khi run thất bại. Hệ thống không mặc nhiên đã gửi email hoặc thông báo ra ngoài. Xử lý lỗi cần dựa vào file, stage và InvocationId đúng, tránh dùng một alert cũ làm kết luận cho lần retry hiện tại.

3.8.3 Retry chạy lại và phục hồi

Controller cho retry tối đa theo pMaxRetries trong khoảng 0 đến 3, mặc định 1. Chỉ các lỗi SQL tạm thời được nhận diện như timeout, deadlock và một số lỗi kết nối mới đủ điều kiện; lỗi schema, dữ liệu hoặc Python không tự retry như lỗi mạng. Structured failure giữ SqlNumber qua tiến trình con và kiểm tra InvocationId để tránh đọc nhầm lỗi cũ.
Resume yêu cầu RunId cũ, cấu hình khớp và stage trước đã StageComplete SUCCEEDED. Stage lỗi chạy lại theo checkpoint của chính nó, không tiếp tục giữa một câu SQL. Controller đã dùng Job Object KILL_ON_JOB_CLOSE để quản lý runner và hậu duệ; Recover-Run chỉ đóng run RUNNING thành FAILED theo thao tác chủ động khi lấy được khóa độc quyền. Chưa có trạng thái CANCELLED hoặc ABANDONED tự động và chưa có nút hủy trên monitor.

3.8.4 Kiểm soát chạy đồng thời

Runner giữ sp_getapplock độc quyền với tài nguyên NYCTaxi Full Pipeline theo session và không chờ khi có khóa khác. Ngoài khóa đang giữ, run RUNNING chưa kết thúc cũng chặn một RunId mới. Hai cơ chế bảo vệ ngăn việc chạy chồng do lịch hoặc do người dùng khởi chạy thêm.
Các đường native package con chưa được xác nhận dùng đầy đủ cùng guard. Vì vậy quy trình vận hành thống nhất qua Master hoặc runner; không Execute package con song song. Không tạo RunId mới hoặc sửa log để vượt guard khi chưa kiểm tra tiến trình cũ đã dừng và khóa đã được giải phóng.

3.8.5 Lập lịch thực thi

Hồ sơ hardening ghi đã đăng ký task NYCTaxi Monthly ETL chạy ngày 15 hàng tháng lúc 10 giờ Bangkok, chọn tháng hiện tại trừ hai và MaxRows 100000. Tháng mục tiêu được tính lúc chạy, không cố định theo ngày đăng ký. Task dùng interactive token nên người dùng phải đăng nhập; StartWhenAvailable không thay thế điều kiện này.

Cấu hình có IgnoreNew và giới hạn thời gian 24 giờ. Mặc định script đăng ký là MaxRows 0, khác giá trị đã truyền khi tạo lịch trong hồ sơ. Chưa có bằng chứng chạy thử lịch thật sau đăng ký; vì vậy báo cáo xác nhận có cấu hình lịch nhưng không kết luận lịch đã vận hành tự động ổn định. Lịch có thể nhận lại tháng cũ nhưng không tự giải quyết mọi file hiệu chỉnh ngoài phạm vi được chọn.

3.9 Kiểm thử và đánh giá kết quả ETL

Hồ sơ rà soát ghi 56 file chuyến được đối chiếu clean fact không lệch số dòng; transaction fact gồm 278.213 YellowGreen, 139.970 FHV và 140.000 HVFHV, tổng 558.183. Lượt no-op được chụp có PendingFiles bằng 0 và cả ba task fact ProcessedFiles bằng 0. Đây là bằng chứng DWH bỏ qua file không đổi trong lần quan sát, không chứng minh Transform, Aggregate và SSAS đều incremental.

Hồ sơ hardening ghi 12 trên 12 kiểm thử Python và metadata, 22 trên 22 ca SQL incremental cách ly, ba ca mapping cờ FHV, bảy ca controller cùng các kiểm tra Job Object và truyền lỗi. Fixture cách ly không phải benchmark full data. Sau thay đổi loader FHV, cần chạy production có kiểm soát và đối soát theo đúng phiên bản trước khi coi chuyển đổi hoàn tất.

Bảng 3.8 Bằng chứng kiểm thử ETL đã lưu

Nhóm	**Kết quả lịch sử**	**Phạm vi chứng minh**
Clean và fact	558183 dòng hợp lệ; 56 file không lệch count	Nhất quán số dòng tại thời điểm rà soát.

Nhóm	Kết quả lịch sử	Phạm vi chứng minh
Incremental SQL fixture	22 trên 22 ca đạt theo hồ sơ hardening	File mới no-op duplicates correction rollback mapping và ForceReload trong DB cách ly.
FHV flag fixture	1 → Y; 0 → N; NULL → U	Logic mapping source cờ theo phiên bản mới.
Controller fixture	7 ca đạt	Success lỗi vĩnh viễn transient retry stale failure và resume.
Python metadata	12 trên 12 theo hồ sơ hardening	Các điều kiện được kiểm thử của converter và metadata.
Lịch production restore UI	Chưa có đủ bằng chứng nghiệm thu	Không suy ra hoàn tất từ unit test và truy vấn dữ liệu.

CHƯƠNG 4 XÂY DỰNG MÔ HÌNH PHÂN TÍCH SSAS

4.1 Mục tiêu và tổ chức mô hình SSAS

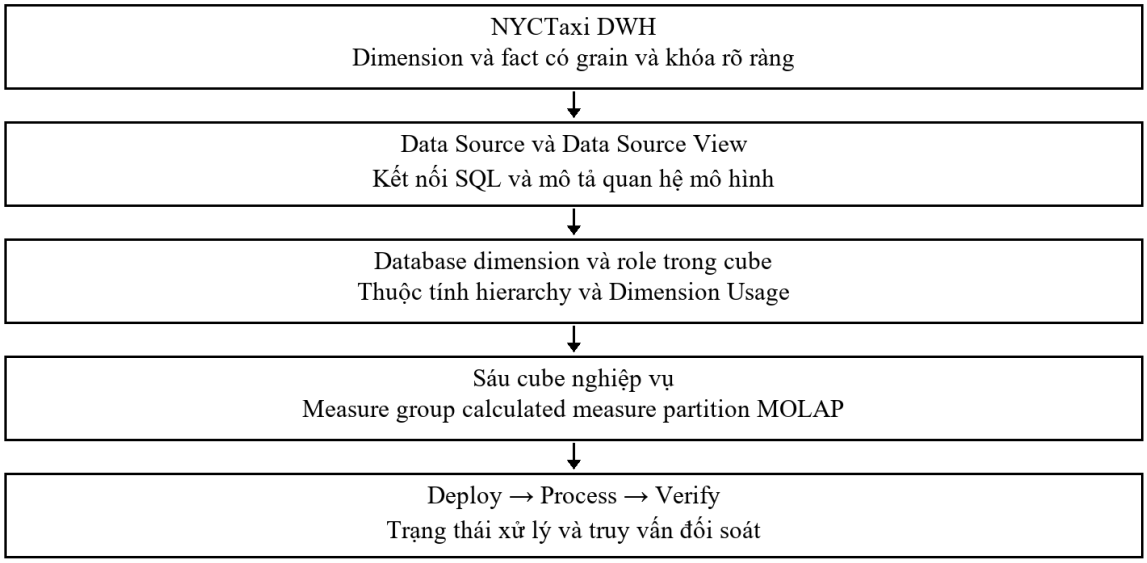
SSAS tạo tầng ngữ nghĩa đa chiều để các báo cáo dùng chung khóa phân tích và công thức. Dự án dùng sáu cube nghiệp vụ, mỗi cube một measure group gắn với fact tương ứng. Các database dimension và DSV được dùng chung; cube chỉ tham gia các dimension phù hợp với phạm vi nghiệp vụ.

Cách tổ chức này làm rõ nguồn của từng phép đo. Một card Trip Count của FHV không phải tổng mọi dịch vụ, còn Daily Financial không bao gồm FHV tài chính. Không có một measure mặc định cộng chéo sáu cube; người đọc phải dùng cùng phạm vi lọc hoặc snapshot phù hợp cho câu hỏi liên nguồn.

4.2 Khởi tạo Data Source và Data Source View

Data Source DS_NYCTaxi_DWH dùng provider MSOLEDBSQL để đọc NYCTaxi_DWH trên SQL Server cấu hình. Windows authentication và danh tính dịch vụ SSAS quyết định quyền truy cập trong lúc process. Đúng connection string nhưng thiếu quyền SELECT vẫn làm xử lý dữ liệu thất bại.

DSV mô tả 15 dimension, ba fact giao dịch và ba snapshot cùng quan hệ khóa. Tệp DSV_NYCTaxi.dsv giữ tên kế thừa, còn ID mô hình là DSV_NYCTaxi_DWH. DSV cũ NYC Taxi DWH được giữ trên đĩa nhưng không dùng làm DSV hoạt động của project. Khi triển khai phải đối chiếu ID tham chiếu trong cube thay vì chọn một file chỉ vì tên gần giống.



Hình 4.1 Tổ chức tầng phân tích SSAS

4.3 Xây dựng các dimension phân tích

4.3.1 Cấu hình thuộc tính khóa và tên hiển thị

Mỗi dimension khai báo KeyColumns xác định member, NameColumn xác định nhãn và thuộc tính phục vụ lọc. Tên hiển thị trùng không chứng minh hai member là một; vùng dùng LocationKey làm khóa nhưng hiển thị ZoneName. Mã nghiệp vụ và nhãn cũng được tách ở provider, base và các chiều phân loại. Các thuộc tính nullable có quy tắc chuyển lỗi hoặc blank riêng ở tầng phân tích. Không mở rộng quy tắc attribute NULL thành một nguyên tắc thay toàn bộ measure NULL bằng 0. Tập dimension được xử lý trước hoặc cùng batch xử lý cube để các tham chiếu member luôn sẵn sàng.

Bảng 4.1 Attribute key và quan hệ Dim Base

Attribute	**KeyColumns**	**NameColumn**	**Attribute relationship**
Base Key	BaseKey	BaseNumber	Base Name, Base Category
Base Name	BaseName	Dùng key theo cấu hình	Không khai báo thêm

Attribute	KeyColumns	NameColumn	Attribute relationship
Base Category	BaseCategory	Dùng key theo cấu hình	Không khai báo thêm

Bảng 4.2 Attribute key và quan hệ Dim Date

Attribute	KeyColumns	NameColumn	Attribute relationship
Date Key	DateKey	FullDate	Day Of Month, Day Of Week Name, Full Date, Is Holiday, Is Weekend, Month Name
Day Of Week Name	DayOfWeekName	Dùng key theo cấu hình	Không khai báo thêm
Day Of Month	DayOfMonth	Dùng key theo cấu hình	Không khai báo thêm
Month Number	YearNumber, MonthNumber	MonthName	Quarter Number
Month Name	MonthNumber	MonthName	Không khai báo thêm
Quarter Number	YearNumber, QuarterNumber	QuarterNumber	Year Number
Year Number	YearNumber	Dùng key theo cấu hình	Không khai báo thêm
Is Weekend	IsWeekend	Dùng key theo cấu hình	Không khai báo thêm
Is Holiday	IsHoliday	Dùng key theo cấu hình	Không khai báo thêm
Full Date	DateKey	FullDate	Month Number

Bảng 4.3 Attribute key và quan hệ Dim FHV Flags

Attribute	**KeyColumns**	**NameColumn**	**Attribute relationship**
FHV Flag Key	FHVFlagKey	IsSharedRide	Không khai báo thêm

Bảng 4.4 Attribute key và quan hệ Dim HVFHS Provider

Attribute	**KeyColumns**	**NameColumn**	**Attribute relationship**
HVFHS Provider Key	HVFHSProviderKey	ProviderName	License Number
License Number	LicenseNumber	Dùng key theo cấu hình	Không khai báo thêm

Bảng 4.5 Attribute key và quan hệ Dim HVFHV Flags

Attribute	**KeyColumns**	**NameColumn**	**Attribute relationship**
HVFHV Flag Key	HVFHVFlagKey	Dùng key theo cấu hình	Is Shared Request, Is Shared Match, Is Access A Ride, Is WAV Request, Is WAV Match
Is Shared Request	IsSharedRequest	Dùng key theo cấu hình	Không khai báo thêm
Is Shared Match	IsSharedMatch	Dùng key theo cấu hình	Không khai báo thêm
Is Access A Ride	IsAccessARide	Dùng key theo cấu hình	Không khai báo thêm
Is WAV Request	IsWAVRequest	Dùng key theo cấu hình	Không khai báo thêm
Is WAV Match	IsWAVMatch	Dùng key theo cấu hình	Không khai báo thêm

Bảng 4.6 Attribute key và quan hệ Dim Load Batch

Attribute	**KeyColumns**	**NameColumn**	**Attribute relationship**
Load Batch Key	LoadBatchKey	BatchID	BatchID, Dataset Name, Source File Name, Source Year, Source Month, Batch Status
BatchID	BatchID	Dùng key theo cấu hình	Không khai báo thêm
Dataset Name	DatasetName	Dùng key theo cấu hình	Không khai báo thêm
Source File Name	SourceFileName	Dùng key theo cấu hình	Không khai báo thêm
Source Year	SourceYear	Dùng key theo cấu hình	Không khai báo thêm
Source Month	SourceMonth	Dùng key theo cấu hình	Không khai báo thêm
Batch Status	BatchStatus	Dùng key theo cấu hình	Không khai báo thêm

Bảng 4.7 Attribute key và quan hệ Dim Location

Attribute	**KeyColumns**	**NameColumn**	**Attribute relationship**
Location Key	LocationKey	ZoneName	Location ID, Service Zone, Zone Name
Location ID	LocationID	Dùng key theo cấu hình	Không khai báo thêm
Borough	Borough	Borough	Không khai báo thêm
Service Zone	ServiceZone	Dùng key theo cấu hình	Không khai báo thêm

Attribute	KeyColumns	NameColumn	Attribute relationship
Zone Name	LocationKey	ZoneName	Borough

Bảng 4.8 Attribute key và quan hệ Dim Passenger Group

Attribute	KeyColumns	NameColumn	Attribute relationship
Passenger Group Key	PassengerGroupKey	GroupName	Passenger Group Code
Passenger Group Code	PassengerGroupCode	Dùng key theo cấu hình	Không khai báo thêm

Bảng 4.9 Attribute key và quan hệ Dim Payment Type

Attribute	KeyColumns	NameColumn	Attribute relationship
Payment Type Key	PaymentTypeKey	PaymentTypeDescription	Không khai báo thêm

Bảng 4.10 Attribute key và quan hệ Dim Rate Code

Attribute	KeyColumns	NameColumn	Attribute relationship
Rate Code Key	RateCodeKey	RateCodeDescription	Rate Code ID
Rate Code ID	RateCodeID	Dùng key theo cấu hình	Không khai báo thêm

Bảng 4.11 Attribute key và quan hệ Dim Service Type

Attribute	KeyColumns	NameColumn	Attribute relationship
Service Type Key	ServiceTypeKey	ServiceName	Service Code, Service Name
Service Code	ServiceCode	Dùng key theo cấu hình	Không khai báo thêm

Attribute	KeyColumns	NameColumn	Attribute relationship
Service Name	ServiceName	Dùng key theo cấu hình	Không khai báo thêm

Bảng 4.12 Attribute key và quan hệ Dim Taxi Flags

Attribute	KeyColumns	NameColumn	Attribute relationship
Taxi Flag Key	TaxiFlagKey	StoreAndFwd	Không khai báo thêm

Bảng 4.13 Attribute key và quan hệ Dim Time

Attribute	KeyColumns	NameColumn	Attribute relationship
Time Key	TimeKey	TimeLabel	Is Peak Hour, Time Label
Hour Number	TimeBand, HourNumber	HourNumber	Time Band
Time Band	TimeBand	TimeBand	Không khai báo thêm
Is Peak Hour	IsPeakHour	IsPeakHour	Không khai báo thêm
Time Label	TimeKey	Dùng key theo cấu hình	Hour Number

Bảng 4.14 Attribute key và quan hệ Dim Trip Type

Attribute	KeyColumns	NameColumn	Attribute relationship
Trip Type Key	TripTypeKey	TripTypeDescription	Trip Type ID
Trip Type ID	TripTypeID	Dùng key theo cấu hình	Không khai báo thêm

Bảng 4.15 Attribute key và quan hệ Dim Vendor

Attribute	**KeyColumns**	**NameColumn**	**Attribute relationship**
Vendor Key	VendorKey	VendorName	Vendor Name
Vendor Name	VendorName	Dùng key theo cấu hình	Không khai báo thêm

4.3.2 Xây dựng hierarchy và quan hệ thuộc tính

Calendar gồm Year, Quarter, Month và Date; Geography gồm Borough và Zone; Time gồm TimeBand, Hour và Time. Key ghép theo năm giúp thứ tự thời gian đúng ở nhiều năm. NameColumn hỗ trợ nhãn đọc được, còn quan hệ thuộc tính mô tả các phụ thuộc tự nhiên để SSAS tổng hợp và điều hướng.
Một hierarchy có tên và thứ tự cấp chưa đủ nếu attribute key hoặc relationship sai. Cần kiểm tra mỗi ngày chỉ thuộc một tháng và năm đúng, mỗi zone thuộc Borough theo danh mục, và Unknown không bị gộp với giờ 0. Các hierarchy này cũng định hướng cách bố trí drill và bộ lọc trên Power BI.

4.3.3 Cấu hình dimension đóng nhiều vai trò

Ở cube, cùng database dimension được đưa vào các role có ID riêng như Pickup Date, Dropoff Date, Request Date hoặc On Scene Time. Dimension Usage nối từng role về cột khóa tương ứng của measure group. Đó là cách giữ một danh mục chuẩn trong khi vẫn cho phép lọc theo nhiều mốc độc lập.
Lọc Pickup Date trả lời các chuyến bắt đầu trong ngày, còn lọc Dropoff Date trả lời các chuyến kết thúc trong ngày. Nếu đặt cả hai, kết quả là giao của hai điều kiện chứ không phải hai cách gọi cho một trường. Dashboard phải chọn role phù hợp với câu hỏi; snapshot sử dụng role Snapshot Date theo grain tổng hợp.

4.3.4 Xử lý Unknown và lỗi khóa

Dimension giữ member khóa 0 như dữ liệu thông thường, đồng thời có UnknownMember bổ sung cho khóa không phân giải. ErrorConfiguration đặt

KeyNotFound, KeyDuplicate và NullKeyNotAllowed theo hướng ReportAndStop. Lỗi tham chiếu không được che bằng cách bỏ fact hoặc tăng ErrorLimit để làm trạng thái Processed xuất hiện.

Khi process lỗi, cần đối chiếu khóa nguồn, DSV, dimension và fact trước khi thử lại. Việc có nhóm Unknown trên báo cáo không tự chứng minh process sai: đó có thể là kết quả loader giữ đúng sự kiện thiếu thuộc tính. Ngược lại, không nhìn thấy Unknown chưa chắc dữ liệu hoàn chỉnh nếu visual đã lọc bỏ nhóm này.

4.4 Xây dựng các cube nghiệp vụ

4.4.1 Cube YellowGreen Trips

Cube sử dụng Fact_YellowGreen_Trips và các vai trò ngày giờ vùng đón trả, cùng Service Type, Vendor, Rate Code, Payment Type, Trip Type, Taxi Flags, Passenger Group và Load Batch. Các measure tài chính, quãng đường và số chuyến chỉ áp dụng phạm vi YellowGreen. Service Type cho phép tách Yellow và Green trong cùng một bề mặt phân tích.

Calculated measure tính trung bình cước, khoảng cách, thời lượng, tip và tốc độ từ các tổng hoặc mẫu số hỗ trợ. Đối với tỷ lệ tip, mẫu số là cước cơ bản ở cùng tập có cả tip và cước, không phải TotalAmount. Measure hỗ trợ được ẩn khỏi danh sách nghiệp vụ khi không cần cho người sử dụng cuối.

Bảng 4.16 Measure và nguồn dữ liệu cube YellowGreen Trips

Measure	**Cột nguồn và hàm**	**Hiển thị**
Trip Count - Fact Yellow Green Trips	TripCount; Sum	Hiện
Passenger Count	PassengerCount; Sum	Hiện
Trip Distance Miles - Fact Yellow Green Trips	TripDistanceMiles; Sum	Hiện

Measure	**Cột nguồn và hàm**	**Hiển thị**
Trip Duration Seconds - Fact Yellow Green Trips	TripDurationSeconds; Sum	Hiện
Fare Amount	FareAmount; Sum	Hiện
Extra Amount	ExtraAmount; Sum	Hiện
MTA Tax Amount	MTATaxAmount; Sum	Hiện
Tip Amount - Fact Yellow Green Trips	TipAmount; Sum	Hiện
Tolls Amount - Fact Yellow Green Trips	TollsAmount; Sum	Hiện
Improvement Surcharge	ImprovementSurcharge; Sum	Hiện
Congestion Surcharge - Fact Yellow Green Trips	CongestionSurcharge; Sum	Hiện
Airport Fee - Fact Yellow Green Trips	AirportFee; Sum	Hiện
CBD Congestion Fee - Fact Yellow Green Trips	CBDCongestionFee; Sum	Hiện
Total Amount	TotalAmount; Sum	Hiện
Fact Yellow Green Trips TripDurationSecondsValidTrips	TripDurationSecondsValidTrips; Sum	Ẩn
Fact Yellow Green Trips TripDistanceMilesValidTrips	TripDistanceMilesValidTrips; Sum	Ẩn
Fact Yellow Green Trips FareAmountValidTrips	FareAmountValidTrips; Sum	Ẩn
Fact Yellow Green Trips TipAmountValidTrips	TipAmountValidTrips; Sum	Ẩn
Fact Yellow Green Trips SpeedDistanceMiles	SpeedDistanceMiles; Sum	Ẩn

Measure	Cột nguồn và hàm	Hiển thị
Fact Yellow Green Trips SpeedDurationSeconds	SpeedDurationSeconds; Sum	Ẩn
Fact Yellow Green Trips TipPairAmount	TipPairAmount; Sum	Ẩn
Fact Yellow Green Trips TipPairFare	TipPairFare; Sum	Ẩn

Bảng 4.17 Dimension Usage cube YellowGreen Trips

Role trong cube	Database dimension	Grain attribute	Cột nguồn
Pickup Date	Dim Date	Date Key	PickupDateKey
Dropoff Date	Dim Date	Date Key	DropoffDateKey
Pickup Time	Dim Time	Time Key	PickupTimeKey
Dropoff Time	Dim Time	Time Key	DropoffTimeKey
Pickup Location	Dim Location	Location Key	PickupLocationKey
Dropoff Location	Dim Location	Location Key	DropoffLocationKey
Dim Service Type	Dim Service Type	Service Type Key	ServiceTypeKey
Dim Vendor	Dim Vendor	Vendor Key	VendorKey
Dim Rate Code	Dim Rate Code	Rate Code Key	RateCodeKey
Dim Payment Type	Dim Payment Type	Payment Type Key	PaymentTypeKey
Dim Trip Type	Dim Trip Type	Trip Type Key	TripTypeKey
Dim Taxi Flags	Dim Taxi Flags	Taxi Flag Key	TaxiFlagKey
Dim Passenger Group	Dim Passenger Group	Passenger Group Key	PassengerGroupKey

Role trong cube	Database dimension	Grain attribute	Cột nguồn
Dim Load Batch	Dim Load Batch	Load Batch Key	LoadBatchKey

4.4.2 Cube FHV Trips

Cube FHV dùng fact FHV để phân tích Trip Count và Trip Duration Seconds, cùng trung bình thời lượng tính từ mẫu số hợp lệ. Dispatch Base và Affiliated Base là các role riêng. Cube không được bổ sung measure tiền hoặc quãng đường không có trong nguồn chỉ để đồng nhất danh sách với YellowGreen. Các phân tích địa lý phải giữ giới hạn Unknown; các phân tích theo tháng dùng Calendar của đúng role. Khi giải thích thời lượng, cần nêu measure gốc là giây và calculated measure đổi sang phút. Đơn vị là một phần của định nghĩa, không chỉ là chuỗi định dạng trên visual.

Bảng 4.18 Measure và nguồn dữ liệu cube FHV Trips

Measure	Cột nguồn và hàm	Hiển thị
Trip Count	TripCount; Sum	Hiện
Trip Duration Seconds	TripDurationSeconds; Sum	Hiện
Fact FHV Trips TripDurationSecondsValidTrips	TripDurationSecondsValidTrips; Sum	Ẩn

Bảng 4.19 Dimension Usage cube FHV Trips

Role trong cube	Database dimension	Grain attribute	Cột nguồn
Pickup Date	Dim Date	Date Key	PickupDateKey
Dropoff Date	Dim Date	Date Key	DropoffDateKey
Pickup Time	Dim Time	Time Key	PickupTimeKey
Dropoff Time	Dim Time	Time Key	DropoffTimeKey
Pickup Location	Dim Location	Location Key	PickupLocationKey

Role trong cube	Database dimension	Grain attribute	Cột nguồn
Dropoff Location	Dim Location	Location Key	DropoffLocationKey
Dim Service Type	Dim Service Type	Service Type Key	ServiceTypeKey
Dispatch Base	Dim Base	Base Key	DispatchBaseKey
Affiliated Base	Dim Base	Base Key	AffiliatedBaseKey
Dim FHV Flags	Dim FHV Flags	FHV Flag Key	FHVFlagKey
Dim Load Batch	Dim Load Batch	Load Batch Key	LoadBatchKey

4.4.3 Cube HVFHV Trips

Cube HVFHV sử dụng các role thời gian Request, On Scene, Pickup, Dropoff cùng địa lý, provider, base và cờ HVFHV. Các measure tài chính gồm cước cơ bản, phí, thuế, tip, tiền tài xế và tổng tiền khách theo DWH. Nhóm thời gian chờ có mẫu số hợp lệ riêng vì không phải mọi dòng đều có request phù hợp. Average Waiting Minutes lấy tổng WaitingTimeSeconds chia số dòng có giá trị hợp lệ rồi đổi phút. On Scene To Pickup Seconds là một đại lượng khác và có giới hạn độ bao phủ của nguồn. Tốc độ dùng cùng tập dòng có quãng đường hợp lệ và thời lượng dương. Không chia tổng quãng đường của mọi dòng cho thời gian chỉ của một phần khác.

Bảng 4.20 Measure và nguồn dữ liệu cube HVFHV Trips

Measure	Cột nguồn và hàm	Hiển thị
Trip Count - Fact HVFHV Trips	TripCount; Sum	Hiện
Trip Distance Miles	TripDistanceMiles; Sum	Hiện
Trip Duration Seconds - Fact HVFHV Trips	TripDurationSeconds; Sum	Hiện
Waiting Time Seconds	WaitingTimeSeconds; Sum	Hiện

Measure	**Cột nguồn và hàm**	**Hiển thị**
On Scene To Pickup Seconds	OnSceneToPickupSeconds; Sum	Hiện
Base Passenger Fare	BasePassengerFare; Sum	Hiện
Tolls Amount	TollsAmount; Sum	Hiện
BCF Amount	BCFAmount; Sum	Hiện
Sales Tax Amount	SalesTaxAmount; Sum	Hiện
Congestion Surcharge	CongestionSurcharge; Sum	Hiện
Airport Fee	AirportFee; Sum	Hiện
Tip Amount	TipAmount; Sum	Hiện
Driver Pay Amount	DriverPayAmount; Sum	Hiện
CBD Congestion Fee	CBDCongestionFee; Sum	Hiện
Gross Passenger Amount	GrossPassengerAmount; Sum	Hiện
Fact HVFHV Trips TripDurationSecondsValidTrips	TripDurationSecondsValidTrips; Sum	Ẩn
Fact HVFHV Trips TripDistanceMilesValidTrips	TripDistanceMilesValidTrips; Sum	Ẩn
Fact HVFHV Trips WaitingTimeSecondsValidTrips	WaitingTimeSecondsValidTrips; Sum	Ẩn
Fact HVFHV Trips BasePassengerFareValidTrips	BasePassengerFareValidTrips; Sum	Ẩn
Fact HVFHV Trips TipAmountValidTrips	TipAmountValidTrips; Sum	Ẩn
Fact HVFHV Trips SpeedDistanceMiles	SpeedDistanceMiles; Sum	Ẩn
Fact HVFHV Trips SpeedDurationSeconds	SpeedDurationSeconds; Sum	Ẩn

Measure	Cột nguồn và hàm	Hiển thị
Fact HVFHV Trips TipPairAmount	TipPairAmount; Sum	Ẩn
Fact HVFHV Trips TipPairFare	TipPairFare; Sum	Ẩn

Bảng 4.21 Dimension Usage cube HVFHV Trips

Role trong cube	Database dimension	Grain attribute	Cột nguồn
Request Date	Dim Date	Date Key	RequestDateKey
On Scene Date	Dim Date	Date Key	OnSceneDateKey
Pickup Date	Dim Date	Date Key	PickupDateKey
Dropoff Date	Dim Date	Date Key	DropoffDateKey
Request Time	Dim Time	Time Key	RequestTimeKey
On Scene Time	Dim Time	Time Key	OnSceneTimeKey
Pickup Time	Dim Time	Time Key	PickupTimeKey
Dropoff Time	Dim Time	Time Key	DropoffTimeKey
Pickup Location	Dim Location	Location Key	PickupLocationKey
Dropoff Location	Dim Location	Location Key	DropoffLocationKey
Dim Service Type	Dim Service Type	Service Type Key	ServiceTypeKey
Dim HVFHS Provider	Dim HVFHS Provider	HVFHS Provider Key	HVFHSProviderKey
Dispatch Base	Dim Base	Base Key	DispatchBaseKey
Origin Base	Dim Base	Base Key	OriginBaseKey
Dim HVFHV Flags	Dim HVFHV Flags	HVFHV Flag Key	HVFHVFlagKey

Role trong cube	Database dimension	Grain attribute	Cột nguồn
Dim Load Batch	Dim Load Batch	Load Batch Key	LoadBatchKey

4.4.4 Cube Daily Financial

Cube Daily Financial đọc snapshot ngày và dịch vụ, dùng các measure tổng tài chính được ETL chuẩn bị. Điều này giúp báo cáo tài chính không phải quét và phối hợp ba transaction fact ở lớp Power BI. Snapshot Date và Service Type xác định phạm vi chính; các chiều thanh toán hoặc provider không được tự gán nếu không có trong grain.

Các ô tiền tài xế thiếu không được hiểu là doanh nghiệp không trả tiền; đó có thể là trường không áp dụng cho nguồn. Daily Financial phục vụ mô tả dòng tiền ghi nhận, chưa phải báo cáo kết quả kinh doanh đầy đủ. Không thêm tổng số chuyến của cube này vào số chuyến transaction.

Bảng 4.22 Measure và nguồn dữ liệu cube Daily Financial

Measure	Cột nguồn và hàm	Hiển thị
Daily Financial Snapshot TotalTrips	TotalTrips; Sum	Hiện
Daily Financial Snapshot TotalDistanceMiles	TotalDistanceMiles; Sum	Hiện
Daily Financial Snapshot TotalPassengerCharges	TotalPassengerCharges; Sum	Hiện
Daily Financial Snapshot TotalTips	TotalTips; Sum	Hiện
Daily Financial Snapshot TotalTaxesAndFees	TotalTaxesAndFees; Sum	Hiện
Daily Financial Snapshot TotalDriverPay	TotalDriverPay; Sum	Hiện

Bảng 4.23 Dimension Usage cube Daily Financial

Role trong cube	**Database dimension**	**Grain attribute**	**Cột nguồn**
Snapshot Date	Dim Date	Date Key	DateKey
Dim Service Type	Dim Service Type	Service Type Key	ServiceTypeKey

4.4.5 Cube Zone Traffic

Cube Zone Traffic đọc snapshot ngày, phút, vùng và dịch vụ, cung cấp TotalPickups và TotalDropoffs. Đây là bề mặt phù hợp để so lưu lượng các dịch vụ theo cùng tập chiều chung. Khi roll up qua giờ hoặc Borough, SSAS cộng các sự kiện thuộc các member ở cấp thấp hơn.

Measure có ý nghĩa theo loại sự kiện nên tiêu đề visual phải ghi rõ lượt đón hoặc lượt trả. Không gọi một visual TotalDropoffs là nhu cầu đón. Trường hợp so sánh nhiều nguồn vẫn cần kiểm tra các file của mỗi nguồn có giới hạn hoặc độ bao phủ tương đương hay không.

Bảng 4.24 Measure và nguồn dữ liệu cube Zone Traffic

Measure	**Cột nguồn và hàm**	**Hiển thị**
Zone Traffic Snapshot TotalPickups	TotalPickups; Sum	Hiện
Zone Traffic Snapshot TotalDropoffs	TotalDropoffs; Sum	Hiện

Bảng 4.25 Dimension Usage cube Zone Traffic

Role trong cube	**Database dimension**	**Grain attribute**	**Cột nguồn**
Snapshot Date	Dim Date	Date Key	DateKey
Snapshot Time	Dim Time	Time Key	TimeKey
Snapshot Location	Dim Location	Location Key	LocationKey

Role trong cube	Database dimension	Grain attribute	Cột nguồn
Dim Service Type	Dim Service Type	Service Type Key	ServiceTypeKey

4.4.6 Cube Base Performance

Cube Base Performance đọc snapshot ngày, dispatch base, dịch vụ và provider. Ngoài tổng chuyến và tiền tài xế, cube lưu các tổng thời lượng và số đếm hợp lệ làm cơ sở tính trung bình. Shared Request Rate chỉ có ý nghĩa ở phạm vi dịch vụ có cờ yêu cầu đi chung.

WAV Match Rate đã được ẩn trong MDX script của bản hardening do phép chia hai tổng chưa có nghĩa tỷ lệ đáp ứng yêu cầu. Các measure số đếm vẫn giữ nguyên. Ẩn chỉ số là kiểm soát ngữ nghĩa sử dụng, không phải sửa dữ liệu để đưa kết quả về dưới 100 phần trăm.

Bảng 4.26 Measure và nguồn dữ liệu cube Base Performance

Measure	Cột nguồn và hàm	Hiển thị
Base Performance Snapshot TotalTrips	TotalTrips; Sum	Hiện
Base Performance Snapshot TotalTripDurationSeconds	TotalTripDurationSeconds; Sum	Hiện
Base Performance Snapshot DurationTripCount	DurationTripCount; Sum	Hiện
Base Performance Snapshot TotalWaitingTimeSeconds	TotalWaitingTimeSeconds; Sum	Hiện
Base Performance Snapshot WaitingTripCount	WaitingTripCount; Sum	Hiện
Base Performance Snapshot TotalSharedRequests	TotalSharedRequests; Sum	Hiện

Measure	Cột nguồn và hàm	Hiển thị
Base Performance Snapshot TotalSharedMatches	TotalSharedMatches; Sum	Hiện
Base Performance Snapshot TotalWAVRequests	TotalWAVRequests; Sum	Hiện
Base Performance Snapshot TotalWAVMatches	TotalWAVMatches; Sum	Hiện
Base Performance Snapshot TotalDriverPay	TotalDriverPay; Sum	Hiện

Bảng 4.27 Dimension Usage cube Base Performance

Role trong cube	Database dimension	Grain attribute	Cột nguồn
Snapshot Date	Dim Date	Date Key	DateKey
Dim Service Type	Dim Service Type	Service Type Key	ServiceTypeKey
Dim HVFHS Provider	Dim HVFHS Provider	HVFHS Provider Key	HVFHSProviderKey
Dispatch Base	Dim Base	Base Key	DispatchBaseKey

4.5 Thiết kế measure và calculated measure

4.5.1 Các độ đo cộng dồn

Measure cộng dồn dùng hàm Sum trên các cột số đếm, khoảng cách, thời gian và tiền ở đúng fact. Giá trị thời gian dùng giây ở measure gốc; khoảng cách dùng mile; tiền dùng USD. Cột AverageSpeedMPH hay AverageTripDurationSeconds không được đưa thành Sum để mô tả tốc độ hoặc thời lượng trung bình của tổng thể.

Các measure hỗ trợ như ValidTrips, TipPairAmount, TipPairFare và các tổng cho tập tính tốc độ được DSV hoặc mô hình dùng để bảo đảm cùng tập tử mẫu.

Chúng có thể bị ẩn vì phục vụ công thức, nhưng vẫn là thành phần quan trọng của thiết kế tính đúng.

4.5.2 Các độ đo trung bình

Trung bình một measure bằng tổng giá trị chia tổng số dòng có giá trị hợp lệ tương ứng. Ví dụ 10 chuyến có 2 chuyến thiếu quãng đường thì trung bình khoảng cách dùng 8 chuyến, không dùng 10 và cũng không coi 2 dòng thiếu là 0. Khi roll up, SSAS cộng tử và mẫu trước khi chia, tránh trung bình các nhóm có kích thước khác nhau.

Với thời lượng và chờ, chia thêm 60 để đổi giây sang phút sau phép chia có kiểm soát. Với tốc độ, dùng tổng mile nhân 3600 chia tổng giây trên cùng tập dòng hợp lệ. Một mẫu số trống hoặc bằng 0 trả NULL để visual thể hiện không có kết quả, không tạo giá trị vô hạn hoặc 0 giả.

4.5.3 Các tỷ lệ nghiệp vụ

Tip Percentage của taxi sử dụng TipAmount trên FareAmount, của HVFHV sử dụng TipAmount trên BasePassengerFare, đều ở tập dòng có đủ hai giá trị. Shared Request Rate dùng số yêu cầu đi chung trên số chuyến thuộc phạm vi nguồn phù hợp. Tỷ lệ phải có tên phản ánh đúng sự kiện và mẫu số, không thể chỉ chọn hai tổng có sẵn rồi gọi là hiệu quả.

Các công thức MDX đầy đủ được lưu trong Phụ lục A; bảng dưới trình bày quy tắc nghiệp vụ để người đọc kiểm tra trước khi xem mã. Việc SQL và MDX trả cùng một giá trị chỉ chứng minh hai tầng nhất quán với nhau; công thức vẫn cần phù hợp định nghĩa nghiệp vụ.

Bảng 4.28 Quy tắc calculated measure

Chỉ tiêu	**Công thức nghiệp vụ**	**Điều kiện**
Trung bình giá trị	Tổng giá trị / số dòng có giá trị hợp lệ	Không dùng tổng chuyến nếu measure có NULL.

Chỉ tiêu	Công thức nghiệp vụ	Điều kiện
Thời lượng hoặc chờ phút	Tổng giây / tổng số chuyến hợp lệ / 60	Mẫu số tương ứng từng measure.
Tốc độ MPH	Tổng mile cùng tập × 3600 / tổng giây cùng tập	Khoảng cách không âm và thời gian dương.
Tip Percentage taxi	Tổng tip của cặp hợp lệ / tổng FareAmount của cặp hợp lệ	Không chia cho TotalAmount.
Tip Percentage HVFHV	Tổng tip của cặp hợp lệ / tổng BasePassengerFare của cặp hợp lệ	Không dùng hai tập dòng khác nhau.
Shared Request Rate	TotalSharedRequests / TotalTrips thuộc nguồn áp dụng	Giữ cùng phạm vi nguồn và bộ lọc.
WAV Match Rate	Đã ẩn; hai cờ chưa đủ nghĩa tỷ lệ đáp ứng	Không clamp để ép kết quả dưới 100 phần trăm.

4.5.4 Xử lý NULL mẫu số bằng không và phạm vi áp dụng

Công thức dùng kiểm tra tập rỗng hoặc mẫu số 0 để trả NULL. Không dùng một giá trị thay thế chung cho mọi trường vì thiếu tiền, không áp dụng và không có giao dịch là các trạng thái khác nhau. Trong biểu đồ, cần diễn giải ô trống theo measure và nguồn, không suy đoán doanh thu bằng 0.
IgnoreUnrelatedDimensions bằng false giúp không lặp một tổng không liên quan theo các member của chiều khác. Payment Type không phải chiều chung của FHV hoặc HVFHV. Mọi so sánh liên cube phải giữ cùng bộ lọc và xác nhận độ bao phủ; chỉ có quan hệ dimension dùng chung không đủ để mọi measure đều so sánh trực tiếp.

4.6 Cấu hình Dimension Usage

Dimension Usage xác định role dimension nối tới measure group qua cột khóa nào và attribute granularity nào. Quan hệ Regular được dùng khi khóa dimension nằm trong fact hoặc nguồn của measure group. Một role đúng tên

nhưng nối nhầm PickupDateKey và DropoffDateKey có thể tạo báo cáo hợp lệ kỹ thuật nhưng sai câu hỏi nghiệp vụ.

Các bảng Dimension Usage được trình bày cùng từng cube ở mục 4.4. Khi kiểm tra, cần đối chiếu ba tầng: role trong cube, granularity attribute của dimension và cột nguồn trong measure group. Các chiều không nằm trong grain snapshot không được thêm tùy ý chỉ để có thêm slicer.

4.7 Thiết kế partition lưu trữ và aggregation

Dự án cấu hình 45 partition MOLAP: 13 FHV, 13 HVFHV, 16 YellowGreen và ba partition snapshot. Các transaction partition phân theo khoảng khóa ngày và nguồn khi cần; Other Dates giữ phần còn lại và Unknown. Tệp partitions có thể lưu State Unprocessed ở source thiết kế, khác trạng thái Processed trên server ở một lần chụp bằng chứng.

QueryDefinition của các partition phải không chồng phạm vi. Khi tách tháng mới, tháng đó phải được loại khỏi catch all để không cộng trùng. Aggregation design Operational chọn các tổ hợp như tháng và dịch vụ, ngày và Borough tùy cube; đây là tối ưu lưu trữ SSAS, khác với ba snapshot do ETL dựng. Planner sinh refresh theo partition đã có, nhưng runner hiện vẫn ProcessFull toàn database.

Bảng 4.29 Partition và phạm vi nguồn Base_Performance

Partition	**Lưu trữ và design**	**Nguồn hoặc QueryDefinition**
Base Performance Snapshot All	Molap; Operational	TableBinding dbo_Fact_Base_Performance_Snapshot

Bảng 4.30 Partition và phạm vi nguồn Daily_Financial

Partition	**Lưu trữ và design**	**Nguồn hoặc QueryDefinition**
Daily Financial Snapshot All	Molap; Operational	TableBinding dbo_Fact_Daily_Financial_Snapshot

Bảng 4.31 Partition và phạm vi nguồn FHV_Trips

Partition	**Lưu trữ và design**	**Nguồn hoặc QueryDefinition**
Fact FHV Trips 202501	Molap; Operational	SELECT * FROM dbo.[Fact_FHV_Trips] WHERE PickupDateKey >= 20250101 AND PickupDateKey < 20250201
Fact FHV Trips 202502	Molap; Operational	SELECT * FROM dbo.[Fact_FHV_Trips] WHERE PickupDateKey >= 20250201 AND PickupDateKey < 20250301
Fact FHV Trips 202503	Molap; Operational	SELECT * FROM dbo.[Fact_FHV_Trips] WHERE PickupDateKey >= 20250301 AND PickupDateKey < 20250401
Fact FHV Trips 202504	Molap; Operational	SELECT * FROM dbo.[Fact_FHV_Trips] WHERE PickupDateKey >= 20250401 AND PickupDateKey < 20250501
Fact FHV Trips 202505	Molap; Operational	SELECT * FROM dbo.[Fact_FHV_Trips] WHERE PickupDateKey >= 20250501 AND PickupDateKey < 20250601
Fact FHV Trips 202506	Molap; Operational	SELECT * FROM dbo.[Fact_FHV_Trips] WHERE PickupDateKey >= 20250601 AND PickupDateKey < 20250701
Fact FHV Trips 202507	Molap; Operational	SELECT * FROM dbo.[Fact_FHV_Trips] WHERE PickupDateKey >= 20250701 AND PickupDateKey < 20250801
Fact FHV Trips 202508	Molap; Operational	SELECT * FROM dbo.[Fact_FHV_Trips] WHERE PickupDateKey >= 20250801 AND PickupDateKey < 20250901
Fact FHV Trips 202509	Molap; Operational	SELECT * FROM dbo.[Fact_FHV_Trips] WHERE PickupDateKey >= 20250901 AND PickupDateKey < 20251001
Fact FHV Trips 202510	Molap; Operational	SELECT * FROM dbo.[Fact_FHV_Trips] WHERE PickupDateKey >= 20251001 AND PickupDateKey < 20251101

Partition	Lưu trữ và design	Nguồn hoặc QueryDefinition
Fact FHV Trips 202511	Molap; Operational	SELECT * FROM dbo.[Fact_FHV_Trips] WHERE PickupDateKey >= 20251101 AND PickupDateKey < 20251201
Fact FHV Trips 202512	Molap; Operational	SELECT * FROM dbo.[Fact_FHV_Trips] WHERE PickupDateKey >= 20251201 AND PickupDateKey < 20260101
Fact FHV Trips Other Dates	Molap; Operational	SELECT * FROM dbo.[Fact_FHV_Trips] WHERE NOT ((PickupDateKey >= 20250101 AND PickupDateKey < 20250201) OR (PickupDateKey >= 20250201 AND PickupDateKey < 20250301) OR (PickupDateKey >= 20250301 AND PickupDateKey < 20250401) OR (PickupDateKey >= 20250401 AND PickupDateKey < 20250501) OR (PickupDateKey >= 20250501 AND PickupDateKey < 20250601) OR (PickupDateKey >= 20250601 AND PickupDateKey < 20250701) OR (PickupDateKey >= 20250701 AND PickupDateKey < 20250801) OR (PickupDateKey >= 20250801 AND PickupDateKey < 20250901) OR (PickupDateKey >= 20250901 AND PickupDateKey < 20251001) OR (PickupDateKey >= 20251001 AND PickupDateKey < 20251101) OR (PickupDateKey >= 20251101 AND PickupDateKey < 20251201) OR (PickupDateKey >= 20251201 AND PickupDateKey < 20260101))

Bảng 4.32 Partition và phạm vi nguồn HVFHV_Trips

Partition	Lưu trữ và design	Nguồn hoặc QueryDefinition
Fact HVFHV Trips 202501	Molap; Operational	SELECT * FROM dbo.[Fact_HVFHV_Trips] WHERE PickupDateKey >= 20250101 AND PickupDateKey < 20250201

Partition	Lưu trữ và design	Nguồn hoặc QueryDefinition
Fact HVFHV Trips 202502	Molap; Operational	SELECT * FROM dbo.[Fact_HVFHV_Trips] WHERE PickupDateKey >= 20250201 AND PickupDateKey < 20250301
Fact HVFHV Trips 202503	Molap; Operational	SELECT * FROM dbo.[Fact_HVFHV_Trips] WHERE PickupDateKey >= 20250301 AND PickupDateKey < 20250401
Fact HVFHV Trips 202504	Molap; Operational	SELECT * FROM dbo.[Fact_HVFHV_Trips] WHERE PickupDateKey >= 20250401 AND PickupDateKey < 20250501
Fact HVFHV Trips 202505	Molap; Operational	SELECT * FROM dbo.[Fact_HVFHV_Trips] WHERE PickupDateKey >= 20250501 AND PickupDateKey < 20250601
Fact HVFHV Trips 202506	Molap; Operational	SELECT * FROM dbo.[Fact_HVFHV_Trips] WHERE PickupDateKey >= 20250601 AND PickupDateKey < 20250701
Fact HVFHV Trips 202507	Molap; Operational	SELECT * FROM dbo.[Fact_HVFHV_Trips] WHERE PickupDateKey >= 20250701 AND PickupDateKey < 20250801
Fact HVFHV Trips 202508	Molap; Operational	SELECT * FROM dbo.[Fact_HVFHV_Trips] WHERE PickupDateKey >= 20250801 AND PickupDateKey < 20250901
Fact HVFHV Trips 202509	Molap; Operational	SELECT * FROM dbo.[Fact_HVFHV_Trips] WHERE PickupDateKey >= 20250901 AND PickupDateKey < 20251001
Fact HVFHV Trips 202510	Molap; Operational	SELECT * FROM dbo.[Fact_HVFHV_Trips] WHERE PickupDateKey >= 20251001 AND PickupDateKey < 20251101
Fact HVFHV Trips 202511	Molap; Operational	SELECT * FROM dbo.[Fact_HVFHV_Trips] WHERE PickupDateKey >= 20251101 AND PickupDateKey < 20251201

Partition	Lưu trữ và design	Nguồn hoặc QueryDefinition
Fact HVFHV Trips 202512	Molap; Operational	SELECT * FROM dbo.[Fact_HVFHV_Trips] WHERE PickupDateKey >= 20251201 AND PickupDateKey < 20260101
Fact HVFHV Trips Other Dates	Molap; Operational	SELECT * FROM dbo.[Fact_HVFHV_Trips] WHERE NOT ((PickupDateKey >= 20250101 AND PickupDateKey < 20250201) OR (PickupDateKey >= 20250201 AND PickupDateKey < 20250301) OR (PickupDateKey >= 20250301 AND PickupDateKey < 20250401) OR (PickupDateKey >= 20250401 AND PickupDateKey < 20250501) OR (PickupDateKey >= 20250501 AND PickupDateKey < 20250601) OR (PickupDateKey >= 20250601 AND PickupDateKey < 20250701) OR (PickupDateKey >= 20250701 AND PickupDateKey < 20250801) OR (PickupDateKey >= 20250801 AND PickupDateKey < 20250901) OR (PickupDateKey >= 20250901 AND PickupDateKey < 20251001) OR (PickupDateKey >= 20251001 AND PickupDateKey < 20251101) OR (PickupDateKey >= 20251101 AND PickupDateKey < 20251201) OR (PickupDateKey >= 20251201 AND PickupDateKey < 20260101))

Bảng 4.33 Partition và phạm vi nguồn YellowGreen_Trips

Partition	Lưu trữ và design	Nguồn hoặc QueryDefinition
Fact Yellow Green Trips 200812	Molap; Operational	SELECT * FROM dbo.[Fact_YellowGreen_Trips] WHERE PickupDateKey >= 20081201 AND PickupDateKey < 20090101

Partition	Lưu trữ và design	Nguồn hoặc QueryDefinition
Fact Yellow Green Trips 202412	Molap; Operational	SELECT * FROM dbo.[Fact_YellowGreen_Trips] WHERE PickupDateKey >= 20241201 AND PickupDateKey < 20250101
Fact Yellow Green Trips 202501	Molap; Operational	SELECT * FROM dbo.[Fact_YellowGreen_Trips] WHERE PickupDateKey >= 20250101 AND PickupDateKey < 20250201
Fact Yellow Green Trips 202502	Molap; Operational	SELECT * FROM dbo.[Fact_YellowGreen_Trips] WHERE PickupDateKey >= 20250201 AND PickupDateKey < 20250301
Fact Yellow Green Trips 202503	Molap; Operational	SELECT * FROM dbo.[Fact_YellowGreen_Trips] WHERE PickupDateKey >= 20250301 AND PickupDateKey < 20250401
Fact Yellow Green Trips 202504	Molap; Operational	SELECT * FROM dbo.[Fact_YellowGreen_Trips] WHERE PickupDateKey >= 20250401 AND PickupDateKey < 20250501
Fact Yellow Green Trips 202505	Molap; Operational	SELECT * FROM dbo.[Fact_YellowGreen_Trips] WHERE PickupDateKey >= 20250501 AND PickupDateKey < 20250601
Fact Yellow Green Trips 202506	Molap; Operational	SELECT * FROM dbo.[Fact_YellowGreen_Trips] WHERE PickupDateKey >= 20250601 AND PickupDateKey < 20250701
Fact Yellow Green Trips 202507	Molap; Operational	SELECT * FROM dbo.[Fact_YellowGreen_Trips] WHERE PickupDateKey >= 20250701 AND PickupDateKey < 20250801

Partition	Lưu trữ và design	Nguồn hoặc QueryDefinition
Fact Yellow Green Trips 202508	Molap; Operational	SELECT * FROM dbo.[Fact_YellowGreen_Trips] WHERE PickupDateKey >= 20250801 AND PickupDateKey < 20250901
Fact Yellow Green Trips 202509	Molap; Operational	SELECT * FROM dbo.[Fact_YellowGreen_Trips] WHERE PickupDateKey >= 20250901 AND PickupDateKey < 20251001
Fact Yellow Green Trips 202510	Molap; Operational	SELECT * FROM dbo.[Fact_YellowGreen_Trips] WHERE PickupDateKey >= 20251001 AND PickupDateKey < 20251101
Fact Yellow Green Trips 202511	Molap; Operational	SELECT * FROM dbo.[Fact_YellowGreen_Trips] WHERE PickupDateKey >= 20251101 AND PickupDateKey < 20251201
Fact Yellow Green Trips 202512	Molap; Operational	SELECT * FROM dbo.[Fact_YellowGreen_Trips] WHERE PickupDateKey >= 20251201 AND PickupDateKey < 20260101
Fact Yellow Green Trips 202601	Molap; Operational	SELECT * FROM dbo.[Fact_YellowGreen_Trips] WHERE PickupDateKey >= 20260101 AND PickupDateKey < 20260201

Partition	Lưu trữ và design	Nguồn hoặc QueryDefinition
Fact Yellow Green Trips Other Dates	Molap; Operational	SELECT * FROM dbo.[Fact_YellowGreen_Trips] WHERE NOT ((PickupDateKey >= 20081201 AND PickupDateKey < 20090101) OR (PickupDateKey >= 20241201 AND PickupDateKey < 20250101) OR (PickupDateKey >= 20250101 AND PickupDateKey < 20250201) OR (PickupDateKey >= 20250201 AND PickupDateKey < 20250301) OR (PickupDateKey >= 20250301 AND PickupDateKey < 20250401) OR (PickupDateKey >= 20250401 AND PickupDateKey < 20250501) OR (PickupDateKey >= 20250501 AND PickupDateKey < 20250601) OR (PickupDateKey >= 20250601 AND PickupDateKey < 20250701) OR (PickupDateKey >= 20250701 AND PickupDateKey < 20250801) OR (PickupDateKey >= 20250801 AND PickupDateKey < 20250901) OR (PickupDateKey >= 20250901 AND PickupDateKey < 20251001) OR (PickupDateKey >= 20251001 AND PickupDateKey < 20251101) OR (PickupDateKey >= 20251101 AND PickupDateKey < 20251201) OR (PickupDateKey >= 20251201 AND PickupDateKey < 20260101) OR (PickupDateKey >= 20260101 AND PickupDateKey < 20260201))

Bảng 4.34 Partition và phạm vi nguồn Zone_Traffic

Partition	Lưu trữ và design	Nguồn hoặc QueryDefinition
Zone Traffic Snapshot All	Molap; Operational	TableBinding dbo_Fact_Zone_Traffic_Snapshot

4.8 Triển khai và xử lý mô hình

Triển khai lần đầu tạo database mô hình; cập nhật mô hình thay metadata có thể làm mất trạng thái process. Hồ sơ dự án ghi build Visual Studio gặp Invalid license data, còn AMO đã kiểm tra, serialize XMLA và triển khai mô hình. Vì vậy không mô tả một lần build Visual Studio thành công nếu bằng chứng chỉ xác nhận đường AMO và XMLA.

Trong pipeline, runner xác nhận đúng Data Source, SQL database và đủ sáu cube, sau đó dùng XMLA Batch Transaction true để ProcessFull database. Kết quả phải không có XmlaError và database trở lại Processed, rồi mới chạy đối soát SQL MDX. DWH không rollback cùng SSAS nếu bước này lỗi; có thể resume bước cuối khi các điều kiện trước vẫn đúng.

4.9 Truy vấn MDX và đối soát với kho dữ liệu

Các truy vấn MDX kiểm tra số chuyến, tổng measure, lát cắt thời gian, địa lý và các công thức nghiệp vụ; SQL tính kết quả tương ứng từ DWH. Chọn Scope Full khi cube đã process toàn kho, không sử dụng test tháng như một bộ lọc tự động cho cube đầy đủ. Trong lúc đối soát cần giữ nguồn ổn định để hai tầng không ở hai thời điểm dữ liệu khác nhau.

Bằng chứng ngày 31 tháng 10 ghi sáu Browse MDX đạt và 102 phép đối soát SQL MDX không thất bại, cùng 15 dimension và 45 partition Processed tại thời điểm chụp. Tài liệu khác có trạng thái Unprocessed sau reset ở thời điểm khác, nên trước demo phải kiểm tra lại nguồn hiện tại. Các ví dụ truy vấn, kết quả và giới hạn kiểm tra được trình bày ở Phụ lục B.

CHƯƠNG 5 XÂY DỰNG DASHBOARD POWER BI

5.1 Mục tiêu và yêu cầu của bộ báo cáo

Bộ báo cáo chuyển các câu hỏi nghiệp vụ thành card, biểu đồ xu hướng, phân bố và bảng chi tiết theo từng cube. Yêu cầu đầu tiên là tên chỉ tiêu và đơn vị rõ ràng; tiếp theo là chọn đúng role ngày hoặc vùng, giữ Unknown và cho phép nhìn phạm vi lọc. Mục tiêu không phải đặt tất cả measure vào một trang mà là giúp trả lời từng nhóm câu hỏi có dữ liệu phù hợp.

Dự án có sáu project PBIP với chín trang theo danh mục source. Một số nhóm nghiệp vụ dùng hai trang, chẳng hạn YellowGreen và HVFHV. Các câu hỏi phân tích tám nhóm trong mục 5.4 được ánh xạ vào những trang hiện có, không tạo thêm một trang chỉ vì mục lục có tám tiêu đề.

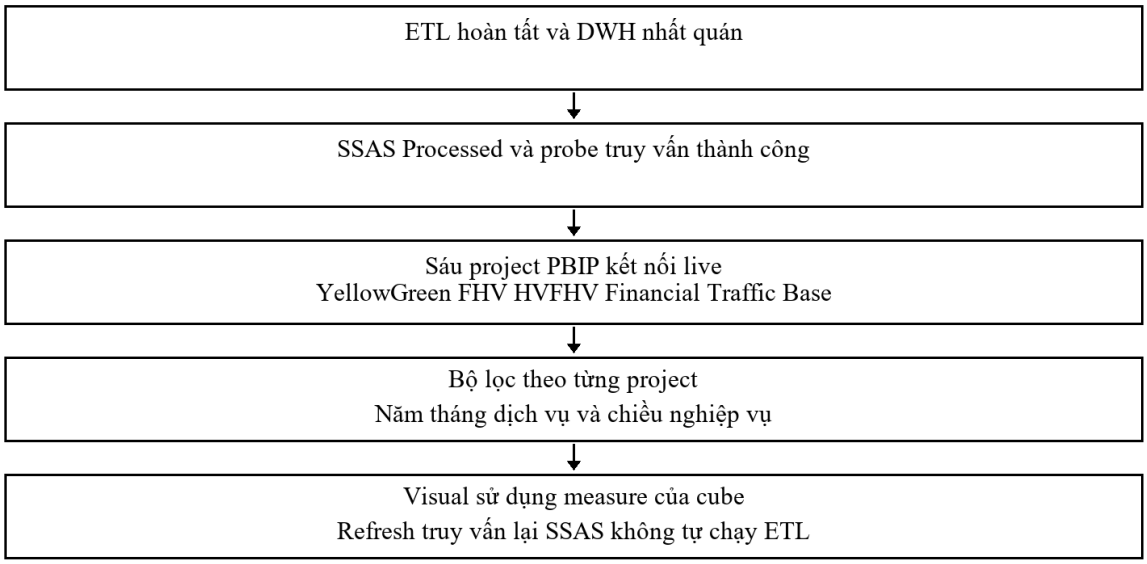
Bảng 5.1 Project Power BI và trang nghiệp vụ

Project	**Các trang**	**Nguồn cube**
YellowGreen Trips	Taxi truyền thống; Hành vi & tiền tip	YellowGreen_Trips
HVFHV Trips	Xe công nghệ; Phí & thu nhập	HVFHV_Trips
FHV Trips	FHV truyền thống	FHV_Trips
Daily Financial	Tài chính theo ngày	Daily_Financial
Zone Traffic	Nhu cầu theo khu vực; Chi tiết không gian & giờ	Zone_Traffic
Base Performance	Hiệu suất điều phối	Base_Performance

5.2 Kết nối Power BI với SSAS

Power BI sử dụng Live Connection đến SSAS Multidimensional, database NYCTaxi_SSAS và cube theo project. Model reference dùng connectionType analysisServicesDatabaseLive, Windows SSPI và tên cube tương ứng. Không chọn Import hoặc chuyển sang local model nếu muốn giữ đúng kiến trúc hiện có.

Tệp .pbip cần đi cùng các thư mục .Report và .SemanticModel. Refresh Power BI yêu cầu kết quả mới từ cube, không chạy ETL hoặc process SSAS. Menu mở nhanh kiểm tra trạng thái và probe chỉ đọc trước khi coi nguồn sẵn sàng. Bộ báo cáo chưa được mô tả như đã publish lên Power BI Service; triển khai Service với live SSAS cần gateway và danh tính phù hợp.



Hình 5.1 Khai thác Power BI qua kết nối live

5.3 Thiết kế bộ lọc bố cục và tương tác

Các trang sử dụng bộ lọc năm, tháng và dịch vụ; tùy nghiệp vụ có thêm Borough, provider, payment type hoặc base. Chọn năm trước tháng để tránh so sánh tháng cùng số ở nhiều năm ngoài ý muốn. Bộ lọc của sáu project độc lập, không tự đồng bộ giữa các cửa sổ Power BI.

Card dùng cho tổng hoặc chỉ tiêu chính, biểu đồ đường thể hiện diễn biến, cột so sánh nhóm, ma trận thể hiện kết hợp chiều và bảng hỗ trợ truy vết chi tiết. Field binding phải dùng member và grouping key theo metadata CSDL mà SSAS công bố. Tiêu đề cần ghi rõ USD, mile, giây hay phút; không dựa riêng vào màu hoặc vị trí để truyền đạt ý nghĩa.

Tương tác chọn điểm và slicer phải được kiểm tra bằng Desktop, vì query hợp lệ không chứng minh cross filter hoặc bố cục đã đúng. Khi nguồn rỗng hay Unknown, visual phải phản ánh trạng thái dữ liệu thay vì được lọc để làm kết

quả đẹp hơn. Các cấu hình title, filter, sort, query role và kích thước hiện có được liệt kê theo từng trang dưới đây.

5.4 Xây dựng các trang phân tích

5.4.1 Phân tích hoạt động taxi Yellow và Green

Trang Taxi truyền thống lấy cube YellowGreen Trips. Các card về số chuyến, tổng tiền và các chỉ tiêu trung bình đi cùng phân bố thời gian, địa lý và dịch vụ. Chọn Service Type để tách Yellow và Green; chọn đúng năm tháng có dữ liệu đã process trước khi so sánh giờ hoặc quận.

Quy trình đọc bắt đầu từ tổng số chuyến trong phạm vi lọc, sau đó xem phân bố theo thời gian và địa lý, cuối cùng kiểm tra các nhóm khác biệt. Nếu kết quả chủ yếu thuộc đầu tháng do giới hạn N dòng đầu, phần nhận xét chỉ mô tả phạm vi đó. Tổng tiền khách và cước cơ bản là hai measure riêng; không đặt cùng tên doanh thu nếu chưa nêu định nghĩa.

Trang Taxi truyền thống; canvas 1440 × 960.

Bảng 5.2 Visual và binding trang Taxi truyền thống

Tiêu đề hoặc ID	Loại visual	Field và measure	Vị trí và filter
Tiền khách trả · USD	card	Fact Yellow Green Trips.Total Amount	x=374; y=210; width=342; height=110; z=9; tabOrder=9
Giờ đón khách cao điểm	clusteredColumnChart	Pickup Time.Hour Number; Fact Yellow Green Trips.Trip Count - Fact Yellow Green Trips	x=728; y=338; width=688; height=256; z=13; tabOrder=13
Số chuyến theo quận đón	clusteredBarChart	Pickup Location.Borough; Fact Yellow Green Trips.Trip Count - Fact Yellow Green Trips	x=24; y=610; width=688; height=256; z=14; tabOrder=14

Tiêu đề hoặc ID	Loại visual	Field và measure	Vị trí và filter
Dịch vụ	slicer	Dim Service Type.Service Code	x=488; y=110; width=250; height=82; z=5; tabOrder=5
Tháng	slicer	Pickup Date.Month Name	x=256; y=110; width=220; height=82; z=4; tabOrder=4
3803a14dd2c35e0a47f5	textbox	Thành phần tiêu đề hoặc bố cục, không có field dữ liệu.	x=24; y=66; width=1392; height=32; z=2; tabOrder=2
39a864d1b00957ec93e3	textbox	Thành phần tiêu đề hoặc bố cục, không có field dữ liệu.	x=24; y=18; width=1392; height=44; z=1; tabOrder=1
Năm	slicer	Pickup Date.Year Number	x=24; y=110; width=220; height=82; z=3; tabOrder=3
Quận đón	slicer	Pickup Location.Borough	x=750; y=110; width=350; height=82; z=6; tabOrder=6
Số chuyến	card	Fact Yellow Green Trips.Trip Count - Fact Yellow Green Trips	x=24; y=210; width=342; height=110; z=8; tabOrder=8
Tiền khách trả theo dịch vụ	clusteredColumnChart	Dim Service Type.Service Code; Fact Yellow Green Trips.Total Amount	x=728; y=610; width=688; height=256; z=15; tabOrder=15

Tiêu đề hoặc ID	Loại visual	Field và measure	Vị trí và filter
987198d94d5ae28f945a	textbox	Thành phần tiêu đề hoặc bố cục, không có field dữ liệu.	x=24; y=884; width=1392; height=58; z=7; tabOrder=7
Cước TB/chuyến · USD	card	Measures.YellowGreen Average Fare Per Trip	x=724; y=210; width=342; height=110; z=10; tabOrder=10
Tip / cước hợp lệ	card	Measures.YellowGreen Tip Percentage	x=1074; y=210; width=342; height=110; z=11; tabOrder=11
Xu hướng số chuyến theo ngày	lineChart	Pickup Date.Full Date; Fact Yellow Green Trips.Trip Count - Fact Yellow Green Trips	x=24; y=338; width=688; height=256; z=12; tabOrder=12

5.4.2 Phân tích thanh toán và tiền tip

Trang Hành vi và tiền tip dùng các chiều thanh toán, thời gian, cuối tuần và địa lý của YellowGreen. Tỷ lệ tip sử dụng calculated measure của cube để giữ cùng tập tử mẫu. Không lấy trung bình các tỷ lệ của từng payment type hoặc từng quận để tạo một tỷ lệ toàn nhóm.

Dữ liệu tip tiền mặt không đầy đủ nên không kết luận khách tiền mặt không tip từ giá trị trường tip bằng 0 hoặc thiếu. Bộ lọc payment type giúp mô tả phần ghi nhận trong hệ thống, không tự tạo một nghiên cứu hành vi toàn bộ hành khách. Nhận xét cần nêu loại thanh toán, thời gian và dịch vụ đã lọc.

Trang Hành vi & tiền tip; canvas 1440 × 960.

Bảng 5.3 Visual và binding trang Hành vi & tiền tip

Tiêu đề hoặc ID	Loại visual	Field và measure	Vị trí và filter
Dịch vụ	slicer	Dim Service Type.Service Code	x=488; y=110; width=250; height=82; z=20; tabOrder=20
0c59e4b88c03ca291b6e	textbox	Thành phần tiêu đề hoặc bố cục, không có field dữ liệu.	x=24; y=66; width=1392; height=32; z=17; tabOrder=17
4b0c41f568cc1aeeb52b	textbox	Thành phần tiêu đề hoặc bố cục, không có field dữ liệu.	x=24; y=18; width=1392; height=44; z=16; tabOrder=16
Thanh toán	slicer	Dim Payment Type.Payment Type Key	x=750; y=110; width=350; height=82; z=21; tabOrder=21
Tỷ lệ tip: ngày thường / cuối tuần	clusteredColumnChart	Pickup Date.Is Weekend; Measures.YellowGreen Tip Percentage	x=24; y=610; width=688; height=256; z=29; tabOrder=29
Tỷ lệ tip theo khu vực đón · xem thêm số chuyến	clusteredBarChart	Pickup Location.Zone Name; Measures.YellowGreen Tip Percentage	x=24; y=338; width=688; height=256; z=27; tabOrder=27
Tiền tip ghi nhận · USD	card	Fact Yellow Green Trips.Tip Amount - Fact Yellow Green Trips	x=24; y=210; width=342; height=110; z=23; tabOrder=23
Năm	slicer	Pickup Date.Year Number	x=24; y=110; width=220; height=82; z=18; tabOrder=18

Tiêu đề hoặc ID	Loại visual	Field và measure	Vị trí và filter
Tỷ lệ tip theo giờ đón	lineChart	Pickup Time.Hour Number; Measures.YellowGreen Tip Percentage	x=728; y=610; width=688; height=256; z=30; tabOrder=30
Tip / cước hợp lệ	card	Measures.YellowGreen Tip Percentage	x=374; y=210; width=342; height=110; z=24; tabOrder=24
Thời lượng TB · phút	card	Measures.YellowGreen Average Trip Duration Minutes	x=1074; y=210; width=342; height=110; z=26; tabOrder=26
Tỷ lệ tip theo hình thức thanh toán	clusteredColumnChart	Dim Payment Type.Payment Type Key; Measures.YellowGreen Tip Percentage	x=728; y=338; width=688; height=256; z=28; tabOrder=28
Tháng	slicer	Pickup Date.Month Name	x=256; y=110; width=220; height=82; z=19; tabOrder=19
f23e8d7f14bc63aa0da9	textbox	Thành phần tiêu đề hoặc bố cục, không có field dữ liệu.	x=24; y=884; width=1392; height=58; z=22; tabOrder=22
Tip TB/chuyến hợp lệ · USD	card	Measures.YellowGreen Average Tip Per Trip	x=724; y=210; width=342; height=110; z=25; tabOrder=25

5.4.3 Phân tích hoạt động xe công nghệ

Trang Xe công nghệ dùng HVFHV Trips, tập trung lượng chuyến, quãng đường, thời lượng, chờ và provider. Các phân tích theo Borough cần dùng đúng Pickup Location cho câu hỏi nhu cầu đón. Khi so sánh provider, bộ lọc thời gian và độ bao phủ nguồn phải giống nhau để tránh khác biệt do phạm vi nạp.

Chỉ tiêu Average Waiting Minutes mô tả từ request đến pickup theo định nghĩa loader và tập hợp lệ. Không đổi tên thành thời gian xe đến nơi khi chưa dùng đúng mốc on scene. Chênh lệch giữa hai provider hoặc quận trong tập mẫu chỉ là quan sát mô tả, chưa loại trừ khác biệt về khu vực, thời điểm hay cơ cấu chuyến.

Trang Xe công nghệ; canvas 1440 × 960.

Bảng 5.4 Visual và binding trang Xe công nghệ

Tiêu đề hoặc ID	Loại visual	Field và measure	Vị trí và filter
Số chuyến	card	Fact HVFHV Trips.Trip Count - Fact HVFHV Trips	x=24; y=210; width=342; height=110; z=8; tabOrder=8
Dịch vụ	slicer	Dim Service Type.Service Code	x=488; y=110; width=250; height=82; z=5; tabOrder=5
Số chuyến theo nhà cung cấp	clusteredBarChart	Dim HVFHS Provider.License Number; Fact HVFHV Trips.Trip Count - Fact HVFHV Trips	x=728; y=610; width=688; height=256; z=15; tabOrder=15
Nhà cung cấp	slicer	Dim HVFHS Provider.License Number	x=750; y=110; width=350; height=82; z=6; tabOrder=6
7100d0e2c05ca870c1a7	textbox	Thành phần tiêu đề hoặc bố cục, không có field dữ liệu.	x=24; y=66; width=1392; height=32; z=2; tabOrder=2
8333bbe13c7cb3ed7b2d	textbox	Thành phần tiêu đề hoặc bố cục, không có field dữ liệu.	x=24; y=18; width=1392; height=44; z=1; tabOrder=1

Tiêu đề hoặc ID	Loại visual	Field và measure	Vị trí và filter
Tháng	slicer	Pickup Date.Month Name	x=256; y=110; width=220; height=82; z=4; tabOrder=4
Chờ TB · phút	card	Measures.HVFHV Average Waiting Minutes	x=724; y=210; width=342; height=110; z=10; tabOrder=10
Thời gian chờ theo quận đón · phút	clusteredBarChart	Pickup Location.Borough; Measures.HVFHV Average Waiting Minutes	x=24; y=610; width=688; height=256; z=14; tabOrder=14
Xu hướng số chuyến theo ngày	lineChart	Pickup Date.Full Date; Fact HVFHV Trips.Trip Count - Fact HVFHV Trips	x=24; y=338; width=688; height=256; z=12; tabOrder=12
Cước TB/chuyến · USD	card	Measures.HVFHV Average Fare Per Trip	x=1074; y=210; width=342; height=110; z=11; tabOrder=11
d4c798cb25b68713bc18	textbox	Thành phần tiêu đề hoặc bố cục, không có field dữ liệu.	x=24; y=884; width=1392; height=58; z=7; tabOrder=7
Tiền tài xế · USD	card	Fact HVFHV Trips.Driver Pay Amount	x=374; y=210; width=342; height=110; z=9; tabOrder=9
Số chuyến theo giờ	clusteredColumnChart	Pickup Time.Hour Number; Fact HVFHV Trips.Trip Count - Fact HVFHV Trips	x=728; y=338; width=688; height=256; z=13; tabOrder=13

Tiêu đề hoặc ID	Loại visual	Field và measure	Vị trí và filter
Năm	slicer	Pickup Date.Year Number	x=24; y=110; width=220; height=82; z=3; tabOrder=3

5.4.4 Phân tích phí và thu nhập

Trang Phí và thu nhập tách cước cơ bản, tổng tiền khách, thuế phí, tip và tiền tài xế. Việc đặt các series cạnh nhau giúp thấy cơ cấu và biến động nhưng không cho phép gọi chênh lệch là lợi nhuận ròng. Driver pay theo nguồn không đồng nghĩa số tiền tài xế mang về sau mọi chi phí vận hành cá nhân.

Phân tích phí theo tháng hoặc dịch vụ phải tính đến thời kỳ xuất hiện trường CBD và phạm vi file. Một biểu đồ có phí tăng và số chuyến giảm không đủ để kết luận quan hệ nhân quả. Muốn đánh giá tác động chính sách cần dữ liệu đầy đủ, thiết kế trước sau và các biến kiểm soát bổ sung.

Trang Phí & thu nhập; canvas 1440 × 960.

Bảng 5.5 Visual và binding trang Phí & thu nhập

Tiêu đề hoặc ID	Loại visual	Field và measure	Vị trí và filter
Số chuyến theo khung giờ	clusteredColumnChart	Pickup Time.Time Band; Fact HVFHV Trips.Trip Count - Fact HVFHV Trips	x=728; y=610; width=688; height=256; z=30; tabOrder=30
0d45e085683239e941fc	textbox	Thành phần tiêu đề hoặc bố cục, không có field dữ liệu.	x=24; y=66; width=1392; height=32; z=17; tabOrder=17
Phí ùn tắc · USD	card	Fact HVFHV Trips.Congestion Surcharge	x=24; y=210; width=342; height=110; z=23; tabOrder=23

Tiêu đề hoặc ID	Loại visual	Field và measure	Vị trí và filter
Phí ùn tắc và CBD theo ngày · USD	lineChart	Pickup Date.Full Date; Fact HVFHV Trips.Congestion Surcharge; Fact HVFHV Trips.CBD Congestion Fee	x=24; y=338; width=688; height=256; z=27; tabOrder=27
Phí CBD · USD	card	Fact HVFHV Trips.CBD Congestion Fee	x=374; y=210; width=342; height=110; z=24; tabOrder=24
Tháng	slicer	Pickup Date.Month Name	x=256; y=110; width=220; height=82; z=19; tabOrder=19
Dịch vụ	slicer	Dim Service Type.Service Code	x=488; y=110; width=250; height=82; z=20; tabOrder=20
7104f4ca8a515bf209d6	textbox	Thành phần tiêu đề hoặc bố cục, không có field dữ liệu.	x=24; y=884; width=1392; height=58; z=22; tabOrder=22
Tiền khách trả · USD	card	Fact HVFHV Trips.Gross Passenger Amount	x=724; y=210; width=342; height=110; z=25; tabOrder=25
Phí ùn tắc theo quận đón · USD	clusteredBarChart	Pickup Location.Borough; Fact HVFHV Trips.Congestion Surcharge	x=24; y=610; width=688; height=256; z=29; tabOrder=29
Tiền tài xế · USD	card	Fact HVFHV Trips.Driver Pay Amount	x=1074; y=210; width=342; height=110; z=26; tabOrder=26

Tiêu đề hoặc ID	Loại visual	Field và measure	Vị trí và filter
Quận đón	slicer	Pickup Location.Borough	x=750; y=110; width=350; height=82; z=21; tabOrder=21
Tiền khách trả và tài xế theo ngày · USD	lineChart	Pickup Date.Full Date; Fact HVFHV Trips.Gross Passenger Amount; Fact HVFHV Trips.Driver Pay Amount	x=728; y=338; width=688; height=256; z=28; tabOrder=28
Năm	slicer	Pickup Date.Year Number	x=24; y=110; width=220; height=82; z=18; tabOrder=18
dd732640730479 75c48f	textbox	Thành phần tiêu đề hoặc bố cục, không có field dữ liệu.	x=24; y=18; width=1392; height=44; z=16; tabOrder=16

5.4.5 Phân tích hoạt động FHV

Trang FHV truyền thống trình bày số chuyến, thời lượng và phân bố theo thời gian, quận hoặc base. Số liệu tài chính không nằm trong phạm vi trang vì nguồn FHV hiện dùng không có các trường đó. Tổng thời lượng gốc có thể dùng giây trong khi trung bình dùng phút, nên mỗi card phải theo tên và định dạng cụ thể. Một base có nhiều chuyến chưa chắc có chất lượng phục vụ cao hơn nếu chưa có tiêu chí về chờ, địa bàn hoặc loại chuyến. Vị trí Unknown làm hạn chế phân tích địa lý, cần giữ trong bảng hoặc tổng để người đọc thấy độ bao phủ. Cờ FHV cần được đọc theo phiên bản loader đã process, nhất là sau bản sửa SourceSRFlag.

Trang FHV truyền thống; canvas 1440 × 960.

Bảng 5.6 Visual và binding trang FHV truyền thống

Tiêu đề hoặc ID	Loại visual	Field và measure	Vị trí và filter
Số chuyến theo base điều phối	clusteredBarChart	Dispatch Base.Base Key; Fact FHV Trips.Trip Count	x=728; y=610; width=688; height=256; z=15; tabOrder=15
Dịch vụ	slicer	Dim Service Type.Service Code	x=488; y=110; width=250; height=82; z=5; tabOrder=5
Số chuyến theo quận đón, gồm Unknown	clusteredBarChart	Pickup Location.Borough; Fact FHV Trips.Trip Count	x=24; y=610; width=688; height=256; z=14; tabOrder=14
Quận đón	slicer	Pickup Location.Borough	x=750; y=110; width=350; height=82; z=6; tabOrder=6
Tháng	slicer	Pickup Date.Month Name	x=256; y=110; width=220; height=82; z=4; tabOrder=4
54141fbe93f01e82c2bf	textbox	Thành phần tiêu đề hoặc bố cục, không có field dữ liệu.	x=24; y=18; width=1392; height=44; z=1; tabOrder=1
Số chuyến	card	Fact FHV Trips.Trip Count	x=24; y=210; width=342; height=110; z=8; tabOrder=8
67d49076f5881790b096	textbox	Thành phần tiêu đề hoặc bố cục, không có field dữ liệu.	x=24; y=884; width=1392; height=58; z=7; tabOrder=7

Tiêu đề hoặc ID	Loại visual	Field và measure	Vị trí và filter
Năm	slicer	Pickup Date.Year Number	x=24; y=110; width=220; height=82; z=3; tabOrder=3
849391d68a402589e096	textbox	Thành phần tiêu đề hoặc bố cục, không có field dữ liệu.	x=24; y=66; width=1392; height=32; z=2; tabOrder=2
Chuyến có thời lượng	card	Fact FHV Trips.Fact FHV Trips TripDurationSecondsValidTrips	x=1074; y=210; width=342; height=110; z=11; tabOrder=11
Xu hướng số chuyến theo ngày	lineChart	Pickup Date.Full Date; Fact FHV Trips.Trip Count	x=24; y=338; width=688; height=256; z=12; tabOrder=12
Số chuyến theo giờ đón	clusteredColumnChart	Pickup Time.Hour Number; Fact FHV Trips.Trip Count	x=728; y=338; width=688; height=256; z=13; tabOrder=13
Thời lượng TB · phút	card	Measures.FHV Average Trip Duration Minutes	x=374; y=210; width=342; height=110; z=9; tabOrder=9
Tổng thời lượng · giây	card	Fact FHV Trips.Trip Duration Seconds	x=724; y=210; width=342; height=110; z=10; tabOrder=10

5.4.6 Phân tích tài chính theo ngày

Trang Tài chính theo ngày sử dụng Daily Financial để theo dõi các tổng ở grain ngày và dịch vụ. Chọn cùng năm tháng và Service Type trước khi đối chiếu tổng tiền khách, tip, thuế phí và tiền tài xế. FHV không thuộc nguồn tài chính của snapshot nên không suy ra giá trị thiếu là 0 USD.

Trang này hỗ trợ xem quy mô và diễn biến tài chính ghi nhận. Nếu cần giải thích sự thay đổi do số chuyến hay giá trị mỗi chuyến, phải kết hợp số chuyến cùng phạm vi và công thức trung bình hợp lệ, không chỉ so độ cao hai đường. Các tổng snapshot được đối chiếu với fact nguồn tại bước aggregate và SQL MDX.

Trang Tài chính theo ngày; canvas 1440 × 960.

Bảng 5.7 Visual và binding trang Tài chính theo ngày

Tiêu đề hoặc ID	Loại visual	Field và measure	Vị trí và filter
Tiền khách trả theo ngày · USD	lineChart	Snapshot Date.Full Date; Daily Financial Snapshot.Daily Financial Snapshot TotalPassengerCharges; Dim Service Type.Service Code	x=24; y=338; width=688; height=256; z=11; tabOrder=11
25676c49db8cfe7a1dfe	textbox	Thành phần tiêu đề hoặc bố cục, không có field dữ liệu.	x=24; y=18; width=1392; height=44; z=1; tabOrder=1
Tiền tip và thuế/phí theo dịch vụ · USD	clusteredColumnChart	Dim Service Type.Service Code; Daily Financial Snapshot.Daily Financial Snapshot TotalTips; Daily Financial Snapshot.Daily Financial Snapshot TotalTaxesAndFees	x=24; y=610; width=688; height=256; z=13; tabOrder=13
Tiền tip · USD	card	Daily Financial Snapshot.Daily Financial Snapshot TotalTips	x=724; y=210; width=342; height=110; z=9; tabOrder=9
Dịch vụ	slicer	Dim Service Type.Service Code	x=488; y=110; width=250; height=82; z=5; tabOrder=5
a2eb0824d628ecee3ab8	textbox	Thành phần tiêu đề hoặc bố cục, không có field dữ liệu.	x=24; y=884; width=1392; height=58; z=6; tabOrder=6

Tiêu đề hoặc ID	Loại visual	Field và measure	Vị trí và filter
Tháng	slicer	Snapshot Date.Month Name	x=256; y=110; width=220; height=82; z=4; tabOrder=4
Tiền tài xế · USD	card	Daily Financial Snapshot.Daily Financial Snapshot TotalDriverPay	x=1074; y=210; width=342; height=110; z=10; tabOrder=10
cb18985fb9a7ece3b62d	textbox	Thành phần tiêu đề hoặc bố cục, không có field dữ liệu.	x=24; y=66; width=1392; height=32; z=2; tabOrder=2
Số chuyến có tài chính	card	Daily Financial Snapshot.Daily Financial Snapshot TotalTrips	x=24; y=210; width=342; height=110; z=7; tabOrder=7
Số chuyến theo dịch vụ	clusteredColumnChart	Dim Service Type.Service Code; Daily Financial Snapshot.Daily Financial Snapshot TotalTrips	x=728; y=338; width=688; height=256; z=12; tabOrder=12
Năm	slicer	Snapshot Date.Year Number	x=24; y=110; width=220; height=82; z=3; tabOrder=3
Tiền tài xế theo ngày · USD	lineChart	Snapshot Date.Full Date; Daily Financial Snapshot.Daily Financial Snapshot TotalDriverPay	x=728; y=610; width=688; height=256; z=14; tabOrder=14
Tiền khách trả · USD	card	Daily Financial Snapshot.Daily Financial Snapshot TotalPassengerCharges	x=374; y=210; width=342; height=110; z=8; tabOrder=8

5.4.7 Phân tích nhu cầu theo khu vực và khung giờ

Zone Traffic có trang Nhu cầu theo khu vực và trang chi tiết không gian giờ. TotalPickups và TotalDropoffs hỗ trợ so phân bố sự kiện theo vùng, dịch vụ, giờ và ngày. Ma trận Borough với giờ giúp xem vị trí tập trung trong tập nạp; drill xuống Zone cần giữ đúng hierarchy và grain.

Đón và trả là hai loại sự kiện khác nhau của một chuyến, có thể nằm ngoài cùng ngày. Một vùng trả nhiều hơn đón trong ngày không có nghĩa lỗi tính tổng. Khi so các dịch vụ, cần nêu số file và giới hạn dòng để không gọi tỷ trọng trong sample là thị phần New York.

Trang Nhu cầu theo khu vực; canvas 1440 × 960.

Bảng 5.8 Visual và binding trang Nhu cầu theo khu vực

Tiêu đề hoặc ID	Loại visual	Field và measure	Vị trí và filter
06507fc1087132b1b1db	textbox	Thành phần tiêu đề hoặc bố cục, không có field dữ liệu.	x=24; y=66; width=1392; height=32; z=2; tabOrder=2
Dịch vụ	slicer	Dim Service Type.Service Code	x=488; y=110; width=250; height=82; z=5; tabOrder=5
Lưu lượng đã nạp theo dịch vụ	clusteredColumnChart	Dim Service Type.Service Code; Zone Traffic Snapshot.Zone Traffic Snapshot TotalPickups	x=728; y=610; width=688; height=256; z=13; tabOrder=13
Tháng	slicer	Snapshot Date.Month Name	x=256; y=110; width=220; height=82; z=4; tabOrder=4
Năm	slicer	Snapshot Date.Year Number	x=24; y=110; width=220; height=82; z=3; tabOrder=3

Tiêu đề hoặc ID	Loại visual	Field và measure	Vị trí và filter
Quận	slicer	Snapshot Location.Borough	x=750; y=110; width=350; height=82; z=6; tabOrder=6
Lượt đón theo giờ và dịch vụ	clusteredColumnChart	Snapshot Time.Hour Number; Zone Traffic Snapshot.Zone Traffic Snapshot TotalPickups; Dim Service Type.Service Code	x=728; y=338; width=688; height=256; z=11; tabOrder=11
8db8b426a8a3587a1bfc	textbox	Thành phần tiêu đề hoặc bố cục, không có field dữ liệu.	x=24; y=18; width=1392; height=44; z=1; tabOrder=1
Lượt trả	card	Zone Traffic Snapshot.Zone Traffic Snapshot TotalDropoffs	x=728; y=210; width=688; height=110; z=9; tabOrder=9
b30f035b477ef0792afe	textbox	Thành phần tiêu đề hoặc bố cục, không có field dữ liệu.	x=24; y=884; width=1392; height=58; z=7; tabOrder=7
Lượt đón theo ngày và dịch vụ	lineChart	Snapshot Date.Full Date; Zone Traffic Snapshot.Zone Traffic Snapshot TotalPickups; Dim Service Type.Service Code	x=24; y=338; width=688; height=256; z=10; tabOrder=10
Lượt đón và trả theo quận	clusteredBarChart	Snapshot Location.Borough; Zone Traffic Snapshot.Zone Traffic Snapshot TotalPickups; Zone Traffic Snapshot.Zone Traffic Snapshot TotalDropoffs	x=24; y=610; width=688; height=256; z=12; tabOrder=12
Lượt đón	card	Zone Traffic Snapshot.Zone Traffic Snapshot TotalPickups	x=24; y=210; width=688; height=110; z=8; tabOrder=8

Trang Chi tiết không gian & giờ; canvas 1440 × 960.

Bảng 5.9 Visual và binding trang Chi tiết không gian & giờ

Tiêu đề hoặc ID	Loại visual	Field và measure	Vị trí và filter
Quận	slicer	Snapshot Location.Borough	x=750; y=110; width=350; height=82; z=19; tabOrder=19
1d9dd53747c099c8c48c	textbox	Thành phần tiêu đề hoặc bố cục, không có field dữ liệu.	x=24; y=18; width=1392; height=44; z=14; tabOrder=14
27276a6b6edb278e42a3	textbox	Thành phần tiêu đề hoặc bố cục, không có field dữ liệu.	x=24; y=66; width=1392; height=32; z=15; tabOrder=15
Lượt đón theo khu vực · cuộn để xem đủ zone	clusteredBarChart	Snapshot Location.Zone Name; Zone Traffic Snapshot.Zone Traffic Snapshot TotalPickups	x=728; y=338; width=688; height=528; z=24; tabOrder=24
Dịch vụ	slicer	Dim Service Type.Service Code	x=488; y=110; width=250; height=82; z=18; tabOrder=18
46d3d49092e26fe8bb33	textbox	Thành phần tiêu đề hoặc bố cục, không có field dữ liệu.	x=24; y=884; width=1392; height=58; z=20; tabOrder=20
Lượt đón	card	Zone Traffic Snapshot.Zone Traffic Snapshot TotalPickups	x=24; y=210; width=688; height=110; z=21; tabOrder=21
Tháng	slicer	Snapshot Date.Month Name	x=256; y=110; width=220; height=82; z=17; tabOrder=17

Tiêu đề hoặc ID	Loại visual	Field và measure	Vị trí và filter
Năm	slicer	Snapshot Date.Year Number	x=24; y=110; width=220; height=82; z=16; tabOrder=16
Lượt đón theo quận × giờ	pivotTable	Snapshot Location.Borough; Snapshot Time.Hour Number; Zone Traffic Snapshot.Zone Traffic Snapshot TotalPickups	x=24; y=338; width=688; height=528; z=23; tabOrder=23
Lượt trả	card	Zone Traffic Snapshot.Zone Traffic Snapshot TotalDropoffs	x=728; y=210; width=688; height=110; z=22; tabOrder=22

5.4.8 Phân tích hiệu suất điều phối

Trang Hiệu suất điều phối dùng Base Performance để xem khối lượng, thời lượng, thời gian chờ và yêu cầu đi chung theo dispatch base, provider và dịch vụ. Trung bình sử dụng measure của cube tính từ tổng và số đếm hợp lệ; không dùng average của các giá trị trung bình đã hiển thị theo base.

Khi drill hoặc chọn một base, phải phân biệt mã base với giấy phép provider. Shared Request Rate mô tả yêu cầu đi chung trong phạm vi có dữ liệu, không phải tỷ lệ ghép thành công. WAV Match Rate không được đưa vào báo cáo vì chưa có định nghĩa tử mẫu phù hợp; không clamp một tỷ lệ vượt 100 phần trăm để tạo kết quả có vẻ hợp lý.

Trang Hiệu suất điều phối; canvas 1440 × 960.

Bảng 5.10 Visual và binding trang Hiệu suất điều phối

Tiêu đề hoặc ID	Loại visual	Field và measure	Vị trí và filter
Tỷ lệ yêu cầu đi chung	card	Measures.Shared Request Rate	x=1074; y=210; width=342; height=110; z=11; tabOrder=11

Tiêu đề hoặc ID	Loại visual	Field và measure	Vị trí và filter
Năm	slicer	Snapshot Date.Year Number	x=24; y=110; width=220; height=82; z=3; tabOrder=3
Số chuyến theo base	clusteredBarChart	Dispatch Base.Base Key; Base Performance Snapshot.Base Performance Snapshot TotalTrips	x=24; y=338; width=688; height=256; z=12; tabOrder=12
31abc79ca2c47b5cd4cd	textbox	Thành phần tiêu đề hoặc bố cục, không có field dữ liệu.	x=24; y=66; width=1392; height=32; z=2; tabOrder=2
31dbdc1a1f8e7b061c18	textbox	Thành phần tiêu đề hoặc bố cục, không có field dữ liệu.	x=24; y=18; width=1392; height=44; z=1; tabOrder=1
36c27ba57b2411091064	textbox	Thành phần tiêu đề hoặc bố cục, không có field dữ liệu.	x=24; y=884; width=1392; height=58; z=7; tabOrder=7
Chờ TB · phút	card	Measures.Base Average Waiting Minutes	x=724; y=210; width=342; height=110; z=10; tabOrder=10
Số chuyến theo ngày và dịch vụ	lineChart	Snapshot Date.Full Date; Base Performance Snapshot.Base Performance Snapshot TotalTrips; Dim Service Type.Service Code	x=24; y=610; width=688; height=256; z=14; tabOrder=14
Thời lượng TB · phút	card	Measures.Base Average Duration Minutes	x=374; y=210; width=342; height=110; z=9; tabOrder=9

Tiêu đề hoặc ID	Loại visual	Field và measure	Vị trí và filter
Yêu cầu đi chung theo nhà cung cấp	clusteredBarChart	Dim HVFHS Provider.License Number; Measures.Shared Request Rate	x=728; y=610; width=688; height=256; z=15; tabOrder=15
Base	slicer	Dispatch Base.Base Key	x=750; y=110; width=350; height=82; z=6; tabOrder=6
Dịch vụ	slicer	Dim Service Type.Service Code	x=488; y=110; width=250; height=82; z=5; tabOrder=5
Số chuyến	card	Base Performance Snapshot.Base Performance Snapshot TotalTrips	x=24; y=210; width=342; height=110; z=8; tabOrder=8
Tháng	slicer	Snapshot Date.Month Name	x=256; y=110; width=220; height=82; z=4; tabOrder=4
Thời gian chờ theo base · phút	clusteredBarChart	Dispatch Base.Base Key; Measures.Base Average Waiting Minutes	x=728; y=338; width=688; height=256; z=13; tabOrder=13

5.5 Khai thác dashboard để trả lời câu hỏi nghiệp vụ

Quy trình khai thác bắt đầu từ câu hỏi, chọn cube hoặc trang đúng phạm vi, xác định bộ lọc và đọc chỉ tiêu cùng đơn vị. Sau khi phát hiện một khác biệt, người dùng drill theo dimension phù hợp rồi đối chiếu card với lát cắt SQL MDX. Kết luận phải ghi rõ khoảng thời gian thực có dữ liệu, dịch vụ, vùng và giới hạn sample.

Ví dụ, để hỏi dịch vụ nào có nhiều lượt đón trong một Borough vào một nhóm giờ, dùng Zone Traffic với TotalPickups, cùng năm tháng và Borough, thay vì

mở hai transaction report rồi cộng card có bộ lọc khác nhau. Để hỏi giá trị tip theo thanh toán taxi, dùng YellowGreen và nêu giới hạn tip tiền mặt. Để hỏi chờ theo provider, dùng HVFHV hoặc snapshot base với cùng phạm vi và số dòng chờ hợp lệ.

Trong bằng chứng hiện có, tổng số chuyến FHV và HVFHV lần lượt 139.970 và 140.000 không phải kết luận hai dịch vụ có quy mô thị trường gần bằng nhau: từng file bị giới hạn theo cùng chính sách N dòng đầu. Đây là ví dụ cho thấy kiểm tra độ bao phủ phải đi trước diễn giải kinh doanh. Báo cáo ưu tiên những kết luận mà cấu trúc và phạm vi dữ liệu thực sự hỗ trợ.

5.6 Kiểm thử dữ liệu và tương tác trên dashboard

Hồ sơ có 70 field binding duy nhất và 101 truy vấn visual đạt ở thời điểm kiểm tra. Schema validation kiểm tra định nghĩa JSON, còn các truy vấn kiểm tra tên trường, measure và grouping key theo CSDL. Các kiểm tra đó hỗ trợ chất lượng đầu vào của visual, nhưng không thay thế mở chín trang trong Power BI Desktop.

Nghiệm thu giao diện phải kiểm tra kết nối live khi cube Processed, tải tất cả trang, chọn năm tháng dịch vụ, thử các bộ lọc đặc thù, tập rỗng và Unknown, đối chiếu card, kiểm tra đơn vị, sort, drill và tương tác. Mỗi ảnh bằng chứng cần có thời gian và phạm vi lọc. Báo cáo không gắn nhãn đã đạt cho những bước chưa có bằng chứng UI tương ứng.

Bảng 5.11 Phân biệt kiểm thử dữ liệu và giao diện

Kiểm tra	**Bằng chứng đã có**	**Giới hạn**
Field binding	70 binding duy nhất trong hồ sơ	Tên field và grouping key hợp lệ tại thời điểm thử.
Truy vấn visual	101 truy vấn đạt trong hồ sơ	Không chứng minh render và tương tác Desktop.
Kết nối live	Sáu cấu hình và health probe được lưu	Phải kiểm tra lại sau reset hoặc process mới.

Kiểm tra	Bằng chứng đã có	Giới hạn
Slicer drill cross filter	Cần nghiệm thu bằng Desktop từng trang	Không gắn nhãn đã đạt nếu chưa có bằng chứng UI.
Publish Service	Không nằm trong triển khai được mô tả	Cần gateway và danh tính khi triển khai thêm.

5.7 Đánh giá kết quả và giới hạn diễn giải

Bộ báo cáo đã có cấu trúc theo sáu nghiệp vụ, liên kết trực tiếp các cube và có các bằng chứng kiểm tra dữ liệu cùng field binding. Việc tách nguồn và chỉ tiêu giúp tránh nhiều sai lầm thường gặp như cộng trùng snapshot với transaction, dùng chiều thanh toán cho FHV hoặc tính trung bình không đúng mẫu số. Các sơ đồ trong báo cáo là minh họa thiết kế, không được trình bày như ảnh chụp dashboard đã chạy.

Giới hạn còn lại gồm dữ liệu sample theo đầu file, chất lượng vị trí và các mốc nguồn không đồng đều, một số thuộc tính lịch hoặc lịch sử dimension chưa đầy đủ, và kiểm thử Desktop chưa bao phủ toàn bộ trang sau các thay đổi cuối. Các biểu đồ có tính mô tả; chưa chứng minh thị phần toàn thành phố, lợi nhuận doanh nghiệp hay tác động nhân quả. Kết luận sử dụng phải giữ các giới hạn này cùng kết quả số.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Đề tài xây dựng chuỗi xử lý NYC Taxi từ dữ liệu file đến báo cáo đa chiều. Kho được thiết kế theo Galaxy Schema với 15 dimension, ba transaction fact và ba snapshot có grain rõ ràng. Pipeline bảo toàn lineage theo file, phân loại chất lượng, hỗ trợ checkpoint và nạp lại khi nguồn thay đổi. SSAS tổ chức sáu cube theo nghiệp vụ và tính các trung bình, tỷ lệ từ tập dữ liệu phù hợp; Power BI cung cấp các trang khai thác tương ứng.

Những bằng chứng lưu trong dự án cho thấy các tập kiểm tra count, khóa ngoại, checkpoint và SQL MDX đã có kết quả nhất quán ở các lần quan sát. Các bản hardening bổ sung mapping FHV, quản lý tiến trình con, truyền lỗi có cấu trúc và lịch tháng. Những kết quả đó cần được phân biệt với kiểm thử production, kiểm thử lịch thật, phục hồi backup và nghiệm thu UI chưa được thực hiện đầy đủ.

Hướng phát triển trước hết là hoàn tất nghiệm thu theo đúng phiên bản, thử phục hồi và thống nhất mọi đường chạy qua guard điều phối. Về tính đúng nghiệp vụ, cần bổ sung lịch ngày lễ đáng tin cậy, xem xét lịch sử thuộc tính base khi có yêu cầu và định nghĩa KPI trên cùng tập tử mẫu. Về hiệu năng, cần đo trên dữ liệu lớn rồi tối ưu Transform theo metadata, aggregate theo ngày bị ảnh hưởng và process partition liên quan, đồng thời giữ đối soát và khả năng phục hồi.

TÀI LIỆU THAM KHẢO

[1] NYC TLC Trip Record Data. https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page. Đối chiếu nguồn dữ liệu và các liên kết metadata khi biên soạn.

[2] NYC TLC Yellow Taxi Trip Records Data Dictionary. https://www.nyc.gov/assets/tlc/downloads/pdf/data_dictionary_trip_records_yellow.pdf.

[3] NYC TLC Green Taxi Trip Records Data Dictionary. https://www.nyc.gov/assets/tlc/downloads/pdf/data_dictionary_trip_records_green.pdf.

[4] NYC TLC FHV Trip Records Data Dictionary. https://www.nyc.gov/assets/tlc/downloads/pdf/data_dictionary_trip_records_fhv.pdf.

[5] NYC TLC High Volume FHV Trip Records Data Dictionary. https://www.nyc.gov/assets/tlc/downloads/pdf/data_dictionary_trip_records_hvfhs.pdf.

[6] NYC TLC Trip Record User Guide. https://www.nyc.gov/assets/tlc/downloads/pdf/trip_record_user_guide.pdf.

[7] Mã nguồn dự án NYC Taxi: etl/packages, etl/python, các script điều phối, mô hình NYCTaxi_SSAS và định nghĩa powerbi/reports. Các đường dẫn trong chương triển khai phản ánh môi trường source đang lưu.

[8] Metadata và bằng chứng: evidence/columns.json, primary-keys.json, foreign-keys.json, review-20261031/REVIEW.md, final-status.json và reconciliation-Full.csv. Bằng chứng có thời điểm chụp, không phải giám sát liên tục.

[9] Hồ sơ vận hành và cập nhật: etl/README.md, incremental/README.md, hardening/STATUS.md, handover/README.md, powerbi/README.md. Khi có khác biệt lịch sử, dùng source và hồ sơ cập nhật tương ứng phiên bản được mô tả.

PHỤ LỤC A SQL VÀ CẤU HÌNH KỸ THUẬT CHI TIẾT

Phụ lục lưu đặc tả để đối chiếu với phần giải thích trong các chương. Mã SQL và MDX được trích từ source, không được thực thi trong lần biên soạn. Tham số, đường dẫn và phiên bản phải được kiểm tra trước khi dùng trên một môi trường khác.

PKG_00_Master

EPT_00_Open_Monitor

Bảng A.1 Cấu hình tác vụ EPT_00_Open_Monitor

Thuộc tính	**Giá trị source**
Executable	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
Arguments	-NoProfile -ExecutionPolicy Bypass -File "E:\NYCTaxi\NYCTaxi_SSAS\etl\Invoke-SsisStage.ps1" -Operation Monitor -Stage Extract -RunnerPath "E:\NYCTaxi\NYCTaxi_SSAS\etl\Invoke-NYCTaxiPipeline.ps1" -RunId "{4BA2DCE7-2944-48C7-9B59-ACB44CDC03AF}" -StartAt "Extract" -StartMonth 2021-02 -EndMonth 2021-02 -MaxRows 10000 -MaxRetries 1 -ShowMonitor 1
WorkingDirectory	E:\NYCTaxi\NYCTaxi_SSAS
StandardOutputVariable	User::vStageStdOut
StandardErrorVariable	User::vStageStdErr
WindowStyle	Hidden
Expression Arguments	"-NoProfile -ExecutionPolicy Bypass -File \"" + @[$Package::pControllerPath] + "\" -Operation Monitor -Stage Extract -RunnerPath \"" + @[$Package::pRunnerPath] + "\" -RunId \"" + @[User::vRunId] + "\" -StartAt \"" + @[$Package::pStartStage] + "\" -StartMonth " + @[$Package::pStartMonth] + " -EndMonth " + @[$Package::pEndMonth] + " -MaxRows " + (DT_WSTR,12)@[$Package::pMaxRows] + " -MaxRetries " + (DT_WSTR,12)@[$Package::pMaxRetries] + " -ShowMonitor " + (DT_WSTR,12)@[$Package::pShowMonitor]

EPT_01_Extract

Bảng A.2 Cấu hình tác vụ EPT_01_Extract

Thuộc tính	**Giá trị source**
Executable	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
Arguments	-NoProfile -ExecutionPolicy Bypass -File "E:\NYCTaxi\NYCTaxi_SSAS\etl\Invoke-SsisStage.ps1" -Operation Stage -Stage Extract -RunnerPath "E:\NYCTaxi\NYCTaxi_SSAS\etl\Invoke-NYCTaxiPipeline.ps1" -RunId "{4BA2DCE7-2944-48C7-9B59-ACB44CDC03AF}" -StartAt "Extract" -StartMonth 2021-02 -EndMonth 2021-02 -MaxRows 10000 -MaxRetries 1 -ShowMonitor 1
WorkingDirectory	E:\NYCTaxi\NYCTaxi_SSAS
StandardOutputVariable	User::vStageStdOut
StandardErrorVariable	User::vStageStdErr
WindowStyle	Hidden
Expression Arguments	"-NoProfile -ExecutionPolicy Bypass -File \"" + @[$Package::pControllerPath] + "\" -Operation Stage -Stage Extract -RunnerPath \"" + @[$Package::pRunnerPath] + "\" -RunId \"" + @[User::vRunId] + "\" -StartAt \"" + @[$Package::pStartStage] + "\" -StartMonth " + @[$Package::pStartMonth] + " -EndMonth " + @[$Package::pEndMonth] + " -MaxRows " + (DT_WSTR,12)@[$Package::pMaxRows] + " -MaxRetries " + (DT_WSTR,12)@[$Package::pMaxRetries] + " -ShowMonitor " + (DT_WSTR,12)@[$Package::pShowMonitor]

EPT_02_Transform

Bảng A.3 Cấu hình tác vụ EPT_02_Transform

Thuộc tính	**Giá trị source**
Executable	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe

Thuộc tính	Giá trị source
Arguments	-NoProfile -ExecutionPolicy Bypass -File "E:\NYCTaxi\NYCTaxi_SSAS\etl\Invoke-SsisStage.ps1" -Operation Stage -Stage Transform -RunnerPath "E:\NYCTaxi\NYCTaxi_SSAS\etl\Invoke-NYCTaxiPipeline.ps1" -RunId "{4BA2DCE7-2944-48C7-9B59-ACB44CDC03AF}" -StartAt "Extract" -StartMonth 2021-02 -EndMonth 2021-02 -MaxRows 10000 -MaxRetries 1 -ShowMonitor 1
WorkingDirectory	E:\NYCTaxi\NYCTaxi_SSAS
StandardOutputVariable	User::vStageStdOut
StandardErrorVariable	User::vStageStdErr
WindowStyle	Hidden
Expression Arguments	"-NoProfile -ExecutionPolicy Bypass -File \"" + @[$Package::pControllerPath] + "\" -Operation Stage -Stage Transform -RunnerPath \"" + @[$Package::pRunnerPath] + "\" -RunId \"" + @[User::vRunId] + "\" -StartAt \"" + @[$Package::pStartStage] + "\" -StartMonth " + @[$Package::pStartMonth] + " -EndMonth " + @[$Package::pEndMonth] + " -MaxRows " + (DT_WSTR,12)@[$Package::pMaxRows] + " -MaxRetries " + (DT_WSTR,12)@[$Package::pMaxRetries] + " -ShowMonitor " + (DT_WSTR,12)@[$Package::pShowMonitor]

EPT_03_Dwh

Bảng A.4 Cấu hình tác vụ EPT_03_Dwh

Thuộc tính	Giá trị source
Executable	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
Arguments	-NoProfile -ExecutionPolicy Bypass -File "E:\NYCTaxi\NYCTaxi_SSAS\etl\Invoke-SsisStage.ps1" -Operation Stage -Stage Dwh -RunnerPath "E:\NYCTaxi\NYCTaxi_SSAS\etl\Invoke-NYCTaxiPipeline.ps1" -RunId "{4BA2DCE7-2944-48C7-9B59-ACB44CDC03AF}" -StartAt "Extract" -StartMonth 2021-02 -EndMonth 2021-02 -MaxRows 10000 -MaxRetries 1 -ShowMonitor 1

Thuộc tính	Giá trị source
WorkingDirectory	E:\NYCTaxi\NYCTaxi_SSAS
StandardOutputVariable	User::vStageStdOut
StandardErrorVariable	User::vStageStdErr
WindowStyle	Hidden
Expression Arguments	"-NoProfile -ExecutionPolicy Bypass -File \"" + @[$Package::pControllerPath] + "\" -Operation Stage -Stage Dwh -RunnerPath \"" + @[$Package::pRunnerPath] + "\" -RunId \"" + @[User::vRunId] + "\" -StartAt \"" + @[$Package::pStartStage] + "\" -StartMonth " + @[$Package::pStartMonth] + " -EndMonth " + @[$Package::pEndMonth] + " -MaxRows " + (DT_WSTR,12)@[$Package::pMaxRows] + " -MaxRetries " + (DT_WSTR,12)@[$Package::pMaxRetries] + " -ShowMonitor " + (DT_WSTR,12)@[$Package::pShowMonitor]

EPT_04_Aggregates

Bảng A.5 Cấu hình tác vụ EPT_04_Aggregates

Thuộc tính	Giá trị source
Executable	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
Arguments	-NoProfile -ExecutionPolicy Bypass -File "E:\NYCTaxi\NYCTaxi_SSAS\etl\Invoke-SsisStage.ps1" -Operation Stage -Stage Aggregates -RunnerPath "E:\NYCTaxi\NYCTaxi_SSAS\etl\Invoke-NYCTaxiPipeline.ps1" -RunId "{4BA2DCE7-2944-48C7-9B59-ACB44CDC03AF}" -StartAt "Extract" -StartMonth 2021-02 -EndMonth 2021-02 -MaxRows 10000 -MaxRetries 1 -ShowMonitor 1
WorkingDirectory	E:\NYCTaxi\NYCTaxi_SSAS
StandardOutputVariable	User::vStageStdOut
StandardErrorVariable	User::vStageStdErr

Thuộc tính	Giá trị source
WindowStyle	Hidden
Expression Arguments	"-NoProfile -ExecutionPolicy Bypass -File \"" + @[$Package::pControllerPath] + "\" -Operation Stage -Stage Aggregates -RunnerPath \"" + @[$Package::pRunnerPath] + "\" -RunId \"" + @[User::vRunId] + "\" -StartAt \"" + @[$Package::pStartStage] + "\" -StartMonth " + @[$Package::pStartMonth] + " -EndMonth " + @[$Package::pEndMonth] + " -MaxRows " + (DT_WSTR,12)@[$Package::pMaxRows] + " -MaxRetries " + (DT_WSTR,12)@[$Package::pMaxRetries] + " -ShowMonitor " + (DT_WSTR,12)@[$Package::pShowMonitor]

EPT_05_Ssas

Bảng A.6 Cấu hình tác vụ EPT_05_Ssas

Thuộc tính	Giá trị source
Executable	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
Arguments	-NoProfile -ExecutionPolicy Bypass -File "E:\NYCTaxi\NYCTaxi_SSAS\etl\Invoke-SsisStage.ps1" -Operation Stage -Stage Ssas -RunnerPath "E:\NYCTaxi\NYCTaxi_SSAS\etl\Invoke-NYCTaxiPipeline.ps1" -RunId "{4BA2DCE7-2944-48C7-9B59-ACB44CDC03AF}" -StartAt "Extract" -StartMonth 2021-02 -EndMonth 2021-02 -MaxRows 10000 -MaxRetries 1 -ShowMonitor 1
WorkingDirectory	E:\NYCTaxi\NYCTaxi_SSAS
StandardOutputVariable	User::vStageStdOut
StandardErrorVariable	User::vStageStdErr
WindowStyle	Hidden

Thuộc tính	Giá trị source
Expression Arguments	"-NoProfile -ExecutionPolicy Bypass -File \"" + @[$Package::pControllerPath] + "\" -Operation Stage -Stage Ssas -RunnerPath \"" + @[$Package::pRunnerPath] + "\" -RunId \"" + @[User::vRunId] + "\" -StartAt \"" + @[$Package::pStartStage] + "\" -StartMonth " + @[$Package::pStartMonth] + " -EndMonth " + @[$Package::pEndMonth] + " -MaxRows " + (DT_WSTR,12)@[$Package::pMaxRows] + " -MaxRetries " + (DT_WSTR,12)@[$Package::pMaxRetries] + " -ShowMonitor " + (DT_WSTR,12)@[$Package::pShowMonitor]

EPT_99_Complete

Bảng A.7 Cấu hình tác vụ EPT_99_Complete

Thuộc tính	Giá trị source
Executable	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
Arguments	-NoProfile -ExecutionPolicy Bypass -File "E:\NYCTaxi\NYCTaxi_SSAS\etl\Invoke-SsisStage.ps1" -Operation Complete -Stage Extract -RunnerPath "E:\NYCTaxi\NYCTaxi_SSAS\etl\Invoke-NYCTaxiPipeline.ps1" -RunId "{4BA2DCE7-2944-48C7-9B59-ACB44CDC03AF}" -StartAt "Extract" -StartMonth 2021-02 -EndMonth 2021-02 -MaxRows 10000 -MaxRetries 1 -ShowMonitor 1
WorkingDirectory	E:\NYCTaxi\NYCTaxi_SSAS
StandardOutputVariable	User::vStageStdOut
StandardErrorVariable	User::vStageStdErr
WindowStyle	Hidden

Thuộc tính	Giá trị source
Expression Arguments	"-NoProfile -ExecutionPolicy Bypass -File \"" + @[$Package::pControllerPath] + "\" -Operation Complete -Stage Extract -RunnerPath \"" + @[$Package::pRunnerPath] + "\" -RunId \"" + @[User::vRunId] + "\" -StartAt \"" + @[$Package::pStartStage] + "\" -StartMonth " + @[$Package::pStartMonth] + " -EndMonth " + @[$Package::pEndMonth] + " -MaxRows " + (DT_WSTR,12)@[$Package::pMaxRows] + " -MaxRetries " + (DT_WSTR,12)@[$Package::pMaxRetries] + " -ShowMonitor " + (DT_WSTR,12)@[$Package::pShowMonitor]

EST_90_Report_Original_Error

```
DECLARE @r nvarchar(64)=?, @e nvarchar(4000)=?; INSERT
etl.PackageError(ExecutionId,PackageName,SourceName,ErrorCode,ErrorDescription)
VALUES(@r,N'PKG_00_Master',N'Master failure branch',55990,@e); DECLARE @m
nvarchar(2047)=LEFT(CONCAT(N'Master failed. RunId=',@r,N'. ',COALESCE(NULLIF(@e,N''),N'See
local ETL monitor/logs.')),2000); RAISERROR(N'%s',16,1,@m);
```

Bảng A.8 Cấu hình tác vụ EST_90_Report_Original_Error

Thuộc tính	Giá trị source
Connection	{4A829E7F-DE39-4A39-83D3-52CE94E192E2}

EST_LogOriginalError

```
INSERT etl.PackageError(ExecutionId,PackageName,SourceName,ErrorCode,ErrorDescription)
VALUES(?,?,?,?,?);
```

Bảng A.9 Cấu hình tác vụ EST_LogOriginalError

Thuộc tính	Giá trị source
Connection	{4A829E7F-DE39-4A39-83D3-52CE94E192E2}

Bảng A.10 Precedence constraint PKG_00_Master

Từ	Đến	Điều kiện
Package\EPT_00_Open_Monitor	Package\EST_90_Report_Original_Error	Value=1
Package\EPT_01_Extract	Package\EST_90_Report_Original_Error	Value=1
Package\EPT_02_Transform	Package\EST_90_Report_Original_Error	Value=1

Từ	Đến	Điều kiện
Package\EPT_03_Dwh	Package\EST_90_Report_Original_Error	Value=1
Package\EPT_04_Aggregates	Package\EST_90_Report_Original_Error	Value=1
Package\EPT_05_Ssas	Package\EST_90_Report_Original_Error	Value=1
Package\EPT_99_Complete	Package\EST_90_Report_Original_Error	Value=1
Package\EPT_00_Open_Monitor	Package\EPT_01_Extract	LogicalAnd=True
Package\EPT_01_Extract	Package\EPT_02_Transform	LogicalAnd=True
Package\EPT_02_Transform	Package\EPT_03_Dwh	LogicalAnd=True
Package\EPT_03_Dwh	Package\EPT_04_Aggregates	LogicalAnd=True
Package\EPT_04_Aggregates	Package\EPT_05_Ssas	LogicalAnd=True
Package\EPT_05_Ssas	Package\EPT_99_Complete	LogicalAnd=True

PKG_01_Extract_Stage

EPT_01_Download_Parquet

Bảng A.11 Cấu hình tác vụ EPT_01_Download_Parquet

Thuộc tính	Giá trị source
Executable	C:\Users\ADMIN\AppData\Local\Programs\Python\Python310\python.exe
Arguments	"D:\Kho_Du_lieu\NYCTaxi\script\download_nyc_parquet.py"
WorkingDirectory	D:\Kho_Du_lieu\NYCTaxi\script

Thuộc tính	Giá trị source
SuccessValue	0
FailTaskIfReturnCodeIsNotSuccessValue	True
RequireFullFileName	True
StandardOutputVariable	User::vProcessOutput
StandardErrorVariable	User::vProcessError
WindowStyle	Hidden
Expression Executable	@[$Package::pPythonExe]
Expression WorkingDirectory	@[$Package::pScriptsRoot]
Expression Arguments	"\"" + @[$Package::pScriptsRoot] + "\\download_nyc_parquet.py\" --start \"" + @[$Package::pStartMonth] + "\" --end \"" + @[$Package::pEndMonth] + "\" --output \"" + @[$Package::pDataRoot] + "\\raw\" --lookup"

EPT_02_Convert_To_CSV

Bảng A.12 Cấu hình tác vụ EPT_02_Convert_To_CSV

Thuộc tính	Giá trị source
Executable	C:\Users\ADMIN\AppData\Local\Programs\Python\Python310\python.exe
Arguments	"D:\Kho_Du_lieu\NYCTaxi\script\convert_nyc_parquet_to_csv.py"
WorkingDirectory	D:\Kho_Du_lieu\NYCTaxi\script
SuccessValue	0

Thuộc tính	Giá trị source
FailTaskIfReturnCodeIsNotSuccessValue	True
RequireFullFileName	True
StandardOutputVariable	User::vProcessOutput
StandardErrorVariable	User::vProcessError
WindowStyle	Hidden
Expression Executable	@[$Package::pPythonExe]
Expression WorkingDirectory	@[$Package::pScriptsRoot]
Expression Arguments	"\"" + @[$Package::pScriptsRoot] + "\\convert_nyc_parquet_to_csv.py\" --input \"" + @[$Package::pDataRoot] + "\\raw\" --output \"" + @[User::vCsvRoot] + "\" --start \"" + @[$Package::pStartMonth] + "\" --end \"" + @[$Package::pEndMonth] + "\" --max-rows " + (DT_WSTR,12)@[$Package::pMaxRows] + " --schema ssis --lookup"

EST_03_Prepare_Staging

```
SET NOCOUNT ON; SET XACT_ABORT ON;
IF DB_NAME() <> N'NYCTaxi_Staging' THROW 51000, 'Use NYCTaxi_Staging connection.', 1;
IF SCHEMA_ID(N'etl') IS NULL EXEC(N'CREATE SCHEMA etl');
IF SCHEMA_ID(N'wrk') IS NULL EXEC(N'CREATE SCHEMA wrk');
IF SCHEMA_ID(N'stg') IS NULL EXEC(N'CREATE SCHEMA stg');


IF OBJECT_ID(N'stg.YellowTrip', N'U') IS NULL
    CREATE TABLE stg.[YellowTrip] (
    [VendorID] VARCHAR(50) NULL,
    [tpep_pickup_datetime] VARCHAR(50) NULL,
    [tpep_dropoff_datetime] VARCHAR(50) NULL,
    [passenger_count] VARCHAR(50) NULL,
    [trip_distance] VARCHAR(50) NULL,
    [RatecodeID] VARCHAR(50) NULL,
    [store_and_fwd_flag] VARCHAR(50) NULL,
    [PULocationID] VARCHAR(50) NULL,
    [DOLocationID] VARCHAR(50) NULL,
    [payment_type] VARCHAR(50) NULL,
    [fare_amount] VARCHAR(50) NULL,
    [extra] VARCHAR(50) NULL,
    [mta_tax] VARCHAR(50) NULL,
    [tip_amount] VARCHAR(50) NULL,
```

```
    [tolls_amount] VARCHAR(50) NULL,
    [improvement_surcharge] VARCHAR(50) NULL,
    [total_amount] VARCHAR(50) NULL,
    [congestion_surcharge] VARCHAR(50) NULL,
    [Airport_fee] VARCHAR(50) NULL,
    [cbd_congestion_fee] VARCHAR(50) NULL
    );
IF COL_LENGTH(N'stg.YellowTrip', N'ETLSourceKey') IS NULL
    ALTER TABLE stg.[YellowTrip] ADD ETLSourceKey NVARCHAR(260) NULL;
IF COL_LENGTH(N'stg.YellowTrip', N'ETLFileHash') IS NULL
    ALTER TABLE stg.[YellowTrip] ADD ETLFileHash CHAR(64) NULL;
IF COL_LENGTH(N'stg.YellowTrip', N'ETLLoadedAt') IS NULL
    ALTER TABLE stg.[YellowTrip] ADD ETLLoadedAt DATETIME2(0) NULL;
IF OBJECT_ID(N'wrk.YellowTrip', N'U') IS NULL
    CREATE TABLE wrk.[YellowTrip] (
    [VendorID] VARCHAR(50) NULL,
    [tpep_pickup_datetime] VARCHAR(50) NULL,
    [tpep_dropoff_datetime] VARCHAR(50) NULL,
    [passenger_count] VARCHAR(50) NULL,
    [trip_distance] VARCHAR(50) NULL,
    [RatecodeID] VARCHAR(50) NULL,
    [store_and_fwd_flag] VARCHAR(50) NULL,
    [PULocationID] VARCHAR(50) NULL,
    [DOLocationID] VARCHAR(50) NULL,
    [payment_type] VARCHAR(50) NULL,
    [fare_amount] VARCHAR(50) NULL,
    [extra] VARCHAR(50) NULL,
    [mta_tax] VARCHAR(50) NULL,
    [tip_amount] VARCHAR(50) NULL,
    [tolls_amount] VARCHAR(50) NULL,
    [improvement_surcharge] VARCHAR(50) NULL,
    [total_amount] VARCHAR(50) NULL,
    [congestion_surcharge] VARCHAR(50) NULL,
    [Airport_fee] VARCHAR(50) NULL,
    [cbd_congestion_fee] VARCHAR(50) NULL
    );


IF COL_LENGTH(N'stg.YellowTrip', N'VendorID') IS NULL
    THROW 51001, 'Missing staging column: stg.YellowTrip.VendorID', 1;


IF COL_LENGTH(N'stg.YellowTrip', N'tpep_pickup_datetime') IS NULL
    THROW 51001, 'Missing staging column: stg.YellowTrip.tpep_pickup_datetime', 1;


IF COL_LENGTH(N'stg.YellowTrip', N'tpep_dropoff_datetime') IS NULL
    THROW 51001, 'Missing staging column: stg.YellowTrip.tpep_dropoff_datetime', 1;


IF COL_LENGTH(N'stg.YellowTrip', N'passenger_count') IS NULL
    THROW 51001, 'Missing staging column: stg.YellowTrip.passenger_count', 1;


IF COL_LENGTH(N'stg.YellowTrip', N'trip_distance') IS NULL
    THROW 51001, 'Missing staging column: stg.YellowTrip.trip_distance', 1;


IF COL_LENGTH(N'stg.YellowTrip', N'RatecodeID') IS NULL
    THROW 51001, 'Missing staging column: stg.YellowTrip.RatecodeID', 1;


IF COL_LENGTH(N'stg.YellowTrip', N'store_and_fwd_flag') IS NULL
    THROW 51001, 'Missing staging column: stg.YellowTrip.store_and_fwd_flag', 1;


IF COL_LENGTH(N'stg.YellowTrip', N'PULocationID') IS NULL
    THROW 51001, 'Missing staging column: stg.YellowTrip.PULocationID', 1;


IF COL_LENGTH(N'stg.YellowTrip', N'DOLocationID') IS NULL
    THROW 51001, 'Missing staging column: stg.YellowTrip.DOLocationID', 1;
```

```
IF COL_LENGTH(N'stg.YellowTrip', N'payment_type') IS NULL
    THROW 51001, 'Missing staging column: stg.YellowTrip.payment_type', 1;

IF COL_LENGTH(N'stg.YellowTrip', N'fare_amount') IS NULL
    THROW 51001, 'Missing staging column: stg.YellowTrip.fare_amount', 1;

IF COL_LENGTH(N'stg.YellowTrip', N'extra') IS NULL
    THROW 51001, 'Missing staging column: stg.YellowTrip.extra', 1;

IF COL_LENGTH(N'stg.YellowTrip', N'mta_tax') IS NULL
    THROW 51001, 'Missing staging column: stg.YellowTrip.mta_tax', 1;

IF COL_LENGTH(N'stg.YellowTrip', N'tip_amount') IS NULL
    THROW 51001, 'Missing staging column: stg.YellowTrip.tip_amount', 1;

IF COL_LENGTH(N'stg.YellowTrip', N'tolls_amount') IS NULL
    THROW 51001, 'Missing staging column: stg.YellowTrip.tolls_amount', 1;

IF COL_LENGTH(N'stg.YellowTrip', N'improvement_surcharge') IS NULL
    THROW 51001, 'Missing staging column: stg.YellowTrip.improvement_surcharge', 1;

IF COL_LENGTH(N'stg.YellowTrip', N'total_amount') IS NULL
    THROW 51001, 'Missing staging column: stg.YellowTrip.total_amount', 1;

IF COL_LENGTH(N'stg.YellowTrip', N'congestion_surcharge') IS NULL
    THROW 51001, 'Missing staging column: stg.YellowTrip.congestion_surcharge', 1;

IF COL_LENGTH(N'stg.YellowTrip', N'Airport_fee') IS NULL
    THROW 51001, 'Missing staging column: stg.YellowTrip.Airport_fee', 1;

IF COL_LENGTH(N'stg.YellowTrip', N'cbd_congestion_fee') IS NULL
    THROW 51001, 'Missing staging column: stg.YellowTrip.cbd_congestion_fee', 1;

IF OBJECT_ID(N'stg.GreenTrip', N'U') IS NULL
    CREATE TABLE stg.[GreenTrip] (
    [VendorID] VARCHAR(50) NULL,
    [lpep_pickup_datetime] VARCHAR(50) NULL,
    [lpep_dropoff_datetime] VARCHAR(50) NULL,
    [store_and_fwd_flag] VARCHAR(50) NULL,
    [RatecodeID] VARCHAR(50) NULL,
    [PULocationID] VARCHAR(50) NULL,
    [DOLocationID] VARCHAR(50) NULL,
    [passenger_count] VARCHAR(50) NULL,
    [trip_distance] VARCHAR(50) NULL,
    [fare_amount] VARCHAR(50) NULL,
    [extra] VARCHAR(50) NULL,
    [mta_tax] VARCHAR(50) NULL,
    [tip_amount] VARCHAR(50) NULL,
    [tolls_amount] VARCHAR(50) NULL,
    [ehail_fee] VARCHAR(50) NULL,
    [improvement_surcharge] VARCHAR(50) NULL,
    [total_amount] VARCHAR(50) NULL,
    [payment_type] VARCHAR(50) NULL,
    [trip_type] VARCHAR(50) NULL,
    [congestion_surcharge] VARCHAR(50) NULL,
    [cbd_congestion_fee] VARCHAR(50) NULL
    );
IF COL_LENGTH(N'stg.GreenTrip', N'ETLSourceKey') IS NULL
    ALTER TABLE stg.[GreenTrip] ADD ETLSourceKey NVARCHAR(260) NULL;
IF COL_LENGTH(N'stg.GreenTrip', N'ETLFileHash') IS NULL
    ALTER TABLE stg.[GreenTrip] ADD ETLFileHash CHAR(64) NULL;
```

```
IF COL_LENGTH(N'stg.GreenTrip', N'ETLLoadedAt') IS NULL
    ALTER TABLE stg.[GreenTrip] ADD ETLLoadedAt DATETIME2(0) NULL;
IF OBJECT_ID(N'wrk.GreenTrip', N'U') IS NULL
    CREATE TABLE wrk.[GreenTrip] (
    [VendorID] VARCHAR(50) NULL,
    [lpep_pickup_datetime] VARCHAR(50) NULL,
    [lpep_dropoff_datetime] VARCHAR(50) NULL,
    [store_and_fwd_flag] VARCHAR(50) NULL,
    [RatecodeID] VARCHAR(50) NULL,
    [PULocationID] VARCHAR(50) NULL,
    [DOLocationID] VARCHAR(50) NULL,
    [passenger_count] VARCHAR(50) NULL,
    [trip_distance] VARCHAR(50) NULL,
    [fare_amount] VARCHAR(50) NULL,
    [extra] VARCHAR(50) NULL,
    [mta_tax] VARCHAR(50) NULL,
    [tip_amount] VARCHAR(50) NULL,
    [tolls_amount] VARCHAR(50) NULL,
    [ehail_fee] VARCHAR(50) NULL,
    [improvement_surcharge] VARCHAR(50) NULL,
    [total_amount] VARCHAR(50) NULL,
    [payment_type] VARCHAR(50) NULL,
    [trip_type] VARCHAR(50) NULL,
    [congestion_surcharge] VARCHAR(50) NULL,
    [cbd_congestion_fee] VARCHAR(50) NULL
    );

IF COL_LENGTH(N'stg.GreenTrip', N'VendorID') IS NULL
    THROW 51001, 'Missing staging column: stg.GreenTrip.VendorID', 1;

IF COL_LENGTH(N'stg.GreenTrip', N'lpep_pickup_datetime') IS NULL
    THROW 51001, 'Missing staging column: stg.GreenTrip.lpep_pickup_datetime', 1;

IF COL_LENGTH(N'stg.GreenTrip', N'lpep_dropoff_datetime') IS NULL
    THROW 51001, 'Missing staging column: stg.GreenTrip.lpep_dropoff_datetime', 1;

IF COL_LENGTH(N'stg.GreenTrip', N'store_and_fwd_flag') IS NULL
    THROW 51001, 'Missing staging column: stg.GreenTrip.store_and_fwd_flag', 1;

IF COL_LENGTH(N'stg.GreenTrip', N'RatecodeID') IS NULL
    THROW 51001, 'Missing staging column: stg.GreenTrip.RatecodeID', 1;

IF COL_LENGTH(N'stg.GreenTrip', N'PULocationID') IS NULL
    THROW 51001, 'Missing staging column: stg.GreenTrip.PULocationID', 1;

IF COL_LENGTH(N'stg.GreenTrip', N'DOLocationID') IS NULL
    THROW 51001, 'Missing staging column: stg.GreenTrip.DOLocationID', 1;

IF COL_LENGTH(N'stg.GreenTrip', N'passenger_count') IS NULL
    THROW 51001, 'Missing staging column: stg.GreenTrip.passenger_count', 1;

IF COL_LENGTH(N'stg.GreenTrip', N'trip_distance') IS NULL
    THROW 51001, 'Missing staging column: stg.GreenTrip.trip_distance', 1;

IF COL_LENGTH(N'stg.GreenTrip', N'fare_amount') IS NULL
    THROW 51001, 'Missing staging column: stg.GreenTrip.fare_amount', 1;

IF COL_LENGTH(N'stg.GreenTrip', N'extra') IS NULL
    THROW 51001, 'Missing staging column: stg.GreenTrip.extra', 1;

IF COL_LENGTH(N'stg.GreenTrip', N'mta_tax') IS NULL
    THROW 51001, 'Missing staging column: stg.GreenTrip.mta_tax', 1;
```

```
IF COL_LENGTH(N'stg.GreenTrip', N'tip_amount') IS NULL
    THROW 51001, 'Missing staging column: stg.GreenTrip.tip_amount', 1;


IF COL_LENGTH(N'stg.GreenTrip', N'tolls_amount') IS NULL
    THROW 51001, 'Missing staging column: stg.GreenTrip.tolls_amount', 1;


IF COL_LENGTH(N'stg.GreenTrip', N'ehail_fee') IS NULL
    THROW 51001, 'Missing staging column: stg.GreenTrip.ehail_fee', 1;


IF COL_LENGTH(N'stg.GreenTrip', N'improvement_surcharge') IS NULL
    THROW 51001, 'Missing staging column: stg.GreenTrip.improvement_surcharge', 1;


IF COL_LENGTH(N'stg.GreenTrip', N'total_amount') IS NULL
    THROW 51001, 'Missing staging column: stg.GreenTrip.total_amount', 1;


IF COL_LENGTH(N'stg.GreenTrip', N'payment_type') IS NULL
    THROW 51001, 'Missing staging column: stg.GreenTrip.payment_type', 1;


IF COL_LENGTH(N'stg.GreenTrip', N'trip_type') IS NULL
    THROW 51001, 'Missing staging column: stg.GreenTrip.trip_type', 1;


IF COL_LENGTH(N'stg.GreenTrip', N'congestion_surcharge') IS NULL
    THROW 51001, 'Missing staging column: stg.GreenTrip.congestion_surcharge', 1;


IF COL_LENGTH(N'stg.GreenTrip', N'cbd_congestion_fee') IS NULL
    THROW 51001, 'Missing staging column: stg.GreenTrip.cbd_congestion_fee', 1;


IF OBJECT_ID(N'stg.FHVTrip', N'U') IS NULL
    CREATE TABLE stg.[FHVTrip] (
    [dispatching_base_num] VARCHAR(50) NULL,
    [pickup_datetime] VARCHAR(50) NULL,
    [dropOff_datetime] VARCHAR(50) NULL,
    [PULocationID] VARCHAR(50) NULL,
    [DOLocationID] VARCHAR(50) NULL,
    [SR_Flag] VARCHAR(50) NULL,
    [Affiliated_base_number] VARCHAR(50) NULL
    );
IF COL_LENGTH(N'stg.FHVTrip', N'ETLSourceKey') IS NULL
    ALTER TABLE stg.[FHVTrip] ADD ETLSourceKey NVARCHAR(260) NULL;
IF COL_LENGTH(N'stg.FHVTrip', N'ETLFileHash') IS NULL
    ALTER TABLE stg.[FHVTrip] ADD ETLFileHash CHAR(64) NULL;
IF COL_LENGTH(N'stg.FHVTrip', N'ETLLoadedAt') IS NULL
    ALTER TABLE stg.[FHVTrip] ADD ETLLoadedAt DATETIME2(0) NULL;
IF OBJECT_ID(N'wrk.FHVTrip', N'U') IS NULL
    CREATE TABLE wrk.[FHVTrip] (
    [dispatching_base_num] VARCHAR(50) NULL,
    [pickup_datetime] VARCHAR(50) NULL,
    [dropOff_datetime] VARCHAR(50) NULL,
    [PULocationID] VARCHAR(50) NULL,
    [DOLocationID] VARCHAR(50) NULL,
    [SR_Flag] VARCHAR(50) NULL,
    [Affiliated_base_number] VARCHAR(50) NULL
    );


IF COL_LENGTH(N'stg.FHVTrip', N'dispatching_base_num') IS NULL
    THROW 51001, 'Missing staging column: stg.FHVTrip.dispatching_base_num', 1;


IF COL_LENGTH(N'stg.FHVTrip', N'pickup_datetime') IS NULL
    THROW 51001, 'Missing staging column: stg.FHVTrip.pickup_datetime', 1;
```

```
IF COL_LENGTH(N'stg.FHVTrip', N'dropOff_datetime') IS NULL
    THROW 51001, 'Missing staging column: stg.FHVTrip.dropOff_datetime', 1;


IF COL_LENGTH(N'stg.FHVTrip', N'PULocationID') IS NULL
    THROW 51001, 'Missing staging column: stg.FHVTrip.PULocationID', 1;


IF COL_LENGTH(N'stg.FHVTrip', N'DOLocationID') IS NULL
    THROW 51001, 'Missing staging column: stg.FHVTrip.DOLocationID', 1;


IF COL_LENGTH(N'stg.FHVTrip', N'SR_Flag') IS NULL
    THROW 51001, 'Missing staging column: stg.FHVTrip.SR_Flag', 1;


IF COL_LENGTH(N'stg.FHVTrip', N'Affiliated_base_number') IS NULL
    THROW 51001, 'Missing staging column: stg.FHVTrip.Affiliated_base_number', 1;


IF OBJECT_ID(N'stg.FHVHVTrip', N'U') IS NULL
    CREATE TABLE stg.[FHVHVTrip] (
    [hvfhs_license_num] VARCHAR(50) NULL,
    [dispatching_base_num] VARCHAR(50) NULL,
    [originating_base_num] VARCHAR(50) NULL,
    [request_datetime] VARCHAR(50) NULL,
    [on_scene_datetime] VARCHAR(50) NULL,
    [pickup_datetime] VARCHAR(50) NULL,
    [dropoff_datetime] VARCHAR(50) NULL,
    [PULocationID] VARCHAR(50) NULL,
    [DOLocationID] VARCHAR(50) NULL,
    [trip_miles] VARCHAR(50) NULL,
    [trip_time] VARCHAR(50) NULL,
    [base_passenger_fare] VARCHAR(50) NULL,
    [tolls] VARCHAR(50) NULL,
    [bcf] VARCHAR(50) NULL,
    [sales_tax] VARCHAR(50) NULL,
    [congestion_surcharge] VARCHAR(50) NULL,
    [airport_fee] VARCHAR(50) NULL,
    [tips] VARCHAR(50) NULL,
    [driver_pay] VARCHAR(50) NULL,
    [shared_request_flag] VARCHAR(50) NULL,
    [shared_match_flag] VARCHAR(50) NULL,
    [access_a_ride_flag] VARCHAR(50) NULL,
    [wav_request_flag] VARCHAR(50) NULL,
    [wav_match_flag] VARCHAR(50) NULL,
    [cbd_congestion_fee] VARCHAR(50) NULL
    );
IF COL_LENGTH(N'stg.FHVHVTrip', N'ETLSourceKey') IS NULL
    ALTER TABLE stg.[FHVHVTrip] ADD ETLSourceKey NVARCHAR(260) NULL;
IF COL_LENGTH(N'stg.FHVHVTrip', N'ETLFileHash') IS NULL
    ALTER TABLE stg.[FHVHVTrip] ADD ETLFileHash CHAR(64) NULL;
IF COL_LENGTH(N'stg.FHVHVTrip', N'ETLLoadedAt') IS NULL
    ALTER TABLE stg.[FHVHVTrip] ADD ETLLoadedAt DATETIME2(0) NULL;
IF OBJECT_ID(N'wrk.FHVHVTrip', N'U') IS NULL
    CREATE TABLE wrk.[FHVHVTrip] (
    [hvfhs_license_num] VARCHAR(50) NULL,
    [dispatching_base_num] VARCHAR(50) NULL,
    [originating_base_num] VARCHAR(50) NULL,
    [request_datetime] VARCHAR(50) NULL,
    [on_scene_datetime] VARCHAR(50) NULL,
    [pickup_datetime] VARCHAR(50) NULL,
    [dropoff_datetime] VARCHAR(50) NULL,
    [PULocationID] VARCHAR(50) NULL,
    [DOLocationID] VARCHAR(50) NULL,
    [trip_miles] VARCHAR(50) NULL,
    [trip_time] VARCHAR(50) NULL,
    [base_passenger_fare] VARCHAR(50) NULL,
    [tolls] VARCHAR(50) NULL,
    [bcf] VARCHAR(50) NULL,
    [sales_tax] VARCHAR(50) NULL,
```

```
    [congestion_surcharge] VARCHAR(50) NULL,
    [airport_fee] VARCHAR(50) NULL,
    [tips] VARCHAR(50) NULL,
    [driver_pay] VARCHAR(50) NULL,
    [shared_request_flag] VARCHAR(50) NULL,
    [shared_match_flag] VARCHAR(50) NULL,
    [access_a_ride_flag] VARCHAR(50) NULL,
    [wav_request_flag] VARCHAR(50) NULL,
    [wav_match_flag] VARCHAR(50) NULL,
    [cbd_congestion_fee] VARCHAR(50) NULL
    );

IF COL_LENGTH(N'stg.FHVHVTrip', N'hvfhs_license_num') IS NULL
    THROW 51001, 'Missing staging column: stg.FHVHVTrip.hvfhs_license_num', 1;

IF COL_LENGTH(N'stg.FHVHVTrip', N'dispatching_base_num') IS NULL
    THROW 51001, 'Missing staging column: stg.FHVHVTrip.dispatching_base_num', 1;

IF COL_LENGTH(N'stg.FHVHVTrip', N'originating_base_num') IS NULL
    THROW 51001, 'Missing staging column: stg.FHVHVTrip.originating_base_num', 1;

IF COL_LENGTH(N'stg.FHVHVTrip', N'request_datetime') IS NULL
    THROW 51001, 'Missing staging column: stg.FHVHVTrip.request_datetime', 1;

IF COL_LENGTH(N'stg.FHVHVTrip', N'on_scene_datetime') IS NULL
    THROW 51001, 'Missing staging column: stg.FHVHVTrip.on_scene_datetime', 1;

IF COL_LENGTH(N'stg.FHVHVTrip', N'pickup_datetime') IS NULL
    THROW 51001, 'Missing staging column: stg.FHVHVTrip.pickup_datetime', 1;

IF COL_LENGTH(N'stg.FHVHVTrip', N'dropoff_datetime') IS NULL
    THROW 51001, 'Missing staging column: stg.FHVHVTrip.dropoff_datetime', 1;

IF COL_LENGTH(N'stg.FHVHVTrip', N'PULocationID') IS NULL
    THROW 51001, 'Missing staging column: stg.FHVHVTrip.PULocationID', 1;

IF COL_LENGTH(N'stg.FHVHVTrip', N'DOLocationID') IS NULL
    THROW 51001, 'Missing staging column: stg.FHVHVTrip.DOLocationID', 1;

IF COL_LENGTH(N'stg.FHVHVTrip', N'trip_miles') IS NULL
    THROW 51001, 'Missing staging column: stg.FHVHVTrip.trip_miles', 1;

IF COL_LENGTH(N'stg.FHVHVTrip', N'trip_time') IS NULL
    THROW 51001, 'Missing staging column: stg.FHVHVTrip.trip_time', 1;

IF COL_LENGTH(N'stg.FHVHVTrip', N'base_passenger_fare') IS NULL
    THROW 51001, 'Missing staging column: stg.FHVHVTrip.base_passenger_fare', 1;

IF COL_LENGTH(N'stg.FHVHVTrip', N'tolls') IS NULL
    THROW 51001, 'Missing staging column: stg.FHVHVTrip.tolls', 1;

IF COL_LENGTH(N'stg.FHVHVTrip', N'bcf') IS NULL
    THROW 51001, 'Missing staging column: stg.FHVHVTrip.bcf', 1;

IF COL_LENGTH(N'stg.FHVHVTrip', N'sales_tax') IS NULL
    THROW 51001, 'Missing staging column: stg.FHVHVTrip.sales_tax', 1;

IF COL_LENGTH(N'stg.FHVHVTrip', N'congestion_surcharge') IS NULL
    THROW 51001, 'Missing staging column: stg.FHVHVTrip.congestion_surcharge', 1;
```

```
IF COL_LENGTH(N'stg.FHVHVTrip', N'airport_fee') IS NULL
    THROW 51001, 'Missing staging column: stg.FHVHVTrip.airport_fee', 1;

IF COL_LENGTH(N'stg.FHVHVTrip', N'tips') IS NULL
    THROW 51001, 'Missing staging column: stg.FHVHVTrip.tips', 1;

IF COL_LENGTH(N'stg.FHVHVTrip', N'driver_pay') IS NULL
    THROW 51001, 'Missing staging column: stg.FHVHVTrip.driver_pay', 1;

IF COL_LENGTH(N'stg.FHVHVTrip', N'shared_request_flag') IS NULL
    THROW 51001, 'Missing staging column: stg.FHVHVTrip.shared_request_flag', 1;

IF COL_LENGTH(N'stg.FHVHVTrip', N'shared_match_flag') IS NULL
    THROW 51001, 'Missing staging column: stg.FHVHVTrip.shared_match_flag', 1;

IF COL_LENGTH(N'stg.FHVHVTrip', N'access_a_ride_flag') IS NULL
    THROW 51001, 'Missing staging column: stg.FHVHVTrip.access_a_ride_flag', 1;

IF COL_LENGTH(N'stg.FHVHVTrip', N'wav_request_flag') IS NULL
    THROW 51001, 'Missing staging column: stg.FHVHVTrip.wav_request_flag', 1;

IF COL_LENGTH(N'stg.FHVHVTrip', N'wav_match_flag') IS NULL
    THROW 51001, 'Missing staging column: stg.FHVHVTrip.wav_match_flag', 1;

IF COL_LENGTH(N'stg.FHVHVTrip', N'cbd_congestion_fee') IS NULL
    THROW 51001, 'Missing staging column: stg.FHVHVTrip.cbd_congestion_fee', 1;

IF OBJECT_ID(N'stg.TaxiZone', N'U') IS NULL
    CREATE TABLE stg.[TaxiZone] (
    [LocationID] VARCHAR(50) NULL,
    [Borough] VARCHAR(255) NULL,
    [Zone] VARCHAR(255) NULL,
    [service_zone] VARCHAR(255) NULL
    );
IF COL_LENGTH(N'stg.TaxiZone', N'ETLSourceKey') IS NULL
    ALTER TABLE stg.[TaxiZone] ADD ETLSourceKey NVARCHAR(260) NULL;
IF COL_LENGTH(N'stg.TaxiZone', N'ETLFileHash') IS NULL
    ALTER TABLE stg.[TaxiZone] ADD ETLFileHash CHAR(64) NULL;
IF COL_LENGTH(N'stg.TaxiZone', N'ETLLoadedAt') IS NULL
    ALTER TABLE stg.[TaxiZone] ADD ETLLoadedAt DATETIME2(0) NULL;
IF OBJECT_ID(N'wrk.TaxiZone', N'U') IS NULL
    CREATE TABLE wrk.[TaxiZone] (
    [LocationID] VARCHAR(50) NULL,
    [Borough] VARCHAR(255) NULL,
    [Zone] VARCHAR(255) NULL,
    [service_zone] VARCHAR(255) NULL
    );

IF COL_LENGTH(N'stg.TaxiZone', N'LocationID') IS NULL
    THROW 51001, 'Missing staging column: stg.TaxiZone.LocationID', 1;

IF COL_LENGTH(N'stg.TaxiZone', N'Borough') IS NULL
    THROW 51001, 'Missing staging column: stg.TaxiZone.Borough', 1;

IF COL_LENGTH(N'stg.TaxiZone', N'Zone') IS NULL
    THROW 51001, 'Missing staging column: stg.TaxiZone.Zone', 1;

IF COL_LENGTH(N'stg.TaxiZone', N'service_zone') IS NULL
    THROW 51001, 'Missing staging column: stg.TaxiZone.service_zone', 1;
```

```
IF OBJECT_ID(N'etl.ExtractFileLog', N'U') IS NULL
CREATE TABLE etl.ExtractFileLog (
    DatasetCode VARCHAR(20) NOT NULL,
    SourceKey NVARCHAR(260) NOT NULL,
    FilePath NVARCHAR(500) NOT NULL,
    FileHash CHAR(64) NOT NULL,
    RowLimit INT NOT NULL,
    ExpectedRows BIGINT NOT NULL,
    RowsLoaded BIGINT NULL,
    LoadStatus VARCHAR(20) NOT NULL,
    StartedAt DATETIME2(0) NOT NULL,
    CompletedAt DATETIME2(0) NULL,
    ErrorMessage NVARCHAR(2000) NULL,
    CONSTRAINT PK_ExtractFileLog PRIMARY KEY (DatasetCode, SourceKey),
    CONSTRAINT CK_ExtractFileLog_Status CHECK (LoadStatus IN
('LOADING','SUCCEEDED','FAILED'))
);

EXEC(N'CREATE OR ALTER PROCEDURE etl.usp_BeginExtractFile
    @DatasetCode VARCHAR(20), @FilePath NVARCHAR(500), @FileHash CHAR(64),
    @ExpectedRows BIGINT, @RowLimit INT
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @Table SYSNAME = CASE @DatasetCode WHEN ''yellow'' THEN ''YellowTrip'' WHEN
''green'' THEN ''GreenTrip'' WHEN ''fhv'' THEN ''FHVTrip'' WHEN ''fhvhv'' THEN
''FHVHVTrip'' WHEN ''lookup'' THEN ''TaxiZone'' END;
    IF @Table IS NULL OR @ExpectedRows IS NULL OR @ExpectedRows < 0 OR @RowLimit IS NULL OR
@RowLimit < 0
        OR @FileHash IS NULL OR LEN(@FileHash) <> 64
        OR @FileHash COLLATE Latin1_General_BIN2 LIKE ''%[^0-9a-f]%''
        THROW 51002, ''Invalid extraction metadata.'', 1;
    DECLARE @SourceKey NVARCHAR(260) = RIGHT(REPLACE(@FilePath, ''/'', ''\''),
        CHARINDEX(''\'', REVERSE(REPLACE(@FilePath, ''/'', ''\'')) + ''\'') - 1);
    DECLARE @Count BIGINT = 0, @WrongHash BIGINT = 0, @AlreadyLoaded INT = 0, @Sql
NVARCHAR(MAX);
    IF EXISTS (SELECT 1 FROM etl.ExtractFileLog WHERE DatasetCode = @DatasetCode
        AND SourceKey = @SourceKey AND FileHash = @FileHash AND RowLimit = @RowLimit
        AND ExpectedRows = @ExpectedRows AND RowsLoaded = @ExpectedRows AND LoadStatus =
''SUCCEEDED'')
    BEGIN
        SET @Sql = N''SELECT @n = COUNT_BIG(*),
                          @bad = COUNT_BIG(CASE WHEN ETLFileHash = @h THEN NULL ELSE 1 END)
                   FROM stg.'' + QUOTENAME(@Table) + N'' WHERE ETLSourceKey = @k;'';
        EXEC sys.sp_executesql @Sql,
            N''@k NVARCHAR(260), @h CHAR(64), @n BIGINT OUTPUT, @bad BIGINT OUTPUT'',
            @SourceKey, @FileHash, @Count OUTPUT, @WrongHash OUTPUT;
        IF @Count = @ExpectedRows AND @WrongHash = 0 SET @AlreadyLoaded = 1;
    END;
    IF @AlreadyLoaded = 0
    BEGIN
        SET @Sql = N''TRUNCATE TABLE wrk.'' + QUOTENAME(@Table) + N'';'';
        EXEC sys.sp_executesql @Sql;
        IF EXISTS (SELECT 1 FROM etl.ExtractFileLog WHERE DatasetCode = @DatasetCode AND
SourceKey = @SourceKey)
            UPDATE etl.ExtractFileLog SET FilePath = @FilePath, FileHash = @FileHash,
                RowLimit = @RowLimit, ExpectedRows = @ExpectedRows, RowsLoaded = NULL,
                LoadStatus = ''LOADING'', StartedAt = SYSDATETIME(), CompletedAt = NULL,
ErrorMessage = NULL
            WHERE DatasetCode = @DatasetCode AND SourceKey = @SourceKey;
        ELSE
            INSERT etl.ExtractFileLog (DatasetCode, SourceKey, FilePath, FileHash,
RowLimit,
                ExpectedRows, LoadStatus, StartedAt)
            VALUES (@DatasetCode, @SourceKey, @FilePath, @FileHash, @RowLimit,
                @ExpectedRows, ''LOADING'', SYSDATETIME());
```

```
    END
    ELSE
        UPDATE etl.ExtractFileLog SET FilePath = @FilePath
        WHERE DatasetCode = @DatasetCode AND SourceKey = @SourceKey;
    SELECT @AlreadyLoaded AS AlreadyLoaded, CAST(0 AS BIGINT) AS RowsLoaded;
END;');


EXEC(N'CREATE OR ALTER PROCEDURE etl.usp_CommitExtractFile
    @DatasetCode VARCHAR(20), @FilePath NVARCHAR(500), @FileHash CHAR(64),
    @ExpectedRows BIGINT, @RowLimit INT, @RowsLoaded BIGINT
AS
BEGIN
    SET NOCOUNT ON; SET XACT_ABORT ON;
    DECLARE @SourceKey NVARCHAR(260) = RIGHT(REPLACE(@FilePath, ''/'', ''\''),
        CHARINDEX(''\'', REVERSE(REPLACE(@FilePath, ''/'', ''\'')) + ''\'') - 1);
    DECLARE @Count BIGINT;
    IF @ExpectedRows IS NULL OR @ExpectedRows < 0 OR @RowsLoaded IS NULL
        THROW 51004, ''Invalid row count.'', 1;
    BEGIN TRY
        BEGIN TRANSACTION;
        IF NOT EXISTS (SELECT 1 FROM etl.ExtractFileLog WITH (UPDLOCK, HOLDLOCK)
            WHERE DatasetCode = @DatasetCode AND SourceKey = @SourceKey
                AND FileHash = @FileHash AND ExpectedRows = @ExpectedRows
                AND RowLimit = @RowLimit AND LoadStatus = ''LOADING'')
            THROW 51009, ''The prepared file load record changed before commit.'', 1;
    IF @DatasetCode = ''yellow''
    BEGIN
        SELECT @Count = COUNT_BIG(*) FROM wrk.[YellowTrip];
        IF @Count <> @ExpectedRows OR @RowsLoaded <> @ExpectedRows
            THROW 51003, ''Loaded row count does not match CSV metadata.'', 1;
        DELETE FROM stg.[YellowTrip] WHERE ETLSourceKey = @SourceKey;
        INSERT stg.[YellowTrip] ([VendorID], [tpep_pickup_datetime],
[tpep_dropoff_datetime], [passenger_count], [trip_distance], [RatecodeID],
[store_and_fwd_flag], [PULocationID], [DOLocationID], [payment_type], [fare_amount],
[extra], [mta_tax], [tip_amount], [tolls_amount], [improvement_surcharge], [total_amount],
[congestion_surcharge], [Airport_fee], [cbd_congestion_fee], ETLSourceKey, ETLFileHash,
ETLLoadedAt)
            SELECT [VendorID], [tpep_pickup_datetime], [tpep_dropoff_datetime],
[passenger_count], [trip_distance], [RatecodeID], [store_and_fwd_flag], [PULocationID],
[DOLocationID], [payment_type], [fare_amount], [extra], [mta_tax], [tip_amount],
[tolls_amount], [improvement_surcharge], [total_amount], [congestion_surcharge],
[Airport_fee], [cbd_congestion_fee], @SourceKey, @FileHash, SYSDATETIME() FROM
wrk.[YellowTrip];
    END
    ELSE IF @DatasetCode = ''green''
    BEGIN
        SELECT @Count = COUNT_BIG(*) FROM wrk.[GreenTrip];
        IF @Count <> @ExpectedRows OR @RowsLoaded <> @ExpectedRows
            THROW 51003, ''Loaded row count does not match CSV metadata.'', 1;
        DELETE FROM stg.[GreenTrip] WHERE ETLSourceKey = @SourceKey;
        INSERT stg.[GreenTrip] ([VendorID], [lpep_pickup_datetime],
[lpep_dropoff_datetime], [store_and_fwd_flag], [RatecodeID], [PULocationID],
[DOLocationID], [passenger_count], [trip_distance], [fare_amount], [extra], [mta_tax],
[tip_amount], [tolls_amount], [ehail_fee], [improvement_surcharge], [total_amount],
[payment_type], [trip_type], [congestion_surcharge], [cbd_congestion_fee], ETLSourceKey,
ETLFileHash, ETLLoadedAt)
            SELECT [VendorID], [lpep_pickup_datetime], [lpep_dropoff_datetime],
[store_and_fwd_flag], [RatecodeID], [PULocationID], [DOLocationID], [passenger_count],
[trip_distance], [fare_amount], [extra], [mta_tax], [tip_amount], [tolls_amount],
[ehail_fee], [improvement_surcharge], [total_amount], [payment_type], [trip_type],
[congestion_surcharge], [cbd_congestion_fee], @SourceKey, @FileHash, SYSDATETIME() FROM
wrk.[GreenTrip];
    END
    ELSE IF @DatasetCode = ''fhv''
    BEGIN
        SELECT @Count = COUNT_BIG(*) FROM wrk.[FHVTrip];
        IF @Count <> @ExpectedRows OR @RowsLoaded <> @ExpectedRows
            THROW 51003, ''Loaded row count does not match CSV metadata.'', 1;
```

```
        DELETE FROM stg.[FHVTrip] WHERE ETLSourceKey = @SourceKey;
        INSERT stg.[FHVTrip] ([dispatching_base_num], [pickup_datetime],
[dropOff_datetime], [PULocationID], [DOLocationID], [SR_Flag], [Affiliated_base_number],
ETLSourceKey, ETLFileHash, ETLLoadedAt)
            SELECT [dispatching_base_num], [pickup_datetime], [dropOff_datetime],
[PULocationID], [DOLocationID], [SR_Flag], [Affiliated_base_number], @SourceKey, @FileHash,
SYSDATETIME() FROM wrk.[FHVTrip];
    END
    ELSE IF @DatasetCode = ''fhvhv''
    BEGIN
        SELECT @Count = COUNT_BIG(*) FROM wrk.[FHVHVTrip];
        IF @Count <> @ExpectedRows OR @RowsLoaded <> @ExpectedRows
            THROW 51003, ''Loaded row count does not match CSV metadata.'', 1;
        DELETE FROM stg.[FHVHVTrip] WHERE ETLSourceKey = @SourceKey;
        INSERT stg.[FHVHVTrip] ([hvfhs_license_num], [dispatching_base_num],
[originating_base_num], [request_datetime], [on_scene_datetime], [pickup_datetime],
[dropoff_datetime], [PULocationID], [DOLocationID], [trip_miles], [trip_time],
[base_passenger_fare], [tolls], [bcf], [sales_tax], [congestion_surcharge], [airport_fee],
[tips], [driver_pay], [shared_request_flag], [shared_match_flag], [access_a_ride_flag],
[wav_request_flag], [wav_match_flag], [cbd_congestion_fee], ETLSourceKey, ETLFileHash,
ETLLoadedAt)
            SELECT [hvfhs_license_num], [dispatching_base_num], [originating_base_num],
[request_datetime], [on_scene_datetime], [pickup_datetime], [dropoff_datetime],
[PULocationID], [DOLocationID], [trip_miles], [trip_time], [base_passenger_fare], [tolls],
[bcf], [sales_tax], [congestion_surcharge], [airport_fee], [tips], [driver_pay],
[shared_request_flag], [shared_match_flag], [access_a_ride_flag], [wav_request_flag],
[wav_match_flag], [cbd_congestion_fee], @SourceKey, @FileHash, SYSDATETIME() FROM
wrk.[FHVHVTrip];
    END
    ELSE IF @DatasetCode = ''lookup''
    BEGIN
        SELECT @Count = COUNT_BIG(*) FROM wrk.[TaxiZone];
        IF @Count <> @ExpectedRows OR @RowsLoaded <> @ExpectedRows
            THROW 51003, ''Loaded row count does not match CSV metadata.'', 1;
        DELETE FROM stg.[TaxiZone] WHERE ETLSourceKey = @SourceKey;
        INSERT stg.[TaxiZone] ([LocationID], [Borough], [Zone], [service_zone],
ETLSourceKey, ETLFileHash, ETLLoadedAt)
            SELECT [LocationID], [Borough], [Zone], [service_zone], @SourceKey, @FileHash,
SYSDATETIME() FROM wrk.[TaxiZone];
    END
    ELSE THROW 51005, ''Unknown dataset.'', 1;
        UPDATE etl.ExtractFileLog SET FilePath = @FilePath, FileHash = @FileHash,
            RowLimit = @RowLimit, ExpectedRows = @ExpectedRows, RowsLoaded = @RowsLoaded,
            LoadStatus = ''SUCCEEDED'', CompletedAt = SYSDATETIME(), ErrorMessage = NULL
        WHERE DatasetCode = @DatasetCode AND SourceKey = @SourceKey;
        IF @@ROWCOUNT <> 1 THROW 51006, ''No matching file load record.'', 1;
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH;
END;');


EXEC(N'IF EXISTS (SELECT 1 FROM stg.[YellowTrip] WHERE ETLSourceKey IS NULL) THROW 51008,
''Existing rows in stg.YellowTrip have no source lineage. Archive or establish an empty
staging baseline before the first refactored load.'', 1;');


IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE object_id = OBJECT_ID(N'stg.YellowTrip') AND
name = N'IX_ExtractSourceKey')
    EXEC(N'CREATE INDEX IX_ExtractSourceKey ON stg.[YellowTrip] (ETLSourceKey) INCLUDE
(ETLFileHash);');


EXEC(N'IF EXISTS (SELECT 1 FROM stg.[GreenTrip] WHERE ETLSourceKey IS NULL) THROW 51008,
''Existing rows in stg.GreenTrip have no source lineage. Archive or establish an empty
staging baseline before the first refactored load.'', 1;');
```

```
IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE object_id = OBJECT_ID(N'stg.GreenTrip') AND
name = N'IX_ExtractSourceKey')
    EXEC(N'CREATE INDEX IX_ExtractSourceKey ON stg.[GreenTrip] (ETLSourceKey) INCLUDE
(ETLFileHash);');

EXEC(N'IF EXISTS (SELECT 1 FROM stg.[FHVTrip] WHERE ETLSourceKey IS NULL) THROW 51008,
''Existing rows in stg.FHVTrip have no source lineage. Archive or establish an empty
staging baseline before the first refactored load.'', 1;');

IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE object_id = OBJECT_ID(N'stg.FHVTrip') AND
name = N'IX_ExtractSourceKey')
    EXEC(N'CREATE INDEX IX_ExtractSourceKey ON stg.[FHVTrip] (ETLSourceKey) INCLUDE
(ETLFileHash);');

EXEC(N'IF EXISTS (SELECT 1 FROM stg.[FHVHVTrip] WHERE ETLSourceKey IS NULL) THROW 51008,
''Existing rows in stg.FHVHVTrip have no source lineage. Archive or establish an empty
staging baseline before the first refactored load.'', 1;');

IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE object_id = OBJECT_ID(N'stg.FHVHVTrip') AND
name = N'IX_ExtractSourceKey')
    EXEC(N'CREATE INDEX IX_ExtractSourceKey ON stg.[FHVHVTrip] (ETLSourceKey) INCLUDE
(ETLFileHash);');

EXEC(N'IF EXISTS (SELECT 1 FROM stg.[TaxiZone] WHERE ETLSourceKey IS NULL) THROW 51008,
''Existing rows in stg.TaxiZone have no source lineage. Archive or establish an empty
staging baseline before the first refactored load.'', 1;');

IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE object_id = OBJECT_ID(N'stg.TaxiZone') AND
name = N'IX_ExtractSourceKey')
    EXEC(N'CREATE INDEX IX_ExtractSourceKey ON stg.[TaxiZone] (ETLSourceKey) INCLUDE
(ETLFileHash);');
```

Bảng A.13 Cấu hình tác vụ EST_03_Prepare_Staging

Thuộc tính	**Giá trị source**
Connection	{428D3CB7-3BB4-4D96-8F32-4EE97109955B}
BypassPrepare	True
ResultType	ResultSetType_None

FLC_04_Load_Yellow_Files

EPT_Inspect_Yellow_CSV

Bảng A.14 Cấu hình tác vụ EPT_Inspect_Yellow_CSV

Thuộc tính	**Giá trị source**
Executable	C:\Users\ADMIN\AppData\Local\Programs\Python\Python310\python.exe
Arguments	"D:\Kho_Du_lieu\NYCTaxi\script\inspect_extract_file.py"

Thuộc tính	Giá trị source
WorkingDirectory	D:\Kho_Du_lieu\NYCTaxi\script
SuccessValue	0
FailTaskIfReturnCodeIsNotSuccessValue	True
RequireFullFileName	True
StandardOutputVariable	User::vFileMeta
StandardErrorVariable	User::vProcessError
WindowStyle	Hidden
Expression Executable	@[$Package::pPythonExe]
Expression WorkingDirectory	@[$Package::pScriptsRoot]
Expression Arguments	"\"" + @[$Package::pScriptsRoot] + "\\inspect_extract_file.py\" --dataset yellow --file \"" + @[User::vCurrentFile] + "\" --value ssis --start \"" + @[$Package::pStartMonth] + "\" --end \"" + @[$Package::pEndMonth] + "\" --row-limit " + (DT_WSTR,12)@[$Package::pMaxRows]

EST_Begin_Yellow_File

```
IF ? = 1 EXEC etl.usp_BeginExtractFile @DatasetCode=N'yellow', @FilePath=?, @FileHash=?, @ExpectedRows=?, @RowLimit=?; ELSE SELECT CAST(1 AS INT) AS AlreadyLoaded, CAST(0 AS BIGINT) AS RowsLoaded;
```

Bảng A.15 Cấu hình tác vụ EST_Begin_Yellow_File

Thuộc tính	Giá trị source
Connection	{428D3CB7-3BB4-4D96-8F32-4EE97109955B}
BypassPrepare	True
ResultType	ResultSetType_SingleRow

DFT_04_Load_Yellow

Bảng A.16 Cấu hình component SRC_Yellow_CSV

Thuộc tính	**Giá trị**
RetainNulls	true
FileNameColumnName	

Bảng A.17 Metadata cột đầu ra SRC_Yellow_CSV

Cột	**Kiểu bề rộng và xử lý lỗi**
VendorID	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
tpep_pickup_datetime	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
tpep_dropoff_datetime	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
passenger_count	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
trip_distance	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
RatecodeID	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
store_and_fwd_flag	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
PULocationID	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent

Cột	**Kiểu bề rộng và xử lý lỗi**
DOLocationID	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
payment_type	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
fare_amount	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
extra	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
mta_tax	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
tip_amount	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
tolls_amount	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
improvement_surcharge	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
total_amount	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
congestion_surcharge	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
Airport_fee	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent

Cột	Kiểu bề rộng và xử lý lỗi
cbd_congestion_fee	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
Flat File Source Error Output Column	codePage=65001; dataType=text
ErrorCode	dataType=i4
ErrorColumn	dataType=i4

Bảng A.18 Cấu hình component RC_Yellow

Thuộc tính	Giá trị
VariableName	User::vRowsLoaded

Bảng A.19 Cấu hình component DST_wrk_YellowTrip

Thuộc tính	Giá trị
CommandTimeout	0
OpenRowset	[wrk].[YellowTrip]
OpenRowsetVariable	
SqlCommand	
DefaultCodePage	65001
AlwaysUseDefaultCodePage	true
AccessMode	3
FastLoadKeepIdentity	false
FastLoadKeepNulls	true
FastLoadOptions	TABLOCK,CHECK_CONSTRAINTS,ROWS_PER_BATCH = 20000

Thuộc tính	Giá trị
FastLoadMaxInsertCommitSize	100000

Bảng A.20 Metadata cột đầu ra DST_wrk_YellowTrip

Cột	Kiểu bề rộng và xử lý lỗi
ErrorCode	dataType=i4
ErrorColumn	dataType=i4

EST_Commit_Yellow_File

```
EXEC etl.usp_CommitExtractFile @DatasetCode=N'yellow', @FilePath=?, @FileHash=?,
@ExpectedRows=?, @RowLimit=?, @RowsLoaded=?;
```

Bảng A.21 Cấu hình tác vụ EST_Commit_Yellow_File

Thuộc tính	Giá trị source
Connection	{428D3CB7-3BB4-4D96-8F32-4EE97109955B}
BypassPrepare	True
ResultType	ResultSetType_None

EST_Mark_Yellow_Failed

```
UPDATE etl.ExtractFileLog SET LoadStatus='FAILED', CompletedAt=SYSDATETIME(),
ErrorMessage=N'SSIS Data Flow or commit failed. See Execution Results for the original
error.' WHERE DatasetCode='yellow' AND FilePath=? AND LoadStatus='LOADING';
```

Bảng A.22 Cấu hình tác vụ EST_Mark_Yellow_Failed

Thuộc tính	Giá trị source
Connection	{428D3CB7-3BB4-4D96-8F32-4EE97109955B}
BypassPrepare	True
ResultType	ResultSetType_None

FLC_05_Load_Green_Files

EPT_Inspect_Green_CSV

Bảng A.23 Cấu hình tác vụ EPT_Inspect_Green_CSV

Thuộc tính	**Giá trị source**
Executable	C:\Users\ADMIN\AppData\Local\Programs\Python\Python310\python.exe
Arguments	"D:\Kho_Du_lieu\NYCTaxi\script\inspect_extract_file.py"
WorkingDirectory	D:\Kho_Du_lieu\NYCTaxi\script
SuccessValue	0
FailTaskIfReturnCodeIsNotSuccessValue	True
RequireFullFileName	True
StandardOutputVariable	User::vFileMeta
StandardErrorVariable	User::vProcessError
WindowStyle	Hidden
Expression Executable	@[$Package::pPythonExe]
Expression WorkingDirectory	@[$Package::pScriptsRoot]
Expression Arguments	"\"" + @[$Package::pScriptsRoot] + "\\inspect_extract_file.py\" --dataset green --file \"" + @[User::vCurrentFile] + "\" --value ssis --start \"" + @[$Package::pStartMonth] + "\" --end \"" + @[$Package::pEndMonth] + "\" --row-limit " + (DT_WSTR,12)@[$Package::pMaxRows]

EST_Begin_Green_File

```
IF ? = 1 EXEC etl.usp_BeginExtractFile @DatasetCode=N'green', @FilePath=?, @FileHash=?, @ExpectedRows=?, @RowLimit=?; ELSE SELECT CAST(1 AS INT) AS AlreadyLoaded, CAST(0 AS BIGINT) AS RowsLoaded;
```

Bảng A.24 Cấu hình tác vụ EST_Begin_Green_File

Thuộc tính	**Giá trị source**
Connection	{428D3CB7-3BB4-4D96-8F32-4EE97109955B}
BypassPrepare	True
ResultType	ResultSetType_SingleRow

DFT_05_Load_Green

Bảng A.25 Cấu hình component SRC_Green_CSV

Thuộc tính	**Giá trị**
RetainNulls	true
FileNameColumnName	

Bảng A.26 Metadata cột đầu ra SRC_Green_CSV

Cột	**Kiểu bề rộng và xử lý lỗi**
VendorID	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
lpep_pickup_datetime	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
lpep_dropoff_datetime	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
store_and_fwd_flag	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
RatecodeID	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent

Cột	**Kiểu bề rộng và xử lý lỗi**
PULocationID	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
DOLocationID	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
passenger_count	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
trip_distance	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
fare_amount	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
extra	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
mta_tax	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
tip_amount	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
tolls_amount	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
ehail_fee	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
improvement_surcharge	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent

Cột	Kiểu bề rộng và xử lý lỗi
total_amount	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
payment_type	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
trip_type	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
congestion_surcharge	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
cbd_congestion_fee	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
Flat File Source Error Output Column	codePage=65001; dataType=text
ErrorCode	dataType=i4
ErrorColumn	dataType=i4

Bảng A.27 Cấu hình component RC_Green

Thuộc tính	Giá trị
VariableName	User::vRowsLoaded

Bảng A.28 Cấu hình component DST_wrk_GreenTrip

Thuộc tính	Giá trị
CommandTimeout	0
OpenRowset	[wrk].[GreenTrip]
OpenRowsetVariable	

Thuộc tính	Giá trị
SqlCommand	
DefaultCodePage	65001
AlwaysUseDefaultCodePage	true
AccessMode	3
FastLoadKeepIdentity	false
FastLoadKeepNulls	true
FastLoadOptions	TABLOCK,CHECK_CONSTRAINTS,ROWS_PER_BATCH = 20000
FastLoadMaxInsertCommitSize	100000

Bảng A.29 Metadata cột đầu ra DST_wrk_GreenTrip

Cột	Kiểu bề rộng và xử lý lỗi
ErrorCode	dataType=i4
ErrorColumn	dataType=i4

EST_Commit_Green_File

```
EXEC etl.usp_CommitExtractFile @DatasetCode=N'green', @FilePath=?, @FileHash=?,
@ExpectedRows=?, @RowLimit=?, @RowsLoaded=?;
```

Bảng A.30 Cấu hình tác vụ EST_Commit_Green_File

Thuộc tính	Giá trị source
Connection	{428D3CB7-3BB4-4D96-8F32-4EE97109955B}
BypassPrepare	True
ResultType	ResultSetType_None

EST_Mark_Green_Failed

```
UPDATE etl.ExtractFileLog SET LoadStatus='FAILED', CompletedAt=SYSDATETIME(),
ErrorMessage=N'SSIS Data Flow or commit failed. See Execution Results for the original
error.' WHERE DatasetCode='green' AND FilePath=? AND LoadStatus='LOADING';
```

Bảng A.31 Cấu hình tác vụ EST_Mark_Green_Failed

Thuộc tính	Giá trị source
Connection	{428D3CB7-3BB4-4D96-8F32-4EE97109955B}
BypassPrepare	True
ResultType	ResultSetType_None

FLC_06_Load_FHV_Files

EPT_Inspect_FHV_CSV

Bảng A.32 Cấu hình tác vụ EPT_Inspect_FHV_CSV

Thuộc tính	Giá trị source
Executable	C:\Users\ADMIN\AppData\Local\Programs\Python\Python310\python.exe
Arguments	"D:\Kho_Du_lieu\NYCTaxi\script\inspect_extract_file.py"
WorkingDirectory	D:\Kho_Du_lieu\NYCTaxi\script
SuccessValue	0
FailTaskIfReturnCodeIsNotSuccessValue	True
RequireFullFileName	True
StandardOutputVariable	User::vFileMeta
StandardErrorVariable	User::vProcessError
WindowStyle	Hidden
Expression Executable	@[$Package::pPythonExe]
Expression WorkingDirectory	@[$Package::pScriptsRoot]

Thuộc tính	Giá trị source
Expression Arguments	"\"" + @[$Package::pScriptsRoot] + "\\inspect_extract_file.py\" --dataset fhv --file \"" + @[User::vCurrentFile] + "\" --value ssis --start \"" + @[$Package::pStartMonth] + "\" --end \"" + @[$Package::pEndMonth] + "\" --row-limit " + (DT_WSTR,12)@[$Package::pMaxRows]

EST_Begin_FHV_File

```
IF ? = 1 EXEC etl.usp_BeginExtractFile @DatasetCode=N'fhv', @FilePath=?, @FileHash=?, @ExpectedRows=?, @RowLimit=?; ELSE SELECT CAST(1 AS INT) AS AlreadyLoaded, CAST(0 AS BIGINT) AS RowsLoaded;
```

Bảng A.33 Cấu hình tác vụ EST_Begin_FHV_File

Thuộc tính	Giá trị source
Connection	{428D3CB7-3BB4-4D96-8F32-4EE97109955B}
BypassPrepare	True
ResultType	ResultSetType_SingleRow

DFT_06_Load_FHV

Bảng A.34 Cấu hình component SRC_FHV_CSV

Thuộc tính	Giá trị
RetainNulls	true
FileNameColumnName	

Bảng A.35 Metadata cột đầu ra SRC_FHV_CSV

Cột	Kiểu bề rộng và xử lý lỗi
dispatching_base_num	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
pickup_datetime	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent

Cột	Kiểu bề rộng và xử lý lỗi
dropOff_datetime	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
PULocationID	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
DOLocationID	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
SR_Flag	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
Affiliated_base_number	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
Flat File Source Error Output Column	codePage=65001; dataType=text
ErrorCode	dataType=i4
ErrorColumn	dataType=i4

Bảng A.36 Cấu hình component RC_FHV

Thuộc tính	Giá trị
VariableName	User::vRowsLoaded

Bảng A.37 Cấu hình component DST_wrk_FHVTrip

Thuộc tính	Giá trị
CommandTimeout	0
OpenRowset	[wrk].[FHVTrip]
OpenRowsetVariable	

Thuộc tính	Giá trị
SqlCommand	
DefaultCodePage	65001
AlwaysUseDefaultCodePage	true
AccessMode	3
FastLoadKeepIdentity	false
FastLoadKeepNulls	true
FastLoadOptions	TABLOCK,CHECK_CONSTRAINTS,ROWS_PER_BATCH = 20000
FastLoadMaxInsertCommitSize	100000

Bảng A.38 Metadata cột đầu ra DST_wrk_FHVTrip

Cột	Kiểu bề rộng và xử lý lỗi
ErrorCode	dataType=i4
ErrorColumn	dataType=i4

EST_Commit_FHV_File

```
EXEC etl.usp_CommitExtractFile @DatasetCode=N'fhv', @FilePath=?, @FileHash=?,
@ExpectedRows=?, @RowLimit=?, @RowsLoaded=?;
```

Bảng A.39 Cấu hình tác vụ EST_Commit_FHV_File

Thuộc tính	Giá trị source
Connection	{428D3CB7-3BB4-4D96-8F32-4EE97109955B}
BypassPrepare	True
ResultType	ResultSetType_None

EST_Mark_FHV_Failed

```
UPDATE etl.ExtractFileLog SET LoadStatus='FAILED', CompletedAt=SYSDATETIME(),
ErrorMessage=N'SSIS Data Flow or commit failed. See Execution Results for the original
error.' WHERE DatasetCode='fhv' AND FilePath=? AND LoadStatus='LOADING';
```

Bảng A.40 Cấu hình tác vụ EST_Mark_FHV_Failed

Thuộc tính	**Giá trị source**
Connection	{428D3CB7-3BB4-4D96-8F32-4EE97109955B}
BypassPrepare	True
ResultType	ResultSetType_None

FLC_07_Load_FHVHV_Files

EPT_Inspect_FHVHV_CSV

Bảng A.41 Cấu hình tác vụ EPT_Inspect_FHVHV_CSV

Thuộc tính	**Giá trị source**
Executable	C:\Users\ADMIN\AppData\Local\Programs\Python\Python310\python.exe
Arguments	"D:\Kho_Du_lieu\NYCTaxi\script\inspect_extract_file.py"
WorkingDirectory	D:\Kho_Du_lieu\NYCTaxi\script
SuccessValue	0
FailTaskIfReturnCodeIsNotSuccessValue	True
RequireFullFileName	True
StandardOutputVariable	User::vFileMeta
StandardErrorVariable	User::vProcessError
WindowStyle	Hidden
Expression Executable	@[$Package::pPythonExe]
Expression WorkingDirectory	@[$Package::pScriptsRoot]

Thuộc tính	Giá trị source
Expression Arguments	"\"" + @[$Package::pScriptsRoot] + "\\inspect_extract_file.py\" --dataset fhvhv --file \"" + @[User::vCurrentFile] + "\" --value ssis --start \"" + @[$Package::pStartMonth] + "\" --end \"" + @[$Package::pEndMonth] + "\" --row-limit " + (DT_WSTR,12)@[$Package::pMaxRows]

EST_Begin_FHVHV_File

```
IF ? = 1 EXEC etl.usp_BeginExtractFile @DatasetCode=N'fhvhv', @FilePath=?, @FileHash=?, @ExpectedRows=?, @RowLimit=?; ELSE SELECT CAST(1 AS INT) AS AlreadyLoaded, CAST(0 AS BIGINT) AS RowsLoaded;
```

Bảng A.42 Cấu hình tác vụ EST_Begin_FHVHV_File

Thuộc tính	Giá trị source
Connection	{428D3CB7-3BB4-4D96-8F32-4EE97109955B}
BypassPrepare	True
ResultType	ResultSetType_SingleRow

DFT_07_Load_FHVHV

Bảng A.43 Cấu hình component SRC_FHVHV_CSV

Thuộc tính	Giá trị
RetainNulls	true
FileNameColumnName	

Bảng A.44 Metadata cột đầu ra SRC_FHVHV_CSV

Cột	Kiểu bề rộng và xử lý lỗi
hvfhs_license_num	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
dispatching_base_num	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent

Cột	**Kiểu bề rộng và xử lý lỗi**
originating_base_num	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
request_datetime	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
on_scene_datetime	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
pickup_datetime	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
dropoff_datetime	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
PULocationID	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
DOLocationID	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
trip_miles	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
trip_time	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
base_passenger_fare	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
tolls	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent

Cột	Kiểu bề rộng và xử lý lỗi
bcf	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
sales_tax	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
congestion_surcharge	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
airport_fee	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
tips	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
driver_pay	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
shared_request_flag	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
shared_match_flag	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
access_a_ride_flag	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
wav_request_flag	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
wav_match_flag	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent

Cột	Kiểu bề rộng và xử lý lỗi
cbd_congestion_fee	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
Flat File Source Error Output Column	codePage=65001; dataType=text
ErrorCode	dataType=i4
ErrorColumn	dataType=i4

Bảng A.45 Cấu hình component RC_FHVHV

Thuộc tính	Giá trị
VariableName	User::vRowsLoaded

Bảng A.46 Cấu hình component DST_wrk_FHVHVTrip

Thuộc tính	Giá trị
CommandTimeout	0
OpenRowset	[wrk].[FHVHVTrip]
OpenRowsetVariable	
SqlCommand	
DefaultCodePage	65001
AlwaysUseDefaultCodePage	true
AccessMode	3
FastLoadKeepIdentity	false
FastLoadKeepNulls	true
FastLoadOptions	TABLOCK,CHECK_CONSTRAINTS,ROWS_PER_BATCH = 20000

Thuộc tính	Giá trị
FastLoadMaxInsertCommitSize	100000

Bảng A.47 Metadata cột đầu ra DST_wrk_FHVHVTrip

Cột	Kiểu bề rộng và xử lý lỗi
ErrorCode	dataType=i4
ErrorColumn	dataType=i4

EST_Commit_FHVHV_File

```
EXEC etl.usp_CommitExtractFile @DatasetCode=N'fhvhv', @FilePath=?, @FileHash=?,
@ExpectedRows=?, @RowLimit=?, @RowsLoaded=?;
```

Bảng A.48 Cấu hình tác vụ EST_Commit_FHVHV_File

Thuộc tính	Giá trị source
Connection	{428D3CB7-3BB4-4D96-8F32-4EE97109955B}
BypassPrepare	True
ResultType	ResultSetType_None

EST_Mark_FHVHV_Failed

```
UPDATE etl.ExtractFileLog SET LoadStatus='FAILED', CompletedAt=SYSDATETIME(),
ErrorMessage=N'SSIS Data Flow or commit failed. See Execution Results for the original
error.' WHERE DatasetCode='fhvhv' AND FilePath=? AND LoadStatus='LOADING';
```

Bảng A.49 Cấu hình tác vụ EST_Mark_FHVHV_Failed

Thuộc tính	Giá trị source
Connection	{428D3CB7-3BB4-4D96-8F32-4EE97109955B}
BypassPrepare	True
ResultType	ResultSetType_None

FLC_08_Load_TaxiZone_Files

EPT_Inspect_TaxiZone_CSV

Bảng A.50 Cấu hình tác vụ EPT_Inspect_TaxiZone_CSV

Thuộc tính	**Giá trị source**
Executable	C:\Users\ADMIN\AppData\Local\Programs\Python\Python310\python.exe
Arguments	"D:\Kho_Du_lieu\NYCTaxi\script\inspect_extract_file.py"
WorkingDirectory	D:\Kho_Du_lieu\NYCTaxi\script
SuccessValue	0
FailTaskIfReturnCodeIsNotSuccessValue	True
RequireFullFileName	True
StandardOutputVariable	User::vFileMeta
StandardErrorVariable	User::vProcessError
WindowStyle	Hidden
Expression Executable	@[$Package::pPythonExe]
Expression WorkingDirectory	@[$Package::pScriptsRoot]
Expression Arguments	"\"" + @[$Package::pScriptsRoot] + "\\inspect_extract_file.py\" --dataset lookup --file \"" + @[User::vCurrentFile] + "\" --value ssis --start \"" + @[$Package::pStartMonth] + "\" --end \"" + @[$Package::pEndMonth] + "\" --row-limit " + (DT_WSTR,12)@[$Package::pMaxRows]

EST_Begin_TaxiZone_File

```
IF ? = 1 EXEC etl.usp_BeginExtractFile @DatasetCode=N'lookup', @FilePath=?, @FileHash=?, @ExpectedRows=?, @RowLimit=?; ELSE SELECT CAST(1 AS INT) AS AlreadyLoaded, CAST(0 AS BIGINT) AS RowsLoaded;
```

Bảng A.51 Cấu hình tác vụ EST_Begin_TaxiZone_File

Thuộc tính	**Giá trị source**
Connection	{428D3CB7-3BB4-4D96-8F32-4EE97109955B}
BypassPrepare	True
ResultType	ResultSetType_SingleRow

DFT_08_Load_TaxiZone

Bảng A.52 Cấu hình component SRC_TaxiZone_CSV

Thuộc tính	**Giá trị**
RetainNulls	true
FileNameColumnName	

Bảng A.53 Metadata cột đầu ra SRC_TaxiZone_CSV

Cột	**Kiểu bề rộng và xử lý lỗi**
LocationID	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=50; truncationRowDisposition=FailComponent
Borough	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=255; truncationRowDisposition=FailComponent
Zone	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=255; truncationRowDisposition=FailComponent
service_zone	codePage=65001; dataType=str; errorRowDisposition=FailComponent; length=255; truncationRowDisposition=FailComponent
Flat File Source Error Output Column	codePage=65001; dataType=text
ErrorCode	dataType=i4

Cột	Kiểu bề rộng và xử lý lỗi
ErrorColumn	dataType=i4

Bảng A.54 Cấu hình component RC_TaxiZone

Thuộc tính	Giá trị
VariableName	User::vRowsLoaded

Bảng A.55 Cấu hình component DST_wrk_TaxiZone

Thuộc tính	Giá trị
CommandTimeout	0
OpenRowset	[wrk].[TaxiZone]
OpenRowsetVariable	
SqlCommand	
DefaultCodePage	65001
AlwaysUseDefaultCodePage	true
AccessMode	3
FastLoadKeepIdentity	false
FastLoadKeepNulls	true
FastLoadOptions	TABLOCK,CHECK_CONSTRAINTS,ROWS_PER_BATCH = 20000
FastLoadMaxInsertCommitSize	100000

Bảng A.56 Metadata cột đầu ra DST_wrk_TaxiZone

Cột	Kiểu bề rộng và xử lý lỗi
ErrorCode	dataType=i4

Cột	Kiểu bề rộng và xử lý lỗi
ErrorColumn	dataType=i4

EST_Commit_TaxiZone_File

```
EXEC etl.usp_CommitExtractFile @DatasetCode=N'lookup', @FilePath=?, @FileHash=?,
@ExpectedRows=?, @RowLimit=?, @RowsLoaded=?;
```

Bảng A.57 Cấu hình tác vụ EST_Commit_TaxiZone_File

Thuộc tính	Giá trị source
Connection	{428D3CB7-3BB4-4D96-8F32-4EE97109955B}
BypassPrepare	True
ResultType	ResultSetType_None

EST_Mark_TaxiZone_Failed

```
UPDATE etl.ExtractFileLog SET LoadStatus='FAILED', CompletedAt=SYSDATETIME(),
ErrorMessage=N'SSIS Data Flow or commit failed. See Execution Results for the original
error.' WHERE DatasetCode='lookup' AND FilePath=? AND LoadStatus='LOADING';
```

Bảng A.58 Cấu hình tác vụ EST_Mark_TaxiZone_Failed

Thuộc tính	Giá trị source
Connection	{428D3CB7-3BB4-4D96-8F32-4EE97109955B}
BypassPrepare	True
ResultType	ResultSetType_None

EST_09_Validate_Staging

```
SET NOCOUNT ON;
DECLARE @StartMonth DATE = CONVERT(DATE, ? + '-01', 23),
        @EndMonth DATE = CONVERT(DATE, ? + '-01', 23), @RowLimit INT = ?;
IF @StartMonth > @EndMonth OR @RowLimit < 0 THROW 51010, 'Invalid extraction range.', 1;
DECLARE @Month DATE = @StartMonth;
DECLARE @Expected TABLE (DatasetCode VARCHAR(20), SourceKey NVARCHAR(260), RowLimit INT);
WHILE @Month <= @EndMonth
BEGIN
    INSERT @Expected (DatasetCode, SourceKey, RowLimit)
    SELECT DatasetCode, DatasetCode + '_tripdata_' + CONVERT(CHAR(7), @Month, 126) +
'.csv', @RowLimit
    FROM (VALUES ('yellow',CONVERT(date,'20090101')), ('green',CONVERT(date,'20130801')),
        ('fhv',CONVERT(date,'20150101')), ('fhvhv',CONVERT(date,'20190201'))) AS
D(DatasetCode,FirstMonth)
    WHERE @Month >= D.FirstMonth;
    IF @Month = @EndMonth BREAK;
    SET @Month = DATEADD(MONTH, 1, @Month);
END;
INSERT @Expected VALUES ('lookup', 'taxi_zone_lookup.csv', 0);
IF EXISTS (
    SELECT 1 FROM @Expected AS E LEFT JOIN etl.ExtractFileLog AS L
```

```
        ON L.DatasetCode = E.DatasetCode AND L.SourceKey = E.SourceKey
    WHERE L.SourceKey IS NULL OR L.LoadStatus <> 'SUCCEEDED'
        OR L.RowLimit <> E.RowLimit OR L.RowsLoaded IS NULL OR L.RowsLoaded <>
L.ExpectedRows)
    THROW 51007, 'File load coverage is incomplete. Check etl.ExtractFileLog.', 1;
```

Bảng A.59 Cấu hình tác vụ EST_09_Validate_Staging

Thuộc tính	**Giá trị source**
Connection	{428D3CB7-3BB4-4D96-8F32-4EE97109955B}
BypassPrepare	True
ResultType	ResultSetType_None

EST_LogOriginalError

```
INSERT etl.PackageError(ExecutionId,PackageName,SourceName,ErrorCode,ErrorDescription)
VALUES(?,?,?,?,?);
```

Bảng A.60 Cấu hình tác vụ EST_LogOriginalError

Thuộc tính	**Giá trị source**
Connection	{4A829E7F-DE39-4A39-83D3-52CE94E192E2}

Bảng A.61 Precedence constraint PKG_01_Extract_Stage

Từ	**Đến**	**Điều kiện**
Package\FLC_04_Load_Yellow_Files\EPT_Inspect_Yellow_CSV	Package\FLC_04_Load_Yellow_Files\EST_Begin_Yellow_File	Value=0; LogicalAnd=True
Package\FLC_04_Load_Yellow_Files\EST_Begin_Yellow_File	Package\FLC_04_Load_Yellow_Files\DFT_04_Load_Yellow	Value=0; LogicalAnd=True; EvalOp=3; Expression=@[User::vFileAlreadyLoaded] == 0
Package\FLC_04_Load_Yellow_Files\DFT_04_Load_Yellow	Package\FLC_04_Load_Yellow_Files\EST_Commit_Yellow_File	Value=0; LogicalAnd=True
Package\FLC_04_Load_Yellow_Files\DFT_04_Load_Yellow	Package\FLC_04_Load_Yellow_Files\EST_Mark_Yellow_Failed	Value=1; LogicalAnd=False

Từ	Đến	Điều kiện
Package\FLC_04_Load_Yellow_Files\EST_Commit_Yellow_File	Package\FLC_04_Load_Yellow_Files\EST_Mark_Yellow_Failed	Value=1; LogicalAnd=False
Package\FLC_05_Load_Green_Files\EPT_Inspect_Green_CSV	Package\FLC_05_Load_Green_Files\EST_Begin_Green_File	Value=0; LogicalAnd=True
Package\FLC_05_Load_Green_Files\EST_Begin_Green_File	Package\FLC_05_Load_Green_Files\DFT_05_Load_Green	Value=0; LogicalAnd=True; EvalOp=3; Expression=@[User::vFileAlreadyLoaded] == 0
Package\FLC_05_Load_Green_Files\DFT_05_Load_Green	Package\FLC_05_Load_Green_Files\EST_Commit_Green_File	Value=0; LogicalAnd=True
Package\FLC_05_Load_Green_Files\DFT_05_Load_Green	Package\FLC_05_Load_Green_Files\EST_Mark_Green_Failed	Value=1; LogicalAnd=False
Package\FLC_05_Load_Green_Files\EST_Commit_Green_File	Package\FLC_05_Load_Green_Files\EST_Mark_Green_Failed	Value=1; LogicalAnd=False
Package\FLC_06_Load_FHV_Files\EPT_Inspect_FHV_CSV	Package\FLC_06_Load_FHV_Files\EST_Begin_FHV_File	Value=0; LogicalAnd=True
Package\FLC_06_Load_FHV_Files\EST_Begin_FHV_File	Package\FLC_06_Load_FHV_Files\DFT_06_Load_FHV	Value=0; LogicalAnd=True; EvalOp=3; Expression=@[User::vFileAlreadyLoaded] == 0
Package\FLC_06_Load_FHV_Files\DFT_06_Load_FHV	Package\FLC_06_Load_FHV_Files\EST_Commit_FHV_File	Value=0; LogicalAnd=True
Package\FLC_06_Load_FHV_Files\DFT_06_Load_FHV	Package\FLC_06_Load_FHV_Files\EST_Mark_FHV_Failed	Value=1; LogicalAnd=False

Từ	Đến	Điều kiện
Package\FLC_06_Load_FHV_Files\EST_Commit_FHV_File	Package\FLC_06_Load_FHV_Files\EST_Mark_FHV_Failed	Value=1; LogicalAnd=False
Package\FLC_07_Load_FHVHV_Files\EPT_Inspect_FHVHV_CSV	Package\FLC_07_Load_FHVHV_Files\EST_Begin_FHVHV_File	Value=0; LogicalAnd=True
Package\FLC_07_Load_FHVHV_Files\EST_Begin_FHVHV_File	Package\FLC_07_Load_FHVHV_Files\DFT_07_Load_FHVHV	Value=0; LogicalAnd=True; EvalOp=3; Expression=@[User::vFileAlreadyLoaded] == 0
Package\FLC_07_Load_FHVHV_Files\DFT_07_Load_FHVHV	Package\FLC_07_Load_FHVHV_Files\EST_Commit_FHVHV_File	Value=0; LogicalAnd=True
Package\FLC_07_Load_FHVHV_Files\DFT_07_Load_FHVHV	Package\FLC_07_Load_FHVHV_Files\EST_Mark_FHVHV_Failed	Value=1; LogicalAnd=False
Package\FLC_07_Load_FHVHV_Files\EST_Commit_FHVHV_File	Package\FLC_07_Load_FHVHV_Files\EST_Mark_FHVHV_Failed	Value=1; LogicalAnd=False
Package\FLC_08_Load_TaxiZone_Files\EPT_Inspect_TaxiZone_CSV	Package\FLC_08_Load_TaxiZone_Files\EST_Begin_TaxiZone_File	Value=0; LogicalAnd=True
Package\FLC_08_Load_TaxiZone_Files\EST_Begin_TaxiZone_File	Package\FLC_08_Load_TaxiZone_Files\DFT_08_Load_TaxiZone	Value=0; LogicalAnd=True; EvalOp=3; Expression=@[User::vFileAlreadyLoaded] == 0
Package\FLC_08_Load_TaxiZone_Files\DFT_08_Load_TaxiZone	Package\FLC_08_Load_TaxiZone_Files\EST_Commit_TaxiZone_File	Value=0; LogicalAnd=True
Package\FLC_08_Load_TaxiZone_Files\DFT_08_Load_TaxiZone	Package\FLC_08_Load_TaxiZone_Files\EST_Mark_TaxiZone_Failed	Value=1; LogicalAnd=False

Từ	Đến	Điều kiện
Package\FLC_08_Load_TaxiZone_Files\EST_Commit_TaxiZone_File	Package\FLC_08_Load_TaxiZone_Files\EST_Mark_TaxiZone_Failed	Value=1; LogicalAnd=False
Package\EPT_01_Download_Parquet	Package\EPT_02_Convert_To_CSV	Value=0; LogicalAnd=True
Package\EPT_02_Convert_To_CSV	Package\EST_03_Prepare_Staging	Value=0; LogicalAnd=True
Package\EST_03_Prepare_Staging	Package\FLC_04_Load_Yellow_Files	Value=0; LogicalAnd=True
Package\FLC_04_Load_Yellow_Files	Package\FLC_05_Load_Green_Files	Value=0; LogicalAnd=True
Package\FLC_05_Load_Green_Files	Package\FLC_06_Load_FHV_Files	Value=0; LogicalAnd=True
Package\FLC_06_Load_FHV_Files	Package\FLC_07_Load_FHVHV_Files	Value=0; LogicalAnd=True
Package\FLC_07_Load_FHVHV_Files	Package\FLC_08_Load_TaxiZone_Files	Value=0; LogicalAnd=True
Package\FLC_08_Load_TaxiZone_Files	Package\EST_09_Validate_Staging	Value=0; LogicalAnd=True

PKG_02_Transform_Clean

EST_01_Setup_Clean

```
SET NOCOUNT ON;
IF DB_NAME() <> 'NYCTaxi_Staging'
    THROW 51000, 'Chon connection NYCTaxi_Staging.', 1;
IF OBJECT_ID('stg.YellowTrip') IS NULL OR OBJECT_ID('stg.GreenTrip') IS NULL
    OR OBJECT_ID('stg.FHVTrip') IS NULL OR OBJECT_ID('stg.FHVHVTrip') IS NULL
    OR OBJECT_ID('stg.TaxiZone') IS NULL
    THROW 51001, 'Chay PKG_01 de tao va nap staging truoc.', 1;
IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name = 'clean')
    EXEC('CREATE SCHEMA clean AUTHORIZATION dbo');

/* ---------------------- 1. BANG CLEAN ---------------------- */

IF OBJECT_ID('clean.TaxiZone', 'U') IS NULL
BEGIN
    CREATE TABLE clean.TaxiZone
    (
        LocationID   INT           NOT NULL,
        Borough      NVARCHAR(100) NULL,
```

```
        Zone         NVARCHAR(200) NULL,
        ServiceZone  NVARCHAR(100) NULL,
        CONSTRAINT PK_CleanTaxiZone PRIMARY KEY (LocationID)
    );
END;

IF OBJECT_ID('clean.Trip', 'U') IS NULL
BEGIN
    CREATE TABLE clean.Trip
    (
        TripCleanID             BIGINT IDENTITY(1,1) NOT NULL,
        TaxiType                VARCHAR(10) NOT NULL,

        VendorID                INT NULL,
        HVFHSLicenseNum         VARCHAR(30) NULL,
        DispatchingBaseNum      VARCHAR(30) NULL,
        OriginatingBaseNum      VARCHAR(30) NULL,
        AffiliatedBaseNum       VARCHAR(30) NULL,

        RequestDateTime         DATETIME2(0) NULL,
        OnSceneDateTime         DATETIME2(0) NULL,
        PickupDateTime          DATETIME2(0) NULL,
        DropoffDateTime         DATETIME2(0) NULL,
        TripDurationSeconds     BIGINT NULL,

        PULocationID            INT NULL,
        DOLocationID            INT NULL,
        PassengerCount          INT NULL,
        TripDistanceMiles       DECIMAL(12,3) NULL,

        RateCodeID              INT NULL,
        PaymentTypeID           INT NULL,
        TripTypeID              INT NULL,
        StoreAndFwdFlag         VARCHAR(5) NULL,

        FareAmount              DECIMAL(18,2) NULL,
        ExtraAmount             DECIMAL(18,2) NULL,
        MTATaxAmount            DECIMAL(18,2) NULL,
        TipAmount               DECIMAL(18,2) NULL,
        TollsAmount             DECIMAL(18,2) NULL,
        ImprovementSurcharge    DECIMAL(18,2) NULL,
        CongestionSurcharge     DECIMAL(18,2) NULL,
        AirportFee              DECIMAL(18,2) NULL,
        CBDCongestionFee        DECIMAL(18,2) NULL,
        BCFAmount               DECIMAL(18,2) NULL,
        SalesTaxAmount          DECIMAL(18,2) NULL,
        DriverPayAmount         DECIMAL(18,2) NULL,
        TotalAmount             DECIMAL(18,2) NULL,
        IsTotalCalculated       BIT NOT NULL,

        SharedRequestFlag       VARCHAR(5) NULL,
        SharedMatchFlag         VARCHAR(5) NULL,
        AccessARideFlag         VARCHAR(5) NULL,
        WAVRequestFlag          VARCHAR(5) NULL,
        WAVMatchFlag            VARCHAR(5) NULL,

        IsCoreValid             BIT NOT NULL,
        QualityIssue            VARCHAR(2000) NULL,
        CleanedAt               DATETIME2(0) NOT NULL
            CONSTRAINT DF_CleanTrip_CleanedAt DEFAULT SYSDATETIME(),

        CONSTRAINT PK_CleanTrip PRIMARY KEY CLUSTERED (TripCleanID),
        CONSTRAINT CK_CleanTrip_TaxiType
            CHECK (TaxiType IN ('YELLOW', 'GREEN', 'FHV', 'FHVHV'))
```

```
    );

    CREATE INDEX IX_CleanTrip_PickupDateTime
        ON clean.Trip (PickupDateTime, TaxiType);

    CREATE INDEX IX_CleanTrip_Location
        ON clean.Trip (PULocationID, DOLocationID);
END;

IF COL_LENGTH('clean.Trip', 'ETLSourceKey') IS NULL
    ALTER TABLE clean.Trip ADD [ETLSourceKey] NVARCHAR(260) NULL;
IF COL_LENGTH('clean.Trip', 'ETLFileHash') IS NULL
    ALTER TABLE clean.Trip ADD [ETLFileHash] CHAR(64) NULL;
IF COL_LENGTH('clean.Trip', 'ETLLoadedAt') IS NULL
    ALTER TABLE clean.Trip ADD [ETLLoadedAt] DATETIME2(0) NULL;
IF COL_LENGTH('clean.Trip', 'TripDistanceKm') IS NULL
    ALTER TABLE clean.Trip ADD [TripDistanceKm] DECIMAL(14,3) NULL;
IF COL_LENGTH('clean.Trip', 'AverageSpeedKmh') IS NULL
    ALTER TABLE clean.Trip ADD [AverageSpeedKmh] DECIMAL(14,3) NULL;
IF COL_LENGTH('clean.Trip', 'SourceTripDurationSeconds') IS NULL
    ALTER TABLE clean.Trip ADD [SourceTripDurationSeconds] BIGINT NULL;
IF COL_LENGTH('clean.Trip', 'RequestWaitSeconds') IS NULL
    ALTER TABLE clean.Trip ADD [RequestWaitSeconds] BIGINT NULL;
IF COL_LENGTH('clean.Trip', 'SourceSRFlag') IS NULL
    ALTER TABLE clean.Trip ADD [SourceSRFlag] VARCHAR(5) NULL;
IF COL_LENGTH('clean.Trip', 'EHailFeeAmount') IS NULL
    ALTER TABLE clean.Trip ADD [EHailFeeAmount] DECIMAL(18,2) NULL;
IF COL_LENGTH('clean.Trip', 'QualityIssue') < 2000
    ALTER TABLE clean.Trip ALTER COLUMN QualityIssue VARCHAR(2000) NULL;

/* Hai view nay duoc dung o buoc Load. */
EXEC('CREATE OR ALTER VIEW clean.ValidTrip AS
      SELECT * FROM clean.Trip WHERE IsCoreValid = 1;');
EXEC('CREATE OR ALTER VIEW clean.RejectedTrip AS
      SELECT * FROM clean.Trip WHERE IsCoreValid = 0;');
EXEC('CREATE OR ALTER VIEW clean.WarningTrip AS
      SELECT * FROM clean.Trip WHERE IsCoreValid = 1 AND QualityIssue IS NOT NULL;');
EXEC('CREATE OR ALTER VIEW clean.RevenueTrip AS
      SELECT * FROM clean.Trip WHERE IsCoreValid = 1 AND TotalAmount IS NOT NULL;');
EXEC('CREATE OR ALTER VIEW clean.TransformSummary AS
      SELECT TaxiType, COUNT_BIG(*) AS TotalRows,
          COUNT_BIG(CASE WHEN IsCoreValid=1 THEN 1 END) AS ValidRows,
          COUNT_BIG(CASE WHEN IsCoreValid=0 THEN 1 END) AS RejectedRows,
          COUNT_BIG(CASE WHEN IsCoreValid=1 AND QualityIssue IS NOT NULL THEN 1 END) AS
WarningRows,
          COUNT_BIG(CASE WHEN IsCoreValid=1 AND TotalAmount IS NOT NULL THEN 1 END) AS
RevenueAvailableRows
      FROM clean.Trip GROUP BY TaxiType;');



/* PKG_01 moi luu lineage cho tung tep. */
IF OBJECT_ID('etl.ExtractFileLog', 'U') IS NULL
    THROW 51200, 'Can etl.ExtractFileLog tu PKG_01 moi.', 1;
IF COL_LENGTH('stg.YellowTrip','ETLSourceKey') IS NULL
    OR COL_LENGTH('stg.GreenTrip','ETLSourceKey') IS NULL
    OR COL_LENGTH('stg.FHVTrip','ETLSourceKey') IS NULL
    OR COL_LENGTH('stg.FHVHVTrip','ETLSourceKey') IS NULL
    OR COL_LENGTH('stg.TaxiZone','ETLSourceKey') IS NULL
    THROW 51201, 'Staging thieu ETLSourceKey. Dung PKG_01 moi de nap staging.', 1;

/* Dung lai dung cau truc log da tao o ban Transform truoc. */
IF OBJECT_ID('etl.TransformFileLog', 'U') IS NULL
    CREATE TABLE etl.TransformFileLog
    (
        DatasetCode VARCHAR(20) NOT NULL,
```

```
        SourceKey NVARCHAR(260) NOT NULL,
        FileHash CHAR(64) NOT NULL,
        RuleVersion VARCHAR(64) NOT NULL,
        ZoneHash CHAR(64) NOT NULL,
        SourceRows BIGINT NOT NULL,
        CleanRows BIGINT NOT NULL,
        ValidRows BIGINT NOT NULL,
        RejectedRows BIGINT NOT NULL,
        WarningRows BIGINT NOT NULL,
        CompletedAt DATETIME2(0) NOT NULL,
        CONSTRAINT PK_TransformFileLog PRIMARY KEY(DatasetCode,SourceKey),
        CONSTRAINT CK_TransformFileLog_Counts CHECK
            (SourceRows=CleanRows AND CleanRows=ValidRows+RejectedRows)
    );

IF COL_LENGTH('etl.TransformFileLog','RuleVersion') IS NULL
    OR COL_LENGTH('etl.TransformFileLog','ZoneHash') IS NULL
    OR COL_LENGTH('etl.TransformFileLog','SourceRows') IS NULL
    OR COL_LENGTH('etl.TransformFileLog','CleanRows') IS NULL
    THROW 51202, 'Cau truc etl.TransformFileLog cu khong tuong thich.', 1;

/* Dem/tra cuu theo tep bang index gon, tranh doc cac cot do luong cu. */
IF NOT EXISTS(SELECT 1 FROM sys.indexes
    WHERE object_id=OBJECT_ID('clean.Trip') AND name='IX_CleanTrip_Source')
    EXEC('CREATE INDEX IX_CleanTrip_Source ON clean.Trip(TaxiType,ETLSourceKey)
          INCLUDE(ETLFileHash);');
```

Bảng A.62 Cấu hình tác vụ EST_01_Setup_Clean

Thuộc tính	**Giá trị source**
Connection	{428D3CB7-3BB4-4D96-8F32-4EE97109955B}

EST_02_Transform_TaxiZone

```
SET NOCOUNT ON;
SET XACT_ABORT ON;
DECLARE @FileHash CHAR(64), @ExpectedRows BIGINT, @CleanRows BIGINT,
        @ZoneText NVARCHAR(MAX), @ZoneHash CHAR(64), @Processed INT=0;
BEGIN TRY
    BEGIN TRANSACTION;

    SELECT @FileHash=FileHash, @ExpectedRows=ExpectedRows
    FROM etl.ExtractFileLog WITH(UPDLOCK,HOLDLOCK)
    WHERE DatasetCode='lookup' AND SourceKey='taxi_zone_lookup.csv'
        AND LoadStatus='SUCCEEDED' AND RowsLoaded=ExpectedRows;
    IF @ExpectedRows IS NULL OR @ExpectedRows=0
        THROW 51210, 'Extract lookup chua hoan tat.', 1;

    SELECT @CleanRows=COUNT_BIG(*) FROM clean.TaxiZone WITH(HOLDLOCK);
SELECT @ZoneText=CONVERT(NVARCHAR(MAX),
    (SELECT LocationID,Borough,Zone,ServiceZone
     FROM clean.TaxiZone WITH(HOLDLOCK)
     ORDER BY LocationID FOR XML RAW('z'),TYPE));
SET @ZoneHash=CONVERT(CHAR(64),HASHBYTES('SHA2_256',@ZoneText),2);

    IF NOT EXISTS(SELECT 1 FROM etl.TransformFileLog WITH(UPDLOCK,HOLDLOCK)
        WHERE DatasetCode='lookup' AND SourceKey='taxi_zone_lookup.csv'
            AND FileHash=@FileHash AND SourceRows=@ExpectedRows
            AND CleanRows=@CleanRows AND CleanRows=@ExpectedRows
            AND RuleVersion='INCR_ZONE_20261019_1' AND ZoneHash=@ZoneHash)
    BEGIN
        /* Chi doc staging lookup neu can xu ly lai. */
        IF (SELECT COUNT_BIG(*) FROM stg.TaxiZone WITH(HOLDLOCK))<>@ExpectedRows
```

```
            OR EXISTS(SELECT 1 FROM stg.TaxiZone WITH(HOLDLOCK)
                WHERE ETLSourceKey IS NULL OR ETLSourceKey<>'taxi_zone_lookup.csv'
                    OR ETLFileHash IS NULL OR ETLFileHash<>@FileHash)
            THROW 51211, 'Staging lookup khong khop log Extract. Chay lai PKG_01.', 1;

DELETE FROM clean.TaxiZone;

;WITH Z AS
(
    SELECT
        TRY_CONVERT(INT, NULLIF(LTRIM(RTRIM(LocationID)), '')) AS LocationID,
        NULLIF(LTRIM(RTRIM(Borough)), '') AS Borough,
        NULLIF(LTRIM(RTRIM(Zone)), '') AS Zone,
        NULLIF(LTRIM(RTRIM(REPLACE(service_zone, CHAR(13), ''))), '') AS ServiceZone
    FROM stg.TaxiZone
), R AS
(
    SELECT *, ROW_NUMBER() OVER
        (PARTITION BY LocationID ORDER BY Borough, Zone, ServiceZone) AS RowNum
    FROM Z
    WHERE LocationID > 0 AND Zone IS NOT NULL
)
INSERT INTO clean.TaxiZone(LocationID, Borough, Zone, ServiceZone)
SELECT LocationID, Borough, Zone, ServiceZone
FROM R
WHERE RowNum = 1;

IF NOT EXISTS(SELECT 1 FROM clean.TaxiZone)
    THROW 51004, 'Khong co TaxiZone hop le; rollback lookup.', 1;




        SELECT @CleanRows=COUNT_BIG(*) FROM clean.TaxiZone;
        IF @CleanRows<>@ExpectedRows
            THROW 51212, 'Lookup co ID loi/trung. Giu lookup cu va kiem tra staging.', 1;
SELECT @ZoneText=CONVERT(NVARCHAR(MAX),
    (SELECT LocationID,Borough,Zone,ServiceZone
     FROM clean.TaxiZone WITH(HOLDLOCK)
     ORDER BY LocationID FOR XML RAW('z'),TYPE));
SET @ZoneHash=CONVERT(CHAR(64),HASHBYTES('SHA2_256',@ZoneText),2);
        DELETE FROM etl.TransformFileLog
        WHERE DatasetCode='lookup' AND SourceKey='taxi_zone_lookup.csv';
        INSERT etl.TransformFileLog
            (DatasetCode,SourceKey,FileHash,RuleVersion,ZoneHash,SourceRows,
             CleanRows,ValidRows,RejectedRows,WarningRows,CompletedAt)
        VALUES('lookup','taxi_zone_lookup.csv',@FileHash,'INCR_ZONE_20261019_1',@ZoneHash,
            @ExpectedRows,@CleanRows,@CleanRows,0,0,SYSDATETIME());
        SET @Processed=1;
    END;
    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT>0 ROLLBACK TRANSACTION;
    THROW;
END CATCH;
SELECT 'lookup' AS DatasetCode,@Processed AS ProcessedFiles,1-@Processed AS SkippedFiles;
```

Bảng A.63 Cấu hình tác vụ EST_02_Transform_TaxiZone

Thuộc tính	**Giá trị source**
Connection	{428D3CB7-3BB4-4D96-8F32-4EE97109955B}

EST_03_Transform_YELLOW

```
SET NOCOUNT ON;
SET XACT_ABORT ON;
DECLARE @Dataset VARCHAR(20)='yellow', @TaxiType VARCHAR(10)='YELLOW',
        @RuleVersion VARCHAR(64)='INCR_20261019_1|hours=48', @MaxDurationHours INT=48,
        @ZoneText NVARCHAR(MAX), @ZoneHash CHAR(64);


DROP TABLE IF EXISTS #Files;
DROP TABLE IF EXISTS #CleanCounts;
DROP TABLE IF EXISTS #Todo;
DROP TABLE IF EXISTS #StageCounts;
DROP TABLE IF EXISTS #Result;


BEGIN TRY
    BEGIN TRANSACTION;
    IF NOT EXISTS(SELECT 1 FROM clean.TaxiZone WITH(HOLDLOCK))
        THROW 51220, 'Chay task lookup truoc.', 1;
    IF EXISTS(SELECT 1 FROM clean.Trip WITH(UPDLOCK,HOLDLOCK)
        WHERE TaxiType=@TaxiType AND ETLSourceKey IS NULL)
        THROW 51221, 'Clean cu thieu ETLSourceKey; can chuan hoa lineage truoc.', 1;
    IF EXISTS(SELECT 1 FROM stg.YellowTrip WITH(HOLDLOCK) WHERE ETLSourceKey IS NULL)
        THROW 51222, 'Staging thieu lineage. Dung PKG_01 moi.', 1;


SELECT @ZoneText=CONVERT(NVARCHAR(MAX),
    (SELECT LocationID,Borough,Zone,ServiceZone
     FROM clean.TaxiZone WITH(HOLDLOCK)
     ORDER BY LocationID FOR XML RAW('z'),TYPE));
SET @ZoneHash=CONVERT(CHAR(64),HASHBYTES('SHA2_256',@ZoneText),2);


    /* 1. Cac tep Extract da nap thanh cong. Khong chuyen kieu du lieu o buoc nay. */
    SELECT SourceKey,FileHash,ExpectedRows
    INTO #Files
    FROM etl.ExtractFileLog WITH(UPDLOCK,HOLDLOCK)
    WHERE DatasetCode=@Dataset AND LoadStatus='SUCCEEDED' AND RowsLoaded=ExpectedRows;


    /* 2. Kiem tra so dong/hash clean thuc te bang index theo tep. */
    SELECT t.ETLSourceKey AS SourceKey,COUNT_BIG(*) AS CleanRows,
        COUNT_BIG(CASE WHEN t.ETLFileHash IS NULL OR t.ETLFileHash<>f.FileHash THEN 1 END)
AS BadHashRows
    INTO #CleanCounts
    FROM clean.Trip AS t WITH(HOLDLOCK)
    INNER JOIN #Files AS f ON f.SourceKey=t.ETLSourceKey
    WHERE t.TaxiType=@TaxiType
    GROUP BY t.ETLSourceKey;


    /* 3. Chi chon tep chua lam, tep thay doi, hoac clean bi thieu. */
SELECT f.SourceKey,f.FileHash,f.ExpectedRows
INTO #Todo
FROM #Files AS f
LEFT JOIN #CleanCounts AS c ON c.SourceKey=f.SourceKey
LEFT JOIN etl.TransformFileLog AS l WITH(UPDLOCK,HOLDLOCK)
    ON l.DatasetCode=@Dataset AND l.SourceKey=f.SourceKey
WHERE l.SourceKey IS NULL
    OR l.FileHash<>f.FileHash
    OR l.SourceRows<>f.ExpectedRows OR l.CleanRows<>f.ExpectedRows
    OR l.RuleVersion<>@RuleVersion OR l.ZoneHash<>@ZoneHash
    OR COALESCE(c.CleanRows,0)<>f.ExpectedRows
    OR COALESCE(c.BadHashRows,0)>0;
    CREATE UNIQUE CLUSTERED INDEX IX_Todo_Source ON #Todo(SourceKey);


    IF EXISTS(SELECT 1 FROM #Todo)
    BEGIN
    /* 4. Kiem tra staging cua DUNG cac tep can xu ly truoc khi sua clean. */
```

```
    SELECT s.ETLSourceKey AS SourceKey,COUNT_BIG(*) AS SourceRows,
        COUNT_BIG(CASE WHEN s.ETLFileHash IS NULL OR s.ETLFileHash<>f.FileHash THEN 1 END)
AS BadHashRows
    INTO #StageCounts
    FROM stg.YellowTrip AS s WITH(HOLDLOCK)
    INNER JOIN #Todo AS f ON f.SourceKey=s.ETLSourceKey
    GROUP BY s.ETLSourceKey;


    IF EXISTS(SELECT 1 FROM #Todo AS f LEFT JOIN #StageCounts AS s ON
s.SourceKey=f.SourceKey
        WHERE COALESCE(s.SourceRows,0)<>f.ExpectedRows OR COALESCE(s.BadHashRows,0)>0)
        THROW 51223, 'Staging cua tep can Transform khong khop log. Chay lai PKG_01.', 1;


    /* 5. Giu nguyen cac tep da hoan tat; chi thay the cac tep trong #Todo. */
    DELETE t FROM clean.Trip AS t
    INNER JOIN #Todo AS f ON f.SourceKey=t.ETLSourceKey
    WHERE t.TaxiType=@TaxiType;


;WITH S AS
(
    SELECT
        TRY_CONVERT(DATETIME2(0), NULLIF(LTRIM(RTRIM([tpep_pickup_datetime])), '')) AS
PickupDateTime,
        TRY_CONVERT(DATETIME2(0), NULLIF(LTRIM(RTRIM([tpep_dropoff_datetime])), '')) AS
DropoffDateTime,
        TRY_CONVERT(DECIMAL(38,18), NULLIF(LTRIM(RTRIM([PULocationID])), '')) AS
PULocationID,
        TRY_CONVERT(DECIMAL(38,18), NULLIF(LTRIM(RTRIM([DOLocationID])), '')) AS
DOLocationID,
        TRY_CONVERT(DECIMAL(38,18), NULLIF(LTRIM(RTRIM([VendorID])), '')) AS VendorID,
        TRY_CONVERT(DECIMAL(38,18), NULLIF(LTRIM(RTRIM([passenger_count])), '')) AS
PassengerCount,
        TRY_CONVERT(DECIMAL(12,3), NULLIF(LTRIM(RTRIM([trip_distance])), '')) AS
TripDistanceMiles,
        TRY_CONVERT(DECIMAL(38,18), NULLIF(LTRIM(RTRIM([RatecodeID])), '')) AS RateCodeID,
        TRY_CONVERT(DECIMAL(38,18), NULLIF(LTRIM(RTRIM([payment_type])), '')) AS
PaymentTypeID,
        CASE WHEN UPPER(NULLIF(LTRIM(RTRIM([store_and_fwd_flag])), '')) IN ('Y','1','TRUE')
THEN 'Y' WHEN UPPER(NULLIF(LTRIM(RTRIM([store_and_fwd_flag])), '')) IN ('N','0','FALSE')
THEN 'N' END AS StoreAndFwdFlag,
        TRY_CONVERT(DECIMAL(18,2), NULLIF(LTRIM(RTRIM([fare_amount])), '')) AS FareAmount,
        TRY_CONVERT(DECIMAL(18,2), NULLIF(LTRIM(RTRIM([extra])), '')) AS ExtraAmount,
        TRY_CONVERT(DECIMAL(18,2), NULLIF(LTRIM(RTRIM([mta_tax])), '')) AS MTATaxAmount,
        TRY_CONVERT(DECIMAL(18,2), NULLIF(LTRIM(RTRIM([tip_amount])), '')) AS TipAmount,
        TRY_CONVERT(DECIMAL(18,2), NULLIF(LTRIM(RTRIM([tolls_amount])), '')) AS
TollsAmount,
        TRY_CONVERT(DECIMAL(18,2), NULLIF(LTRIM(RTRIM([improvement_surcharge])), '')) AS
ImprovementSurcharge,
        TRY_CONVERT(DECIMAL(18,2), NULLIF(LTRIM(RTRIM([congestion_surcharge])), '')) AS
CongestionSurcharge,
        TRY_CONVERT(DECIMAL(18,2), NULLIF(LTRIM(RTRIM([cbd_congestion_fee])), '')) AS
CBDCongestionFee,
        TRY_CONVERT(DECIMAL(18,2), NULLIF(LTRIM(RTRIM([total_amount])), '')) AS
TotalAmount,
        TRY_CONVERT(DECIMAL(18,2), NULLIF(LTRIM(RTRIM([Airport_fee])), '')) AS AirportFee,
        ETLSourceKey,
        ETLFileHash,
        ETLLoadedAt
    FROM stg.YellowTrip AS s WITH(HOLDLOCK)
    INNER JOIN #Todo AS f ON f.SourceKey=s.ETLSourceKey
)
INSERT INTO clean.Trip
(
    TaxiType, PickupDateTime, DropoffDateTime, PULocationID, DOLocationID, VendorID,
PassengerCount, TripDistanceMiles, RateCodeID, PaymentTypeID, StoreAndFwdFlag, FareAmount,
ExtraAmount, MTATaxAmount, TipAmount, TollsAmount, ImprovementSurcharge,
```

```
CongestionSurcharge, CBDCongestionFee, TotalAmount, AirportFee, TripDurationSeconds,
IsTotalCalculated, IsCoreValid, ETLSourceKey, ETLFileHash, ETLLoadedAt
)
SELECT
    'YELLOW',
    PickupDateTime,
    DropoffDateTime,
    CASE WHEN PULocationID = FLOOR(PULocationID) THEN TRY_CONVERT(INT, PULocationID) END,
    CASE WHEN DOLocationID = FLOOR(DOLocationID) THEN TRY_CONVERT(INT, DOLocationID) END,
    CASE WHEN VendorID = FLOOR(VendorID) THEN TRY_CONVERT(INT, VendorID) END,
    CASE WHEN PassengerCount = FLOOR(PassengerCount) THEN TRY_CONVERT(INT, PassengerCount)
END,
    TripDistanceMiles,
    CASE WHEN RateCodeID = FLOOR(RateCodeID) THEN TRY_CONVERT(INT, RateCodeID) END,
    CASE WHEN PaymentTypeID = FLOOR(PaymentTypeID) THEN TRY_CONVERT(INT, PaymentTypeID)
END,
    StoreAndFwdFlag,
    FareAmount,
    ExtraAmount,
    MTATaxAmount,
    TipAmount,
    TollsAmount,
    ImprovementSurcharge,
    CongestionSurcharge,
    CBDCongestionFee,
    TotalAmount,
    AirportFee,
    DATEDIFF_BIG(SECOND, PickupDateTime, DropoffDateTime),
    0,
    0,
    ETLSourceKey,
    ETLFileHash,
    ETLLoadedAt
FROM S;


    /* 6. Chi tinh do do/gan loi cho cac tep vua nap. */
UPDATE t
SET
    TripDistanceKm = TRY_CONVERT(DECIMAL(14,3), t.TripDistanceMiles * 1.609344),
    AverageSpeedKmh = CASE WHEN t.TripDurationSeconds > 0 AND t.TripDistanceMiles >= 0
        THEN TRY_CONVERT(DECIMAL(14,3), t.TripDistanceMiles * 1.609344 * 3600.0
            / NULLIF(t.TripDurationSeconds, 0)) END,
    RequestWaitSeconds = CASE WHEN t.RequestDateTime <= t.PickupDateTime
        THEN DATEDIFF_BIG(SECOND, t.RequestDateTime, t.PickupDateTime) END,
    IsCoreValid = CASE
        WHEN t.PickupDateTime IS NULL OR t.DropoffDateTime IS NULL
          OR t.DropoffDateTime <= t.PickupDateTime
          OR t.TripDurationSeconds > CONVERT(BIGINT, @MaxDurationHours) * 3600
          OR (t.TaxiType <> 'FHV' AND (t.TripDistanceMiles IS NULL OR t.TripDistanceMiles <
0))
        THEN 0 ELSE 1 END,
    QualityIssue = NULLIF(CONCAT(
        CASE WHEN t.PickupDateTime IS NULL THEN 'PICKUP_INVALID;' ELSE '' END,
        CASE WHEN t.DropoffDateTime IS NULL THEN 'DROPOFF_INVALID;' ELSE '' END,
        CASE WHEN t.DropoffDateTime <= t.PickupDateTime THEN 'DURATION_NON_POSITIVE;' ELSE
'' END,
        CASE WHEN t.TripDurationSeconds > CONVERT(BIGINT, @MaxDurationHours) * 3600 THEN
'DURATION_OVER_LIMIT;' ELSE '' END,
        CASE WHEN t.TaxiType <> 'FHV' AND t.TripDistanceMiles IS NULL THEN
'DISTANCE_MISSING;' ELSE '' END,
        CASE WHEN t.TripDistanceMiles < 0 THEN 'DISTANCE_NEGATIVE;' ELSE '' END,
        CASE WHEN t.TripDistanceMiles = 0 THEN 'DISTANCE_ZERO;' ELSE '' END,
        CASE WHEN t.TaxiType IN ('YELLOW','GREEN') AND (t.PassengerCount IS NULL OR
t.PassengerCount <= 0) THEN 'PASSENGER_INVALID;' ELSE '' END,
        CASE WHEN pu.LocationID IS NULL THEN 'PU_LOCATION_UNKNOWN;' ELSE '' END,
        CASE WHEN dz.LocationID IS NULL THEN 'DO_LOCATION_UNKNOWN;' ELSE '' END,
        CASE WHEN t.TotalAmount < 0 THEN 'TOTAL_NEGATIVE;' ELSE '' END,
```

```
        CASE WHEN t.TaxiType <> 'FHV' AND t.TotalAmount IS NULL THEN 'TOTAL_MISSING;' ELSE
'' END,
        CASE WHEN t.TaxiType = 'FHVHV' AND t.RequestDateTime IS NULL THEN
'REQUEST_MISSING;' ELSE '' END,
        CASE WHEN t.RequestDateTime > t.PickupDateTime THEN 'REQUEST_AFTER_PICKUP;' ELSE ''
END,
        CASE WHEN t.OnSceneDateTime > t.PickupDateTime THEN 'ONSCENE_AFTER_PICKUP;' ELSE ''
END,
        CASE WHEN ABS(CONVERT(DECIMAL(38,0), t.SourceTripDurationSeconds) -
t.TripDurationSeconds) > 60
            THEN 'SOURCE_DURATION_MISMATCH;' ELSE '' END
    ), '')
FROM clean.Trip AS t
INNER JOIN #Todo AS scope ON scope.SourceKey=t.ETLSourceKey
LEFT JOIN clean.TaxiZone AS pu ON pu.LocationID = t.PULocationID
LEFT JOIN clean.TaxiZone AS dz ON dz.LocationID = t.DOLocationID
WHERE t.TaxiType=@TaxiType;


    /* 7. Doi soat va ghi log trong CUNG transaction voi du lieu clean. */
    SELECT t.ETLSourceKey AS SourceKey,COUNT_BIG(*) AS CleanRows,
        COUNT_BIG(CASE WHEN t.IsCoreValid=1 THEN 1 END) AS ValidRows,
        COUNT_BIG(CASE WHEN t.IsCoreValid=0 THEN 1 END) AS RejectedRows,
        COUNT_BIG(CASE WHEN t.IsCoreValid=1 AND t.QualityIssue IS NOT NULL THEN 1 END) AS
WarningRows
    INTO #Result
    FROM clean.Trip AS t
    INNER JOIN #Todo AS f ON f.SourceKey=t.ETLSourceKey
    WHERE t.TaxiType=@TaxiType
    GROUP BY t.ETLSourceKey;


    IF EXISTS(SELECT 1 FROM #Todo AS f LEFT JOIN #Result AS r ON r.SourceKey=f.SourceKey
        WHERE COALESCE(r.CleanRows,0)<>f.ExpectedRows
            OR COALESCE(r.ValidRows,0)+COALESCE(r.RejectedRows,0)<>f.ExpectedRows)
        THROW 51224, 'So dong sau Transform khong khop. Rollback tep dang xu ly.', 1;


    DELETE l FROM etl.TransformFileLog AS l
    INNER JOIN #Todo AS f ON f.SourceKey=l.SourceKey
    WHERE l.DatasetCode=@Dataset;


    INSERT etl.TransformFileLog
        (DatasetCode,SourceKey,FileHash,RuleVersion,ZoneHash,SourceRows,
         CleanRows,ValidRows,RejectedRows,WarningRows,CompletedAt)
    SELECT @Dataset,f.SourceKey,f.FileHash,@RuleVersion,@ZoneHash,f.ExpectedRows,
        COALESCE(r.CleanRows,0),COALESCE(r.ValidRows,0),COALESCE(r.RejectedRows,0),
        COALESCE(r.WarningRows,0),SYSDATETIME()
    FROM #Todo AS f LEFT JOIN #Result AS r ON r.SourceKey=f.SourceKey;
    END;


    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT>0 ROLLBACK TRANSACTION;
    THROW;
END CATCH;


SELECT @Dataset AS DatasetCode,
    (SELECT COUNT(*) FROM #Todo) AS ProcessedFiles,
    (SELECT COUNT(*) FROM #Files)-(SELECT COUNT(*) FROM #Todo) AS SkippedFiles;


DROP TABLE IF EXISTS #Files;
DROP TABLE IF EXISTS #CleanCounts;
DROP TABLE IF EXISTS #Todo;
DROP TABLE IF EXISTS #StageCounts;
DROP TABLE IF EXISTS #Result;
```

Bảng A.64 Cấu hình tác vụ EST_03_Transform_YELLOW

Thuộc tính	Giá trị source
Connection	{428D3CB7-3BB4-4D96-8F32-4EE97109955B}

EST_04_Transform_GREEN

```
SET NOCOUNT ON;
SET XACT_ABORT ON;
DECLARE @Dataset VARCHAR(20)='green', @TaxiType VARCHAR(10)='GREEN',
        @RuleVersion VARCHAR(64)='INCR_20261019_1|hours=48', @MaxDurationHours INT=48,
        @ZoneText NVARCHAR(MAX), @ZoneHash CHAR(64);


DROP TABLE IF EXISTS #Files;
DROP TABLE IF EXISTS #CleanCounts;
DROP TABLE IF EXISTS #Todo;
DROP TABLE IF EXISTS #StageCounts;
DROP TABLE IF EXISTS #Result;


BEGIN TRY
    BEGIN TRANSACTION;
    IF NOT EXISTS(SELECT 1 FROM clean.TaxiZone WITH(HOLDLOCK))
        THROW 51220, 'Chay task lookup truoc.', 1;
    IF EXISTS(SELECT 1 FROM clean.Trip WITH(UPDLOCK,HOLDLOCK)
        WHERE TaxiType=@TaxiType AND ETLSourceKey IS NULL)
        THROW 51221, 'Clean cu thieu ETLSourceKey; can chuan hoa lineage truoc.', 1;
    IF EXISTS(SELECT 1 FROM stg.GreenTrip WITH(HOLDLOCK) WHERE ETLSourceKey IS NULL)
        THROW 51222, 'Staging thieu lineage. Dung PKG_01 moi.', 1;


SELECT @ZoneText=CONVERT(NVARCHAR(MAX),
    (SELECT LocationID,Borough,Zone,ServiceZone
     FROM clean.TaxiZone WITH(HOLDLOCK)
     ORDER BY LocationID FOR XML RAW('z'),TYPE));
SET @ZoneHash=CONVERT(CHAR(64),HASHBYTES('SHA2_256',@ZoneText),2);


    /* 1. Cac tep Extract da nap thanh cong. Khong chuyen kieu du lieu o buoc nay. */
    SELECT SourceKey,FileHash,ExpectedRows
    INTO #Files
    FROM etl.ExtractFileLog WITH(UPDLOCK,HOLDLOCK)
    WHERE DatasetCode=@Dataset AND LoadStatus='SUCCEEDED' AND RowsLoaded=ExpectedRows;


    /* 2. Kiem tra so dong/hash clean thuc te bang index theo tep. */
    SELECT t.ETLSourceKey AS SourceKey,COUNT_BIG(*) AS CleanRows,
        COUNT_BIG(CASE WHEN t.ETLFileHash IS NULL OR t.ETLFileHash<>f.FileHash THEN 1 END)
AS BadHashRows
    INTO #CleanCounts
    FROM clean.Trip AS t WITH(HOLDLOCK)
    INNER JOIN #Files AS f ON f.SourceKey=t.ETLSourceKey
    WHERE t.TaxiType=@TaxiType
    GROUP BY t.ETLSourceKey;


    /* 3. Chi chon tep chua lam, tep thay doi, hoac clean bi thieu. */
SELECT f.SourceKey,f.FileHash,f.ExpectedRows
INTO #Todo
FROM #Files AS f
LEFT JOIN #CleanCounts AS c ON c.SourceKey=f.SourceKey
LEFT JOIN etl.TransformFileLog AS l WITH(UPDLOCK,HOLDLOCK)
    ON l.DatasetCode=@Dataset AND l.SourceKey=f.SourceKey
WHERE l.SourceKey IS NULL
    OR l.FileHash<>f.FileHash
    OR l.SourceRows<>f.ExpectedRows OR l.CleanRows<>f.ExpectedRows
```

```
    OR l.RuleVersion<>@RuleVersion OR l.ZoneHash<>@ZoneHash
    OR COALESCE(c.CleanRows,0)<>f.ExpectedRows
    OR COALESCE(c.BadHashRows,0)>0;
    CREATE UNIQUE CLUSTERED INDEX IX_Todo_Source ON #Todo(SourceKey);


    IF EXISTS(SELECT 1 FROM #Todo)
    BEGIN
    /* 4. Kiem tra staging cua DUNG cac tep can xu ly truoc khi sua clean. */
    SELECT s.ETLSourceKey AS SourceKey,COUNT_BIG(*) AS SourceRows,
        COUNT_BIG(CASE WHEN s.ETLFileHash IS NULL OR s.ETLFileHash<>f.FileHash THEN 1 END)
AS BadHashRows
    INTO #StageCounts
    FROM stg.GreenTrip AS s WITH(HOLDLOCK)
    INNER JOIN #Todo AS f ON f.SourceKey=s.ETLSourceKey
    GROUP BY s.ETLSourceKey;


    IF EXISTS(SELECT 1 FROM #Todo AS f LEFT JOIN #StageCounts AS s ON
s.SourceKey=f.SourceKey
        WHERE COALESCE(s.SourceRows,0)<>f.ExpectedRows OR COALESCE(s.BadHashRows,0)>0)
        THROW 51223, 'Staging cua tep can Transform khong khop log. Chay lai PKG_01.', 1;


    /* 5. Giu nguyen cac tep da hoan tat; chi thay the cac tep trong #Todo. */
    DELETE t FROM clean.Trip AS t
    INNER JOIN #Todo AS f ON f.SourceKey=t.ETLSourceKey
    WHERE t.TaxiType=@TaxiType;


;WITH S AS
(
    SELECT
        TRY_CONVERT(DATETIME2(0), NULLIF(LTRIM(RTRIM([lpep_pickup_datetime])), '')) AS
PickupDateTime,
        TRY_CONVERT(DATETIME2(0), NULLIF(LTRIM(RTRIM([lpep_dropoff_datetime])), '')) AS
DropoffDateTime,
        TRY_CONVERT(DECIMAL(38,18), NULLIF(LTRIM(RTRIM([PULocationID])), '')) AS
PULocationID,
        TRY_CONVERT(DECIMAL(38,18), NULLIF(LTRIM(RTRIM([DOLocationID])), '')) AS
DOLocationID,
        TRY_CONVERT(DECIMAL(38,18), NULLIF(LTRIM(RTRIM([VendorID])), '')) AS VendorID,
        TRY_CONVERT(DECIMAL(38,18), NULLIF(LTRIM(RTRIM([passenger_count])), '')) AS
PassengerCount,
        TRY_CONVERT(DECIMAL(12,3), NULLIF(LTRIM(RTRIM([trip_distance])), '')) AS
TripDistanceMiles,
        TRY_CONVERT(DECIMAL(38,18), NULLIF(LTRIM(RTRIM([RatecodeID])), '')) AS RateCodeID,
        TRY_CONVERT(DECIMAL(38,18), NULLIF(LTRIM(RTRIM([payment_type])), '')) AS
PaymentTypeID,
        CASE WHEN UPPER(NULLIF(LTRIM(RTRIM([store_and_fwd_flag])), '')) IN ('Y','1','TRUE')
THEN 'Y' WHEN UPPER(NULLIF(LTRIM(RTRIM([store_and_fwd_flag])), '')) IN ('N','0','FALSE')
THEN 'N' END AS StoreAndFwdFlag,
        TRY_CONVERT(DECIMAL(18,2), NULLIF(LTRIM(RTRIM([fare_amount])), '')) AS FareAmount,
        TRY_CONVERT(DECIMAL(18,2), NULLIF(LTRIM(RTRIM([extra])), '')) AS ExtraAmount,
        TRY_CONVERT(DECIMAL(18,2), NULLIF(LTRIM(RTRIM([mta_tax])), '')) AS MTATaxAmount,
        TRY_CONVERT(DECIMAL(18,2), NULLIF(LTRIM(RTRIM([tip_amount])), '')) AS TipAmount,
        TRY_CONVERT(DECIMAL(18,2), NULLIF(LTRIM(RTRIM([tolls_amount])), '')) AS
TollsAmount,
        TRY_CONVERT(DECIMAL(18,2), NULLIF(LTRIM(RTRIM([improvement_surcharge])), '')) AS
ImprovementSurcharge,
        TRY_CONVERT(DECIMAL(18,2), NULLIF(LTRIM(RTRIM([congestion_surcharge])), '')) AS
CongestionSurcharge,
        TRY_CONVERT(DECIMAL(18,2), NULLIF(LTRIM(RTRIM([cbd_congestion_fee])), '')) AS
CBDCongestionFee,
        TRY_CONVERT(DECIMAL(18,2), NULLIF(LTRIM(RTRIM([total_amount])), '')) AS
TotalAmount,
        TRY_CONVERT(DECIMAL(38,18), NULLIF(LTRIM(RTRIM([trip_type])), '')) AS TripTypeID,
        TRY_CONVERT(DECIMAL(18,2), NULLIF(LTRIM(RTRIM([ehail_fee])), '')) AS
EHailFeeAmount,
        ETLSourceKey,
```

```
        ETLFileHash,
        ETLLoadedAt
    FROM stg.GreenTrip AS s WITH(HOLDLOCK)
    INNER JOIN #Todo AS f ON f.SourceKey=s.ETLSourceKey
)
INSERT INTO clean.Trip
(
    TaxiType, PickupDateTime, DropoffDateTime, PULocationID, DOLocationID, VendorID,
PassengerCount, TripDistanceMiles, RateCodeID, PaymentTypeID, StoreAndFwdFlag, FareAmount,
ExtraAmount, MTATaxAmount, TipAmount, TollsAmount, ImprovementSurcharge,
CongestionSurcharge, CBDCongestionFee, TotalAmount, TripTypeID, EHailFeeAmount,
TripDurationSeconds, IsTotalCalculated, IsCoreValid, ETLSourceKey, ETLFileHash, ETLLoadedAt
)
SELECT
    'GREEN',
    PickupDateTime,
    DropoffDateTime,
    CASE WHEN PULocationID = FLOOR(PULocationID) THEN TRY_CONVERT(INT, PULocationID) END,
    CASE WHEN DOLocationID = FLOOR(DOLocationID) THEN TRY_CONVERT(INT, DOLocationID) END,
    CASE WHEN VendorID = FLOOR(VendorID) THEN TRY_CONVERT(INT, VendorID) END,
    CASE WHEN PassengerCount = FLOOR(PassengerCount) THEN TRY_CONVERT(INT, PassengerCount)
END,
    TripDistanceMiles,
    CASE WHEN RateCodeID = FLOOR(RateCodeID) THEN TRY_CONVERT(INT, RateCodeID) END,
    CASE WHEN PaymentTypeID = FLOOR(PaymentTypeID) THEN TRY_CONVERT(INT, PaymentTypeID)
END,
    StoreAndFwdFlag,
    FareAmount,
    ExtraAmount,
    MTATaxAmount,
    TipAmount,
    TollsAmount,
    ImprovementSurcharge,
    CongestionSurcharge,
    CBDCongestionFee,
    TotalAmount,
    CASE WHEN TripTypeID = FLOOR(TripTypeID) THEN TRY_CONVERT(INT, TripTypeID) END,
    EHailFeeAmount,
    DATEDIFF_BIG(SECOND, PickupDateTime, DropoffDateTime),
    0,
    0,
    ETLSourceKey,
    ETLFileHash,
    ETLLoadedAt
FROM S;


    /* 6. Chi tinh do do/gan loi cho cac tep vua nap. */
UPDATE t
SET
    TripDistanceKm = TRY_CONVERT(DECIMAL(14,3), t.TripDistanceMiles * 1.609344),
    AverageSpeedKmh = CASE WHEN t.TripDurationSeconds > 0 AND t.TripDistanceMiles >= 0
        THEN TRY_CONVERT(DECIMAL(14,3), t.TripDistanceMiles * 1.609344 * 3600.0
            / NULLIF(t.TripDurationSeconds, 0)) END,
    RequestWaitSeconds = CASE WHEN t.RequestDateTime <= t.PickupDateTime
        THEN DATEDIFF_BIG(SECOND, t.RequestDateTime, t.PickupDateTime) END,
    IsCoreValid = CASE
        WHEN t.PickupDateTime IS NULL OR t.DropoffDateTime IS NULL
          OR t.DropoffDateTime <= t.PickupDateTime
          OR t.TripDurationSeconds > CONVERT(BIGINT, @MaxDurationHours) * 3600
          OR (t.TaxiType <> 'FHV' AND (t.TripDistanceMiles IS NULL OR t.TripDistanceMiles <
0))
        THEN 0 ELSE 1 END,
    QualityIssue = NULLIF(CONCAT(
        CASE WHEN t.PickupDateTime IS NULL THEN 'PICKUP_INVALID;' ELSE '' END,
        CASE WHEN t.DropoffDateTime IS NULL THEN 'DROPOFF_INVALID;' ELSE '' END,
        CASE WHEN t.DropoffDateTime <= t.PickupDateTime THEN 'DURATION_NON_POSITIVE;' ELSE
'' END,
```

```
        CASE WHEN t.TripDurationSeconds > CONVERT(BIGINT, @MaxDurationHours) * 3600 THEN
'DURATION_OVER_LIMIT;' ELSE '' END,
        CASE WHEN t.TaxiType <> 'FHV' AND t.TripDistanceMiles IS NULL THEN
'DISTANCE_MISSING;' ELSE '' END,
        CASE WHEN t.TripDistanceMiles < 0 THEN 'DISTANCE_NEGATIVE;' ELSE '' END,
        CASE WHEN t.TripDistanceMiles = 0 THEN 'DISTANCE_ZERO;' ELSE '' END,
        CASE WHEN t.TaxiType IN ('YELLOW','GREEN') AND (t.PassengerCount IS NULL OR
t.PassengerCount <= 0) THEN 'PASSENGER_INVALID;' ELSE '' END,
        CASE WHEN pu.LocationID IS NULL THEN 'PU_LOCATION_UNKNOWN;' ELSE '' END,
        CASE WHEN dz.LocationID IS NULL THEN 'DO_LOCATION_UNKNOWN;' ELSE '' END,
        CASE WHEN t.TotalAmount < 0 THEN 'TOTAL_NEGATIVE;' ELSE '' END,
        CASE WHEN t.TaxiType <> 'FHV' AND t.TotalAmount IS NULL THEN 'TOTAL_MISSING;' ELSE
'' END,
        CASE WHEN t.TaxiType = 'FHVHV' AND t.RequestDateTime IS NULL THEN
'REQUEST_MISSING;' ELSE '' END,
        CASE WHEN t.RequestDateTime > t.PickupDateTime THEN 'REQUEST_AFTER_PICKUP;' ELSE ''
END,
        CASE WHEN t.OnSceneDateTime > t.PickupDateTime THEN 'ONSCENE_AFTER_PICKUP;' ELSE ''
END,
        CASE WHEN ABS(CONVERT(DECIMAL(38,0), t.SourceTripDurationSeconds) -
t.TripDurationSeconds) > 60
             THEN 'SOURCE_DURATION_MISMATCH;' ELSE '' END
    ), '')
FROM clean.Trip AS t
INNER JOIN #Todo AS scope ON scope.SourceKey=t.ETLSourceKey
LEFT JOIN clean.TaxiZone AS pu ON pu.LocationID = t.PULocationID
LEFT JOIN clean.TaxiZone AS dz ON dz.LocationID = t.DOLocationID
WHERE t.TaxiType=@TaxiType;


    /* 7. Doi soat va ghi log trong CUNG transaction voi du lieu clean. */
    SELECT t.ETLSourceKey AS SourceKey,COUNT_BIG(*) AS CleanRows,
        COUNT_BIG(CASE WHEN t.IsCoreValid=1 THEN 1 END) AS ValidRows,
        COUNT_BIG(CASE WHEN t.IsCoreValid=0 THEN 1 END) AS RejectedRows,
        COUNT_BIG(CASE WHEN t.IsCoreValid=1 AND t.QualityIssue IS NOT NULL THEN 1 END) AS
WarningRows
    INTO #Result
    FROM clean.Trip AS t
    INNER JOIN #Todo AS f ON f.SourceKey=t.ETLSourceKey
    WHERE t.TaxiType=@TaxiType
    GROUP BY t.ETLSourceKey;


    IF EXISTS(SELECT 1 FROM #Todo AS f LEFT JOIN #Result AS r ON r.SourceKey=f.SourceKey
        WHERE COALESCE(r.CleanRows,0)<>f.ExpectedRows
            OR COALESCE(r.ValidRows,0)+COALESCE(r.RejectedRows,0)<>f.ExpectedRows)
        THROW 51224, 'So dong sau Transform khong khop. Rollback tep dang xu ly.', 1;


    DELETE l FROM etl.TransformFileLog AS l
    INNER JOIN #Todo AS f ON f.SourceKey=l.SourceKey
    WHERE l.DatasetCode=@Dataset;


    INSERT etl.TransformFileLog
        (DatasetCode,SourceKey,FileHash,RuleVersion,ZoneHash,SourceRows,
         CleanRows,ValidRows,RejectedRows,WarningRows,CompletedAt)
    SELECT @Dataset,f.SourceKey,f.FileHash,@RuleVersion,@ZoneHash,f.ExpectedRows,
        COALESCE(r.CleanRows,0),COALESCE(r.ValidRows,0),COALESCE(r.RejectedRows,0),
        COALESCE(r.WarningRows,0),SYSDATETIME()
    FROM #Todo AS f LEFT JOIN #Result AS r ON r.SourceKey=f.SourceKey;
    END;


    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT>0 ROLLBACK TRANSACTION;
    THROW;
END CATCH;
```

```
SELECT @Dataset AS DatasetCode,
    (SELECT COUNT(*) FROM #Todo) AS ProcessedFiles,
    (SELECT COUNT(*) FROM #Files)-(SELECT COUNT(*) FROM #Todo) AS SkippedFiles;


DROP TABLE IF EXISTS #Files;
DROP TABLE IF EXISTS #CleanCounts;
DROP TABLE IF EXISTS #Todo;
DROP TABLE IF EXISTS #StageCounts;
DROP TABLE IF EXISTS #Result;
```

Bảng A.65 Cấu hình tác vụ EST_04_Transform_GREEN

Thuộc tính	**Giá trị source**
Connection	{428D3CB7-3BB4-4D96-8F32-4EE97109955B}

EST_05_Transform_FHV

```
SET NOCOUNT ON;
SET XACT_ABORT ON;
DECLARE @Dataset VARCHAR(20)='fhv', @TaxiType VARCHAR(10)='FHV',
        @RuleVersion VARCHAR(64)='INCR_20261019_1|hours=48', @MaxDurationHours INT=48,
        @ZoneText NVARCHAR(MAX), @ZoneHash CHAR(64);


DROP TABLE IF EXISTS #Files;
DROP TABLE IF EXISTS #CleanCounts;
DROP TABLE IF EXISTS #Todo;
DROP TABLE IF EXISTS #StageCounts;
DROP TABLE IF EXISTS #Result;


BEGIN TRY
    BEGIN TRANSACTION;
    IF NOT EXISTS(SELECT 1 FROM clean.TaxiZone WITH(HOLDLOCK))
        THROW 51220, 'Chay task lookup truoc.', 1;
    IF EXISTS(SELECT 1 FROM clean.Trip WITH(UPDLOCK,HOLDLOCK)
        WHERE TaxiType=@TaxiType AND ETLSourceKey IS NULL)
        THROW 51221, 'Clean cu thieu ETLSourceKey; can chuan hoa lineage truoc.', 1;
    IF EXISTS(SELECT 1 FROM stg.FHVTrip WITH(HOLDLOCK) WHERE ETLSourceKey IS NULL)
        THROW 51222, 'Staging thieu lineage. Dung PKG_01 moi.', 1;


SELECT @ZoneText=CONVERT(NVARCHAR(MAX),
    (SELECT LocationID,Borough,Zone,ServiceZone
     FROM clean.TaxiZone WITH(HOLDLOCK)
     ORDER BY LocationID FOR XML RAW('z'),TYPE));
SET @ZoneHash=CONVERT(CHAR(64),HASHBYTES('SHA2_256',@ZoneText),2);


    /* 1. Cac tep Extract da nap thanh cong. Khong chuyen kieu du lieu o buoc nay. */
    SELECT SourceKey,FileHash,ExpectedRows
    INTO #Files
    FROM etl.ExtractFileLog WITH(UPDLOCK,HOLDLOCK)
    WHERE DatasetCode=@Dataset AND LoadStatus='SUCCEEDED' AND RowsLoaded=ExpectedRows;


    /* 2. Kiem tra so dong/hash clean thuc te bang index theo tep. */
    SELECT t.ETLSourceKey AS SourceKey,COUNT_BIG(*) AS CleanRows,
        COUNT_BIG(CASE WHEN t.ETLFileHash IS NULL OR t.ETLFileHash<>f.FileHash THEN 1 END)
AS BadHashRows
    INTO #CleanCounts
    FROM clean.Trip AS t WITH(HOLDLOCK)
    INNER JOIN #Files AS f ON f.SourceKey=t.ETLSourceKey
    WHERE t.TaxiType=@TaxiType
    GROUP BY t.ETLSourceKey;
```

```
    /* 3. Chi chon tep chua lam, tep thay doi, hoac clean bi thieu. */
SELECT f.SourceKey,f.FileHash,f.ExpectedRows
INTO #Todo
FROM #Files AS f
LEFT JOIN #CleanCounts AS c ON c.SourceKey=f.SourceKey
LEFT JOIN etl.TransformFileLog AS l WITH(UPDLOCK,HOLDLOCK)
    ON l.DatasetCode=@Dataset AND l.SourceKey=f.SourceKey
WHERE l.SourceKey IS NULL
    OR l.FileHash<>f.FileHash
    OR l.SourceRows<>f.ExpectedRows OR l.CleanRows<>f.ExpectedRows
    OR l.RuleVersion<>@RuleVersion OR l.ZoneHash<>@ZoneHash
    OR COALESCE(c.CleanRows,0)<>f.ExpectedRows
    OR COALESCE(c.BadHashRows,0)>0;
    CREATE UNIQUE CLUSTERED INDEX IX_Todo_Source ON #Todo(SourceKey);


    IF EXISTS(SELECT 1 FROM #Todo)
    BEGIN
    /* 4. Kiem tra staging cua DUNG cac tep can xu ly truoc khi sua clean. */
    SELECT s.ETLSourceKey AS SourceKey,COUNT_BIG(*) AS SourceRows,
        COUNT_BIG(CASE WHEN s.ETLFileHash IS NULL OR s.ETLFileHash<>f.FileHash THEN 1 END)
AS BadHashRows
    INTO #StageCounts
    FROM stg.FHVTrip AS s WITH(HOLDLOCK)
    INNER JOIN #Todo AS f ON f.SourceKey=s.ETLSourceKey
    GROUP BY s.ETLSourceKey;


    IF EXISTS(SELECT 1 FROM #Todo AS f LEFT JOIN #StageCounts AS s ON
s.SourceKey=f.SourceKey
        WHERE COALESCE(s.SourceRows,0)<>f.ExpectedRows OR COALESCE(s.BadHashRows,0)>0)
        THROW 51223, 'Staging cua tep can Transform khong khop log. Chay lai PKG_01.', 1;


    /* 5. Giu nguyen cac tep da hoan tat; chi thay the cac tep trong #Todo. */
    DELETE t FROM clean.Trip AS t
    INNER JOIN #Todo AS f ON f.SourceKey=t.ETLSourceKey
    WHERE t.TaxiType=@TaxiType;


;WITH S AS
(
    SELECT
        TRY_CONVERT(DATETIME2(0), NULLIF(LTRIM(RTRIM([pickup_datetime])), '')) AS
PickupDateTime,
        TRY_CONVERT(DATETIME2(0), NULLIF(LTRIM(RTRIM([dropOff_datetime])), '')) AS
DropoffDateTime,
        TRY_CONVERT(DECIMAL(38,18), NULLIF(LTRIM(RTRIM([PULocationID])), '')) AS
PULocationID,
        TRY_CONVERT(DECIMAL(38,18), NULLIF(LTRIM(RTRIM([DOLocationID])), '')) AS
DOLocationID,
        UPPER(NULLIF(LTRIM(RTRIM([dispatching_base_num])), '')) AS DispatchingBaseNum,
        UPPER(NULLIF(LTRIM(RTRIM([Affiliated_base_number])), '')) AS AffiliatedBaseNum,
        CASE WHEN NULLIF(LTRIM(RTRIM([SR_Flag])), '') IN ('1','1.0') THEN '1' WHEN
NULLIF(LTRIM(RTRIM([SR_Flag])), '') IN ('0','0.0') THEN '0' END AS SourceSRFlag,
        ETLSourceKey,
        ETLFileHash,
        ETLLoadedAt
    FROM stg.FHVTrip AS s WITH(HOLDLOCK)
    INNER JOIN #Todo AS f ON f.SourceKey=s.ETLSourceKey
)
INSERT INTO clean.Trip
(
    TaxiType, PickupDateTime, DropoffDateTime, PULocationID, DOLocationID,
DispatchingBaseNum, AffiliatedBaseNum, SourceSRFlag, TripDurationSeconds,
IsTotalCalculated, IsCoreValid, ETLSourceKey, ETLFileHash, ETLLoadedAt
)
SELECT
    'FHV',
```

```
    PickupDateTime,
    DropoffDateTime,
    CASE WHEN PULocationID = FLOOR(PULocationID) THEN TRY_CONVERT(INT, PULocationID) END,
    CASE WHEN DOLocationID = FLOOR(DOLocationID) THEN TRY_CONVERT(INT, DOLocationID) END,
    DispatchingBaseNum,
    AffiliatedBaseNum,
    SourceSRFlag,
    DATEDIFF_BIG(SECOND, PickupDateTime, DropoffDateTime),
    0,
    0,
    ETLSourceKey,
    ETLFileHash,
    ETLLoadedAt
FROM S;


    /* 6. Chi tinh do do/gan loi cho cac tep vua nap. */
UPDATE t
SET
    TripDistanceKm = TRY_CONVERT(DECIMAL(14,3), t.TripDistanceMiles * 1.609344),
    AverageSpeedKmh = CASE WHEN t.TripDurationSeconds > 0 AND t.TripDistanceMiles >= 0
        THEN TRY_CONVERT(DECIMAL(14,3), t.TripDistanceMiles * 1.609344 * 3600.0
            / NULLIF(t.TripDurationSeconds, 0)) END,
    RequestWaitSeconds = CASE WHEN t.RequestDateTime <= t.PickupDateTime
        THEN DATEDIFF_BIG(SECOND, t.RequestDateTime, t.PickupDateTime) END,
    IsCoreValid = CASE
        WHEN t.PickupDateTime IS NULL OR t.DropoffDateTime IS NULL
          OR t.DropoffDateTime <= t.PickupDateTime
          OR t.TripDurationSeconds > CONVERT(BIGINT, @MaxDurationHours) * 3600
          OR (t.TaxiType <> 'FHV' AND (t.TripDistanceMiles IS NULL OR t.TripDistanceMiles <
0))
        THEN 0 ELSE 1 END,
    QualityIssue = NULLIF(CONCAT(
        CASE WHEN t.PickupDateTime IS NULL THEN 'PICKUP_INVALID;' ELSE '' END,
        CASE WHEN t.DropoffDateTime IS NULL THEN 'DROPOFF_INVALID;' ELSE '' END,
        CASE WHEN t.DropoffDateTime <= t.PickupDateTime THEN 'DURATION_NON_POSITIVE;' ELSE
'' END,
        CASE WHEN t.TripDurationSeconds > CONVERT(BIGINT, @MaxDurationHours) * 3600 THEN
'DURATION_OVER_LIMIT;' ELSE '' END,
        CASE WHEN t.TaxiType <> 'FHV' AND t.TripDistanceMiles IS NULL THEN
'DISTANCE_MISSING;' ELSE '' END,
        CASE WHEN t.TripDistanceMiles < 0 THEN 'DISTANCE_NEGATIVE;' ELSE '' END,
        CASE WHEN t.TripDistanceMiles = 0 THEN 'DISTANCE_ZERO;' ELSE '' END,
        CASE WHEN t.TaxiType IN ('YELLOW','GREEN') AND (t.PassengerCount IS NULL OR
t.PassengerCount <= 0) THEN 'PASSENGER_INVALID;' ELSE '' END,
        CASE WHEN pu.LocationID IS NULL THEN 'PU_LOCATION_UNKNOWN;' ELSE '' END,
        CASE WHEN dz.LocationID IS NULL THEN 'DO_LOCATION_UNKNOWN;' ELSE '' END,
        CASE WHEN t.TotalAmount < 0 THEN 'TOTAL_NEGATIVE;' ELSE '' END,
        CASE WHEN t.TaxiType <> 'FHV' AND t.TotalAmount IS NULL THEN 'TOTAL_MISSING;' ELSE
'' END,
        CASE WHEN t.TaxiType = 'FHVHV' AND t.RequestDateTime IS NULL THEN
'REQUEST_MISSING;' ELSE '' END,
        CASE WHEN t.RequestDateTime > t.PickupDateTime THEN 'REQUEST_AFTER_PICKUP;' ELSE ''
END,
        CASE WHEN t.OnSceneDateTime > t.PickupDateTime THEN 'ONSCENE_AFTER_PICKUP;' ELSE ''
END,
        CASE WHEN ABS(CONVERT(DECIMAL(38,0), t.SourceTripDurationSeconds) -
t.TripDurationSeconds) > 60
            THEN 'SOURCE_DURATION_MISMATCH;' ELSE '' END
    ), '')
FROM clean.Trip AS t
INNER JOIN #Todo AS scope ON scope.SourceKey=t.ETLSourceKey
LEFT JOIN clean.TaxiZone AS pu ON pu.LocationID = t.PULocationID
LEFT JOIN clean.TaxiZone AS dz ON dz.LocationID = t.DOLocationID
WHERE t.TaxiType=@TaxiType;


    /* 7. Doi soat va ghi log trong CUNG transaction voi du lieu clean. */
    SELECT t.ETLSourceKey AS SourceKey,COUNT_BIG(*) AS CleanRows,
```

```
        COUNT_BIG(CASE WHEN t.IsCoreValid=1 THEN 1 END) AS ValidRows,
        COUNT_BIG(CASE WHEN t.IsCoreValid=0 THEN 1 END) AS RejectedRows,
        COUNT_BIG(CASE WHEN t.IsCoreValid=1 AND t.QualityIssue IS NOT NULL THEN 1 END) AS
WarningRows
    INTO #Result
    FROM clean.Trip AS t
    INNER JOIN #Todo AS f ON f.SourceKey=t.ETLSourceKey
    WHERE t.TaxiType=@TaxiType
    GROUP BY t.ETLSourceKey;

    IF EXISTS(SELECT 1 FROM #Todo AS f LEFT JOIN #Result AS r ON r.SourceKey=f.SourceKey
        WHERE COALESCE(r.CleanRows,0)<>f.ExpectedRows
            OR COALESCE(r.ValidRows,0)+COALESCE(r.RejectedRows,0)<>f.ExpectedRows)
        THROW 51224, 'So dong sau Transform khong khop. Rollback tep dang xu ly.', 1;

    DELETE l FROM etl.TransformFileLog AS l
    INNER JOIN #Todo AS f ON f.SourceKey=l.SourceKey
    WHERE l.DatasetCode=@Dataset;

    INSERT etl.TransformFileLog
        (DatasetCode,SourceKey,FileHash,RuleVersion,ZoneHash,SourceRows,
         CleanRows,ValidRows,RejectedRows,WarningRows,CompletedAt)
    SELECT @Dataset,f.SourceKey,f.FileHash,@RuleVersion,@ZoneHash,f.ExpectedRows,
        COALESCE(r.CleanRows,0),COALESCE(r.ValidRows,0),COALESCE(r.RejectedRows,0),
        COALESCE(r.WarningRows,0),SYSDATETIME()
    FROM #Todo AS f LEFT JOIN #Result AS r ON r.SourceKey=f.SourceKey;
    END;

    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT>0 ROLLBACK TRANSACTION;
    THROW;
END CATCH;

SELECT @Dataset AS DatasetCode,
    (SELECT COUNT(*) FROM #Todo) AS ProcessedFiles,
    (SELECT COUNT(*) FROM #Files)-(SELECT COUNT(*) FROM #Todo) AS SkippedFiles;

DROP TABLE IF EXISTS #Files;
DROP TABLE IF EXISTS #CleanCounts;
DROP TABLE IF EXISTS #Todo;
DROP TABLE IF EXISTS #StageCounts;
DROP TABLE IF EXISTS #Result;
```

Bảng A.66 Cấu hình tác vụ EST_05_Transform_FHV

Thuộc tính	**Giá trị source**
Connection	{428D3CB7-3BB4-4D96-8F32-4EE97109955B}

EST_06_Transform_FHVHV

```
SET NOCOUNT ON;
SET XACT_ABORT ON;
DECLARE @Dataset VARCHAR(20)='fhvhv', @TaxiType VARCHAR(10)='FHVHV',
        @RuleVersion VARCHAR(64)='INCR_20261019_1|hours=48', @MaxDurationHours INT=48,
        @ZoneText NVARCHAR(MAX), @ZoneHash CHAR(64);

DROP TABLE IF EXISTS #Files;
DROP TABLE IF EXISTS #CleanCounts;
DROP TABLE IF EXISTS #Todo;
DROP TABLE IF EXISTS #StageCounts;
```

```
DROP TABLE IF EXISTS #Result;

BEGIN TRY
    BEGIN TRANSACTION;
    IF NOT EXISTS(SELECT 1 FROM clean.TaxiZone WITH(HOLDLOCK))
        THROW 51220, 'Chay task lookup truoc.', 1;
    IF EXISTS(SELECT 1 FROM clean.Trip WITH(UPDLOCK,HOLDLOCK)
        WHERE TaxiType=@TaxiType AND ETLSourceKey IS NULL)
        THROW 51221, 'Clean cu thieu ETLSourceKey; can chuan hoa lineage truoc.', 1;
    IF EXISTS(SELECT 1 FROM stg.FHVHVTrip WITH(HOLDLOCK) WHERE ETLSourceKey IS NULL)
        THROW 51222, 'Staging thieu lineage. Dung PKG_01 moi.', 1;

SELECT @ZoneText=CONVERT(NVARCHAR(MAX),
    (SELECT LocationID,Borough,Zone,ServiceZone
     FROM clean.TaxiZone WITH(HOLDLOCK)
     ORDER BY LocationID FOR XML RAW('z'),TYPE));
SET @ZoneHash=CONVERT(CHAR(64),HASHBYTES('SHA2_256',@ZoneText),2);

    /* 1. Cac tep Extract da nap thanh cong. Khong chuyen kieu du lieu o buoc nay. */
    SELECT SourceKey,FileHash,ExpectedRows
    INTO #Files
    FROM etl.ExtractFileLog WITH(UPDLOCK,HOLDLOCK)
    WHERE DatasetCode=@Dataset AND LoadStatus='SUCCEEDED' AND RowsLoaded=ExpectedRows;

    /* 2. Kiem tra so dong/hash clean thuc te bang index theo tep. */
    SELECT t.ETLSourceKey AS SourceKey,COUNT_BIG(*) AS CleanRows,
        COUNT_BIG(CASE WHEN t.ETLFileHash IS NULL OR t.ETLFileHash<>f.FileHash THEN 1 END)
AS BadHashRows
    INTO #CleanCounts
    FROM clean.Trip AS t WITH(HOLDLOCK)
    INNER JOIN #Files AS f ON f.SourceKey=t.ETLSourceKey
    WHERE t.TaxiType=@TaxiType
    GROUP BY t.ETLSourceKey;

    /* 3. Chi chon tep chua lam, tep thay doi, hoac clean bi thieu. */
SELECT f.SourceKey,f.FileHash,f.ExpectedRows
INTO #Todo
FROM #Files AS f
LEFT JOIN #CleanCounts AS c ON c.SourceKey=f.SourceKey
LEFT JOIN etl.TransformFileLog AS l WITH(UPDLOCK,HOLDLOCK)
    ON l.DatasetCode=@Dataset AND l.SourceKey=f.SourceKey
WHERE l.SourceKey IS NULL
    OR l.FileHash<>f.FileHash
    OR l.SourceRows<>f.ExpectedRows OR l.CleanRows<>f.ExpectedRows
    OR l.RuleVersion<>@RuleVersion OR l.ZoneHash<>@ZoneHash
    OR COALESCE(c.CleanRows,0)<>f.ExpectedRows
    OR COALESCE(c.BadHashRows,0)>0;
    CREATE UNIQUE CLUSTERED INDEX IX_Todo_Source ON #Todo(SourceKey);

    IF EXISTS(SELECT 1 FROM #Todo)
    BEGIN
    /* 4. Kiem tra staging cua DUNG cac tep can xu ly truoc khi sua clean. */
    SELECT s.ETLSourceKey AS SourceKey,COUNT_BIG(*) AS SourceRows,
        COUNT_BIG(CASE WHEN s.ETLFileHash IS NULL OR s.ETLFileHash<>f.FileHash THEN 1 END)
AS BadHashRows
    INTO #StageCounts
    FROM stg.FHVHVTrip AS s WITH(HOLDLOCK)
    INNER JOIN #Todo AS f ON f.SourceKey=s.ETLSourceKey
    GROUP BY s.ETLSourceKey;

    IF EXISTS(SELECT 1 FROM #Todo AS f LEFT JOIN #StageCounts AS s ON
s.SourceKey=f.SourceKey
        WHERE COALESCE(s.SourceRows,0)<>f.ExpectedRows OR COALESCE(s.BadHashRows,0)>0)
        THROW 51223, 'Staging cua tep can Transform khong khop log. Chay lai PKG_01.', 1;
```

```
    /* 5. Giu nguyen cac tep da hoan tat; chi thay the cac tep trong #Todo. */
    DELETE t FROM clean.Trip AS t
    INNER JOIN #Todo AS f ON f.SourceKey=t.ETLSourceKey
    WHERE t.TaxiType=@TaxiType;

;WITH S AS
(
    SELECT
        TRY_CONVERT(DATETIME2(0), NULLIF(LTRIM(RTRIM([pickup_datetime])), '')) AS
PickupDateTime,
        TRY_CONVERT(DATETIME2(0), NULLIF(LTRIM(RTRIM([dropoff_datetime])), '')) AS
DropoffDateTime,
        TRY_CONVERT(DECIMAL(38,18), NULLIF(LTRIM(RTRIM([PULocationID])), '')) AS
PULocationID,
        TRY_CONVERT(DECIMAL(38,18), NULLIF(LTRIM(RTRIM([DOLocationID])), '')) AS
DOLocationID,
        UPPER(NULLIF(LTRIM(RTRIM([hvfhs_license_num])), '')) AS HVFHSLicenseNum,
        UPPER(NULLIF(LTRIM(RTRIM([dispatching_base_num])), '')) AS DispatchingBaseNum,
        UPPER(NULLIF(LTRIM(RTRIM([originating_base_num])), '')) AS OriginatingBaseNum,
        TRY_CONVERT(DATETIME2(0), NULLIF(LTRIM(RTRIM([request_datetime])), '')) AS
RequestDateTime,
        TRY_CONVERT(DATETIME2(0), NULLIF(LTRIM(RTRIM([on_scene_datetime])), '')) AS
OnSceneDateTime,
        TRY_CONVERT(DECIMAL(12,3), NULLIF(LTRIM(RTRIM([trip_miles])), '')) AS
TripDistanceMiles,
        TRY_CONVERT(DECIMAL(38,18), NULLIF(LTRIM(RTRIM([trip_time])), '')) AS
SourceTripDurationSeconds,
        TRY_CONVERT(DECIMAL(18,2), NULLIF(LTRIM(RTRIM([base_passenger_fare])), '')) AS
FareAmount,
        TRY_CONVERT(DECIMAL(18,2), NULLIF(LTRIM(RTRIM([tolls])), '')) AS TollsAmount,
        TRY_CONVERT(DECIMAL(18,2), NULLIF(LTRIM(RTRIM([bcf])), '')) AS BCFAmount,
        TRY_CONVERT(DECIMAL(18,2), NULLIF(LTRIM(RTRIM([sales_tax])), '')) AS
SalesTaxAmount,
        TRY_CONVERT(DECIMAL(18,2), NULLIF(LTRIM(RTRIM([congestion_surcharge])), '')) AS
CongestionSurcharge,
        TRY_CONVERT(DECIMAL(18,2), NULLIF(LTRIM(RTRIM([airport_fee])), '')) AS AirportFee,
        TRY_CONVERT(DECIMAL(18,2), NULLIF(LTRIM(RTRIM([tips])), '')) AS TipAmount,
        TRY_CONVERT(DECIMAL(18,2), NULLIF(LTRIM(RTRIM([driver_pay])), '')) AS
DriverPayAmount,
        TRY_CONVERT(DECIMAL(18,2), NULLIF(LTRIM(RTRIM([cbd_congestion_fee])), '')) AS
CBDCongestionFee,
        CASE WHEN UPPER(NULLIF(LTRIM(RTRIM([shared_request_flag])), '')) IN
('Y','1','TRUE') THEN 'Y' WHEN UPPER(NULLIF(LTRIM(RTRIM([shared_request_flag])), '')) IN
('N','0','FALSE') THEN 'N' END AS SharedRequestFlag,
        CASE WHEN UPPER(NULLIF(LTRIM(RTRIM([shared_match_flag])), '')) IN ('Y','1','TRUE')
THEN 'Y' WHEN UPPER(NULLIF(LTRIM(RTRIM([shared_match_flag])), '')) IN ('N','0','FALSE')
THEN 'N' END AS SharedMatchFlag,
        CASE WHEN UPPER(NULLIF(LTRIM(RTRIM([access_a_ride_flag])), '')) IN ('Y','1','TRUE')
THEN 'Y' WHEN UPPER(NULLIF(LTRIM(RTRIM([access_a_ride_flag])), '')) IN ('N','0','FALSE')
THEN 'N' END AS AccessARideFlag,
        CASE WHEN UPPER(NULLIF(LTRIM(RTRIM([wav_request_flag])), '')) IN ('Y','1','TRUE')
THEN 'Y' WHEN UPPER(NULLIF(LTRIM(RTRIM([wav_request_flag])), '')) IN ('N','0','FALSE') THEN
'N' END AS WAVRequestFlag,
        CASE WHEN UPPER(NULLIF(LTRIM(RTRIM([wav_match_flag])), '')) IN ('Y','1','TRUE')
THEN 'Y' WHEN UPPER(NULLIF(LTRIM(RTRIM([wav_match_flag])), '')) IN ('N','0','FALSE') THEN
'N' END AS WAVMatchFlag,
        ETLSourceKey,
        ETLFileHash,
        ETLLoadedAt,
        TRY_CONVERT(DATE, SUBSTRING(ETLSourceKey, 16, 7) + '-01', 23) AS SourceMonth
    FROM stg.FHVHVTrip AS s WITH(HOLDLOCK)
    INNER JOIN #Todo AS f ON f.SourceKey=s.ETLSourceKey
)
INSERT INTO clean.Trip
(
```

```
    TaxiType, PickupDateTime, DropoffDateTime, PULocationID, DOLocationID, HVFHSLicenseNum,
DispatchingBaseNum, OriginatingBaseNum, RequestDateTime, OnSceneDateTime,
TripDistanceMiles, SourceTripDurationSeconds, FareAmount, TollsAmount, BCFAmount,
SalesTaxAmount, CongestionSurcharge, AirportFee, TipAmount, DriverPayAmount,
CBDCongestionFee, SharedRequestFlag, SharedMatchFlag, AccessARideFlag, WAVRequestFlag,
WAVMatchFlag, TripDurationSeconds, IsTotalCalculated, IsCoreValid, ETLSourceKey,
ETLFileHash, ETLLoadedAt, TotalAmount
)
SELECT
    'FHVHV',
    PickupDateTime,
    DropoffDateTime,
    CASE WHEN PULocationID = FLOOR(PULocationID) THEN TRY_CONVERT(INT, PULocationID) END,
    CASE WHEN DOLocationID = FLOOR(DOLocationID) THEN TRY_CONVERT(INT, DOLocationID) END,
    HVFHSLicenseNum,
    DispatchingBaseNum,
    OriginatingBaseNum,
    RequestDateTime,
    OnSceneDateTime,
    TripDistanceMiles,
    CASE WHEN SourceTripDurationSeconds = FLOOR(SourceTripDurationSeconds) THEN
TRY_CONVERT(BIGINT, SourceTripDurationSeconds) END,
    FareAmount,
    TollsAmount,
    BCFAmount,
    SalesTaxAmount,
    CongestionSurcharge,
    AirportFee,
    TipAmount,
    DriverPayAmount,
    CBDCongestionFee,
    SharedRequestFlag,
    SharedMatchFlag,
    AccessARideFlag,
    WAVRequestFlag,
    WAVMatchFlag,
    DATEDIFF_BIG(SECOND, PickupDateTime, DropoffDateTime),
    1,
    0,
    ETLSourceKey,
    ETLFileHash,
    ETLLoadedAt,
    CASE WHEN FareAmount IS NOT NULL AND TollsAmount IS NOT NULL
            AND BCFAmount IS NOT NULL AND SalesTaxAmount IS NOT NULL
            AND CongestionSurcharge IS NOT NULL AND AirportFee IS NOT NULL AND TipAmount IS
NOT NULL
            AND (CBDCongestionFee IS NOT NULL OR SourceMonth < '20250101')
        THEN TRY_CONVERT(DECIMAL(18,2), FareAmount + TollsAmount + BCFAmount +
SalesTaxAmount
            + CongestionSurcharge + AirportFee + TipAmount + COALESCE(CBDCongestionFee, 0))
END
FROM S;


    /* 6. Chi tinh do do/gan loi cho cac tep vua nap. */
UPDATE t
SET
    TripDistanceKm = TRY_CONVERT(DECIMAL(14,3), t.TripDistanceMiles * 1.609344),
    AverageSpeedKmh = CASE WHEN t.TripDurationSeconds > 0 AND t.TripDistanceMiles >= 0
        THEN TRY_CONVERT(DECIMAL(14,3), t.TripDistanceMiles * 1.609344 * 3600.0
            / NULLIF(t.TripDurationSeconds, 0)) END,
    RequestWaitSeconds = CASE WHEN t.RequestDateTime <= t.PickupDateTime
        THEN DATEDIFF_BIG(SECOND, t.RequestDateTime, t.PickupDateTime) END,
    IsCoreValid = CASE
        WHEN t.PickupDateTime IS NULL OR t.DropoffDateTime IS NULL
          OR t.DropoffDateTime <= t.PickupDateTime
          OR t.TripDurationSeconds > CONVERT(BIGINT, @MaxDurationHours) * 3600
          OR (t.TaxiType <> 'FHV' AND (t.TripDistanceMiles IS NULL OR t.TripDistanceMiles <
0))
```

```
        THEN 0 ELSE 1 END,
    QualityIssue = NULLIF(CONCAT(
        CASE WHEN t.PickupDateTime IS NULL THEN 'PICKUP_INVALID;' ELSE '' END,
        CASE WHEN t.DropoffDateTime IS NULL THEN 'DROPOFF_INVALID;' ELSE '' END,
        CASE WHEN t.DropoffDateTime <= t.PickupDateTime THEN 'DURATION_NON_POSITIVE;' ELSE
'' END,
        CASE WHEN t.TripDurationSeconds > CONVERT(BIGINT, @MaxDurationHours) * 3600 THEN
'DURATION_OVER_LIMIT;' ELSE '' END,
        CASE WHEN t.TaxiType <> 'FHV' AND t.TripDistanceMiles IS NULL THEN
'DISTANCE_MISSING;' ELSE '' END,
        CASE WHEN t.TripDistanceMiles < 0 THEN 'DISTANCE_NEGATIVE;' ELSE '' END,
        CASE WHEN t.TripDistanceMiles = 0 THEN 'DISTANCE_ZERO;' ELSE '' END,
        CASE WHEN t.TaxiType IN ('YELLOW','GREEN') AND (t.PassengerCount IS NULL OR
t.PassengerCount <= 0) THEN 'PASSENGER_INVALID;' ELSE '' END,
        CASE WHEN pu.LocationID IS NULL THEN 'PU_LOCATION_UNKNOWN;' ELSE '' END,
        CASE WHEN dz.LocationID IS NULL THEN 'DO_LOCATION_UNKNOWN;' ELSE '' END,
        CASE WHEN t.TotalAmount < 0 THEN 'TOTAL_NEGATIVE;' ELSE '' END,
        CASE WHEN t.TaxiType <> 'FHV' AND t.TotalAmount IS NULL THEN 'TOTAL_MISSING;' ELSE
'' END,
        CASE WHEN t.TaxiType = 'FHVHV' AND t.RequestDateTime IS NULL THEN
'REQUEST_MISSING;' ELSE '' END,
        CASE WHEN t.RequestDateTime > t.PickupDateTime THEN 'REQUEST_AFTER_PICKUP;' ELSE ''
END,
        CASE WHEN t.OnSceneDateTime > t.PickupDateTime THEN 'ONSCENE_AFTER_PICKUP;' ELSE ''
END,
        CASE WHEN ABS(CONVERT(DECIMAL(38,0), t.SourceTripDurationSeconds) -
t.TripDurationSeconds) > 60
            THEN 'SOURCE_DURATION_MISMATCH;' ELSE '' END
    ), '')
FROM clean.Trip AS t
INNER JOIN #Todo AS scope ON scope.SourceKey=t.ETLSourceKey
LEFT JOIN clean.TaxiZone AS pu ON pu.LocationID = t.PULocationID
LEFT JOIN clean.TaxiZone AS dz ON dz.LocationID = t.DOLocationID
WHERE t.TaxiType=@TaxiType;


    /* 7. Doi soat va ghi log trong CUNG transaction voi du lieu clean. */
    SELECT t.ETLSourceKey AS SourceKey,COUNT_BIG(*) AS CleanRows,
        COUNT_BIG(CASE WHEN t.IsCoreValid=1 THEN 1 END) AS ValidRows,
        COUNT_BIG(CASE WHEN t.IsCoreValid=0 THEN 1 END) AS RejectedRows,
        COUNT_BIG(CASE WHEN t.IsCoreValid=1 AND t.QualityIssue IS NOT NULL THEN 1 END) AS
WarningRows
    INTO #Result
    FROM clean.Trip AS t
    INNER JOIN #Todo AS f ON f.SourceKey=t.ETLSourceKey
    WHERE t.TaxiType=@TaxiType
    GROUP BY t.ETLSourceKey;


    IF EXISTS(SELECT 1 FROM #Todo AS f LEFT JOIN #Result AS r ON r.SourceKey=f.SourceKey
        WHERE COALESCE(r.CleanRows,0)<>f.ExpectedRows
            OR COALESCE(r.ValidRows,0)+COALESCE(r.RejectedRows,0)<>f.ExpectedRows)
        THROW 51224, 'So dong sau Transform khong khop. Rollback tep dang xu ly.', 1;


    DELETE l FROM etl.TransformFileLog AS l
    INNER JOIN #Todo AS f ON f.SourceKey=l.SourceKey
    WHERE l.DatasetCode=@Dataset;


    INSERT etl.TransformFileLog
        (DatasetCode,SourceKey,FileHash,RuleVersion,ZoneHash,SourceRows,
         CleanRows,ValidRows,RejectedRows,WarningRows,CompletedAt)
    SELECT @Dataset,f.SourceKey,f.FileHash,@RuleVersion,@ZoneHash,f.ExpectedRows,
        COALESCE(r.CleanRows,0),COALESCE(r.ValidRows,0),COALESCE(r.RejectedRows,0),
        COALESCE(r.WarningRows,0),SYSDATETIME()
    FROM #Todo AS f LEFT JOIN #Result AS r ON r.SourceKey=f.SourceKey;
    END;


    COMMIT TRANSACTION;
```

```
END TRY
BEGIN CATCH
    IF @@TRANCOUNT>0 ROLLBACK TRANSACTION;
    THROW;
END CATCH;


SELECT @Dataset AS DatasetCode,
    (SELECT COUNT(*) FROM #Todo) AS ProcessedFiles,
    (SELECT COUNT(*) FROM #Files)-(SELECT COUNT(*) FROM #Todo) AS SkippedFiles;


DROP TABLE IF EXISTS #Files;
DROP TABLE IF EXISTS #CleanCounts;
DROP TABLE IF EXISTS #Todo;
DROP TABLE IF EXISTS #StageCounts;
DROP TABLE IF EXISTS #Result;
```

Bảng A.67 Cấu hình tác vụ EST_06_Transform_FHVHV

Thuộc tính	**Giá trị source**
Connection	{428D3CB7-3BB4-4D96-8F32-4EE97109955B}

EST_07_Validate_Transform

```
SET NOCOUNT ON;
DECLARE @ZoneText NVARCHAR(MAX),@ZoneHash CHAR(64);
SELECT @ZoneText=CONVERT(NVARCHAR(MAX),
    (SELECT LocationID,Borough,Zone,ServiceZone
     FROM clean.TaxiZone WITH(HOLDLOCK)
     ORDER BY LocationID FOR XML RAW('z'),TYPE));
SET @ZoneHash=CONVERT(CHAR(64),HASHBYTES('SHA2_256',@ZoneText),2);


/* Doc log nho de kiem tra coverage; khong UPDATE lai toan bo clean. */
IF NOT EXISTS(SELECT 1 FROM etl.ExtractFileLog
    WHERE DatasetCode IN ('yellow','green','fhv','fhvhv') AND LoadStatus='SUCCEEDED')
    THROW 51230, 'Chua co tep chuyen di Extract thanh cong.', 1;


IF EXISTS(SELECT 1 FROM etl.ExtractFileLog AS e
    LEFT JOIN etl.TransformFileLog AS t ON t.DatasetCode=e.DatasetCode AND
t.SourceKey=e.SourceKey
    WHERE e.DatasetCode IN ('yellow','green','fhv','fhvhv') AND e.LoadStatus='SUCCEEDED'
        AND (t.SourceKey IS NULL OR t.FileHash<>e.FileHash
            OR t.SourceRows<>e.ExpectedRows OR t.CleanRows<>e.ExpectedRows
            OR t.RuleVersion<>'INCR_20261019_1|hours=48' OR t.ZoneHash<>@ZoneHash
            OR t.CleanRows<>t.ValidRows+t.RejectedRows))
    THROW 51231, 'Con tep chua Transform dung phien ban hien tai. Kiem tra log.', 1;


SELECT t.DatasetCode,COUNT_BIG(*) AS CompletedFiles,
    SUM(t.SourceRows) AS SourceRows,SUM(t.CleanRows) AS CleanRows,
    SUM(t.ValidRows) AS ValidRows,SUM(t.RejectedRows) AS RejectedRows,
    SUM(t.WarningRows) AS WarningRows
FROM etl.TransformFileLog AS t
INNER JOIN etl.ExtractFileLog AS e ON e.DatasetCode=t.DatasetCode AND
e.SourceKey=t.SourceKey
WHERE e.LoadStatus='SUCCEEDED' AND t.FileHash=e.FileHash
GROUP BY t.DatasetCode ORDER BY t.DatasetCode;
```

Bảng A.68 Cấu hình tác vụ EST_07_Validate_Transform

Thuộc tính	Giá trị source
Connection	{428D3CB7-3BB4-4D96-8F32-4EE97109955B}

EST_LogOriginalError

```
INSERT etl.PackageError(ExecutionId,PackageName,SourceName,ErrorCode,ErrorDescription)
VALUES(?,?,?,?,?);
```

Bảng A.69 Cấu hình tác vụ EST_LogOriginalError

Thuộc tính	Giá trị source
Connection	{4A829E7F-DE39-4A39-83D3-52CE94E192E2}

Bảng A.70 Precedence constraint PKG_02_Transform_Clean

Từ	Đến	Điều kiện
Package\EST_01_Setup_Clean	Package\EST_02_Transform_TaxiZone	LogicalAnd=True
Package\EST_02_Transform_TaxiZone	Package\EST_03_Transform_YELLOW	LogicalAnd=True
Package\EST_03_Transform_YELLOW	Package\EST_04_Transform_GREEN	LogicalAnd=True
Package\EST_04_Transform_GREEN	Package\EST_05_Transform_FHV	LogicalAnd=True
Package\EST_05_Transform_FHV	Package\EST_06_Transform_FHVHV	LogicalAnd=True
Package\EST_06_Transform_FHVHV	Package\EST_07_Validate_Transform	LogicalAnd=True

PKG_03_Load_DWH

EST_00_Precheck_DWH

```
SET NOCOUNT ON;
SET XACT_ABORT ON;
IF DB_ID(N'NYCTaxi_Staging') IS NULL OR DB_ID(N'NYCTaxi_DWH') IS NULL
    THROW 55001, 'Missing database NYCTaxi_Staging or NYCTaxi_DWH.', 1;
IF OBJECT_ID(N'NYCTaxi_Staging.clean.ValidTrip',N'V') IS NULL
    THROW 55002, 'Missing NYCTaxi_Staging.clean.ValidTrip; run Transform first.', 1;
IF OBJECT_ID(N'NYCTaxi_Staging.clean.TaxiZone',N'U') IS NULL
    THROW 55003, 'Missing clean.TaxiZone.', 1;
IF OBJECT_ID(N'dbo.Fact_YellowGreen_Trips',N'U') IS NULL
 OR OBJECT_ID(N'dbo.Fact_FHV_Trips',N'U') IS NULL
```

```
 OR OBJECT_ID(N'dbo.Fact_HVFHV_Trips',N'U') IS NULL
    THROW 55004, 'Missing transactional Fact tables; run 04_create_nyctaxi_dwh.sql.', 1;
IF NOT EXISTS(SELECT 1 FROM dbo.Dim_Time WHERE MinuteOfDay=0 AND TimeKey=1)
 OR NOT EXISTS(SELECT 1 FROM dbo.Dim_Time WHERE MinuteOfDay=1439 AND TimeKey=1440)
    THROW 55005, 'Dim_Time not initialized (TimeKey must be MinuteOfDay + 1).',1;
IF NOT EXISTS(SELECT 1 FROM NYCTaxi_Staging.clean.ValidTrip)
    THROW 55006, 'No valid trips in clean.ValidTrip.', 1;


IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name = N'etl')
    EXEC(N'CREATE SCHEMA etl AUTHORIZATION dbo');
IF OBJECT_ID(N'etl.LoadStepLog',N'U') IS NULL
    CREATE TABLE etl.LoadStepLog
    (
        LogID BIGINT IDENTITY(1,1) NOT NULL PRIMARY KEY,
        StepName NVARCHAR(100) NOT NULL,
        InsertedRows BIGINT NULL,
        CompletedAt DATETIME2(0) NOT NULL DEFAULT SYSDATETIME()
    );
IF NOT EXISTS(SELECT 1 FROM sys.indexes WHERE
object_id=OBJECT_ID(N'dbo.Fact_YellowGreen_Trips') AND name=N'UX_FYG_RecordHash')
    CREATE UNIQUE INDEX UX_FYG_RecordHash ON dbo.Fact_YellowGreen_Trips(RecordHash) WHERE
RecordHash IS NOT NULL;
IF NOT EXISTS(SELECT 1 FROM sys.indexes WHERE object_id=OBJECT_ID(N'dbo.Fact_FHV_Trips')
AND name=N'UX_FHV_RecordHash')
    CREATE UNIQUE INDEX UX_FHV_RecordHash ON dbo.Fact_FHV_Trips(RecordHash) WHERE
RecordHash IS NOT NULL;
IF NOT EXISTS(SELECT 1 FROM sys.indexes WHERE object_id=OBJECT_ID(N'dbo.Fact_HVFHV_Trips')
AND name=N'UX_HVFHV_RecordHash')
    CREATE UNIQUE INDEX UX_HVFHV_RecordHash ON dbo.Fact_HVFHV_Trips(RecordHash) WHERE
RecordHash IS NOT NULL;


IF OBJECT_ID('etl.FactChangeAudit','U') IS NULL CREATE TABLE etl.FactChangeAudit(
 AuditId bigint IDENTITY PRIMARY KEY,TableName sysname NOT NULL,RowJson nvarchar(max) NOT
NULL,
 DeletedAt datetime2 NOT NULL DEFAULT SYSUTCDATETIME());
IF COL_LENGTH('dbo.Fact_YellowGreen_Trips','SourceDataset') IS NULL ALTER TABLE
dbo.Fact_YellowGreen_Trips ADD SourceDataset varchar(20) NULL;
IF COL_LENGTH('dbo.Fact_YellowGreen_Trips','SourceKey') IS NULL ALTER TABLE
dbo.Fact_YellowGreen_Trips ADD SourceKey nvarchar(260) NULL;
IF COL_LENGTH('dbo.Fact_FHV_Trips','SourceDataset') IS NULL ALTER TABLE dbo.Fact_FHV_Trips
ADD SourceDataset varchar(20) NULL;
IF COL_LENGTH('dbo.Fact_FHV_Trips','SourceKey') IS NULL ALTER TABLE dbo.Fact_FHV_Trips ADD
SourceKey nvarchar(260) NULL;
IF COL_LENGTH('dbo.Fact_HVFHV_Trips','SourceDataset') IS NULL ALTER TABLE
dbo.Fact_HVFHV_Trips ADD SourceDataset varchar(20) NULL;
IF COL_LENGTH('dbo.Fact_HVFHV_Trips','SourceKey') IS NULL ALTER TABLE dbo.Fact_HVFHV_Trips
ADD SourceKey nvarchar(260) NULL;


-- FILE_V1_20261031
SET NOCOUNT ON;
IF OBJECT_ID('etl.DwhFileLog','U') IS NULL
CREATE TABLE etl.DwhFileLog(
 DatasetCode varchar(20) NOT NULL, SourceKey nvarchar(260) NOT NULL,
 SourceVersion char(64) NULL, LoaderVersion varchar(40) NULL,
 Status varchar(20) NOT NULL, ForceReload bit NOT NULL DEFAULT 0,
 ValidRows bigint NULL, FactRows bigint NULL, InsertedRows bigint NULL,
 UpdatedRows bigint NULL, DeletedRows bigint NULL,
 UnknownMask int NOT NULL DEFAULT 0,
 LocationMapHash char(64) NULL, BaseMapHash char(64) NULL, OtherMapHash char(64) NULL,
 CompletedAt datetime2 NULL, ErrorMessage nvarchar(max) NULL,
 CONSTRAINT PK_DwhFileLog PRIMARY KEY(DatasetCode,SourceKey));
EXEC(N'CREATE OR ALTER VIEW etl.DwhSourceState AS
 SELECT T.DatasetCode,T.SourceKey,T.FileHash,T.ValidRows,T.SourceRows,T.CompletedAt,

CONVERT(char(64),HASHBYTES(''SHA2_256'',CONCAT(T.FileHash,''|'',T.RuleVersion,''|'',T.ZoneH
ash,
```

```
''|'',T.SourceRows,''|'',T.ValidRows,''|'',T.RejectedRows,''|'',CONVERT(varchar(33),T.Compl
etedAt,126))),2) AS SourceVersion
 FROM NYCTaxi_Staging.etl.TransformFileLog T
 JOIN NYCTaxi_Staging.etl.ExtractFileLog E ON E.DatasetCode=T.DatasetCode AND
E.SourceKey=T.SourceKey
 WHERE E.DatasetCode IN(''yellow'',''green'',''fhv'',''fhvhv'') AND
E.LoadStatus=''SUCCEEDED'' AND E.RowsLoaded=E.ExpectedRows
 AND E.FileHash=T.FileHash AND E.ExpectedRows=T.SourceRows AND T.CleanRows=T.SourceRows
 AND T.ValidRows+T.RejectedRows=T.CleanRows;');


IF NOT EXISTS(SELECT 1 FROM sys.indexes WHERE
object_id=OBJECT_ID('dbo.Fact_YellowGreen_Trips') AND name='IX_DwhFile_Source') CREATE
INDEX IX_DwhFile_Source ON dbo.Fact_YellowGreen_Trips(SourceDataset,SourceKey)
INCLUDE(RecordHash);
IF EXISTS(SELECT 1 FROM dbo.Fact_YellowGreen_Trips WHERE SourceKey IS NULL OR SourceDataset
IS NULL) THROW 55946,'Legacy facts have no source. Migrate lineage before incremental
loading.',1;


IF NOT EXISTS(SELECT 1 FROM sys.indexes WHERE object_id=OBJECT_ID('dbo.Fact_FHV_Trips') AND
name='IX_DwhFile_Source') CREATE INDEX IX_DwhFile_Source ON
dbo.Fact_FHV_Trips(SourceDataset,SourceKey) INCLUDE(RecordHash);
IF EXISTS(SELECT 1 FROM dbo.Fact_FHV_Trips WHERE SourceKey IS NULL OR SourceDataset IS
NULL) THROW 55946,'Legacy facts have no source. Migrate lineage before incremental
loading.',1;


IF NOT EXISTS(SELECT 1 FROM sys.indexes WHERE object_id=OBJECT_ID('dbo.Fact_HVFHV_Trips')
AND name='IX_DwhFile_Source') CREATE INDEX IX_DwhFile_Source ON
dbo.Fact_HVFHV_Trips(SourceDataset,SourceKey) INCLUDE(RecordHash);
IF EXISTS(SELECT 1 FROM dbo.Fact_HVFHV_Trips WHERE SourceKey IS NULL OR SourceDataset IS
NULL) THROW 55946,'Legacy facts have no source. Migrate lineage before incremental
loading.',1;
```

Bảng A.71 Cấu hình tác vụ EST_00_Precheck_DWH

Thuộc tính	**Giá trị source**
Connection	{4A829E7F-DE39-4A39-83D3-52CE94E192E2}

EST_01_Load_Dimensions

```
SET NOCOUNT ON;
SET XACT_ABORT ON;
BEGIN TRY
 BEGIN TRAN;


DECLARE @LocationMapHash char(64),@BaseMapHash char(64),@OtherMapHash char(64),@MapText
nvarchar(max);
SET @MapText=(SELECT LocationKey,LocationID FROM dbo.Dim_Location ORDER BY LocationKey FOR
JSON PATH);
SET @LocationMapHash=CONVERT(char(64),HASHBYTES('SHA2_256',@MapText),2);
SET @MapText=(SELECT BaseKey,BaseNumber FROM dbo.Dim_Base ORDER BY BaseKey FOR JSON PATH);
SET @BaseMapHash=CONVERT(char(64),HASHBYTES('SHA2_256',@MapText),2);
SET @MapText=CONCAT(
 (SELECT ServiceTypeKey,ServiceCode FROM dbo.Dim_ServiceType ORDER BY ServiceTypeKey FOR
JSON PATH),
 (SELECT VendorKey,VendorID FROM dbo.Dim_Vendor ORDER BY VendorKey FOR JSON PATH),
 (SELECT RateCodeKey,RateCodeID FROM dbo.Dim_RateCode ORDER BY RateCodeKey FOR JSON PATH),
 (SELECT PaymentTypeKey,PaymentTypeID FROM dbo.Dim_PaymentType ORDER BY PaymentTypeKey FOR
JSON PATH),
 (SELECT TripTypeKey,TripTypeID FROM dbo.Dim_TripType ORDER BY TripTypeKey FOR JSON PATH),
 (SELECT HVFHSProviderKey,LicenseNumber FROM dbo.Dim_HVFHSProvider ORDER BY
HVFHSProviderKey FOR JSON PATH));
SET @OtherMapHash=CONVERT(char(64),HASHBYTES('SHA2_256',@MapText),2);
```

```
DROP TABLE IF EXISTS #DwhTodo;
SELECT S.* INTO #DwhTodo FROM etl.DwhSourceState S
LEFT JOIN etl.DwhFileLog L ON L.DatasetCode=S.DatasetCode AND L.SourceKey=S.SourceKey
WHERE L.SourceKey IS NULL OR L.Status<>'SUCCEEDED' OR L.ForceReload=1
 OR L.SourceVersion<>S.SourceVersion OR L.LoaderVersion<>CASE WHEN S.DatasetCode='fhv' THEN
'FILE_FHV_V2_20261031' ELSE 'FILE_V1_20261031' END
 OR (L.UnknownMask&1=1 AND L.LocationMapHash<>@LocationMapHash)
 OR (L.UnknownMask&2=2 AND L.BaseMapHash<>@BaseMapHash)
 OR (L.UnknownMask&4=4 AND L.OtherMapHash<>@OtherMapHash);
CREATE UNIQUE CLUSTERED INDEX IX_DwhTodo ON #DwhTodo(DatasetCode,SourceKey);


 DROP TABLE IF EXISTS #DeltaTrip;
 SELECT C.* INTO #DeltaTrip
 FROM #DwhTodo T JOIN NYCTaxi_Staging.clean.ValidTrip C
 ON C.TaxiType=UPPER(T.DatasetCode) AND C.ETLSourceKey=T.SourceKey;


 -- All physical Dimension tables and their Unknown members come from
04_create_nyctaxi_dwh.sql.
 -- 1) SCD Type 1: Taxi Zone lookup
 INSERT INTO dbo.Dim_Location(LocationID,Borough,ZoneName,ServiceZone)
 SELECT S.LocationID,S.Borough,S.Zone,S.ServiceZone
 FROM NYCTaxi_Staging.clean.TaxiZone S
 WHERE S.LocationID IS NOT NULL
   AND NOT EXISTS(SELECT 1 FROM dbo.Dim_Location D WITH (UPDLOCK,HOLDLOCK) WHERE
D.LocationID=S.LocationID);
 UPDATE D SET D.Borough=S.Borough, D.ZoneName=S.Zone, D.ServiceZone=S.ServiceZone
 FROM dbo.Dim_Location D JOIN NYCTaxi_Staging.clean.TaxiZone S ON S.LocationID=D.LocationID
 WHERE EXISTS (SELECT D.Borough,D.ZoneName,D.ServiceZone EXCEPT SELECT
S.Borough,S.Zone,S.ServiceZone);
 -- 2) Codes used by all four source types
 INSERT dbo.Dim_ServiceType(ServiceCode,ServiceName)
 SELECT V.Code,V.Name
 FROM (VALUES ('YELLOW',N'Yellow Taxi'),('GREEN',N'Green Taxi'),
              ('FHV',N'For-Hire Vehicle'),('FHVHV',N'High Volume FHV')) V(Code,Name)
 WHERE NOT EXISTS(SELECT 1 FROM dbo.Dim_ServiceType D WHERE D.ServiceCode=V.Code);
 INSERT dbo.Dim_Vendor(VendorID)
 SELECT DISTINCT C.VendorID FROM #DeltaTrip C
 WHERE C.TaxiType IN('YELLOW','GREEN') AND C.VendorID IS NOT NULL
   AND NOT EXISTS(SELECT 1 FROM dbo.Dim_Vendor D WHERE D.VendorID=C.VendorID);
 INSERT dbo.Dim_RateCode(RateCodeID)
 SELECT DISTINCT C.RateCodeID FROM #DeltaTrip C
 WHERE C.TaxiType IN('YELLOW','GREEN') AND C.RateCodeID IS NOT NULL
   AND NOT EXISTS(SELECT 1 FROM dbo.Dim_RateCode D WHERE D.RateCodeID=C.RateCodeID);
 INSERT dbo.Dim_PaymentType(PaymentTypeID)
 SELECT DISTINCT C.PaymentTypeID FROM #DeltaTrip C
 WHERE C.TaxiType IN('YELLOW','GREEN') AND C.PaymentTypeID IS NOT NULL
   AND NOT EXISTS(SELECT 1 FROM dbo.Dim_PaymentType D WHERE
D.PaymentTypeID=C.PaymentTypeID);
 INSERT dbo.Dim_TripType(TripTypeID)
 SELECT DISTINCT C.TripTypeID FROM #DeltaTrip C
 WHERE C.TaxiType='GREEN' AND C.TripTypeID IS NOT NULL
   AND NOT EXISTS(SELECT 1 FROM dbo.Dim_TripType D WHERE D.TripTypeID=C.TripTypeID);
 INSERT dbo.Dim_HVFHSProvider(LicenseNumber)
 SELECT DISTINCT C.HVFHSLicenseNum FROM #DeltaTrip C
 WHERE C.TaxiType='FHVHV' AND C.HVFHSLicenseNum IS NOT NULL
   AND NOT EXISTS(SELECT 1 FROM dbo.Dim_HVFHSProvider D WHERE
D.LicenseNumber=C.HVFHSLicenseNum);
 ;WITH Bases AS
 (
   SELECT DispatchingBaseNum AS BaseNumber FROM #DeltaTrip
   UNION SELECT OriginatingBaseNum FROM #DeltaTrip
   UNION SELECT AffiliatedBaseNum FROM #DeltaTrip
 )
 INSERT dbo.Dim_Base(BaseNumber)
 SELECT B.BaseNumber FROM Bases B
 WHERE B.BaseNumber IS NOT NULL
```

```
   AND NOT EXISTS(SELECT 1 FROM dbo.Dim_Base D WHERE D.BaseNumber=B.BaseNumber);
 -- 3) Junk dimensions
 INSERT dbo.Dim_TaxiFlags(StoreAndFwd)
 SELECT DISTINCT CASE WHEN UPPER(C.StoreAndFwdFlag) IN('Y','N') THEN
UPPER(C.StoreAndFwdFlag) ELSE 'U' END
 FROM #DeltaTrip C WHERE C.TaxiType IN('YELLOW','GREEN')
 AND NOT EXISTS (SELECT 1 FROM dbo.Dim_TaxiFlags D
     WHERE D.StoreAndFwd=CASE WHEN UPPER(C.StoreAndFwdFlag) IN('Y','N') THEN
UPPER(C.StoreAndFwdFlag) ELSE 'U' END);
 INSERT dbo.Dim_FHVFlags(IsSharedRide)
 SELECT DISTINCT CASE C.SourceSRFlag WHEN '1' THEN 'Y' WHEN '0' THEN 'N' ELSE 'U' END
 FROM #DeltaTrip C WHERE C.TaxiType='FHV'
 AND NOT EXISTS (SELECT 1 FROM dbo.Dim_FHVFlags D
     WHERE D.IsSharedRide=CASE C.SourceSRFlag WHEN '1' THEN 'Y' WHEN '0' THEN 'N' ELSE 'U'
END);
 ;WITH F AS
 (
   SELECT DISTINCT
    CASE WHEN UPPER(SharedRequestFlag) IN('Y','N') THEN UPPER(SharedRequestFlag) ELSE 'U'
END AS R,
    CASE WHEN UPPER(SharedMatchFlag) IN('Y','N') THEN UPPER(SharedMatchFlag) ELSE 'U' END
AS M,
    CASE WHEN UPPER(AccessARideFlag) IN('Y','N') THEN UPPER(AccessARideFlag) ELSE 'U' END
AS A,
    CASE WHEN UPPER(WAVRequestFlag) IN('Y','N') THEN UPPER(WAVRequestFlag) ELSE 'U' END AS
WR,
    CASE WHEN UPPER(WAVMatchFlag) IN('Y','N') THEN UPPER(WAVMatchFlag) ELSE 'U' END AS WM
   FROM #DeltaTrip WHERE TaxiType='FHVHV'
 )
 INSERT
dbo.Dim_HVFHVFlags(IsSharedRequest,IsSharedMatch,IsAccessARide,IsWAVRequest,IsWAVMatch)
 SELECT F.R,F.M,F.A,F.WR,F.WM FROM F
 WHERE NOT EXISTS (SELECT 1 FROM dbo.Dim_HVFHVFlags D
    WHERE D.IsSharedRequest=F.R AND D.IsSharedMatch=F.M AND D.IsAccessARide=F.A
    AND D.IsWAVRequest=F.WR AND D.IsWAVMatch=F.WM);
 -- 4) Add dates outside seed period; IsHoliday=0 until a reliable calendar is provided.
 ;WITH Dates AS
 (
    SELECT CONVERT(date,PickupDateTime) AS FullDate FROM #DeltaTrip
    UNION SELECT CONVERT(date,DropoffDateTime) FROM #DeltaTrip
    UNION SELECT CONVERT(date,RequestDateTime) FROM #DeltaTrip WHERE RequestDateTime IS NOT
NULL
    UNION SELECT CONVERT(date,OnSceneDateTime) FROM #DeltaTrip WHERE OnSceneDateTime IS NOT
NULL
 )
 INSERT dbo.Dim_Date(DateKey,FullDate,DayOfWeekNumber,DayOfWeekName,DayOfMonth,MonthNumber,
                     MonthName,QuarterNumber,YearNumber,IsWeekend,IsHoliday)
 SELECT YEAR(X.FullDate)*10000+MONTH(X.FullDate)*100+DAY(X.FullDate),X.FullDate,
        CONVERT(tinyint,DATEDIFF(DAY,'19000101',X.FullDate)%7+1),
        DATENAME(WEEKDAY,X.FullDate),CONVERT(tinyint,DAY(X.FullDate)),
        CONVERT(tinyint,MONTH(X.FullDate)),DATENAME(MONTH,X.FullDate),
        CONVERT(tinyint,DATEPART(QUARTER,X.FullDate)),CONVERT(smallint,YEAR(X.FullDate)),
        CASE WHEN DATEDIFF(DAY,'19000101',X.FullDate)%7 IN(5,6) THEN 1 ELSE 0 END,0
 FROM Dates X WHERE X.FullDate BETWEEN '20000101' AND '20991231'
 AND NOT EXISTS(SELECT 1 FROM dbo.Dim_Date D WHERE D.FullDate=X.FullDate);
 -- 5) Monthly processing groups inferred from pickup dates (NOT actual file lineage).
 ;WITH B AS
 (
    SELECT DISTINCT C.TaxiType,
         YEAR(C.PickupDateTime) AS YY,MONTH(C.PickupDateTime) AS MM,
         CONCAT('CLEAN-',C.TaxiType,'-',CONVERT(char(6),C.PickupDateTime,112)) AS BatchID
    FROM #DeltaTrip C
    WHERE C.PickupDateTime IS NOT NULL
 )
 INSERT
dbo.Dim_LoadBatch(BatchID,DatasetName,SourceFileName,SourceYear,SourceMonth,LoadedAt,BatchS
tatus)
 SELECT B.BatchID,B.TaxiType,NULL,B.YY,B.MM,SYSDATETIME(),'READY'
```

```
 FROM B WHERE NOT EXISTS(SELECT 1 FROM dbo.Dim_LoadBatch D WHERE D.BatchID=B.BatchID);

 SELECT (SELECT COUNT(*) FROM #DwhTodo) AS PendingFiles,
        (SELECT COUNT_BIG(*) FROM #DeltaTrip) AS DimensionInputRows;
 DROP TABLE #DeltaTrip; DROP TABLE #DwhTodo;
 COMMIT TRAN;
END TRY
BEGIN CATCH
 IF @@TRANCOUNT>0 ROLLBACK;
 THROW;
END CATCH;
```

Bảng A.72 Cấu hình tác vụ EST_01_Load_Dimensions

Thuộc tính	**Giá trị source**
Connection	{4A829E7F-DE39-4A39-83D3-52CE94E192E2}

EST_02_Load_FactYellowGreen

```
SET NOCOUNT ON; SET XACT_ABORT ON;

DECLARE @LocationMapHash char(64),@BaseMapHash char(64),@OtherMapHash char(64),@MapText
nvarchar(max);
SET @MapText=(SELECT LocationKey,LocationID FROM dbo.Dim_Location ORDER BY LocationKey FOR
JSON PATH);
SET @LocationMapHash=CONVERT(char(64),HASHBYTES('SHA2_256',@MapText),2);
SET @MapText=(SELECT BaseKey,BaseNumber FROM dbo.Dim_Base ORDER BY BaseKey FOR JSON PATH);
SET @BaseMapHash=CONVERT(char(64),HASHBYTES('SHA2_256',@MapText),2);
SET @MapText=CONCAT(
 (SELECT ServiceTypeKey,ServiceCode FROM dbo.Dim_ServiceType ORDER BY ServiceTypeKey FOR
JSON PATH),
 (SELECT VendorKey,VendorID FROM dbo.Dim_Vendor ORDER BY VendorKey FOR JSON PATH),
 (SELECT RateCodeKey,RateCodeID FROM dbo.Dim_RateCode ORDER BY RateCodeKey FOR JSON PATH),
 (SELECT PaymentTypeKey,PaymentTypeID FROM dbo.Dim_PaymentType ORDER BY PaymentTypeKey FOR
JSON PATH),
 (SELECT TripTypeKey,TripTypeID FROM dbo.Dim_TripType ORDER BY TripTypeKey FOR JSON PATH),
 (SELECT HVFHSProviderKey,LicenseNumber FROM dbo.Dim_HVFHSProvider ORDER BY
HVFHSProviderKey FOR JSON PATH));
SET @OtherMapHash=CONVERT(char(64),HASHBYTES('SHA2_256',@MapText),2);



DROP TABLE IF EXISTS #DwhTodo;
SELECT S.* INTO #DwhTodo FROM etl.DwhSourceState S
LEFT JOIN etl.DwhFileLog L ON L.DatasetCode=S.DatasetCode AND L.SourceKey=S.SourceKey
WHERE L.SourceKey IS NULL OR L.Status<>'SUCCEEDED' OR L.ForceReload=1
 OR L.SourceVersion<>S.SourceVersion OR L.LoaderVersion<>CASE WHEN S.DatasetCode='fhv' THEN
'FILE_FHV_V2_20261031' ELSE 'FILE_V1_20261031' END
 OR (L.UnknownMask&1=1 AND L.LocationMapHash<>@LocationMapHash)
 OR (L.UnknownMask&2=2 AND L.BaseMapHash<>@BaseMapHash)
 OR (L.UnknownMask&4=4 AND L.OtherMapHash<>@OtherMapHash);
CREATE UNIQUE CLUSTERED INDEX IX_DwhTodo ON #DwhTodo(DatasetCode,SourceKey);

DELETE FROM #DwhTodo WHERE DatasetCode NOT IN ('yellow','green');
DECLARE @Dataset varchar(20),@Source nvarchar(260),@Expected bigint,@Token char(64),
 @Inserted bigint=0,@Updated bigint=0,@Deleted bigint=0,@FactRows bigint,@Unknown int,
 @Files int=0,@Input bigint=0,@Error nvarchar(max);
DECLARE FileCursor CURSOR LOCAL FAST_FORWARD FOR SELECT
DatasetCode,SourceKey,ValidRows,SourceVersion FROM #DwhTodo ORDER BY DatasetCode,SourceKey;
OPEN FileCursor; FETCH NEXT FROM FileCursor INTO @Dataset,@Source,@Expected,@Token;
WHILE @@FETCH_STATUS=0
BEGIN
 BEGIN TRY
  BEGIN TRANSACTION;
```

```
  DROP TABLE IF EXISTS #CurrentFile; DROP TABLE IF EXISTS #Desired;
  SELECT * INTO #CurrentFile FROM #DwhTodo WHERE DatasetCode=@Dataset AND
SourceKey=@Source;
  IF NOT EXISTS(SELECT 1 FROM etl.DwhSourceState WITH(HOLDLOCK) WHERE DatasetCode=@Dataset
AND SourceKey=@Source AND SourceVersion=@Token)
   THROW 55940,'Source changed after DWH planning; retry after Transform.',1;
  IF EXISTS(SELECT 1 FROM NYCTaxi_Staging.clean.Trip C JOIN #CurrentFile P
    ON C.TaxiType=UPPER(P.DatasetCode) AND C.ETLSourceKey=P.SourceKey
    WHERE C.ETLFileHash IS NULL OR C.ETLFileHash<>P.FileHash)
   THROW 55947,'File clean lineage hash disagrees with Transform checkpoint.',1;
  IF (SELECT COUNT_BIG(*) FROM NYCTaxi_Staging.clean.Trip WHERE TaxiType=UPPER(@Dataset)
AND ETLSourceKey=@Source)
   <>(SELECT SourceRows FROM #CurrentFile)
   THROW 55948,'File clean row coverage is incomplete.',1;
;WITH SourceRows AS
(
  SELECT C.*,
    HASHBYTES('SHA2_256',
      (SELECT C.TaxiType AS TaxiType,
            C.VendorID AS VendorID,
            C.HVFHSLicenseNum AS HVFHSLicenseNum,
            C.DispatchingBaseNum AS DispatchingBaseNum,
            C.OriginatingBaseNum AS OriginatingBaseNum,
            C.AffiliatedBaseNum AS AffiliatedBaseNum,
            C.RequestDateTime AS RequestDateTime,
            C.OnSceneDateTime AS OnSceneDateTime,
            C.PickupDateTime AS PickupDateTime,
            C.DropoffDateTime AS DropoffDateTime,
            C.TripDurationSeconds AS TripDurationSeconds,
            C.PULocationID AS PULocationID,
            C.DOLocationID AS DOLocationID,
            C.PassengerCount AS PassengerCount,
            C.TripDistanceMiles AS TripDistanceMiles,
            C.RateCodeID AS RateCodeID,
            C.PaymentTypeID AS PaymentTypeID,
            C.TripTypeID AS TripTypeID,
            C.StoreAndFwdFlag AS StoreAndFwdFlag,
            C.FareAmount AS FareAmount,
            C.ExtraAmount AS ExtraAmount,
            C.MTATaxAmount AS MTATaxAmount,
            C.TipAmount AS TipAmount,
            C.TollsAmount AS TollsAmount,
            C.ImprovementSurcharge AS ImprovementSurcharge,
            C.CongestionSurcharge AS CongestionSurcharge,
            C.AirportFee AS AirportFee,
            C.CBDCongestionFee AS CBDCongestionFee,
            C.BCFAmount AS BCFAmount,
            C.SalesTaxAmount AS SalesTaxAmount,
            C.DriverPayAmount AS DriverPayAmount,
            C.TotalAmount AS TotalAmount,
            C.IsTotalCalculated AS IsTotalCalculated,
            C.SharedRequestFlag AS SharedRequestFlag,
            C.SharedMatchFlag AS SharedMatchFlag,
            C.AccessARideFlag AS AccessARideFlag,
            C.WAVRequestFlag AS WAVRequestFlag,
            C.WAVMatchFlag AS WAVMatchFlag
       FOR JSON PATH, WITHOUT_ARRAY_WRAPPER, INCLUDE_NULL_VALUES)
    ) AS PayloadHash
  FROM NYCTaxi_Staging.clean.ValidTrip C
  INNER JOIN #CurrentFile F ON C.TaxiType=UPPER(F.DatasetCode) AND
C.ETLSourceKey=F.SourceKey
  WHERE C.TaxiType IN ('YELLOW','GREEN')
),
Ranked AS
(
  SELECT S.*, ROW_NUMBER() OVER(PARTITION BY S.TaxiType,S.ETLSourceKey,S.PayloadHash ORDER
BY S.TripCleanID) AS DuplicateOrdinal
  FROM SourceRows S
```

```
),
Tagged AS
(
  SELECT R.*,
    HASHBYTES('SHA2_256',
CONCAT(R.TaxiType,'|',R.ETLSourceKey,'|',CONVERT(varchar(64),R.PayloadHash,2),':',R.Duplica
teOrdinal)) AS RecordHash
  FROM Ranked R
),
FactValues AS (
SELECT
    COALESCE(PickupDate.DateKey, 0) AS PickupDateKey,
    COALESCE(DropoffDate.DateKey, 0) AS DropoffDateKey,


    COALESCE(PickupTime.TimeKey, 0) AS PickupTimeKey,
    COALESCE(DropoffTime.TimeKey, 0) AS DropoffTimeKey,


    COALESCE(PickupZone.LocationKey, 0) AS PickupLocationKey,
    COALESCE(DropoffZone.LocationKey, 0) AS DropoffLocationKey,


    COALESCE(ServiceType.ServiceTypeKey, 0) AS ServiceTypeKey,
    COALESCE(Vendor.VendorKey, 0) AS VendorKey,
    COALESCE(RateCode.RateCodeKey, 0) AS RateCodeKey,
    COALESCE(Payment.PaymentTypeKey, 0) AS PaymentTypeKey,
    COALESCE(TripType.TripTypeKey, 0) AS TripTypeKey,
    COALESCE(TaxiFlag.TaxiFlagKey, 0) AS TaxiFlagKey,
    COALESCE(PassengerGroup.PassengerGroupKey, 0)
        AS PassengerGroupKey,


    COALESCE(LoadBatch.LoadBatchKey,0) AS LoadBatchKey,
    1 AS TripCount,


    C.PassengerCount,
    C.TripDistanceMiles,
    C.TripDurationSeconds,


    CASE
        WHEN C.TripDurationSeconds > 0
             AND C.TripDistanceMiles >= 0
        THEN CONVERT
        (
            DECIMAL(12,3),
            C.TripDistanceMiles * 3600.0
            / C.TripDurationSeconds
        )
        ELSE NULL
    END AS AverageSpeedMPH,


    C.FareAmount,
    C.ExtraAmount,
    C.MTATaxAmount,
    C.TipAmount,
    C.TollsAmount,
    C.ImprovementSurcharge,
    C.CongestionSurcharge,
    C.AirportFee,
    C.CBDCongestionFee,
    C.TotalAmount,
    C.RecordHash AS RecordHash, LOWER(C.TaxiType) AS SourceDataset, C.ETLSourceKey AS
SourceKey,
    CASE WHEN (C.PULocationID IS NOT NULL AND COALESCE(PickupZone.LocationKey,0)=0) OR
(C.DOLocationID IS NOT NULL AND COALESCE(DropoffZone.LocationKey,0)=0) THEN 1 ELSE 0 END +
CASE WHEN (C.VendorID IS NOT NULL AND COALESCE(Vendor.VendorKey,0)=0) OR (C.RateCodeID IS
NOT NULL AND COALESCE(RateCode.RateCodeKey,0)=0) OR (C.PaymentTypeID IS NOT NULL AND
```

```
COALESCE(Payment.PaymentTypeKey,0)=0) OR (C.TripTypeID IS NOT NULL AND
COALESCE(TripType.TripTypeKey,0)=0) THEN 4 ELSE 0 END AS UnresolvedMask

FROM Tagged AS C

LEFT JOIN dbo.Dim_Date AS PickupDate
    ON PickupDate.FullDate =
       CONVERT(DATE, C.PickupDateTime)

LEFT JOIN dbo.Dim_Date AS DropoffDate
    ON DropoffDate.FullDate =
       CONVERT(DATE, C.DropoffDateTime)

LEFT JOIN dbo.Dim_Time AS PickupTime
    ON PickupTime.MinuteOfDay =
       DATEPART(HOUR, C.PickupDateTime) * 60
       + DATEPART(MINUTE, C.PickupDateTime)

LEFT JOIN dbo.Dim_Time AS DropoffTime
    ON DropoffTime.MinuteOfDay =
       DATEPART(HOUR, C.DropoffDateTime) * 60
       + DATEPART(MINUTE, C.DropoffDateTime)

LEFT JOIN dbo.Dim_Location AS PickupZone
    ON PickupZone.LocationID = C.PULocationID

LEFT JOIN dbo.Dim_Location AS DropoffZone
    ON DropoffZone.LocationID = C.DOLocationID

LEFT JOIN dbo.Dim_ServiceType AS ServiceType
    ON ServiceType.ServiceCode = C.TaxiType

LEFT JOIN dbo.Dim_Vendor AS Vendor
    ON Vendor.VendorID = C.VendorID

LEFT JOIN dbo.Dim_RateCode AS RateCode
    ON RateCode.RateCodeID = C.RateCodeID

LEFT JOIN dbo.Dim_PaymentType AS Payment
    ON Payment.PaymentTypeID = C.PaymentTypeID

LEFT JOIN dbo.Dim_TripType AS TripType
    ON TripType.TripTypeID = C.TripTypeID

LEFT JOIN dbo.Dim_TaxiFlags AS TaxiFlag
    ON TaxiFlag.StoreAndFwd =
       CASE
           WHEN UPPER(C.StoreAndFwdFlag) IN ('Y', 'N')
               THEN UPPER(C.StoreAndFwdFlag)
           ELSE 'U'
       END

LEFT JOIN dbo.Dim_PassengerGroup AS PassengerGroup
    ON C.PassengerCount BETWEEN
       PassengerGroup.MinCount AND PassengerGroup.MaxCount

LEFT JOIN dbo.Dim_LoadBatch AS LoadBatch
    ON LoadBatch.BatchID = CONCAT('CLEAN-',C.TaxiType,'-
',CONVERT(char(6),C.PickupDateTime,112))
```

```
WHERE C.TaxiType IN ('YELLOW', 'GREEN')
)


  SELECT * INTO #Desired FROM FactValues;
  CREATE UNIQUE CLUSTERED INDEX IX_Desired_Hash ON #Desired(RecordHash);
  IF (SELECT COUNT_BIG(*) FROM #Desired)<>@Expected
   THROW 55941,'File clean rows disagree with Transform checkpoint.',1;
  IF EXISTS(SELECT 1 FROM #Desired WHERE ServiceTypeKey=0)
   THROW 55942,'Unknown ServiceTypeKey in changed file.',1;
  INSERT etl.FactChangeAudit(TableName,RowJson)
   SELECT N'Fact_YellowGreen_Trips',(SELECT F.* FOR JSON PATH,WITHOUT_ARRAY_WRAPPER)
   FROM dbo.Fact_YellowGreen_Trips F
   WHERE F.SourceDataset=@Dataset AND F.SourceKey=@Source
     AND NOT EXISTS(SELECT 1 FROM #Desired V WHERE V.RecordHash=F.RecordHash);
  DELETE F FROM dbo.Fact_YellowGreen_Trips F
   WHERE F.SourceDataset=@Dataset AND F.SourceKey=@Source
     AND NOT EXISTS(SELECT 1 FROM #Desired V WHERE V.RecordHash=F.RecordHash);
  SET @Deleted=@@ROWCOUNT;
  UPDATE F SET
PickupDateKey=V.PickupDateKey,DropoffDateKey=V.DropoffDateKey,PickupTimeKey=V.PickupTimeKey
,DropoffTimeKey=V.DropoffTimeKey,PickupLocationKey=V.PickupLocationKey,DropoffLocationKey=V
.DropoffLocationKey,ServiceTypeKey=V.ServiceTypeKey,VendorKey=V.VendorKey,RateCodeKey=V.Rat
eCodeKey,PaymentTypeKey=V.PaymentTypeKey,TripTypeKey=V.TripTypeKey,TaxiFlagKey=V.TaxiFlagKe
y,PassengerGroupKey=V.PassengerGroupKey,LoadBatchKey=V.LoadBatchKey,TripCount=V.TripCount,P
assengerCount=V.PassengerCount,TripDistanceMiles=V.TripDistanceMiles,TripDurationSeconds=V.
TripDurationSeconds,AverageSpeedMPH=V.AverageSpeedMPH,FareAmount=V.FareAmount,ExtraAmount=V
.ExtraAmount,MTATaxAmount=V.MTATaxAmount,TipAmount=V.TipAmount,TollsAmount=V.TollsAmount,Im
provementSurcharge=V.ImprovementSurcharge,CongestionSurcharge=V.CongestionSurcharge,Airport
Fee=V.AirportFee,CBDCongestionFee=V.CBDCongestionFee,TotalAmount=V.TotalAmount,SourceDatase
t=V.SourceDataset,SourceKey=V.SourceKey
   FROM dbo.Fact_YellowGreen_Trips F JOIN #Desired V ON V.RecordHash=F.RecordHash
   WHERE F.SourceDataset=@Dataset AND F.SourceKey=@Source
     AND EXISTS(SELECT
F.PickupDateKey,F.DropoffDateKey,F.PickupTimeKey,F.DropoffTimeKey,F.PickupLocationKey,F.Dro
poffLocationKey,F.ServiceTypeKey,F.VendorKey,F.RateCodeKey,F.PaymentTypeKey,F.TripTypeKey,F
.TaxiFlagKey,F.PassengerGroupKey,F.LoadBatchKey,F.TripCount,F.PassengerCount,F.TripDistance
Miles,F.TripDurationSeconds,F.AverageSpeedMPH,F.FareAmount,F.ExtraAmount,F.MTATaxAmount,F.T
ipAmount,F.TollsAmount,F.ImprovementSurcharge,F.CongestionSurcharge,F.AirportFee,F.CBDConge
stionFee,F.TotalAmount,F.SourceDataset,F.SourceKey EXCEPT SELECT
V.PickupDateKey,V.DropoffDateKey,V.PickupTimeKey,V.DropoffTimeKey,V.PickupLocationKey,V.Dro
poffLocationKey,V.ServiceTypeKey,V.VendorKey,V.RateCodeKey,V.PaymentTypeKey,V.TripTypeKey,V
.TaxiFlagKey,V.PassengerGroupKey,V.LoadBatchKey,V.TripCount,V.PassengerCount,V.TripDistance
Miles,V.TripDurationSeconds,V.AverageSpeedMPH,V.FareAmount,V.ExtraAmount,V.MTATaxAmount,V.T
ipAmount,V.TollsAmount,V.ImprovementSurcharge,V.CongestionSurcharge,V.AirportFee,V.CBDConge
stionFee,V.TotalAmount,V.SourceDataset,V.SourceKey);
  SET @Updated=@@ROWCOUNT;
    INSERT INTO dbo.Fact_YellowGreen_Trips
    (PickupDateKey, DropoffDateKey, PickupTimeKey, DropoffTimeKey, PickupLocationKey,
DropoffLocationKey, ServiceTypeKey, VendorKey, RateCodeKey, PaymentTypeKey, TripTypeKey,
TaxiFlagKey, PassengerGroupKey, LoadBatchKey, TripCount, PassengerCount, TripDistanceMiles,
TripDurationSeconds, AverageSpeedMPH, FareAmount, ExtraAmount, MTATaxAmount, TipAmount,
TollsAmount, ImprovementSurcharge, CongestionSurcharge, AirportFee, CBDCongestionFee,
TotalAmount, RecordHash, SourceDataset, SourceKey)
    SELECT V.PickupDateKey, V.DropoffDateKey, V.PickupTimeKey, V.DropoffTimeKey,
V.PickupLocationKey, V.DropoffLocationKey, V.ServiceTypeKey, V.VendorKey, V.RateCodeKey,
V.PaymentTypeKey, V.TripTypeKey, V.TaxiFlagKey, V.PassengerGroupKey, V.LoadBatchKey,
V.TripCount, V.PassengerCount, V.TripDistanceMiles, V.TripDurationSeconds,
V.AverageSpeedMPH, V.FareAmount, V.ExtraAmount, V.MTATaxAmount, V.TipAmount, V.TollsAmount,
V.ImprovementSurcharge, V.CongestionSurcharge, V.AirportFee, V.CBDCongestionFee,
V.TotalAmount, V.RecordHash, V.SourceDataset, V.SourceKey
    FROM #Desired V
    WHERE NOT EXISTS
      (SELECT 1 FROM dbo.Fact_YellowGreen_Trips F WITH (UPDLOCK,HOLDLOCK) WHERE
F.RecordHash=V.RecordHash);


  SET @Inserted=@@ROWCOUNT;
  IF TRY_CONVERT(int,SESSION_CONTEXT(N'NYCTaxiFailAfterFactWrite'))=1
```

```
   THROW 55949,'Injected failure after Fact write, before checkpoint.',1;
  SELECT @FactRows=COUNT_BIG(*) FROM dbo.Fact_YellowGreen_Trips WHERE
SourceDataset=@Dataset AND SourceKey=@Source;
  IF @FactRows<>@Expected THROW 55943,'File Fact reconciliation failed.',1;
  SELECT
@Unknown=COALESCE(MAX(UnresolvedMask&1),0)+COALESCE(MAX(UnresolvedMask&2),0)+COALESCE(MAX(U
nresolvedMask&4),0) FROM #Desired;
  DELETE etl.DwhFileLog WHERE DatasetCode=@Dataset AND SourceKey=@Source;
  INSERT
etl.DwhFileLog(DatasetCode,SourceKey,SourceVersion,LoaderVersion,Status,ForceReload,

ValidRows,FactRows,InsertedRows,UpdatedRows,DeletedRows,UnknownMask,LocationMapHash,BaseMap
Hash,OtherMapHash,CompletedAt)

VALUES(@Dataset,@Source,@Token,'FILE_V1_20261031','SUCCEEDED',0,@Expected,@FactRows,@Insert
ed,@Updated,@Deleted,@Unknown,
   @LocationMapHash,@BaseMapHash,@OtherMapHash,SYSUTCDATETIME());
  INSERT etl.LoadStepLog(StepName,InsertedRows)
VALUES(N'Fact_YellowGreen_Trips',@Inserted);
  COMMIT TRANSACTION;
  SET @Files+=1; SET @Input+=@Expected;
 END TRY
 BEGIN CATCH
  SET @Error=ERROR_MESSAGE(); IF @@TRANCOUNT>0 ROLLBACK;
  IF NOT EXISTS(SELECT 1 FROM etl.DwhFileLog WHERE DatasetCode=@Dataset AND
SourceKey=@Source)
   INSERT etl.DwhFileLog(DatasetCode,SourceKey,Status,ErrorMessage)
VALUES(@Dataset,@Source,'FAILED',@Error);
  ELSE UPDATE etl.DwhFileLog SET Status='FAILED',ErrorMessage=@Error WHERE
DatasetCode=@Dataset AND SourceKey=@Source;
  CLOSE FileCursor; DEALLOCATE FileCursor; THROW;
 END CATCH;
 FETCH NEXT FROM FileCursor INTO @Dataset,@Source,@Expected,@Token;
END;
CLOSE FileCursor; DEALLOCATE FileCursor;
DROP TABLE IF EXISTS #Desired; DROP TABLE IF EXISTS #CurrentFile; DROP TABLE #DwhTodo;
SELECT @Files AS ProcessedFiles,@Input AS ValidInputRows;
```

Bảng A.73 Cấu hình tác vụ EST_02_Load_FactYellowGreen

Thuộc tính	**Giá trị source**
Connection	{4A829E7F-DE39-4A39-83D3-52CE94E192E2}

EST_03_Load_FactFHV

```
SET NOCOUNT ON; SET XACT_ABORT ON;


DECLARE @LocationMapHash char(64),@BaseMapHash char(64),@OtherMapHash char(64),@MapText
nvarchar(max);
SET @MapText=(SELECT LocationKey,LocationID FROM dbo.Dim_Location ORDER BY LocationKey FOR
JSON PATH);
SET @LocationMapHash=CONVERT(char(64),HASHBYTES('SHA2_256',@MapText),2);
SET @MapText=(SELECT BaseKey,BaseNumber FROM dbo.Dim_Base ORDER BY BaseKey FOR JSON PATH);
SET @BaseMapHash=CONVERT(char(64),HASHBYTES('SHA2_256',@MapText),2);
SET @MapText=CONCAT(
 (SELECT ServiceTypeKey,ServiceCode FROM dbo.Dim_ServiceType ORDER BY ServiceTypeKey FOR
JSON PATH),
 (SELECT VendorKey,VendorID FROM dbo.Dim_Vendor ORDER BY VendorKey FOR JSON PATH),
 (SELECT RateCodeKey,RateCodeID FROM dbo.Dim_RateCode ORDER BY RateCodeKey FOR JSON PATH),
 (SELECT PaymentTypeKey,PaymentTypeID FROM dbo.Dim_PaymentType ORDER BY PaymentTypeKey FOR
JSON PATH),
 (SELECT TripTypeKey,TripTypeID FROM dbo.Dim_TripType ORDER BY TripTypeKey FOR JSON PATH),
 (SELECT HVFHSProviderKey,LicenseNumber FROM dbo.Dim_HVFHSProvider ORDER BY
HVFHSProviderKey FOR JSON PATH));
SET @OtherMapHash=CONVERT(char(64),HASHBYTES('SHA2_256',@MapText),2);
```

```
DROP TABLE IF EXISTS #DwhTodo;
SELECT S.* INTO #DwhTodo FROM etl.DwhSourceState S
LEFT JOIN etl.DwhFileLog L ON L.DatasetCode=S.DatasetCode AND L.SourceKey=S.SourceKey
WHERE L.SourceKey IS NULL OR L.Status<>'SUCCEEDED' OR L.ForceReload=1
 OR L.SourceVersion<>S.SourceVersion OR L.LoaderVersion<>CASE WHEN S.DatasetCode='fhv' THEN
'FILE_FHV_V2_20261031' ELSE 'FILE_V1_20261031' END
 OR (L.UnknownMask&1=1 AND L.LocationMapHash<>@LocationMapHash)
 OR (L.UnknownMask&2=2 AND L.BaseMapHash<>@BaseMapHash)
 OR (L.UnknownMask&4=4 AND L.OtherMapHash<>@OtherMapHash);
CREATE UNIQUE CLUSTERED INDEX IX_DwhTodo ON #DwhTodo(DatasetCode,SourceKey);


DELETE FROM #DwhTodo WHERE DatasetCode NOT IN ('fhv');
DECLARE @Dataset varchar(20),@Source nvarchar(260),@Expected bigint,@Token char(64),
 @Inserted bigint=0,@Updated bigint=0,@Deleted bigint=0,@FactRows bigint,@Unknown int,
 @Files int=0,@Input bigint=0,@Error nvarchar(max);
DECLARE FileCursor CURSOR LOCAL FAST_FORWARD FOR SELECT
DatasetCode,SourceKey,ValidRows,SourceVersion FROM #DwhTodo ORDER BY DatasetCode,SourceKey;
OPEN FileCursor; FETCH NEXT FROM FileCursor INTO @Dataset,@Source,@Expected,@Token;
WHILE @@FETCH_STATUS=0
BEGIN
 BEGIN TRY
  BEGIN TRANSACTION;
  DROP TABLE IF EXISTS #CurrentFile; DROP TABLE IF EXISTS #Desired;
  SELECT * INTO #CurrentFile FROM #DwhTodo WHERE DatasetCode=@Dataset AND
SourceKey=@Source;
  IF NOT EXISTS(SELECT 1 FROM etl.DwhSourceState WITH(HOLDLOCK) WHERE DatasetCode=@Dataset
AND SourceKey=@Source AND SourceVersion=@Token)
   THROW 55940,'Source changed after DWH planning; retry after Transform.',1;
  IF EXISTS(SELECT 1 FROM NYCTaxi_Staging.clean.Trip C JOIN #CurrentFile P
    ON C.TaxiType=UPPER(P.DatasetCode) AND C.ETLSourceKey=P.SourceKey
    WHERE C.ETLFileHash IS NULL OR C.ETLFileHash<>P.FileHash)
   THROW 55947,'File clean lineage hash disagrees with Transform checkpoint.',1;
  IF (SELECT COUNT_BIG(*) FROM NYCTaxi_Staging.clean.Trip WHERE TaxiType=UPPER(@Dataset)
AND ETLSourceKey=@Source)
   <>(SELECT SourceRows FROM #CurrentFile)
   THROW 55948,'File clean row coverage is incomplete.',1;
;WITH SourceRows AS
(
  SELECT C.*,
    HASHBYTES('SHA2_256',
      (SELECT C.TaxiType AS TaxiType,
            C.VendorID AS VendorID,
            C.HVFHSLicenseNum AS HVFHSLicenseNum,
            C.DispatchingBaseNum AS DispatchingBaseNum,
            C.OriginatingBaseNum AS OriginatingBaseNum,
            C.AffiliatedBaseNum AS AffiliatedBaseNum,
            C.RequestDateTime AS RequestDateTime,
            C.OnSceneDateTime AS OnSceneDateTime,
            C.PickupDateTime AS PickupDateTime,
            C.DropoffDateTime AS DropoffDateTime,
            C.TripDurationSeconds AS TripDurationSeconds,
            C.PULocationID AS PULocationID,
            C.DOLocationID AS DOLocationID,
            C.PassengerCount AS PassengerCount,
            C.TripDistanceMiles AS TripDistanceMiles,
            C.RateCodeID AS RateCodeID,
            C.PaymentTypeID AS PaymentTypeID,
            C.TripTypeID AS TripTypeID,
            C.StoreAndFwdFlag AS StoreAndFwdFlag,
            C.FareAmount AS FareAmount,
            C.ExtraAmount AS ExtraAmount,
            C.MTATaxAmount AS MTATaxAmount,
            C.TipAmount AS TipAmount,
            C.TollsAmount AS TollsAmount,
            C.ImprovementSurcharge AS ImprovementSurcharge,
```

```
            C.CongestionSurcharge AS CongestionSurcharge,
            C.AirportFee AS AirportFee,
            C.CBDCongestionFee AS CBDCongestionFee,
            C.BCFAmount AS BCFAmount,
            C.SalesTaxAmount AS SalesTaxAmount,
            C.DriverPayAmount AS DriverPayAmount,
            C.TotalAmount AS TotalAmount,
            C.IsTotalCalculated AS IsTotalCalculated,
            C.SharedRequestFlag AS SharedRequestFlag, C.SourceSRFlag AS SourceSRFlag,
            C.SharedMatchFlag AS SharedMatchFlag,
            C.AccessARideFlag AS AccessARideFlag,
            C.WAVRequestFlag AS WAVRequestFlag,
            C.WAVMatchFlag AS WAVMatchFlag
       FOR JSON PATH, WITHOUT_ARRAY_WRAPPER, INCLUDE_NULL_VALUES)
    ) AS PayloadHash
  FROM NYCTaxi_Staging.clean.ValidTrip C
  INNER JOIN #CurrentFile F ON C.TaxiType=UPPER(F.DatasetCode) AND
C.ETLSourceKey=F.SourceKey
  WHERE C.TaxiType='FHV'
),
Ranked AS
(
  SELECT S.*, ROW_NUMBER() OVER(PARTITION BY S.TaxiType,S.ETLSourceKey,S.PayloadHash ORDER
BY S.TripCleanID) AS DuplicateOrdinal
  FROM SourceRows S
),
Tagged AS
(
  SELECT R.*,
    HASHBYTES('SHA2_256',
CONCAT(R.TaxiType,'|',R.ETLSourceKey,'|',CONVERT(varchar(64),R.PayloadHash,2),':',R.Duplica
teOrdinal)) AS RecordHash
  FROM Ranked R
),
FactValues AS (
SELECT
    COALESCE(PickupDate.DateKey, 0) AS PickupDateKey,
    COALESCE(DropoffDate.DateKey, 0) AS DropoffDateKey,


    COALESCE(PickupTime.TimeKey, 0) AS PickupTimeKey,
    COALESCE(DropoffTime.TimeKey, 0) AS DropoffTimeKey,


    COALESCE(PickupZone.LocationKey, 0)
        AS PickupLocationKey,


    COALESCE(DropoffZone.LocationKey, 0)
        AS DropoffLocationKey,


    COALESCE(ServiceType.ServiceTypeKey, 0)
        AS ServiceTypeKey,


    COALESCE(DispatchBase.BaseKey, 0)
        AS DispatchBaseKey,


    COALESCE(AffiliatedBase.BaseKey, 0)
        AS AffiliatedBaseKey,


    COALESCE(FHVFlag.FHVFlagKey, 0)
        AS FHVFlagKey,


    COALESCE(LoadBatch.LoadBatchKey,0) AS LoadBatchKey,
    1 AS TripCount,


    C.TripDurationSeconds,
```

```
    C.RecordHash AS RecordHash, LOWER(C.TaxiType) AS SourceDataset, C.ETLSourceKey AS
SourceKey,
    CASE WHEN (C.PULocationID IS NOT NULL AND COALESCE(PickupZone.LocationKey,0)=0) OR
(C.DOLocationID IS NOT NULL AND COALESCE(DropoffZone.LocationKey,0)=0) THEN 1 ELSE 0 END +
CASE WHEN (C.DispatchingBaseNum IS NOT NULL AND COALESCE(DispatchBase.BaseKey,0)=0) OR
(C.AffiliatedBaseNum IS NOT NULL AND COALESCE(AffiliatedBase.BaseKey,0)=0) THEN 2 ELSE 0
END AS UnresolvedMask

FROM Tagged AS C

LEFT JOIN dbo.Dim_Date AS PickupDate
    ON PickupDate.FullDate =
       CONVERT(DATE, C.PickupDateTime)

LEFT JOIN dbo.Dim_Date AS DropoffDate
    ON DropoffDate.FullDate =
       CONVERT(DATE, C.DropoffDateTime)

LEFT JOIN dbo.Dim_Time AS PickupTime
    ON PickupTime.MinuteOfDay =
       DATEPART(HOUR, C.PickupDateTime) * 60
       + DATEPART(MINUTE, C.PickupDateTime)

LEFT JOIN dbo.Dim_Time AS DropoffTime
    ON DropoffTime.MinuteOfDay =
       DATEPART(HOUR, C.DropoffDateTime) * 60
       + DATEPART(MINUTE, C.DropoffDateTime)

LEFT JOIN dbo.Dim_Location AS PickupZone
    ON PickupZone.LocationID = C.PULocationID

LEFT JOIN dbo.Dim_Location AS DropoffZone
    ON DropoffZone.LocationID = C.DOLocationID

LEFT JOIN dbo.Dim_ServiceType AS ServiceType
    ON ServiceType.ServiceCode = C.TaxiType

LEFT JOIN dbo.Dim_Base AS DispatchBase
    ON DispatchBase.BaseNumber = C.DispatchingBaseNum

LEFT JOIN dbo.Dim_Base AS AffiliatedBase
    ON AffiliatedBase.BaseNumber = C.AffiliatedBaseNum

LEFT JOIN dbo.Dim_FHVFlags AS FHVFlag
    ON FHVFlag.IsSharedRide =
       CASE C.SourceSRFlag WHEN '1' THEN 'Y' WHEN '0' THEN 'N' ELSE 'U' END

LEFT JOIN dbo.Dim_LoadBatch AS LoadBatch
    ON LoadBatch.BatchID = CONCAT('CLEAN-',C.TaxiType,'-
',CONVERT(char(6),C.PickupDateTime,112))

WHERE C.TaxiType = 'FHV'
)

  SELECT * INTO #Desired FROM FactValues;
  CREATE UNIQUE CLUSTERED INDEX IX_Desired_Hash ON #Desired(RecordHash);
  IF (SELECT COUNT_BIG(*) FROM #Desired)<>@Expected
   THROW 55941,'File clean rows disagree with Transform checkpoint.',1;
  IF EXISTS(SELECT 1 FROM #Desired WHERE ServiceTypeKey=0)
   THROW 55942,'Unknown ServiceTypeKey in changed file.',1;
  INSERT etl.FactChangeAudit(TableName,RowJson)
```

```
   SELECT N'Fact_FHV_Trips',(SELECT F.* FOR JSON PATH,WITHOUT_ARRAY_WRAPPER)
   FROM dbo.Fact_FHV_Trips F
   WHERE F.SourceDataset=@Dataset AND F.SourceKey=@Source
     AND NOT EXISTS(SELECT 1 FROM #Desired V WHERE V.RecordHash=F.RecordHash);
  DELETE F FROM dbo.Fact_FHV_Trips F
   WHERE F.SourceDataset=@Dataset AND F.SourceKey=@Source
     AND NOT EXISTS(SELECT 1 FROM #Desired V WHERE V.RecordHash=F.RecordHash);
  SET @Deleted=@@ROWCOUNT;
  UPDATE F SET
PickupDateKey=V.PickupDateKey,DropoffDateKey=V.DropoffDateKey,PickupTimeKey=V.PickupTimeKey
,DropoffTimeKey=V.DropoffTimeKey,PickupLocationKey=V.PickupLocationKey,DropoffLocationKey=V
.DropoffLocationKey,ServiceTypeKey=V.ServiceTypeKey,DispatchBaseKey=V.DispatchBaseKey,Affil
iatedBaseKey=V.AffiliatedBaseKey,FHVFlagKey=V.FHVFlagKey,LoadBatchKey=V.LoadBatchKey,TripCo
unt=V.TripCount,TripDurationSeconds=V.TripDurationSeconds,SourceDataset=V.SourceDataset,Sou
rceKey=V.SourceKey
   FROM dbo.Fact_FHV_Trips F JOIN #Desired V ON V.RecordHash=F.RecordHash
   WHERE F.SourceDataset=@Dataset AND F.SourceKey=@Source
     AND EXISTS(SELECT
F.PickupDateKey,F.DropoffDateKey,F.PickupTimeKey,F.DropoffTimeKey,F.PickupLocationKey,F.Dro
poffLocationKey,F.ServiceTypeKey,F.DispatchBaseKey,F.AffiliatedBaseKey,F.FHVFlagKey,F.LoadB
atchKey,F.TripCount,F.TripDurationSeconds,F.SourceDataset,F.SourceKey EXCEPT SELECT
V.PickupDateKey,V.DropoffDateKey,V.PickupTimeKey,V.DropoffTimeKey,V.PickupLocationKey,V.Dro
poffLocationKey,V.ServiceTypeKey,V.DispatchBaseKey,V.AffiliatedBaseKey,V.FHVFlagKey,V.LoadB
atchKey,V.TripCount,V.TripDurationSeconds,V.SourceDataset,V.SourceKey);
  SET @Updated=@@ROWCOUNT;
    INSERT INTO dbo.Fact_FHV_Trips
    (PickupDateKey, DropoffDateKey, PickupTimeKey, DropoffTimeKey, PickupLocationKey,
DropoffLocationKey, ServiceTypeKey, DispatchBaseKey, AffiliatedBaseKey, FHVFlagKey,
LoadBatchKey, TripCount, TripDurationSeconds, RecordHash, SourceDataset, SourceKey)
    SELECT V.PickupDateKey, V.DropoffDateKey, V.PickupTimeKey, V.DropoffTimeKey,
V.PickupLocationKey, V.DropoffLocationKey, V.ServiceTypeKey, V.DispatchBaseKey,
V.AffiliatedBaseKey, V.FHVFlagKey, V.LoadBatchKey, V.TripCount, V.TripDurationSeconds,
V.RecordHash, V.SourceDataset, V.SourceKey
    FROM #Desired V
    WHERE NOT EXISTS
      (SELECT 1 FROM dbo.Fact_FHV_Trips F WITH (UPDLOCK,HOLDLOCK) WHERE
F.RecordHash=V.RecordHash);


  SET @Inserted=@@ROWCOUNT;
  IF TRY_CONVERT(int,SESSION_CONTEXT(N'NYCTaxiFailAfterFactWrite'))=1
   THROW 55949,'Injected failure after Fact write, before checkpoint.',1;
  SELECT @FactRows=COUNT_BIG(*) FROM dbo.Fact_FHV_Trips WHERE SourceDataset=@Dataset AND
SourceKey=@Source;
  IF @FactRows<>@Expected THROW 55943,'File Fact reconciliation failed.',1;
  SELECT
@Unknown=COALESCE(MAX(UnresolvedMask&1),0)+COALESCE(MAX(UnresolvedMask&2),0)+COALESCE(MAX(U
nresolvedMask&4),0) FROM #Desired;
  DELETE etl.DwhFileLog WHERE DatasetCode=@Dataset AND SourceKey=@Source;
  INSERT
etl.DwhFileLog(DatasetCode,SourceKey,SourceVersion,LoaderVersion,Status,ForceReload,

ValidRows,FactRows,InsertedRows,UpdatedRows,DeletedRows,UnknownMask,LocationMapHash,BaseMap
Hash,OtherMapHash,CompletedAt)

VALUES(@Dataset,@Source,@Token,'FILE_FHV_V2_20261031','SUCCEEDED',0,@Expected,@FactRows,@In
serted,@Updated,@Deleted,@Unknown,
   @LocationMapHash,@BaseMapHash,@OtherMapHash,SYSUTCDATETIME());
  INSERT etl.LoadStepLog(StepName,InsertedRows) VALUES(N'Fact_FHV_Trips',@Inserted);
  COMMIT TRANSACTION;
  SET @Files+=1; SET @Input+=@Expected;
 END TRY
 BEGIN CATCH
  SET @Error=ERROR_MESSAGE(); IF @@TRANCOUNT>0 ROLLBACK;
  IF NOT EXISTS(SELECT 1 FROM etl.DwhFileLog WHERE DatasetCode=@Dataset AND
SourceKey=@Source)
   INSERT etl.DwhFileLog(DatasetCode,SourceKey,Status,ErrorMessage)
VALUES(@Dataset,@Source,'FAILED',@Error);
```

```
  ELSE UPDATE etl.DwhFileLog SET Status='FAILED',ErrorMessage=@Error WHERE
DatasetCode=@Dataset AND SourceKey=@Source;
  CLOSE FileCursor; DEALLOCATE FileCursor; THROW;
 END CATCH;
 FETCH NEXT FROM FileCursor INTO @Dataset,@Source,@Expected,@Token;
END;
CLOSE FileCursor; DEALLOCATE FileCursor;
DROP TABLE IF EXISTS #Desired; DROP TABLE IF EXISTS #CurrentFile; DROP TABLE #DwhTodo;
SELECT @Files AS ProcessedFiles,@Input AS ValidInputRows;
```

Bảng A.74 Cấu hình tác vụ EST_03_Load_FactFHV

Thuộc tính	**Giá trị source**
Connection	{4A829E7F-DE39-4A39-83D3-52CE94E192E2}

EST_04_Load_FactHVFHV

```
SET NOCOUNT ON; SET XACT_ABORT ON;


DECLARE @LocationMapHash char(64),@BaseMapHash char(64),@OtherMapHash char(64),@MapText
nvarchar(max);
SET @MapText=(SELECT LocationKey,LocationID FROM dbo.Dim_Location ORDER BY LocationKey FOR
JSON PATH);
SET @LocationMapHash=CONVERT(char(64),HASHBYTES('SHA2_256',@MapText),2);
SET @MapText=(SELECT BaseKey,BaseNumber FROM dbo.Dim_Base ORDER BY BaseKey FOR JSON PATH);
SET @BaseMapHash=CONVERT(char(64),HASHBYTES('SHA2_256',@MapText),2);
SET @MapText=CONCAT(
 (SELECT ServiceTypeKey,ServiceCode FROM dbo.Dim_ServiceType ORDER BY ServiceTypeKey FOR
JSON PATH),
 (SELECT VendorKey,VendorID FROM dbo.Dim_Vendor ORDER BY VendorKey FOR JSON PATH),
 (SELECT RateCodeKey,RateCodeID FROM dbo.Dim_RateCode ORDER BY RateCodeKey FOR JSON PATH),
 (SELECT PaymentTypeKey,PaymentTypeID FROM dbo.Dim_PaymentType ORDER BY PaymentTypeKey FOR
JSON PATH),
 (SELECT TripTypeKey,TripTypeID FROM dbo.Dim_TripType ORDER BY TripTypeKey FOR JSON PATH),
 (SELECT HVFHSProviderKey,LicenseNumber FROM dbo.Dim_HVFHSProvider ORDER BY
HVFHSProviderKey FOR JSON PATH));
SET @OtherMapHash=CONVERT(char(64),HASHBYTES('SHA2_256',@MapText),2);



DROP TABLE IF EXISTS #DwhTodo;
SELECT S.* INTO #DwhTodo FROM etl.DwhSourceState S
LEFT JOIN etl.DwhFileLog L ON L.DatasetCode=S.DatasetCode AND L.SourceKey=S.SourceKey
WHERE L.SourceKey IS NULL OR L.Status<>'SUCCEEDED' OR L.ForceReload=1
 OR L.SourceVersion<>S.SourceVersion OR L.LoaderVersion<>CASE WHEN S.DatasetCode='fhv' THEN
'FILE_FHV_V2_20261031' ELSE 'FILE_V1_20261031' END
 OR (L.UnknownMask&1=1 AND L.LocationMapHash<>@LocationMapHash)
 OR (L.UnknownMask&2=2 AND L.BaseMapHash<>@BaseMapHash)
 OR (L.UnknownMask&4=4 AND L.OtherMapHash<>@OtherMapHash);
CREATE UNIQUE CLUSTERED INDEX IX_DwhTodo ON #DwhTodo(DatasetCode,SourceKey);


DELETE FROM #DwhTodo WHERE DatasetCode NOT IN ('fhvhv');
DECLARE @Dataset varchar(20),@Source nvarchar(260),@Expected bigint,@Token char(64),
 @Inserted bigint=0,@Updated bigint=0,@Deleted bigint=0,@FactRows bigint,@Unknown int,
 @Files int=0,@Input bigint=0,@Error nvarchar(max);
DECLARE FileCursor CURSOR LOCAL FAST_FORWARD FOR SELECT
DatasetCode,SourceKey,ValidRows,SourceVersion FROM #DwhTodo ORDER BY DatasetCode,SourceKey;
OPEN FileCursor; FETCH NEXT FROM FileCursor INTO @Dataset,@Source,@Expected,@Token;
WHILE @@FETCH_STATUS=0
BEGIN
 BEGIN TRY
  BEGIN TRANSACTION;
  DROP TABLE IF EXISTS #CurrentFile; DROP TABLE IF EXISTS #Desired;
```

```
  SELECT * INTO #CurrentFile FROM #DwhTodo WHERE DatasetCode=@Dataset AND
SourceKey=@Source;
  IF NOT EXISTS(SELECT 1 FROM etl.DwhSourceState WITH(HOLDLOCK) WHERE DatasetCode=@Dataset
AND SourceKey=@Source AND SourceVersion=@Token)
   THROW 55940,'Source changed after DWH planning; retry after Transform.',1;
  IF EXISTS(SELECT 1 FROM NYCTaxi_Staging.clean.Trip C JOIN #CurrentFile P
    ON C.TaxiType=UPPER(P.DatasetCode) AND C.ETLSourceKey=P.SourceKey
    WHERE C.ETLFileHash IS NULL OR C.ETLFileHash<>P.FileHash)
   THROW 55947,'File clean lineage hash disagrees with Transform checkpoint.',1;
  IF (SELECT COUNT_BIG(*) FROM NYCTaxi_Staging.clean.Trip WHERE TaxiType=UPPER(@Dataset)
AND ETLSourceKey=@Source)
   <>(SELECT SourceRows FROM #CurrentFile)
   THROW 55948,'File clean row coverage is incomplete.',1;
;WITH SourceRows AS
(
  SELECT C.*,
    HASHBYTES('SHA2_256',
      (SELECT C.TaxiType AS TaxiType,
            C.VendorID AS VendorID,
            C.HVFHSLicenseNum AS HVFHSLicenseNum,
            C.DispatchingBaseNum AS DispatchingBaseNum,
            C.OriginatingBaseNum AS OriginatingBaseNum,
            C.AffiliatedBaseNum AS AffiliatedBaseNum,
            C.RequestDateTime AS RequestDateTime,
            C.OnSceneDateTime AS OnSceneDateTime,
            C.PickupDateTime AS PickupDateTime,
            C.DropoffDateTime AS DropoffDateTime,
            C.TripDurationSeconds AS TripDurationSeconds,
            C.PULocationID AS PULocationID,
            C.DOLocationID AS DOLocationID,
            C.PassengerCount AS PassengerCount,
            C.TripDistanceMiles AS TripDistanceMiles,
            C.RateCodeID AS RateCodeID,
            C.PaymentTypeID AS PaymentTypeID,
            C.TripTypeID AS TripTypeID,
            C.StoreAndFwdFlag AS StoreAndFwdFlag,
            C.FareAmount AS FareAmount,
            C.ExtraAmount AS ExtraAmount,
            C.MTATaxAmount AS MTATaxAmount,
            C.TipAmount AS TipAmount,
            C.TollsAmount AS TollsAmount,
            C.ImprovementSurcharge AS ImprovementSurcharge,
            C.CongestionSurcharge AS CongestionSurcharge,
            C.AirportFee AS AirportFee,
            C.CBDCongestionFee AS CBDCongestionFee,
            C.BCFAmount AS BCFAmount,
            C.SalesTaxAmount AS SalesTaxAmount,
            C.DriverPayAmount AS DriverPayAmount,
            C.TotalAmount AS TotalAmount,
            C.IsTotalCalculated AS IsTotalCalculated,
            C.SharedRequestFlag AS SharedRequestFlag,
            C.SharedMatchFlag AS SharedMatchFlag,
            C.AccessARideFlag AS AccessARideFlag,
            C.WAVRequestFlag AS WAVRequestFlag,
            C.WAVMatchFlag AS WAVMatchFlag
       FOR JSON PATH, WITHOUT_ARRAY_WRAPPER, INCLUDE_NULL_VALUES)
    ) AS PayloadHash
  FROM NYCTaxi_Staging.clean.ValidTrip C
  INNER JOIN #CurrentFile F ON C.TaxiType=UPPER(F.DatasetCode) AND
C.ETLSourceKey=F.SourceKey
  WHERE C.TaxiType='FHVHV'
),
Ranked AS
(
  SELECT S.*, ROW_NUMBER() OVER(PARTITION BY S.TaxiType,S.ETLSourceKey,S.PayloadHash ORDER
BY S.TripCleanID) AS DuplicateOrdinal
  FROM SourceRows S
),
```

```
Tagged AS
(
  SELECT R.*,
    HASHBYTES('SHA2_256',
CONCAT(R.TaxiType,'|',R.ETLSourceKey,'|',CONVERT(varchar(64),R.PayloadHash,2),':',R.Duplica
teOrdinal)) AS RecordHash
  FROM Ranked R
),
C AS
(
    SELECT
        T.*,

        CASE
            WHEN UPPER(T.SharedRequestFlag) IN ('Y', 'N')
                THEN UPPER(T.SharedRequestFlag)
            ELSE 'U'
        END AS NormalizedSharedRequest,

        CASE
            WHEN UPPER(T.SharedMatchFlag) IN ('Y', 'N')
                THEN UPPER(T.SharedMatchFlag)
            ELSE 'U'
        END AS NormalizedSharedMatch,

        CASE
            WHEN UPPER(T.AccessARideFlag) IN ('Y', 'N')
                THEN UPPER(T.AccessARideFlag)
            ELSE 'U'
        END AS NormalizedAccessARide,

        CASE
            WHEN UPPER(T.WAVRequestFlag) IN ('Y', 'N')
                THEN UPPER(T.WAVRequestFlag)
            ELSE 'U'
        END AS NormalizedWAVRequest,

        CASE
            WHEN UPPER(T.WAVMatchFlag) IN ('Y', 'N')
                THEN UPPER(T.WAVMatchFlag)
            ELSE 'U'
        END AS NormalizedWAVMatch

    FROM Tagged AS T
    WHERE T.TaxiType = 'FHVHV'
),
FactValues AS (
SELECT
    COALESCE(RequestDate.DateKey, 0) AS RequestDateKey,
    COALESCE(OnSceneDate.DateKey, 0) AS OnSceneDateKey,
    COALESCE(PickupDate.DateKey, 0) AS PickupDateKey,
    COALESCE(DropoffDate.DateKey, 0) AS DropoffDateKey,

    COALESCE(RequestTime.TimeKey, 0) AS RequestTimeKey,
    COALESCE(OnSceneTime.TimeKey, 0) AS OnSceneTimeKey,
    COALESCE(PickupTime.TimeKey, 0) AS PickupTimeKey,
    COALESCE(DropoffTime.TimeKey, 0) AS DropoffTimeKey,

    COALESCE(PickupZone.LocationKey, 0)
        AS PickupLocationKey,

    COALESCE(DropoffZone.LocationKey, 0)
        AS DropoffLocationKey,
```

```
    COALESCE(ServiceType.ServiceTypeKey, 0)
        AS ServiceTypeKey,

    COALESCE(Provider.HVFHSProviderKey, 0)
        AS HVFHSProviderKey,

    COALESCE(DispatchBase.BaseKey, 0)
        AS DispatchBaseKey,

    COALESCE(OriginBase.BaseKey, 0)
        AS OriginBaseKey,

    COALESCE(Flags.HVFHVFlagKey, 0)
        AS HVFHVFlagKey,

    COALESCE(LoadBatch.LoadBatchKey,0) AS LoadBatchKey,
    1 AS TripCount,

    C.TripDistanceMiles,
    C.TripDurationSeconds,

    CASE
        WHEN C.RequestDateTime IS NOT NULL
         AND C.PickupDateTime >= C.RequestDateTime
        THEN DATEDIFF_BIG(
            SECOND,
            C.RequestDateTime,
            C.PickupDateTime
        )
    END AS WaitingTimeSeconds,

    CASE
        WHEN C.OnSceneDateTime IS NOT NULL
         AND C.PickupDateTime >= C.OnSceneDateTime
        THEN DATEDIFF_BIG(
            SECOND,
            C.OnSceneDateTime,
            C.PickupDateTime
        )
    END AS OnSceneToPickupSeconds,

    C.FareAmount AS BasePassengerFare,
    C.TollsAmount,
    C.BCFAmount,
    C.SalesTaxAmount,
    C.CongestionSurcharge,
    C.AirportFee,
    C.TipAmount,
    C.DriverPayAmount,
    C.CBDCongestionFee,
    C.TotalAmount AS GrossPassengerAmount,
    C.RecordHash AS RecordHash, LOWER(C.TaxiType) AS SourceDataset, C.ETLSourceKey AS
SourceKey,
    CASE WHEN (C.PULocationID IS NOT NULL AND COALESCE(PickupZone.LocationKey,0)=0) OR
(C.DOLocationID IS NOT NULL AND COALESCE(DropoffZone.LocationKey,0)=0) THEN 1 ELSE 0 END +
CASE WHEN (C.DispatchingBaseNum IS NOT NULL AND COALESCE(DispatchBase.BaseKey,0)=0) OR
(C.OriginatingBaseNum IS NOT NULL AND COALESCE(OriginBase.BaseKey,0)=0) THEN 2 ELSE 0 END +
CASE WHEN (C.HVFHSLicenseNum IS NOT NULL AND COALESCE(Provider.HVFHSProviderKey,0)=0) THEN
4 ELSE 0 END AS UnresolvedMask

FROM C
```

```
LEFT JOIN dbo.Dim_Date AS RequestDate
    ON RequestDate.FullDate =
       CONVERT(DATE, C.RequestDateTime)

LEFT JOIN dbo.Dim_Date AS OnSceneDate
    ON OnSceneDate.FullDate =
       CONVERT(DATE, C.OnSceneDateTime)

LEFT JOIN dbo.Dim_Date AS PickupDate
    ON PickupDate.FullDate =
       CONVERT(DATE, C.PickupDateTime)

LEFT JOIN dbo.Dim_Date AS DropoffDate
    ON DropoffDate.FullDate =
       CONVERT(DATE, C.DropoffDateTime)

LEFT JOIN dbo.Dim_Time AS RequestTime
    ON RequestTime.MinuteOfDay =
       DATEPART(HOUR, C.RequestDateTime) * 60
       + DATEPART(MINUTE, C.RequestDateTime)

LEFT JOIN dbo.Dim_Time AS OnSceneTime
    ON OnSceneTime.MinuteOfDay =
       DATEPART(HOUR, C.OnSceneDateTime) * 60
       + DATEPART(MINUTE, C.OnSceneDateTime)

LEFT JOIN dbo.Dim_Time AS PickupTime
    ON PickupTime.MinuteOfDay =
       DATEPART(HOUR, C.PickupDateTime) * 60
       + DATEPART(MINUTE, C.PickupDateTime)

LEFT JOIN dbo.Dim_Time AS DropoffTime
    ON DropoffTime.MinuteOfDay =
       DATEPART(HOUR, C.DropoffDateTime) * 60
       + DATEPART(MINUTE, C.DropoffDateTime)

LEFT JOIN dbo.Dim_Location AS PickupZone
    ON PickupZone.LocationID = C.PULocationID

LEFT JOIN dbo.Dim_Location AS DropoffZone
    ON DropoffZone.LocationID = C.DOLocationID

LEFT JOIN dbo.Dim_ServiceType AS ServiceType
    ON ServiceType.ServiceCode = C.TaxiType

LEFT JOIN dbo.Dim_HVFHSProvider AS Provider
    ON Provider.LicenseNumber = C.HVFHSLicenseNum

LEFT JOIN dbo.Dim_Base AS DispatchBase
    ON DispatchBase.BaseNumber = C.DispatchingBaseNum

LEFT JOIN dbo.Dim_Base AS OriginBase
    ON OriginBase.BaseNumber = C.OriginatingBaseNum

LEFT JOIN dbo.Dim_HVFHVFlags AS Flags
    ON Flags.IsSharedRequest = C.NormalizedSharedRequest
   AND Flags.IsSharedMatch = C.NormalizedSharedMatch
   AND Flags.IsAccessARide = C.NormalizedAccessARide
   AND Flags.IsWAVRequest = C.NormalizedWAVRequest
```

```
   AND Flags.IsWAVMatch = C.NormalizedWAVMatch
LEFT JOIN dbo.Dim_LoadBatch AS LoadBatch
    ON LoadBatch.BatchID = CONCAT('CLEAN-',C.TaxiType,'-
',CONVERT(char(6),C.PickupDateTime,112))

)

  SELECT * INTO #Desired FROM FactValues;
  CREATE UNIQUE CLUSTERED INDEX IX_Desired_Hash ON #Desired(RecordHash);
  IF (SELECT COUNT_BIG(*) FROM #Desired)<>@Expected
   THROW 55941,'File clean rows disagree with Transform checkpoint.',1;
  IF EXISTS(SELECT 1 FROM #Desired WHERE ServiceTypeKey=0)
   THROW 55942,'Unknown ServiceTypeKey in changed file.',1;
  INSERT etl.FactChangeAudit(TableName,RowJson)
   SELECT N'Fact_HVFHV_Trips',(SELECT F.* FOR JSON PATH,WITHOUT_ARRAY_WRAPPER)
   FROM dbo.Fact_HVFHV_Trips F
   WHERE F.SourceDataset=@Dataset AND F.SourceKey=@Source
     AND NOT EXISTS(SELECT 1 FROM #Desired V WHERE V.RecordHash=F.RecordHash);
  DELETE F FROM dbo.Fact_HVFHV_Trips F
   WHERE F.SourceDataset=@Dataset AND F.SourceKey=@Source
     AND NOT EXISTS(SELECT 1 FROM #Desired V WHERE V.RecordHash=F.RecordHash);
  SET @Deleted=@@ROWCOUNT;
  UPDATE F SET
RequestDateKey=V.RequestDateKey,OnSceneDateKey=V.OnSceneDateKey,PickupDateKey=V.PickupDateK
ey,DropoffDateKey=V.DropoffDateKey,RequestTimeKey=V.RequestTimeKey,OnSceneTimeKey=V.OnScene
TimeKey,PickupTimeKey=V.PickupTimeKey,DropoffTimeKey=V.DropoffTimeKey,PickupLocationKey=V.P
ickupLocationKey,DropoffLocationKey=V.DropoffLocationKey,ServiceTypeKey=V.ServiceTypeKey,HV
FHSProviderKey=V.HVFHSProviderKey,DispatchBaseKey=V.DispatchBaseKey,OriginBaseKey=V.OriginB
aseKey,HVFHVFlagKey=V.HVFHVFlagKey,LoadBatchKey=V.LoadBatchKey,TripCount=V.TripCount,TripDi
stanceMiles=V.TripDistanceMiles,TripDurationSeconds=V.TripDurationSeconds,WaitingTimeSecond
s=V.WaitingTimeSeconds,OnSceneToPickupSeconds=V.OnSceneToPickupSeconds,BasePassengerFare=V.
BasePassengerFare,TollsAmount=V.TollsAmount,BCFAmount=V.BCFAmount,SalesTaxAmount=V.SalesTax
Amount,CongestionSurcharge=V.CongestionSurcharge,AirportFee=V.AirportFee,TipAmount=V.TipAmo
unt,DriverPayAmount=V.DriverPayAmount,CBDCongestionFee=V.CBDCongestionFee,GrossPassengerAmo
unt=V.GrossPassengerAmount,SourceDataset=V.SourceDataset,SourceKey=V.SourceKey
   FROM dbo.Fact_HVFHV_Trips F JOIN #Desired V ON V.RecordHash=F.RecordHash
   WHERE F.SourceDataset=@Dataset AND F.SourceKey=@Source
     AND EXISTS(SELECT
F.RequestDateKey,F.OnSceneDateKey,F.PickupDateKey,F.DropoffDateKey,F.RequestTimeKey,F.OnSce
neTimeKey,F.PickupTimeKey,F.DropoffTimeKey,F.PickupLocationKey,F.DropoffLocationKey,F.Servi
ceTypeKey,F.HVFHSProviderKey,F.DispatchBaseKey,F.OriginBaseKey,F.HVFHVFlagKey,F.LoadBatchKe
y,F.TripCount,F.TripDistanceMiles,F.TripDurationSeconds,F.WaitingTimeSeconds,F.OnSceneToPic
kupSeconds,F.BasePassengerFare,F.TollsAmount,F.BCFAmount,F.SalesTaxAmount,F.CongestionSurch
arge,F.AirportFee,F.TipAmount,F.DriverPayAmount,F.CBDCongestionFee,F.GrossPassengerAmount,F
.SourceDataset,F.SourceKey EXCEPT SELECT
V.RequestDateKey,V.OnSceneDateKey,V.PickupDateKey,V.DropoffDateKey,V.RequestTimeKey,V.OnSce
neTimeKey,V.PickupTimeKey,V.DropoffTimeKey,V.PickupLocationKey,V.DropoffLocationKey,V.Servi
ceTypeKey,V.HVFHSProviderKey,V.DispatchBaseKey,V.OriginBaseKey,V.HVFHVFlagKey,V.LoadBatchKe
y,V.TripCount,V.TripDistanceMiles,V.TripDurationSeconds,V.WaitingTimeSeconds,V.OnSceneToPic
kupSeconds,V.BasePassengerFare,V.TollsAmount,V.BCFAmount,V.SalesTaxAmount,V.CongestionSurch
arge,V.AirportFee,V.TipAmount,V.DriverPayAmount,V.CBDCongestionFee,V.GrossPassengerAmount,V
.SourceDataset,V.SourceKey);
  SET @Updated=@@ROWCOUNT;
    INSERT INTO dbo.Fact_HVFHV_Trips
    (RequestDateKey, OnSceneDateKey, PickupDateKey, DropoffDateKey, RequestTimeKey,
OnSceneTimeKey, PickupTimeKey, DropoffTimeKey, PickupLocationKey, DropoffLocationKey,
ServiceTypeKey, HVFHSProviderKey, DispatchBaseKey, OriginBaseKey, HVFHVFlagKey,
LoadBatchKey, TripCount, TripDistanceMiles, TripDurationSeconds, WaitingTimeSeconds,
OnSceneToPickupSeconds, BasePassengerFare, TollsAmount, BCFAmount, SalesTaxAmount,
CongestionSurcharge, AirportFee, TipAmount, DriverPayAmount, CBDCongestionFee,
GrossPassengerAmount, RecordHash, SourceDataset, SourceKey)
    SELECT V.RequestDateKey, V.OnSceneDateKey, V.PickupDateKey, V.DropoffDateKey,
V.RequestTimeKey, V.OnSceneTimeKey, V.PickupTimeKey, V.DropoffTimeKey, V.PickupLocationKey,
V.DropoffLocationKey, V.ServiceTypeKey, V.HVFHSProviderKey, V.DispatchBaseKey,
V.OriginBaseKey, V.HVFHVFlagKey, V.LoadBatchKey, V.TripCount, V.TripDistanceMiles,
V.TripDurationSeconds, V.WaitingTimeSeconds, V.OnSceneToPickupSeconds, V.BasePassengerFare,
V.TollsAmount, V.BCFAmount, V.SalesTaxAmount, V.CongestionSurcharge, V.AirportFee,
```

```
V.TipAmount, V.DriverPayAmount, V.CBDCongestionFee, V.GrossPassengerAmount, V.RecordHash,
V.SourceDataset, V.SourceKey
    FROM #Desired V
    WHERE NOT EXISTS
      (SELECT 1 FROM dbo.Fact_HVFHV_Trips F WITH (UPDLOCK,HOLDLOCK) WHERE
F.RecordHash=V.RecordHash);


  SET @Inserted=@@ROWCOUNT;
  IF TRY_CONVERT(int,SESSION_CONTEXT(N'NYCTaxiFailAfterFactWrite'))=1
   THROW 55949,'Injected failure after Fact write, before checkpoint.',1;
  SELECT @FactRows=COUNT_BIG(*) FROM dbo.Fact_HVFHV_Trips WHERE SourceDataset=@Dataset AND
SourceKey=@Source;
  IF @FactRows<>@Expected THROW 55943,'File Fact reconciliation failed.',1;
  SELECT
@Unknown=COALESCE(MAX(UnresolvedMask&1),0)+COALESCE(MAX(UnresolvedMask&2),0)+COALESCE(MAX(U
nresolvedMask&4),0) FROM #Desired;
  DELETE etl.DwhFileLog WHERE DatasetCode=@Dataset AND SourceKey=@Source;
  INSERT
etl.DwhFileLog(DatasetCode,SourceKey,SourceVersion,LoaderVersion,Status,ForceReload,

ValidRows,FactRows,InsertedRows,UpdatedRows,DeletedRows,UnknownMask,LocationMapHash,BaseMap
Hash,OtherMapHash,CompletedAt)

VALUES(@Dataset,@Source,@Token,'FILE_V1_20261031','SUCCEEDED',0,@Expected,@FactRows,@Insert
ed,@Updated,@Deleted,@Unknown,
   @LocationMapHash,@BaseMapHash,@OtherMapHash,SYSUTCDATETIME());
  INSERT etl.LoadStepLog(StepName,InsertedRows) VALUES(N'Fact_HVFHV_Trips',@Inserted);
  COMMIT TRANSACTION;
  SET @Files+=1; SET @Input+=@Expected;
 END TRY
 BEGIN CATCH
  SET @Error=ERROR_MESSAGE(); IF @@TRANCOUNT>0 ROLLBACK;
  IF NOT EXISTS(SELECT 1 FROM etl.DwhFileLog WHERE DatasetCode=@Dataset AND
SourceKey=@Source)
   INSERT etl.DwhFileLog(DatasetCode,SourceKey,Status,ErrorMessage)
VALUES(@Dataset,@Source,'FAILED',@Error);
  ELSE UPDATE etl.DwhFileLog SET Status='FAILED',ErrorMessage=@Error WHERE
DatasetCode=@Dataset AND SourceKey=@Source;
  CLOSE FileCursor; DEALLOCATE FileCursor; THROW;
 END CATCH;
 FETCH NEXT FROM FileCursor INTO @Dataset,@Source,@Expected,@Token;
END;
CLOSE FileCursor; DEALLOCATE FileCursor;
DROP TABLE IF EXISTS #Desired; DROP TABLE IF EXISTS #CurrentFile; DROP TABLE #DwhTodo;
SELECT @Files AS ProcessedFiles,@Input AS ValidInputRows;
```

Bảng A.75 Cấu hình tác vụ EST_04_Load_FactHVFHV

Thuộc tính	**Giá trị source**
Connection	{4A829E7F-DE39-4A39-83D3-52CE94E192E2}

EST_05_Validate_DWH

```
SET NOCOUNT ON;
IF EXISTS(SELECT 1 FROM NYCTaxi_Staging.etl.ExtractFileLog E
 LEFT JOIN etl.DwhSourceState S ON S.DatasetCode=E.DatasetCode AND S.SourceKey=E.SourceKey
 WHERE E.DatasetCode IN('yellow','green','fhv','fhvhv') AND E.LoadStatus='SUCCEEDED' AND
S.SourceKey IS NULL)
 THROW 55944,'Extract/Transform coverage is incomplete; DWH checkpoint cannot be
accepted.',1;


DECLARE @LocationMapHash char(64),@BaseMapHash char(64),@OtherMapHash char(64),@MapText
nvarchar(max);
```

```
SET @MapText=(SELECT LocationKey,LocationID FROM dbo.Dim_Location ORDER BY LocationKey FOR
JSON PATH);
SET @LocationMapHash=CONVERT(char(64),HASHBYTES('SHA2_256',@MapText),2);
SET @MapText=(SELECT BaseKey,BaseNumber FROM dbo.Dim_Base ORDER BY BaseKey FOR JSON PATH);
SET @BaseMapHash=CONVERT(char(64),HASHBYTES('SHA2_256',@MapText),2);
SET @MapText=CONCAT(
 (SELECT ServiceTypeKey,ServiceCode FROM dbo.Dim_ServiceType ORDER BY ServiceTypeKey FOR
JSON PATH),
 (SELECT VendorKey,VendorID FROM dbo.Dim_Vendor ORDER BY VendorKey FOR JSON PATH),
 (SELECT RateCodeKey,RateCodeID FROM dbo.Dim_RateCode ORDER BY RateCodeKey FOR JSON PATH),
 (SELECT PaymentTypeKey,PaymentTypeID FROM dbo.Dim_PaymentType ORDER BY PaymentTypeKey FOR
JSON PATH),
 (SELECT TripTypeKey,TripTypeID FROM dbo.Dim_TripType ORDER BY TripTypeKey FOR JSON PATH),
 (SELECT HVFHSProviderKey,LicenseNumber FROM dbo.Dim_HVFHSProvider ORDER BY
HVFHSProviderKey FOR JSON PATH));
SET @OtherMapHash=CONVERT(char(64),HASHBYTES('SHA2_256',@MapText),2);



DROP TABLE IF EXISTS #DwhTodo;
SELECT S.* INTO #DwhTodo FROM etl.DwhSourceState S
LEFT JOIN etl.DwhFileLog L ON L.DatasetCode=S.DatasetCode AND L.SourceKey=S.SourceKey
WHERE L.SourceKey IS NULL OR L.Status<>'SUCCEEDED' OR L.ForceReload=1
 OR L.SourceVersion<>S.SourceVersion OR L.LoaderVersion<>CASE WHEN S.DatasetCode='fhv' THEN
'FILE_FHV_V2_20261031' ELSE 'FILE_V1_20261031' END
 OR (L.UnknownMask&1=1 AND L.LocationMapHash<>@LocationMapHash)
 OR (L.UnknownMask&2=2 AND L.BaseMapHash<>@BaseMapHash)
 OR (L.UnknownMask&4=4 AND L.OtherMapHash<>@OtherMapHash);
CREATE UNIQUE CLUSTERED INDEX IX_DwhTodo ON #DwhTodo(DatasetCode,SourceKey);


IF EXISTS(SELECT 1 FROM #DwhTodo) THROW 55945,'DWH file checkpoints are incomplete.',1;
DROP TABLE #DwhTodo;
UPDATE dbo.Dim_LoadBatch SET BatchStatus='SUCCEEDED',LoadedAt=SYSDATETIME() WHERE
BatchStatus='READY' AND BatchID LIKE 'CLEAN-%';
SELECT COUNT(*) AS CompletedFiles, SUM(FactRows) AS CheckpointFactRows FROM etl.DwhFileLog
WHERE Status='SUCCEEDED';
```

Bảng A.76 Cấu hình tác vụ EST_05_Validate_DWH

Thuộc tính	**Giá trị source**
Connection	{4A829E7F-DE39-4A39-83D3-52CE94E192E2}

EST_LogOriginalError

```
INSERT etl.PackageError(ExecutionId,PackageName,SourceName,ErrorCode,ErrorDescription)
VALUES(?,?,?,?,?);
```

Bảng A.77 Cấu hình tác vụ EST_LogOriginalError

Thuộc tính	**Giá trị source**
Connection	{4A829E7F-DE39-4A39-83D3-52CE94E192E2}

Bảng A.78 Precedence constraint PKG_03_Load_DWH

Từ	**Đến**	**Điều kiện**
Package\EST_00_Precheck_DWH	Package\EST_01_Load_Dimensions	LogicalAnd=True

Từ	Đến	Điều kiện
Package\EST_01_Load_Dimensions	Package\EST_02_Load_FactYellowGreen	LogicalAnd=True
Package\EST_02_Load_FactYellowGreen	Package\EST_03_Load_FactFHV	LogicalAnd=True
Package\EST_03_Load_FactFHV	Package\EST_04_Load_FactHVFHV	LogicalAnd=True
Package\EST_04_Load_FactHVFHV	Package\EST_05_Validate_DWH	LogicalAnd=True

PKG_04_Load_Aggregates

EST_00_Atomic_Rebuild_And_Validate

```
SET XACT_ABORT ON; BEGIN TRY BEGIN TRANSACTION;
SET NOCOUNT ON;
SET XACT_ABORT ON;
IF DB_ID(N'NYCTaxi_DWH') IS NULL
    THROW 54001, 'NYCTaxi_DWH does not exist', 1;
IF OBJECT_ID(N'dbo.Fact_YellowGreen_Trips', N'U') IS NULL
 OR OBJECT_ID(N'dbo.Fact_FHV_Trips', N'U') IS NULL
 OR OBJECT_ID(N'dbo.Fact_HVFHV_Trips', N'U') IS NULL
    THROW 54002, 'Transactional facts are missing', 1;
IF OBJECT_ID(N'dbo.Fact_Daily_Financial_Snapshot', N'U') IS NULL
BEGIN
    CREATE TABLE dbo.Fact_Daily_Financial_Snapshot
    (
        DateKey INT NOT NULL,
        ServiceTypeKey INT NOT NULL,
        TotalTrips BIGINT NOT NULL,
        TotalDistanceMiles DECIMAL(24,3) NULL,
        TotalPassengerCharges DECIMAL(28,2) NULL,
        TotalTips DECIMAL(28,2) NULL,
        TotalTaxesAndFees DECIMAL(28,2) NULL,
        TotalDriverPay DECIMAL(28,2) NULL,
        SnapshotUpdatedAt DATETIME2(0) NOT NULL DEFAULT SYSDATETIME(),
        CONSTRAINT PK_Fact_Daily_Financial_Snapshot PRIMARY KEY (DateKey, ServiceTypeKey),
        CONSTRAINT FK_DFS_Date FOREIGN KEY (DateKey) REFERENCES dbo.Dim_Date(DateKey),
        CONSTRAINT FK_DFS_Service FOREIGN KEY (ServiceTypeKey) REFERENCES
dbo.Dim_ServiceType(ServiceTypeKey)
    );
END;
IF OBJECT_ID(N'dbo.Fact_Zone_Traffic_Snapshot', N'U') IS NULL
BEGIN
    CREATE TABLE dbo.Fact_Zone_Traffic_Snapshot
    (
        DateKey INT NOT NULL,
        TimeKey SMALLINT NOT NULL,
        LocationKey INT NOT NULL,
        ServiceTypeKey INT NOT NULL,
        TotalPickups BIGINT NOT NULL,
        TotalDropoffs BIGINT NOT NULL,
        SnapshotUpdatedAt DATETIME2(0) NOT NULL DEFAULT SYSDATETIME(),
        CONSTRAINT PK_Fact_Zone_Traffic_Snapshot PRIMARY KEY (DateKey, TimeKey,
LocationKey, ServiceTypeKey),
        CONSTRAINT FK_ZTS_Date FOREIGN KEY (DateKey) REFERENCES dbo.Dim_Date(DateKey),
        CONSTRAINT FK_ZTS_Time FOREIGN KEY (TimeKey) REFERENCES dbo.Dim_Time(TimeKey),
```

```
        CONSTRAINT FK_ZTS_Location FOREIGN KEY (LocationKey) REFERENCES
dbo.Dim_Location(LocationKey),
        CONSTRAINT FK_ZTS_Service FOREIGN KEY (ServiceTypeKey) REFERENCES
dbo.Dim_ServiceType(ServiceTypeKey)
    );
END;
IF OBJECT_ID(N'dbo.Fact_Base_Performance_Snapshot', N'U') IS NULL
BEGIN
    CREATE TABLE dbo.Fact_Base_Performance_Snapshot
    (
        DateKey INT NOT NULL,
        DispatchBaseKey INT NOT NULL,
        ServiceTypeKey INT NOT NULL,
        HVFHSProviderKey INT NOT NULL,
        TotalTrips BIGINT NOT NULL,
        TotalTripDurationSeconds BIGINT NULL,
        DurationTripCount BIGINT NOT NULL,
        AverageTripDurationSeconds DECIMAL(18,2) NULL,
        TotalWaitingTimeSeconds BIGINT NULL,
        WaitingTripCount BIGINT NOT NULL,
        AverageWaitingTimeSeconds DECIMAL(18,2) NULL,
        TotalSharedRequests BIGINT NULL,
        TotalSharedMatches BIGINT NULL,
        TotalWAVRequests BIGINT NULL,
        TotalWAVMatches BIGINT NULL,
        TotalDriverPay DECIMAL(28,2) NULL,
        SnapshotUpdatedAt DATETIME2(0) NOT NULL DEFAULT SYSDATETIME(),
        CONSTRAINT PK_Fact_Base_Performance_Snapshot PRIMARY KEY (DateKey, DispatchBaseKey,
ServiceTypeKey, HVFHSProviderKey),
        CONSTRAINT FK_BPS_Date FOREIGN KEY (DateKey) REFERENCES dbo.Dim_Date(DateKey),
        CONSTRAINT FK_BPS_Base FOREIGN KEY (DispatchBaseKey) REFERENCES
dbo.Dim_Base(BaseKey),
        CONSTRAINT FK_BPS_Service FOREIGN KEY (ServiceTypeKey) REFERENCES
dbo.Dim_ServiceType(ServiceTypeKey),
        CONSTRAINT FK_BPS_Provider FOREIGN KEY (HVFHSProviderKey) REFERENCES
dbo.Dim_HVFHSProvider(HVFHSProviderKey)
    );
END;


BEGIN TRY
BEGIN TRANSACTION;
IF NOT EXISTS(SELECT 1 FROM sys.schemas WHERE name=N'etl')
    EXEC(N'CREATE SCHEMA etl AUTHORIZATION dbo');
IF OBJECT_ID(N'etl.AggregateWatermark',N'U') IS NULL
    CREATE TABLE etl.AggregateWatermark
    (
       AggregateName varchar(50) NOT NULL,
       FactName varchar(50) NOT NULL,
       LastTripKey bigint NOT NULL DEFAULT 0,
       UpdatedAt datetime2(0) NOT NULL DEFAULT SYSDATETIME(),
       CONSTRAINT PK_AggregateWatermark PRIMARY KEY (AggregateName,FactName)
    );
-- If a snapshot already exists from the prior full-refresh package,
-- rebuild it ONCE from all facts, without touching transactional rows.
IF NOT EXISTS(SELECT 1 FROM etl.AggregateWatermark WHERE AggregateName='DailyFinancial')
BEGIN
  DELETE FROM dbo.Fact_Daily_Financial_Snapshot;
  INSERT etl.AggregateWatermark(AggregateName,FactName,LastTripKey)
  VALUES('DailyFinancial','YellowGreen',0),('DailyFinancial','HVFHV',0);
END;
IF NOT EXISTS(SELECT 1 FROM etl.AggregateWatermark WHERE AggregateName='ZoneTraffic')
BEGIN
  DELETE FROM dbo.Fact_Zone_Traffic_Snapshot;
  INSERT etl.AggregateWatermark(AggregateName,FactName,LastTripKey)
  VALUES('ZoneTraffic','YellowGreen',0),('ZoneTraffic','FHV',0),('ZoneTraffic','HVFHV',0);
END;
IF NOT EXISTS(SELECT 1 FROM etl.AggregateWatermark WHERE AggregateName='BasePerformance')
BEGIN
```

```
  DELETE FROM dbo.Fact_Base_Performance_Snapshot;
  INSERT etl.AggregateWatermark(AggregateName,FactName,LastTripKey)
  VALUES('BasePerformance','FHV',0),('BasePerformance','HVFHV',0);
END;
IF (SELECT COUNT(*) FROM etl.AggregateWatermark WHERE AggregateName='DailyFinancial')<>2
 OR (SELECT COUNT(*) FROM etl.AggregateWatermark WHERE AggregateName='ZoneTraffic')<>3
 OR (SELECT COUNT(*) FROM etl.AggregateWatermark WHERE AggregateName='BasePerformance')<>2
 THROW 55211,'AggregateWatermark has an incomplete state; inspect before loading.',1;
COMMIT TRANSACTION;
END TRY
BEGIN CATCH
 IF @@TRANCOUNT>0 ROLLBACK TRANSACTION;
 THROW;
END CATCH;


DELETE FROM dbo.Fact_Daily_Financial_Snapshot; DELETE FROM dbo.Fact_Zone_Traffic_Snapshot;
DELETE FROM dbo.Fact_Base_Performance_Snapshot; UPDATE etl.AggregateWatermark SET
LastTripKey=0,UpdatedAt=SYSDATETIME();
EXEC sys.sp_executesql N'SET NOCOUNT ON;
SET XACT_ABORT ON;
BEGIN TRY
 BEGIN TRAN;
 DECLARE @YellowGreenPrev bigint = (SELECT LastTripKey FROM etl.AggregateWatermark WITH
(UPDLOCK,HOLDLOCK) WHERE AggregateName=''DailyFinancial'' AND FactName=''YellowGreen'');
 DECLARE @YellowGreenMax bigint = ISNULL((SELECT MAX(TripKey) FROM
dbo.Fact_YellowGreen_Trips),0);
 IF @YellowGreenPrev IS NULL OR @YellowGreenMax < @YellowGreenPrev THROW 55101, ''Missing
watermark or transactional Fact keys reset; manual rebuild required.'',1;
 DECLARE @HVFHVPrev bigint = (SELECT LastTripKey FROM etl.AggregateWatermark WITH
(UPDLOCK,HOLDLOCK) WHERE AggregateName=''DailyFinancial'' AND FactName=''HVFHV'');
 DECLARE @HVFHVMax bigint = ISNULL((SELECT MAX(TripKey) FROM dbo.Fact_HVFHV_Trips),0);
 IF @HVFHVPrev IS NULL OR @HVFHVMax < @HVFHVPrev THROW 55101, ''Missing watermark or
transactional Fact keys reset; manual rebuild required.'',1;
;WITH FinancialSource AS
    (
        SELECT
            PickupDateKey AS DateKey, ServiceTypeKey,
            CONVERT(BIGINT, TripCount) AS Trips,
            CONVERT(DECIMAL(28,3), TripDistanceMiles) AS DistanceMiles,
            CONVERT(DECIMAL(28,2), TotalAmount) AS PassengerCharges,
            CONVERT(DECIMAL(28,2), TipAmount) AS Tips,
            CASE WHEN ExtraAmount IS NULL AND MTATaxAmount IS NULL
                   AND ImprovementSurcharge IS NULL AND CongestionSurcharge IS NULL
                   AND AirportFee IS NULL AND CBDCongestionFee IS NULL
                 THEN CAST(NULL AS DECIMAL(28,2))
                 ELSE CONVERT(DECIMAL(28,2),
                   COALESCE(ExtraAmount, 0) + COALESCE(MTATaxAmount, 0)
                   + COALESCE(ImprovementSurcharge, 0) + COALESCE(CongestionSurcharge, 0)
                   + COALESCE(AirportFee, 0) + COALESCE(CBDCongestionFee, 0)) END AS
TaxesAndFees,
            CAST(NULL AS DECIMAL(28,2)) AS DriverPay
        FROM dbo.Fact_YellowGreen_Trips WHERE TripKey>@YellowGreenPrev AND
TripKey<=@YellowGreenMax
        UNION ALL
        SELECT PickupDateKey, ServiceTypeKey,
            CONVERT(BIGINT, TripCount),
            CONVERT(DECIMAL(28,3), TripDistanceMiles),
            CONVERT(DECIMAL(28,2), GrossPassengerAmount),
            CONVERT(DECIMAL(28,2), TipAmount),
            CASE WHEN BCFAmount IS NULL AND SalesTaxAmount IS NULL
                   AND CongestionSurcharge IS NULL AND AirportFee IS NULL
                   AND CBDCongestionFee IS NULL
                 THEN CAST(NULL AS DECIMAL(28,2))
                 ELSE CONVERT(DECIMAL(28,2), COALESCE(BCFAmount, 0)
                    + COALESCE(SalesTaxAmount, 0) + COALESCE(CongestionSurcharge, 0)
                    + COALESCE(AirportFee, 0) + COALESCE(CBDCongestionFee, 0)) END,
            CONVERT(DECIMAL(28,2), DriverPayAmount)
        FROM dbo.Fact_HVFHV_Trips WHERE TripKey>@HVFHVPrev AND TripKey<=@HVFHVMax
```

```
    )

 SELECT DateKey,ServiceTypeKey,
        SUM(Trips) AS TotalTrips, SUM(DistanceMiles) AS TotalDistanceMiles,
        SUM(PassengerCharges) AS TotalPassengerCharges,SUM(Tips) AS TotalTips,
        SUM(TaxesAndFees) AS TotalTaxesAndFees,SUM(DriverPay) AS TotalDriverPay
 INTO #Delta FROM FinancialSource GROUP BY DateKey,ServiceTypeKey;


 UPDATE T SET
    T.TotalTrips=T.TotalTrips+S.TotalTrips,
    T.TotalDistanceMiles=CASE WHEN T.TotalDistanceMiles IS NULL AND S.TotalDistanceMiles IS
NULL THEN NULL ELSE COALESCE(T.TotalDistanceMiles,0)+COALESCE(S.TotalDistanceMiles,0) END,
    T.TotalPassengerCharges=CASE WHEN T.TotalPassengerCharges IS NULL AND
S.TotalPassengerCharges IS NULL THEN NULL ELSE
COALESCE(T.TotalPassengerCharges,0)+COALESCE(S.TotalPassengerCharges,0) END,
    T.TotalTips=CASE WHEN T.TotalTips IS NULL AND S.TotalTips IS NULL THEN NULL ELSE
COALESCE(T.TotalTips,0)+COALESCE(S.TotalTips,0) END,
    T.TotalTaxesAndFees=CASE WHEN T.TotalTaxesAndFees IS NULL AND S.TotalTaxesAndFees IS
NULL THEN NULL ELSE COALESCE(T.TotalTaxesAndFees,0)+COALESCE(S.TotalTaxesAndFees,0) END,
    T.TotalDriverPay=CASE WHEN T.TotalDriverPay IS NULL AND S.TotalDriverPay IS NULL THEN
NULL ELSE COALESCE(T.TotalDriverPay,0)+COALESCE(S.TotalDriverPay,0) END,
    T.SnapshotUpdatedAt=SYSDATETIME()
 FROM dbo.Fact_Daily_Financial_Snapshot T JOIN #Delta S ON T.DateKey=S.DateKey AND
T.ServiceTypeKey=S.ServiceTypeKey;
 INSERT dbo.Fact_Daily_Financial_Snapshot(DateKey, ServiceTypeKey, TotalTrips,
TotalDistanceMiles, TotalPassengerCharges, TotalTips, TotalTaxesAndFees, TotalDriverPay)
 SELECT S.DateKey, S.ServiceTypeKey, S.TotalTrips, S.TotalDistanceMiles,
S.TotalPassengerCharges, S.TotalTips, S.TotalTaxesAndFees, S.TotalDriverPay
 FROM #Delta S WHERE NOT EXISTS(SELECT 1 FROM dbo.Fact_Daily_Financial_Snapshot T
WITH(UPDLOCK,HOLDLOCK) WHERE T.DateKey=S.DateKey AND T.ServiceTypeKey=S.ServiceTypeKey);
 UPDATE etl.AggregateWatermark SET LastTripKey=@YellowGreenMax,UpdatedAt=SYSDATETIME()
WHERE AggregateName=''DailyFinancial'' AND FactName=''YellowGreen'';
UPDATE etl.AggregateWatermark SET LastTripKey=@HVFHVMax,UpdatedAt=SYSDATETIME() WHERE
AggregateName=''DailyFinancial'' AND FactName=''HVFHV'';
 COMMIT TRAN;
END TRY
BEGIN CATCH
 IF @@TRANCOUNT>0 ROLLBACK;
 THROW;
END CATCH;
';
EXEC sys.sp_executesql N'SET NOCOUNT ON;
SET XACT_ABORT ON;
BEGIN TRY
 BEGIN TRAN;
 DECLARE @YellowGreenPrev bigint = (SELECT LastTripKey FROM etl.AggregateWatermark WITH
(UPDLOCK,HOLDLOCK) WHERE AggregateName=''ZoneTraffic'' AND FactName=''YellowGreen'');
 DECLARE @YellowGreenMax bigint = ISNULL((SELECT MAX(TripKey) FROM
dbo.Fact_YellowGreen_Trips),0);
 IF @YellowGreenPrev IS NULL OR @YellowGreenMax < @YellowGreenPrev THROW 55101, ''Missing
watermark or transactional Fact keys reset; manual rebuild required.'',1;
 DECLARE @FHVPrev bigint = (SELECT LastTripKey FROM etl.AggregateWatermark WITH
(UPDLOCK,HOLDLOCK) WHERE AggregateName=''ZoneTraffic'' AND FactName=''FHV'');
 DECLARE @FHVMax bigint = ISNULL((SELECT MAX(TripKey) FROM dbo.Fact_FHV_Trips),0);
 IF @FHVPrev IS NULL OR @FHVMax < @FHVPrev THROW 55101, ''Missing watermark or
transactional Fact keys reset; manual rebuild required.'',1;
 DECLARE @HVFHVPrev bigint = (SELECT LastTripKey FROM etl.AggregateWatermark WITH
(UPDLOCK,HOLDLOCK) WHERE AggregateName=''ZoneTraffic'' AND FactName=''HVFHV'');
 DECLARE @HVFHVMax bigint = ISNULL((SELECT MAX(TripKey) FROM dbo.Fact_HVFHV_Trips),0);
 IF @HVFHVPrev IS NULL OR @HVFHVMax < @HVFHVPrev THROW 55101, ''Missing watermark or
transactional Fact keys reset; manual rebuild required.'',1;
;WITH ZoneEvents AS
    (
        SELECT PickupDateKey AS DateKey, PickupTimeKey AS TimeKey,
               PickupLocationKey AS LocationKey, ServiceTypeKey,
               CONVERT(BIGINT, TripCount) AS Pickups, CONVERT(BIGINT,0) AS Dropoffs
```

```
        FROM dbo.Fact_YellowGreen_Trips WHERE TripKey>@YellowGreenPrev AND
TripKey<=@YellowGreenMax
        UNION ALL
        SELECT DropoffDateKey, DropoffTimeKey, DropoffLocationKey, ServiceTypeKey,
               CONVERT(BIGINT,0), CONVERT(BIGINT, TripCount)
        FROM dbo.Fact_YellowGreen_Trips WHERE TripKey>@YellowGreenPrev AND
TripKey<=@YellowGreenMax
        UNION ALL
        SELECT PickupDateKey, PickupTimeKey, PickupLocationKey, ServiceTypeKey,
               CONVERT(BIGINT, TripCount), CONVERT(BIGINT,0)
        FROM dbo.Fact_FHV_Trips WHERE TripKey>@FHVPrev AND TripKey<=@FHVMax
        UNION ALL
        SELECT DropoffDateKey, DropoffTimeKey, DropoffLocationKey, ServiceTypeKey,
               CONVERT(BIGINT,0), CONVERT(BIGINT, TripCount)
        FROM dbo.Fact_FHV_Trips WHERE TripKey>@FHVPrev AND TripKey<=@FHVMax
        UNION ALL
        SELECT PickupDateKey, PickupTimeKey, PickupLocationKey, ServiceTypeKey,
               CONVERT(BIGINT, TripCount), CONVERT(BIGINT,0)
        FROM dbo.Fact_HVFHV_Trips WHERE TripKey>@HVFHVPrev AND TripKey<=@HVFHVMax
        UNION ALL
        SELECT DropoffDateKey, DropoffTimeKey, DropoffLocationKey, ServiceTypeKey,
               CONVERT(BIGINT,0), CONVERT(BIGINT, TripCount)
        FROM dbo.Fact_HVFHV_Trips WHERE TripKey>@HVFHVPrev AND TripKey<=@HVFHVMax
    )

 SELECT DateKey,TimeKey,LocationKey,ServiceTypeKey,
        SUM(Pickups) AS TotalPickups,SUM(Dropoffs) AS TotalDropoffs
 INTO #Delta FROM ZoneEvents GROUP BY DateKey,TimeKey,LocationKey,ServiceTypeKey;


 UPDATE T SET
    T.TotalPickups=T.TotalPickups+S.TotalPickups,
    T.TotalDropoffs=T.TotalDropoffs+S.TotalDropoffs,
    T.SnapshotUpdatedAt=SYSDATETIME()
 FROM dbo.Fact_Zone_Traffic_Snapshot T JOIN #Delta S ON T.DateKey=S.DateKey AND
T.TimeKey=S.TimeKey AND T.LocationKey=S.LocationKey AND T.ServiceTypeKey=S.ServiceTypeKey;
 INSERT dbo.Fact_Zone_Traffic_Snapshot(DateKey, TimeKey, LocationKey, ServiceTypeKey,
TotalPickups, TotalDropoffs)
 SELECT S.DateKey, S.TimeKey, S.LocationKey, S.ServiceTypeKey, S.TotalPickups,
S.TotalDropoffs
 FROM #Delta S WHERE NOT EXISTS(SELECT 1 FROM dbo.Fact_Zone_Traffic_Snapshot T
WITH(UPDLOCK,HOLDLOCK) WHERE T.DateKey=S.DateKey AND T.TimeKey=S.TimeKey AND
T.LocationKey=S.LocationKey AND T.ServiceTypeKey=S.ServiceTypeKey);
 UPDATE etl.AggregateWatermark SET LastTripKey=@YellowGreenMax,UpdatedAt=SYSDATETIME()
WHERE AggregateName=''ZoneTraffic'' AND FactName=''YellowGreen'';
UPDATE etl.AggregateWatermark SET LastTripKey=@FHVMax,UpdatedAt=SYSDATETIME() WHERE
AggregateName=''ZoneTraffic'' AND FactName=''FHV'';
UPDATE etl.AggregateWatermark SET LastTripKey=@HVFHVMax,UpdatedAt=SYSDATETIME() WHERE
AggregateName=''ZoneTraffic'' AND FactName=''HVFHV'';
 COMMIT TRAN;
END TRY
BEGIN CATCH
 IF @@TRANCOUNT>0 ROLLBACK;
 THROW;
END CATCH;
';
EXEC sys.sp_executesql N'SET NOCOUNT ON;
SET XACT_ABORT ON;
BEGIN TRY
 BEGIN TRAN;
 DECLARE @FHVPrev bigint = (SELECT LastTripKey FROM etl.AggregateWatermark WITH
(UPDLOCK,HOLDLOCK) WHERE AggregateName=''BasePerformance'' AND FactName=''FHV'');
 DECLARE @FHVMax bigint = ISNULL((SELECT MAX(TripKey) FROM dbo.Fact_FHV_Trips),0);
 IF @FHVPrev IS NULL OR @FHVMax < @FHVPrev THROW 55101, ''Missing watermark or
transactional Fact keys reset; manual rebuild required.'',1;
 DECLARE @HVFHVPrev bigint = (SELECT LastTripKey FROM etl.AggregateWatermark WITH
(UPDLOCK,HOLDLOCK) WHERE AggregateName=''BasePerformance'' AND FactName=''HVFHV'');
 DECLARE @HVFHVMax bigint = ISNULL((SELECT MAX(TripKey) FROM dbo.Fact_HVFHV_Trips),0);
```

```
 IF @HVFHVPrev IS NULL OR @HVFHVMax < @HVFHVPrev THROW 55101, ''Missing watermark or
transactional Fact keys reset; manual rebuild required.'',1;
;WITH BaseRows AS
    (
        SELECT PickupDateKey AS DateKey, DispatchBaseKey, ServiceTypeKey,
               CONVERT(INT, 0) AS HVFHSProviderKey,
               CONVERT(BIGINT, TripCount) AS Trips,
               TripDurationSeconds AS DurationSeconds,
               CAST(NULL AS BIGINT) AS WaitingSeconds,
               CAST(NULL AS BIGINT) AS SharedRequests,
               CAST(NULL AS BIGINT) AS SharedMatches,
               CAST(NULL AS BIGINT) AS WAVRequests,
               CAST(NULL AS BIGINT) AS WAVMatches,
               CAST(NULL AS DECIMAL(28,2)) AS DriverPay
        FROM dbo.Fact_FHV_Trips WHERE TripKey>@FHVPrev AND TripKey<=@FHVMax
        UNION ALL
        SELECT F.PickupDateKey, F.DispatchBaseKey, F.ServiceTypeKey,
               F.HVFHSProviderKey, CONVERT(BIGINT, F.TripCount),
               F.TripDurationSeconds, F.WaitingTimeSeconds,
               CONVERT(BIGINT, CASE WHEN FL.IsSharedRequest = ''Y'' THEN 1 ELSE 0 END),
               CONVERT(BIGINT, CASE WHEN FL.IsSharedMatch = ''Y'' THEN 1 ELSE 0 END),
               CONVERT(BIGINT, CASE WHEN FL.IsWAVRequest = ''Y'' THEN 1 ELSE 0 END),
               CONVERT(BIGINT, CASE WHEN FL.IsWAVMatch = ''Y'' THEN 1 ELSE 0 END),
               CONVERT(DECIMAL(28,2), F.DriverPayAmount)
        FROM dbo.Fact_HVFHV_Trips F
        LEFT JOIN dbo.Dim_HVFHVFlags FL
            ON FL.HVFHVFlagKey = F.HVFHVFlagKey
        WHERE F.TripKey>@HVFHVPrev AND F.TripKey<=@HVFHVMax
    )


 SELECT DateKey,DispatchBaseKey,ServiceTypeKey,HVFHSProviderKey,
        SUM(Trips) AS TotalTrips,
        SUM(DurationSeconds) AS TotalTripDurationSeconds,
        COUNT_BIG(DurationSeconds) AS DurationTripCount,

CONVERT(decimal(18,2),SUM(CONVERT(decimal(28,2),DurationSeconds))/NULLIF(COUNT_BIG(Duration
Seconds),0)) AS AverageTripDurationSeconds,
        SUM(WaitingSeconds) AS TotalWaitingTimeSeconds,
        COUNT_BIG(WaitingSeconds) AS WaitingTripCount,

CONVERT(decimal(18,2),SUM(CONVERT(decimal(28,2),WaitingSeconds))/NULLIF(COUNT_BIG(WaitingSe
conds),0)) AS AverageWaitingTimeSeconds,
        SUM(SharedRequests) AS TotalSharedRequests,
        SUM(SharedMatches) AS TotalSharedMatches,
        SUM(WAVRequests) AS TotalWAVRequests,
        SUM(WAVMatches) AS TotalWAVMatches,
        SUM(DriverPay) AS TotalDriverPay
 INTO #Delta FROM BaseRows GROUP BY
DateKey,DispatchBaseKey,ServiceTypeKey,HVFHSProviderKey;


 UPDATE T SET
    T.TotalTrips=T.TotalTrips+S.TotalTrips,
    T.TotalTripDurationSeconds=CASE WHEN T.TotalTripDurationSeconds IS NULL AND
S.TotalTripDurationSeconds IS NULL THEN NULL ELSE
COALESCE(T.TotalTripDurationSeconds,0)+COALESCE(S.TotalTripDurationSeconds,0) END,
    T.DurationTripCount=T.DurationTripCount+S.DurationTripCount,
    T.TotalWaitingTimeSeconds=CASE WHEN T.TotalWaitingTimeSeconds IS NULL AND
S.TotalWaitingTimeSeconds IS NULL THEN NULL ELSE
COALESCE(T.TotalWaitingTimeSeconds,0)+COALESCE(S.TotalWaitingTimeSeconds,0) END,
    T.WaitingTripCount=T.WaitingTripCount+S.WaitingTripCount,
    T.TotalSharedRequests=CASE WHEN T.TotalSharedRequests IS NULL AND S.TotalSharedRequests
IS NULL THEN NULL ELSE COALESCE(T.TotalSharedRequests,0)+COALESCE(S.TotalSharedRequests,0)
END,
    T.TotalSharedMatches=CASE WHEN T.TotalSharedMatches IS NULL AND S.TotalSharedMatches IS
NULL THEN NULL ELSE COALESCE(T.TotalSharedMatches,0)+COALESCE(S.TotalSharedMatches,0) END,
    T.TotalWAVRequests=CASE WHEN T.TotalWAVRequests IS NULL AND S.TotalWAVRequests IS NULL
THEN NULL ELSE COALESCE(T.TotalWAVRequests,0)+COALESCE(S.TotalWAVRequests,0) END,
```

```
    T.TotalWAVMatches=CASE WHEN T.TotalWAVMatches IS NULL AND S.TotalWAVMatches IS NULL
THEN NULL ELSE COALESCE(T.TotalWAVMatches,0)+COALESCE(S.TotalWAVMatches,0) END,
    T.TotalDriverPay=CASE WHEN T.TotalDriverPay IS NULL AND S.TotalDriverPay IS NULL THEN
NULL ELSE COALESCE(T.TotalDriverPay,0)+COALESCE(S.TotalDriverPay,0) END,

T.AverageTripDurationSeconds=CONVERT(decimal(18,2),(CONVERT(decimal(28,2),COALESCE(T.TotalT
ripDurationSeconds,0))+COALESCE(S.TotalTripDurationSeconds,0))/NULLIF(T.DurationTripCount+S
.DurationTripCount,0)),

T.AverageWaitingTimeSeconds=CONVERT(decimal(18,2),(CONVERT(decimal(28,2),COALESCE(T.TotalWa
itingTimeSeconds,0))+COALESCE(S.TotalWaitingTimeSeconds,0))/NULLIF(T.WaitingTripCount+S.Wai
tingTripCount,0)),
    T.SnapshotUpdatedAt=SYSDATETIME()
 FROM dbo.Fact_Base_Performance_Snapshot T JOIN #Delta S ON T.DateKey=S.DateKey AND
T.DispatchBaseKey=S.DispatchBaseKey AND T.ServiceTypeKey=S.ServiceTypeKey AND
T.HVFHSProviderKey=S.HVFHSProviderKey;
 INSERT dbo.Fact_Base_Performance_Snapshot(DateKey, DispatchBaseKey, ServiceTypeKey,
HVFHSProviderKey, TotalTrips, TotalTripDurationSeconds, DurationTripCount,
AverageTripDurationSeconds, TotalWaitingTimeSeconds, WaitingTripCount,
AverageWaitingTimeSeconds, TotalSharedRequests, TotalSharedMatches, TotalWAVRequests,
TotalWAVMatches, TotalDriverPay)
 SELECT S.DateKey, S.DispatchBaseKey, S.ServiceTypeKey, S.HVFHSProviderKey, S.TotalTrips,
S.TotalTripDurationSeconds, S.DurationTripCount, S.AverageTripDurationSeconds,
S.TotalWaitingTimeSeconds, S.WaitingTripCount, S.AverageWaitingTimeSeconds,
S.TotalSharedRequests, S.TotalSharedMatches, S.TotalWAVRequests, S.TotalWAVMatches,
S.TotalDriverPay
 FROM #Delta S WHERE NOT EXISTS(SELECT 1 FROM dbo.Fact_Base_Performance_Snapshot T
WITH(UPDLOCK,HOLDLOCK) WHERE T.DateKey=S.DateKey AND T.DispatchBaseKey=S.DispatchBaseKey
AND T.ServiceTypeKey=S.ServiceTypeKey AND T.HVFHSProviderKey=S.HVFHSProviderKey);
 UPDATE etl.AggregateWatermark SET LastTripKey=@FHVMax,UpdatedAt=SYSDATETIME() WHERE
AggregateName=''BasePerformance'' AND FactName=''FHV'';
UPDATE etl.AggregateWatermark SET LastTripKey=@HVFHVMax,UpdatedAt=SYSDATETIME() WHERE
AggregateName=''BasePerformance'' AND FactName=''HVFHV'';
 COMMIT TRAN;
END TRY
BEGIN CATCH
 IF @@TRANCOUNT>0 ROLLBACK;
 THROW;
END CATCH;
';
EXEC sys.sp_executesql N'SET NOCOUNT ON;
-- Financial excludes ordinary FHV by design.
DECLARE @FinancialSource BIGINT =
    (SELECT COALESCE(SUM(CONVERT(BIGINT,TripCount)),0)
     FROM dbo.Fact_YellowGreen_Trips) +
    (SELECT COALESCE(SUM(CONVERT(BIGINT,TripCount)),0)
     FROM dbo.Fact_HVFHV_Trips);
DECLARE @AllSource BIGINT = @FinancialSource +
    (SELECT COALESCE(SUM(CONVERT(BIGINT,TripCount)),0)
     FROM dbo.Fact_FHV_Trips);
IF @FinancialSource <> (SELECT COALESCE(SUM(TotalTrips),0) FROM
dbo.Fact_Daily_Financial_Snapshot)
    THROW 54101, ''Financial trip totals do not match'', 1;
IF @AllSource <> (SELECT COALESCE(SUM(TotalPickups),0) FROM dbo.Fact_Zone_Traffic_Snapshot)
    THROW 54102, ''Zone pickup totals do not match'', 1;
IF @AllSource <> (SELECT COALESCE(SUM(TotalDropoffs),0) FROM
dbo.Fact_Zone_Traffic_Snapshot)
    THROW 54103, ''Zone dropoff totals do not match'', 1;
DECLARE @BaseSource BIGINT =
    (SELECT COALESCE(SUM(CONVERT(BIGINT,TripCount)),0) FROM dbo.Fact_FHV_Trips) +
    (SELECT COALESCE(SUM(CONVERT(BIGINT,TripCount)),0) FROM dbo.Fact_HVFHV_Trips);
IF @BaseSource <> (SELECT COALESCE(SUM(TotalTrips),0) FROM
dbo.Fact_Base_Performance_Snapshot)
    THROW 54104, ''Base performance trip totals do not match'', 1;


IF EXISTS(SELECT 1 FROM etl.AggregateWatermark W
WHERE (W.FactName=''YellowGreen'' AND W.LastTripKey < (SELECT ISNULL(MAX(TripKey),0) FROM
dbo.Fact_YellowGreen_Trips))
```

```
   OR (W.FactName=''FHV'' AND W.LastTripKey < (SELECT ISNULL(MAX(TripKey),0) FROM
dbo.Fact_FHV_Trips))
   OR (W.FactName=''HVFHV'' AND W.LastTripKey < (SELECT ISNULL(MAX(TripKey),0) FROM
dbo.Fact_HVFHV_Trips)))
 THROW 55112,''Aggregate watermarks lag transactional facts.'',1;
';
COMMIT TRANSACTION; END TRY BEGIN CATCH IF @@TRANCOUNT>0 ROLLBACK; THROW; END CATCH;
```

Bảng A.79 Cấu hình tác vụ EST_00_Atomic_Rebuild_And_Validate

Thuộc tính	**Giá trị source**
Connection	{4A829E7F-DE39-4A39-83D3-52CE94E192E2}

EST_LogOriginalError

```
INSERT etl.PackageError(ExecutionId,PackageName,SourceName,ErrorCode,ErrorDescription)
VALUES(?,?,?,?,?);
```

Bảng A.80 Cấu hình tác vụ EST_LogOriginalError

Thuộc tính	**Giá trị source**
Connection	{4A829E7F-DE39-4A39-83D3-52CE94E192E2}

MDX script Base Performance

```
CALCULATE;
CREATE MEMBER CURRENTCUBE.[Measures].[Base Average Duration Minutes] AS
IIF(IsEmpty([Measures].[Base Performance Snapshot DurationTripCount]) OR [Measures].[Base
Performance Snapshot DurationTripCount] = 0, NULL, ([Measures].[Base Performance Snapshot
TotalTripDurationSeconds]) / ([Measures].[Base Performance Snapshot DurationTripCount]) *
0.016666666666666666), FORMAT_STRING = '#,##0.00', DISPLAY_FOLDER = 'Base Performance',
VISIBLE = 1;
CREATE MEMBER CURRENTCUBE.[Measures].[Base Average Waiting Minutes] AS
IIF(IsEmpty([Measures].[Base Performance Snapshot WaitingTripCount]) OR [Measures].[Base
Performance Snapshot WaitingTripCount] = 0, NULL, ([Measures].[Base Performance Snapshot
TotalWaitingTimeSeconds]) / ([Measures].[Base Performance Snapshot WaitingTripCount]) *
0.016666666666666666), FORMAT_STRING = '#,##0.00', DISPLAY_FOLDER = 'Base Performance',
VISIBLE = 1;
CREATE MEMBER CURRENTCUBE.[Measures].[Shared Request Rate] AS IIF(IsEmpty(([Measures].[Base
Performance Snapshot TotalTrips], [Dim Service Type].[Service Code].&[FHVHV])) OR
([Measures].[Base Performance Snapshot TotalTrips], [Dim Service Type].[Service
Code].&[FHVHV]) = 0, NULL, (([Measures].[Base Performance Snapshot TotalSharedRequests],
[Dim Service Type].[Service Code].&[FHVHV])) / (([Measures].[Base Performance Snapshot
TotalTrips], [Dim Service Type].[Service Code].&[FHVHV])) * 1), FORMAT_STRING = '0.00%',
DISPLAY_FOLDER = 'Base Performance', VISIBLE = 1;
CREATE MEMBER CURRENTCUBE.[Measures].[WAV Match Rate] AS IIF(IsEmpty([Measures].[Base
Performance Snapshot TotalWAVRequests]) OR [Measures].[Base Performance Snapshot
TotalWAVRequests] = 0, NULL, ([Measures].[Base Performance Snapshot TotalWAVMatches]) /
([Measures].[Base Performance Snapshot TotalWAVRequests]) * 1), FORMAT_STRING = '0.00%',
DISPLAY_FOLDER = 'Base Performance', VISIBLE = 0;
```

MDX script Daily Financial

```
CALCULATE;
```

MDX script FHV Trips

```
CALCULATE;
CREATE MEMBER CURRENTCUBE.[Measures].[FHV Average Trip Duration Minutes] AS
IIF(IsEmpty([Measures].[Fact FHV Trips TripDurationSecondsValidTrips]) OR [Measures].[Fact
FHV Trips TripDurationSecondsValidTrips] = 0, NULL, ([Measures].[Trip Duration Seconds]) /
([Measures].[Fact FHV Trips TripDurationSecondsValidTrips]) * 0.016666666666666666),
FORMAT_STRING = '#,##0.00', DISPLAY_FOLDER = 'FHV', VISIBLE = 1;
```

MDX script HVFHV Trips

```
CALCULATE;
CREATE MEMBER CURRENTCUBE.[Measures].[HVFHV Average Trip Duration Minutes] AS
IIF(IsEmpty([Measures].[Fact HVFHV Trips TripDurationSecondsValidTrips]) OR
[Measures].[Fact HVFHV Trips TripDurationSecondsValidTrips] = 0, NULL, ([Measures].[Trip
Duration Seconds - Fact HVFHV Trips]) / ([Measures].[Fact HVFHV Trips
TripDurationSecondsValidTrips]) * 0.016666666666666666), FORMAT_STRING = '#,##0.00',
DISPLAY_FOLDER = 'HVFHV', VISIBLE = 1;
CREATE MEMBER CURRENTCUBE.[Measures].[HVFHV Average Trip Distance] AS
IIF(IsEmpty([Measures].[Fact HVFHV Trips TripDistanceMilesValidTrips]) OR [Measures].[Fact
HVFHV Trips TripDistanceMilesValidTrips] = 0, NULL, ([Measures].[Trip Distance Miles]) /
([Measures].[Fact HVFHV Trips TripDistanceMilesValidTrips]) * 1), FORMAT_STRING =
'#,##0.00', DISPLAY_FOLDER = 'HVFHV', VISIBLE = 1;
CREATE MEMBER CURRENTCUBE.[Measures].[HVFHV Average Speed MPH] AS
IIF(IsEmpty([Measures].[Fact HVFHV Trips SpeedDurationSeconds]) OR [Measures].[Fact HVFHV
Trips SpeedDurationSeconds] = 0, NULL, ([Measures].[Fact HVFHV Trips SpeedDistanceMiles]) /
([Measures].[Fact HVFHV Trips SpeedDurationSeconds]) * 3600), FORMAT_STRING = '#,##0.00',
DISPLAY_FOLDER = 'HVFHV', VISIBLE = 1;
CREATE MEMBER CURRENTCUBE.[Measures].[HVFHV Average Fare Per Trip] AS
IIF(IsEmpty([Measures].[Fact HVFHV Trips BasePassengerFareValidTrips]) OR [Measures].[Fact
HVFHV Trips BasePassengerFareValidTrips] = 0, NULL, ([Measures].[Base Passenger Fare]) /
([Measures].[Fact HVFHV Trips BasePassengerFareValidTrips]) * 1), FORMAT_STRING =
'#,##0.00', DISPLAY_FOLDER = 'HVFHV', VISIBLE = 1;
CREATE MEMBER CURRENTCUBE.[Measures].[HVFHV Average Tip Per Trip] AS
IIF(IsEmpty([Measures].[Fact HVFHV Trips TipAmountValidTrips]) OR [Measures].[Fact HVFHV
Trips TipAmountValidTrips] = 0, NULL, ([Measures].[Tip Amount]) / ([Measures].[Fact HVFHV
Trips TipAmountValidTrips]) * 1), FORMAT_STRING = '#,##0.00', DISPLAY_FOLDER = 'HVFHV',
VISIBLE = 1;
CREATE MEMBER CURRENTCUBE.[Measures].[HVFHV Tip Percentage] AS IIF(IsEmpty([Measures].[Fact
HVFHV Trips TipPairFare]) OR [Measures].[Fact HVFHV Trips TipPairFare] = 0, NULL,
([Measures].[Fact HVFHV Trips TipPairAmount]) / ([Measures].[Fact HVFHV Trips TipPairFare])
* 1), FORMAT_STRING = '0.00%', DISPLAY_FOLDER = 'HVFHV', VISIBLE = 1;
CREATE MEMBER CURRENTCUBE.[Measures].[HVFHV Average Waiting Minutes] AS
IIF(IsEmpty([Measures].[Fact HVFHV Trips WaitingTimeSecondsValidTrips]) OR [Measures].[Fact
HVFHV Trips WaitingTimeSecondsValidTrips] = 0, NULL, ([Measures].[Waiting Time Seconds]) /
([Measures].[Fact HVFHV Trips WaitingTimeSecondsValidTrips]) * 0.016666666666666666),
FORMAT_STRING = '#,##0.00', DISPLAY_FOLDER = 'HVFHV', VISIBLE = 1;
```

MDX script YellowGreen Trips

```
CALCULATE;
CREATE MEMBER CURRENTCUBE.[Measures].[YellowGreen Average Trip Duration Minutes] AS
IIF(IsEmpty([Measures].[Fact Yellow Green Trips TripDurationSecondsValidTrips]) OR
[Measures].[Fact Yellow Green Trips TripDurationSecondsValidTrips] = 0, NULL,
([Measures].[Trip Duration Seconds - Fact Yellow Green Trips]) / ([Measures].[Fact Yellow
Green Trips TripDurationSecondsValidTrips]) * 0.016666666666666666), FORMAT_STRING =
'#,##0.00', DISPLAY_FOLDER = 'YellowGreen', VISIBLE = 1;
CREATE MEMBER CURRENTCUBE.[Measures].[YellowGreen Average Trip Distance] AS
IIF(IsEmpty([Measures].[Fact Yellow Green Trips TripDistanceMilesValidTrips]) OR
[Measures].[Fact Yellow Green Trips TripDistanceMilesValidTrips] = 0, NULL,
([Measures].[Trip Distance Miles - Fact Yellow Green Trips]) / ([Measures].[Fact Yellow
Green Trips TripDistanceMilesValidTrips]) * 1), FORMAT_STRING = '#,##0.00', DISPLAY_FOLDER
= 'YellowGreen', VISIBLE = 1;
CREATE MEMBER CURRENTCUBE.[Measures].[YellowGreen Average Speed MPH] AS
IIF(IsEmpty([Measures].[Fact Yellow Green Trips SpeedDurationSeconds]) OR [Measures].[Fact
Yellow Green Trips SpeedDurationSeconds] = 0, NULL, ([Measures].[Fact Yellow Green Trips
SpeedDistanceMiles]) / ([Measures].[Fact Yellow Green Trips SpeedDurationSeconds]) * 3600),
FORMAT_STRING = '#,##0.00', DISPLAY_FOLDER = 'YellowGreen', VISIBLE = 1;
CREATE MEMBER CURRENTCUBE.[Measures].[YellowGreen Average Fare Per Trip] AS
IIF(IsEmpty([Measures].[Fact Yellow Green Trips FareAmountValidTrips]) OR [Measures].[Fact
Yellow Green Trips FareAmountValidTrips] = 0, NULL, ([Measures].[Fare Amount]) /
([Measures].[Fact Yellow Green Trips FareAmountValidTrips]) * 1), FORMAT_STRING =
'#,##0.00', DISPLAY_FOLDER = 'YellowGreen', VISIBLE = 1;
CREATE MEMBER CURRENTCUBE.[Measures].[YellowGreen Average Tip Per Trip] AS
IIF(IsEmpty([Measures].[Fact Yellow Green Trips TipAmountValidTrips]) OR [Measures].[Fact
Yellow Green Trips TipAmountValidTrips] = 0, NULL, ([Measures].[Tip Amount - Fact Yellow
Green Trips]) / ([Measures].[Fact Yellow Green Trips TipAmountValidTrips]) * 1),
FORMAT_STRING = '#,##0.00', DISPLAY_FOLDER = 'YellowGreen', VISIBLE = 1;
```

```
CREATE MEMBER CURRENTCUBE.[Measures].[YellowGreen Tip Percentage] AS
IIF(IsEmpty([Measures].[Fact Yellow Green Trips TipPairFare]) OR [Measures].[Fact Yellow
Green Trips TipPairFare] = 0, NULL, ([Measures].[Fact Yellow Green Trips TipPairAmount]) /
([Measures].[Fact Yellow Green Trips TipPairFare]) * 1), FORMAT_STRING = '0.00%',
DISPLAY_FOLDER = 'YellowGreen', VISIBLE = 1;
```

MDX script Zone Traffic

```
CALCULATE;
```

PHỤ LỤC B TRUY VẤN KIỂM TRA VÀ KẾT QUẢ ĐỐI SOÁT

Các bảng bằng chứng dưới đây là kết quả đã lưu trong dự án. Chúng được dùng để đánh giá phạm vi kiểm chứng của hệ thống, không phải kết quả chạy lại tại ngày biên soạn. Mỗi nhóm kiểm tra có mục đích riêng: đếm dòng, toàn vẹn khóa, trạng thái xử lý hoặc sự nhất quán SQL MDX.

Bảng row-counts.csv là ảnh chụp metadata ngày 19 tháng 10 năm 2026, trước lần rà soát ngày 31 tháng 10. Số dòng trong bảng này được giữ như bằng chứng lịch sử; không dùng thay tổng 558.183 chuyến hợp lệ của lần rà soát về sau. Các bảng kiểm tra khác cần được đọc theo thời điểm bằng chứng tương ứng.

Bảng B.1 Bằng chứng lưu trong row-counts.csv

TableName	**RowCount**
Dim_Base	921
Dim_Date	5846
Dim_FHVFlags	3
Dim_HVFHSProvider	3
Dim_HVFHVFlags	15
Dim_LoadBatch	53
Dim_Location	266
Dim_PassengerGroup	6
Dim_PaymentType	7
Dim_RateCode	8
Dim_ServiceType	5
Dim_TaxiFlags	3
Dim_Time	1441

TableName	RowCount
Dim_TripType	3
Dim_Vendor	4
Fact_Base_Performance_Snapshot	8015
Fact_Daily_Financial_Snapshot	411
Fact_FHV_Trips	1199560
Fact_HVFHV_Trips	1200000
Fact_YellowGreen_Trips	1772699
Fact_Zone_Traffic_Snapshot	3564940

Bảng B.2 Bằng chứng lưu trong foreign-key-audit.csv

Fact cột và FK	FactRows	Orphan	Unknown dòng và %	Enabled và Trusted
Fact_FHV_Trips.PickupDateKey FK_FHV_PickupDate	139970	0	0; 0%	True / True
Fact_FHV_Trips.DropoffDateKey FK_FHV_DropoffDate	139970	0	0; 0%	True / True
Fact_HVFHV_Trips.RequestDateKey FK_HVFHV_RequestDate	140000	0	0; 0%	True / True
Fact_HVFHV_Trips.OnSceneDateKey FK_HVFHV_OnSceneDate	140000	0	13753; 9.82357142857143%	True / True
Fact_HVFHV_Trips.PickupDateKey FK_HVFHV_PickupDate	140000	0	0; 0%	True / True
Fact_HVFHV_Trips.DropoffDateKey FK_HVFHV_DropoffDate	140000	0	0; 0%	True / True
Fact_Daily_Financial_Snapshot.DateKey FK_DFS_Date	203	0	0; 0%	True / True

Fact cột và FK	FactRows	Orphan	Unknown dòng và %	Enabled và Trusted
Fact_Zone_Traffic_Snapshot.DateKey FK_ZTS_Date	672766	0	0; 0%	True / True
Fact_Base_Performance_Snapshot.DateKey FK_BPS_Date	3703	0	0; 0%	True / True
Fact_YellowGreen_Trips.PickupDateKey FK_FYG_PickupDate	278213	0	0; 0%	True / True
Fact_YellowGreen_Trips.DropoffDateKey FK_FYG_DropoffDate	278213	0	0; 0%	True / True
Fact_FHV_Trips.PickupTimeKey FK_FHV_PickupTime	139970	0	0; 0%	True / True
Fact_FHV_Trips.DropoffTimeKey FK_FHV_DropoffTime	139970	0	0; 0%	True / True
Fact_HVFHV_Trips.RequestTimeKey FK_HVFHV_RequestTime	140000	0	0; 0%	True / True
Fact_HVFHV_Trips.OnSceneTimeKey FK_HVFHV_OnSceneTime	140000	0	13753; 9.823571428571433%	True / True
Fact_HVFHV_Trips.PickupTimeKey FK_HVFHV_PickupTime	140000	0	0; 0%	True / True
Fact_HVFHV_Trips.DropoffTimeKey FK_HVFHV_DropoffTime	140000	0	0; 0%	True / True
Fact_Zone_Traffic_Snapshot.TimeKey FK_ZTS_Time	672766	0	0; 0%	True / True
Fact_YellowGreen_Trips.PickupTimeKey FK_FYG_PickupTime	278213	0	0; 0%	True / True
Fact_YellowGreen_Trips.DropoffTimeKey FK_FYG_DropoffTime	278213	0	0; 0%	True / True

Fact cột và FK	FactRows	Orphan	Unknown dòng và %	Enabled và Trusted
Fact_FHV_Trips.PickupLocationKey FK_FHV_PickupLocation	139970	0	121337; 86.6878616846467%	True / True
Fact_FHV_Trips.DropoffLocationKey FK_FHV_DropoffLocation	139970	0	20006; 14.2930627991713%	True / True
Fact_HVFHV_Trips.PickupLocationKey FK_HVFHV_PickupLocation	140000	0	0; 0%	True / True
Fact_HVFHV_Trips.DropoffLocationKey FK_HVFHV_DropoffLocation	140000	0	0; 0%	True / True
Fact_Zone_Traffic_Snapshot.LocationKey FK_ZTS_Location	672766	0	8556; 1.27176462544183%	True / True
Fact_YellowGreen_Trips.PickupLocationKey FK_FYG_PickupLocation	278213	0	0; 0%	True / True
Fact_YellowGreen_Trips.DropoffLocationKey FK_FYG_DropoffLocation	278213	0	0; 0%	True / True
Fact_FHV_Trips.ServiceTypeKey FK_FHV_ServiceType	139970	0	0; 0%	True / True
Fact_HVFHV_Trips.ServiceTypeKey FK_HVFHV_ServiceType	140000	0	0; 0%	True / True
Fact_Daily_Financial_Snapshot.ServiceTypeKey FK_DFS_Service	203	0	0; 0%	True / True
Fact_Zone_Traffic_Snapshot.ServiceTypeKey FK_ZTS_Service	672766	0	0; 0%	True / True
Fact_Base_Performance_Snapshot.ServiceTypeKey FK_BPS_Service	3703	0	0; 0%	True / True
Fact_YellowGreen_Trips.ServiceTypeKey FK_FYG_ServiceType	278213	0	0; 0%	True / True

Fact cột và FK	FactRows	Orphan	Unknown dòng và %	Enabled và Trusted
Fact_YellowGreen_Trips.VendorKey FK_FYG_Vendor	278213	0	0; 0%	True / True
Fact_HVFHV_Trips.HVFHSProviderKey FK_HVFHV_Provider	140000	0	0; 0%	True / True
Fact_Base_Performance_Snapshot.HVFHSProviderKey FK_BPS_Provider	3703	0	3618; 97.7045638671348%	True / True
Fact_FHV_Trips.DispatchBaseKey FK_FHV_DispatchBase	139970	0	0; 0%	True / True
Fact_FHV_Trips.AffiliatedBaseKey FK_FHV_AffiliatedBase	139970	0	2472; 1.766092734157352%	True / True
Fact_HVFHV_Trips.DispatchBaseKey FK_HVFHV_DispatchBase	140000	0	0; 0%	True / True
Fact_HVFHV_Trips.OriginBaseKey FK_HVFHV_OriginBase	140000	0	41980; 29.9857142857143%	True / True
Fact_Base_Performance_Snapshot.DispatchBaseKey FK_BPS_Base	3703	0	0; 0%	True / True
Fact_YellowGreen_Trips.RateCodeKey FK_FYG_RateCode	278213	0	0; 0%	True / True
Fact_YellowGreen_Trips.PaymentTypeKey FK_FYG_PaymentType	278213	0	0; 0%	True / True
Fact_YellowGreen_Trips.TripTypeKey FK_FYG_TripType	278213	0	138335; 49.722694482285 2%	True / True
Fact_YellowGreen_Trips.TaxiFlagKey FK_FYG_TaxiFlag	278213	0	0; 0%	True / True
Fact_FHV_Trips.FHVFlagKey FK_FHV_Flag	139970	0	139970; 100%	True / True

Fact cột và FK	FactRows	Orphan	Unknown dòng và %	Enabled và Trusted
Fact_HVFHV_Trips.HVFHVFlagKey FK_HVFHV_Flag	140000	0	0; 0%	True / True
Fact_YellowGreen_Trips.PassengerGroupKey FK_FYG_PassengerGroup	278213	0	0; 0%	True / True
Fact_FHV_Trips.LoadBatchKey FK_FHV_LoadBatch	139970	0	0; 0%	True / True
Fact_HVFHV_Trips.LoadBatchKey FK_HVFHV_LoadBatch	140000	0	0; 0%	True / True
Fact_YellowGreen_Trips.LoadBatchKey FK_FYG_LoadBatch	278213	0	0; 0%	True / True

Bảng B.3 Bằng chứng lưu trong browse-checks.csv

Query	Cells	Passed
SELECT {[Measures].[Trip Count]} ON COLUMNS FROM [FHV Trips]	1	True
SELECT {[Measures].[Gross Passenger Amount]} ON COLUMNS FROM [HVFHV Trips]	1	True
SELECT {[Measures].[Fact Yellow Green Trips TripDurationSecondsValidTrips]} ON COLUMNS FROM [YellowGreen Trips]	1	True
SELECT {[Measures].[Daily Financial Snapshot TotalDistanceMiles]} ON COLUMNS FROM [Daily Financial]	1	True

Query	Cells	Passed
SELECT {[Measures].[Zone Traffic Snapshot TotalDropoffs]} ON COLUMNS FROM [Zone Traffic]	1	True
SELECT {[Measures].[Base Performance Snapshot TotalWAVMatches]} ON COLUMNS FROM [Base Performance]	1	True

Bảng B.4 Bằng chứng lưu trong reconciliation-Full.csv

Check	SQLValue	CubeValue	Passed
FHV trips	140970	140970	True
FHV date 20250101	9998	9998	True
FHV date 20250115	NULL	NULL	True
FHV pickup location 0	122225	122225	True
FHV pickup location 1	50	50	True
FHV pickup location 132	175	175	True
FHV pickup location 138	108	108	True
FHV borough Manhattan	3036	3036	True
FHV borough Queens	5423	5423	True
FHV avg duration	23.147444018822	23.147444018822	True
HVFHV trips	141000	141000	True

Check	SQLValue	CubeValue	Passed
HVFHV date 20250101	10000	10000	True
HVFHV date 20250115	NULL	NULL	True
HVFHV pickup location 0	NULL	NULL	True
HVFHV pickup location 1	NULL	NULL	True
HVFHV pickup location 132	2541	2541	True
HVFHV pickup location 138	3377	3377	True
HVFHV borough Manhattan	52414	52414	True
HVFHV borough Queens	27740	27740	True
HVFHV avg duration	17.291456501182	17.291456501182	True
HVFHV avg distance	5.019193	5.01919315602837	True
HVFHV speed	17.4162072085205	17.4162072085203	True
HVFHV avg fare	25.396676	25.3966766666667	True
HVFHV avg tip	0.883538	0.883538652482269	True
HVFHV tip percentage	0.034789	0.0347895381777222	True
YellowGreen trips	280208	280208	True

Check	SQLValue	CubeValue	Passed
YellowGreen date 20250101	10938	10938	True
YellowGreen date 20250115	NULL	NULL	True
YellowGreen pickup location 0	NULL	NULL	True
YellowGreen pickup location 1	33	33	True
YellowGreen pickup location 132	10241	10241	True
YellowGreen pickup location 138	3933	3933	True
YellowGreen borough Manhattan	205161	205161	True
YellowGreen borough Queens	48337	48337	True
YellowGreen avg duration	18.2126691600525	18.2126691600525	True
YellowGreen avg distance	4.889733	4.88973312682009	True
YellowGreen speed	16.1087858693832	16.1087858693832	True
YellowGreen avg fare	18.341510	18.3415102352538	True
YellowGreen avg tip	2.909045	2.90904503083424	True
YellowGreen tip percentage	0.158604	0.158604443882861	True
Service YELLOW	139321	139321	True

Check	SQLValue	CubeValue	Passed
Service GREEN	140887	140887	True
Service FHV	140970	140970	True
Service FHVHV	141000	141000	True
YellowGreen revenue	7136096.19	7136096.18999999	True
HVFHV driver pay	2825515.00	2825515	True
HVFHV avg waiting	5.83893615156275	5.83893615156275	True
Daily Financial Snapshot TotalTrips	421208	421208	True
Daily Financial Snapshot TotalDistanceMiles	2077848.575	2077848.575	True
Daily Financial Snapshot TotalPassengerCharges	11075652.05	11075652.05	True
Daily Financial Snapshot TotalTips	939716.64	939716.64	True
Daily Financial Snapshot TotalTaxesAndFees	1792308.04	1792308.04	True
Daily Financial Snapshot TotalDriverPay	2825515.00	2825515	True
Zone Traffic Snapshot TotalPickups	562178	562178	True

Check	SQLValue	CubeValue	Passed
Zone Traffic Snapshot TotalDropoffs	562178	562178	True
Base Performance Snapshot TotalTrips	281970	281970	True
Base Performance Snapshot TotalTripDurationSeconds	342071433	342071433	True
Base Performance Snapshot DurationTripCount	281970	281970	True
Base Performance Snapshot TotalWaitingTimeSeconds	49293350	49293350	True
Base Performance Snapshot WaitingTripCount	140703	140703	True
Base Performance Snapshot TotalSharedRequests	5886	5886	True
Base Performance Snapshot TotalSharedMatches	3804	3804	True
Base Performance Snapshot TotalWAVRequests	81	81	True
Base Performance Snapshot TotalWAVMatches	7482	7482	True

Check	SQLValue	CubeValue	Passed
Base Performance Snapshot TotalDriverPay	2825515.00	2825515	True
Shared request rate	0.0417446808510638	0.0417446808510638	True
WAV match rate	92.3703703703704	92.3703703703704	True
Dispatch base 0	NULL	NULL	True
Dispatch base 1	NULL	NULL	True
Dispatch base 2	NULL	NULL	True
Dispatch base 3	NULL	NULL	True
Base avg duration	20.2191387381636	20.2191387381636	True
Base avg waiting	5.83893615156275	5.83893615156275	True
Snapshot pickups date 20250101	30936	30936	True
Snapshot pickups date 20250115	NULL	NULL	True
Snapshot dropoffs location 0	20066	20066	True
Snapshot dropoffs location 1	799	799	True
Snapshot dropoffs location 132	5198	5198	True
Fact_FHV_Trips Unknown batch	NULL	NULL	True
Fact_FHV_Trips Unknown time	NULL	NULL	True

Check	SQLValue	CubeValue	Passed
Fact_FHV_Trips Unknown date	NULL	NULL	True
Fact_FHV_Trips hour 0	17609	17609	True
Fact_FHV_Trips hour 8	6381	6381	True
Fact_FHV_Trips hour 17	299	299	True
Fact_FHV_Trips calendar Jan2025	9998	9998	True
Fact_FHV_Trips calendar Q12025	29998	29998	True
Fact_HVFHV_Trips Unknown batch	NULL	NULL	True
Fact_HVFHV_Trips Unknown time	NULL	NULL	True
Fact_HVFHV_Trips Unknown date	NULL	NULL	True
Fact_HVFHV_Trips hour 0	138772	138772	True
Fact_HVFHV_Trips hour 8	NULL	NULL	True
Fact_HVFHV_Trips hour 17	NULL	NULL	True
Fact_HVFHV_Trips calendar Jan2025	10000	10000	True
Fact_HVFHV_Trips calendar Q12025	30000	30000	True

Check	SQLValue	CubeValue	Passed
Fact_YellowGreen_Trips Unknown batch	NULL	NULL	True
Fact_YellowGreen_Trips Unknown time	NULL	NULL	True
Fact_YellowGreen_Trips Unknown date	NULL	NULL	True
Fact_YellowGreen_Trips hour 0	42881	42881	True
Fact_YellowGreen_Trips hour 8	14985	14985	True
Fact_YellowGreen_Trips hour 17	11028	11028	True
Fact_YellowGreen_Trips calendar Jan2025	19997	19997	True
Fact_YellowGreen_Trips calendar Q12025	59938	59938	True

Truy vấn dwh-integrity.sql

```
-- FK_FHV_PickupDate
SELECT COUNT_BIG(*) FROM dbo.[Fact_FHV_Trips] f LEFT JOIN dbo.[Dim_Date] d ON
f.[PickupDateKey]=d.[DateKey] WHERE d.[DateKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [PickupDateKey]=0 THEN 1 ELSE 0 END AS
bigint)) AS UnknownRows FROM dbo.[Fact_FHV_Trips];
-- FK_FHV_DropoffDate
SELECT COUNT_BIG(*) FROM dbo.[Fact_FHV_Trips] f LEFT JOIN dbo.[Dim_Date] d ON
f.[DropoffDateKey]=d.[DateKey] WHERE d.[DateKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [DropoffDateKey]=0 THEN 1 ELSE 0 END AS
bigint)) AS UnknownRows FROM dbo.[Fact_FHV_Trips];
-- FK_HVFHV_RequestDate
SELECT COUNT_BIG(*) FROM dbo.[Fact_HVFHV_Trips] f LEFT JOIN dbo.[Dim_Date] d ON
f.[RequestDateKey]=d.[DateKey] WHERE d.[DateKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [RequestDateKey]=0 THEN 1 ELSE 0 END AS
bigint)) AS UnknownRows FROM dbo.[Fact_HVFHV_Trips];
-- FK_HVFHV_OnSceneDate
SELECT COUNT_BIG(*) FROM dbo.[Fact_HVFHV_Trips] f LEFT JOIN dbo.[Dim_Date] d ON
f.[OnSceneDateKey]=d.[DateKey] WHERE d.[DateKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [OnSceneDateKey]=0 THEN 1 ELSE 0 END AS
bigint)) AS UnknownRows FROM dbo.[Fact_HVFHV_Trips];
-- FK_HVFHV_PickupDate
SELECT COUNT_BIG(*) FROM dbo.[Fact_HVFHV_Trips] f LEFT JOIN dbo.[Dim_Date] d ON
f.[PickupDateKey]=d.[DateKey] WHERE d.[DateKey] IS NULL;
```

```
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [PickupDateKey]=0 THEN 1 ELSE 0 END AS
bigint)) AS UnknownRows FROM dbo.[Fact_HVFHV_Trips];
-- FK_HVFHV_DropoffDate
SELECT COUNT_BIG(*) FROM dbo.[Fact_HVFHV_Trips] f LEFT JOIN dbo.[Dim_Date] d ON
f.[DropoffDateKey]=d.[DateKey] WHERE d.[DateKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [DropoffDateKey]=0 THEN 1 ELSE 0 END AS
bigint)) AS UnknownRows FROM dbo.[Fact_HVFHV_Trips];
-- FK_DFS_Date
SELECT COUNT_BIG(*) FROM dbo.[Fact_Daily_Financial_Snapshot] f LEFT JOIN dbo.[Dim_Date] d
ON f.[DateKey]=d.[DateKey] WHERE d.[DateKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [DateKey]=0 THEN 1 ELSE 0 END AS
bigint)) AS UnknownRows FROM dbo.[Fact_Daily_Financial_Snapshot];
-- FK_ZTS_Date
SELECT COUNT_BIG(*) FROM dbo.[Fact_Zone_Traffic_Snapshot] f LEFT JOIN dbo.[Dim_Date] d ON
f.[DateKey]=d.[DateKey] WHERE d.[DateKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [DateKey]=0 THEN 1 ELSE 0 END AS
bigint)) AS UnknownRows FROM dbo.[Fact_Zone_Traffic_Snapshot];
-- FK_BPS_Date
SELECT COUNT_BIG(*) FROM dbo.[Fact_Base_Performance_Snapshot] f LEFT JOIN dbo.[Dim_Date] d
ON f.[DateKey]=d.[DateKey] WHERE d.[DateKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [DateKey]=0 THEN 1 ELSE 0 END AS
bigint)) AS UnknownRows FROM dbo.[Fact_Base_Performance_Snapshot];
-- FK_FYG_PickupDate
SELECT COUNT_BIG(*) FROM dbo.[Fact_YellowGreen_Trips] f LEFT JOIN dbo.[Dim_Date] d ON
f.[PickupDateKey]=d.[DateKey] WHERE d.[DateKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [PickupDateKey]=0 THEN 1 ELSE 0 END AS
bigint)) AS UnknownRows FROM dbo.[Fact_YellowGreen_Trips];
-- FK_FYG_DropoffDate
SELECT COUNT_BIG(*) FROM dbo.[Fact_YellowGreen_Trips] f LEFT JOIN dbo.[Dim_Date] d ON
f.[DropoffDateKey]=d.[DateKey] WHERE d.[DateKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [DropoffDateKey]=0 THEN 1 ELSE 0 END AS
bigint)) AS UnknownRows FROM dbo.[Fact_YellowGreen_Trips];
-- FK_FHV_PickupTime
SELECT COUNT_BIG(*) FROM dbo.[Fact_FHV_Trips] f LEFT JOIN dbo.[Dim_Time] d ON
f.[PickupTimeKey]=d.[TimeKey] WHERE d.[TimeKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [PickupTimeKey]=0 THEN 1 ELSE 0 END AS
bigint)) AS UnknownRows FROM dbo.[Fact_FHV_Trips];
-- FK_FHV_DropoffTime
SELECT COUNT_BIG(*) FROM dbo.[Fact_FHV_Trips] f LEFT JOIN dbo.[Dim_Time] d ON
f.[DropoffTimeKey]=d.[TimeKey] WHERE d.[TimeKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [DropoffTimeKey]=0 THEN 1 ELSE 0 END AS
bigint)) AS UnknownRows FROM dbo.[Fact_FHV_Trips];
-- FK_HVFHV_RequestTime
SELECT COUNT_BIG(*) FROM dbo.[Fact_HVFHV_Trips] f LEFT JOIN dbo.[Dim_Time] d ON
f.[RequestTimeKey]=d.[TimeKey] WHERE d.[TimeKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [RequestTimeKey]=0 THEN 1 ELSE 0 END AS
bigint)) AS UnknownRows FROM dbo.[Fact_HVFHV_Trips];
-- FK_HVFHV_OnSceneTime
SELECT COUNT_BIG(*) FROM dbo.[Fact_HVFHV_Trips] f LEFT JOIN dbo.[Dim_Time] d ON
f.[OnSceneTimeKey]=d.[TimeKey] WHERE d.[TimeKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [OnSceneTimeKey]=0 THEN 1 ELSE 0 END AS
bigint)) AS UnknownRows FROM dbo.[Fact_HVFHV_Trips];
-- FK_HVFHV_PickupTime
SELECT COUNT_BIG(*) FROM dbo.[Fact_HVFHV_Trips] f LEFT JOIN dbo.[Dim_Time] d ON
f.[PickupTimeKey]=d.[TimeKey] WHERE d.[TimeKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [PickupTimeKey]=0 THEN 1 ELSE 0 END AS
bigint)) AS UnknownRows FROM dbo.[Fact_HVFHV_Trips];
-- FK_HVFHV_DropoffTime
SELECT COUNT_BIG(*) FROM dbo.[Fact_HVFHV_Trips] f LEFT JOIN dbo.[Dim_Time] d ON
f.[DropoffTimeKey]=d.[TimeKey] WHERE d.[TimeKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [DropoffTimeKey]=0 THEN 1 ELSE 0 END AS
bigint)) AS UnknownRows FROM dbo.[Fact_HVFHV_Trips];
-- FK_ZTS_Time
SELECT COUNT_BIG(*) FROM dbo.[Fact_Zone_Traffic_Snapshot] f LEFT JOIN dbo.[Dim_Time] d ON
f.[TimeKey]=d.[TimeKey] WHERE d.[TimeKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [TimeKey]=0 THEN 1 ELSE 0 END AS
bigint)) AS UnknownRows FROM dbo.[Fact_Zone_Traffic_Snapshot];
-- FK_FYG_PickupTime
```

```
SELECT COUNT_BIG(*) FROM dbo.[Fact_YellowGreen_Trips] f LEFT JOIN dbo.[Dim_Time] d ON
f.[PickupTimeKey]=d.[TimeKey] WHERE d.[TimeKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [PickupTimeKey]=0 THEN 1 ELSE 0 END AS
bigint)) AS UnknownRows FROM dbo.[Fact_YellowGreen_Trips];
-- FK_FYG_DropoffTime
SELECT COUNT_BIG(*) FROM dbo.[Fact_YellowGreen_Trips] f LEFT JOIN dbo.[Dim_Time] d ON
f.[DropoffTimeKey]=d.[TimeKey] WHERE d.[TimeKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [DropoffTimeKey]=0 THEN 1 ELSE 0 END AS
bigint)) AS UnknownRows FROM dbo.[Fact_YellowGreen_Trips];
-- FK_FHV_PickupLocation
SELECT COUNT_BIG(*) FROM dbo.[Fact_FHV_Trips] f LEFT JOIN dbo.[Dim_Location] d ON
f.[PickupLocationKey]=d.[LocationKey] WHERE d.[LocationKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [PickupLocationKey]=0 THEN 1 ELSE 0 END
AS bigint)) AS UnknownRows FROM dbo.[Fact_FHV_Trips];
-- FK_FHV_DropoffLocation
SELECT COUNT_BIG(*) FROM dbo.[Fact_FHV_Trips] f LEFT JOIN dbo.[Dim_Location] d ON
f.[DropoffLocationKey]=d.[LocationKey] WHERE d.[LocationKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [DropoffLocationKey]=0 THEN 1 ELSE 0
END AS bigint)) AS UnknownRows FROM dbo.[Fact_FHV_Trips];
-- FK_HVFHV_PickupLocation
SELECT COUNT_BIG(*) FROM dbo.[Fact_HVFHV_Trips] f LEFT JOIN dbo.[Dim_Location] d ON
f.[PickupLocationKey]=d.[LocationKey] WHERE d.[LocationKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [PickupLocationKey]=0 THEN 1 ELSE 0 END
AS bigint)) AS UnknownRows FROM dbo.[Fact_HVFHV_Trips];
-- FK_HVFHV_DropoffLocation
SELECT COUNT_BIG(*) FROM dbo.[Fact_HVFHV_Trips] f LEFT JOIN dbo.[Dim_Location] d ON
f.[DropoffLocationKey]=d.[LocationKey] WHERE d.[LocationKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [DropoffLocationKey]=0 THEN 1 ELSE 0
END AS bigint)) AS UnknownRows FROM dbo.[Fact_HVFHV_Trips];
-- FK_ZTS_Location
SELECT COUNT_BIG(*) FROM dbo.[Fact_Zone_Traffic_Snapshot] f LEFT JOIN dbo.[Dim_Location] d
ON f.[LocationKey]=d.[LocationKey] WHERE d.[LocationKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [LocationKey]=0 THEN 1 ELSE 0 END AS
bigint)) AS UnknownRows FROM dbo.[Fact_Zone_Traffic_Snapshot];
-- FK_FYG_PickupLocation
SELECT COUNT_BIG(*) FROM dbo.[Fact_YellowGreen_Trips] f LEFT JOIN dbo.[Dim_Location] d ON
f.[PickupLocationKey]=d.[LocationKey] WHERE d.[LocationKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [PickupLocationKey]=0 THEN 1 ELSE 0 END
AS bigint)) AS UnknownRows FROM dbo.[Fact_YellowGreen_Trips];
-- FK_FYG_DropoffLocation
SELECT COUNT_BIG(*) FROM dbo.[Fact_YellowGreen_Trips] f LEFT JOIN dbo.[Dim_Location] d ON
f.[DropoffLocationKey]=d.[LocationKey] WHERE d.[LocationKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [DropoffLocationKey]=0 THEN 1 ELSE 0
END AS bigint)) AS UnknownRows FROM dbo.[Fact_YellowGreen_Trips];
-- FK_FHV_ServiceType
SELECT COUNT_BIG(*) FROM dbo.[Fact_FHV_Trips] f LEFT JOIN dbo.[Dim_ServiceType] d ON
f.[ServiceTypeKey]=d.[ServiceTypeKey] WHERE d.[ServiceTypeKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [ServiceTypeKey]=0 THEN 1 ELSE 0 END AS
bigint)) AS UnknownRows FROM dbo.[Fact_FHV_Trips];
-- FK_HVFHV_ServiceType
SELECT COUNT_BIG(*) FROM dbo.[Fact_HVFHV_Trips] f LEFT JOIN dbo.[Dim_ServiceType] d ON
f.[ServiceTypeKey]=d.[ServiceTypeKey] WHERE d.[ServiceTypeKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [ServiceTypeKey]=0 THEN 1 ELSE 0 END AS
bigint)) AS UnknownRows FROM dbo.[Fact_HVFHV_Trips];
-- FK_DFS_Service
SELECT COUNT_BIG(*) FROM dbo.[Fact_Daily_Financial_Snapshot] f LEFT JOIN
dbo.[Dim_ServiceType] d ON f.[ServiceTypeKey]=d.[ServiceTypeKey] WHERE d.[ServiceTypeKey]
IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [ServiceTypeKey]=0 THEN 1 ELSE 0 END AS
bigint)) AS UnknownRows FROM dbo.[Fact_Daily_Financial_Snapshot];
-- FK_ZTS_Service
SELECT COUNT_BIG(*) FROM dbo.[Fact_Zone_Traffic_Snapshot] f LEFT JOIN dbo.[Dim_ServiceType]
d ON f.[ServiceTypeKey]=d.[ServiceTypeKey] WHERE d.[ServiceTypeKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [ServiceTypeKey]=0 THEN 1 ELSE 0 END AS
bigint)) AS UnknownRows FROM dbo.[Fact_Zone_Traffic_Snapshot];
-- FK_BPS_Service
```

```
SELECT COUNT_BIG(*) FROM dbo.[Fact_Base_Performance_Snapshot] f LEFT JOIN
dbo.[Dim_ServiceType] d ON f.[ServiceTypeKey]=d.[ServiceTypeKey] WHERE d.[ServiceTypeKey]
IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [ServiceTypeKey]=0 THEN 1 ELSE 0 END AS
bigint)) AS UnknownRows FROM dbo.[Fact_Base_Performance_Snapshot];
-- FK_FYG_ServiceType
SELECT COUNT_BIG(*) FROM dbo.[Fact_YellowGreen_Trips] f LEFT JOIN dbo.[Dim_ServiceType] d
ON f.[ServiceTypeKey]=d.[ServiceTypeKey] WHERE d.[ServiceTypeKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [ServiceTypeKey]=0 THEN 1 ELSE 0 END AS
bigint)) AS UnknownRows FROM dbo.[Fact_YellowGreen_Trips];
-- FK_FYG_Vendor
SELECT COUNT_BIG(*) FROM dbo.[Fact_YellowGreen_Trips] f LEFT JOIN dbo.[Dim_Vendor] d ON
f.[VendorKey]=d.[VendorKey] WHERE d.[VendorKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [VendorKey]=0 THEN 1 ELSE 0 END AS
bigint)) AS UnknownRows FROM dbo.[Fact_YellowGreen_Trips];
-- FK_HVFHV_Provider
SELECT COUNT_BIG(*) FROM dbo.[Fact_HVFHV_Trips] f LEFT JOIN dbo.[Dim_HVFHSProvider] d ON
f.[HVFHSProviderKey]=d.[HVFHSProviderKey] WHERE d.[HVFHSProviderKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [HVFHSProviderKey]=0 THEN 1 ELSE 0 END
AS bigint)) AS UnknownRows FROM dbo.[Fact_HVFHV_Trips];
-- FK_BPS_Provider
SELECT COUNT_BIG(*) FROM dbo.[Fact_Base_Performance_Snapshot] f LEFT JOIN
dbo.[Dim_HVFHSProvider] d ON f.[HVFHSProviderKey]=d.[HVFHSProviderKey] WHERE
d.[HVFHSProviderKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [HVFHSProviderKey]=0 THEN 1 ELSE 0 END
AS bigint)) AS UnknownRows FROM dbo.[Fact_Base_Performance_Snapshot];
-- FK_FHV_DispatchBase
SELECT COUNT_BIG(*) FROM dbo.[Fact_FHV_Trips] f LEFT JOIN dbo.[Dim_Base] d ON
f.[DispatchBaseKey]=d.[BaseKey] WHERE d.[BaseKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [DispatchBaseKey]=0 THEN 1 ELSE 0 END
AS bigint)) AS UnknownRows FROM dbo.[Fact_FHV_Trips];
-- FK_FHV_AffiliatedBase
SELECT COUNT_BIG(*) FROM dbo.[Fact_FHV_Trips] f LEFT JOIN dbo.[Dim_Base] d ON
f.[AffiliatedBaseKey]=d.[BaseKey] WHERE d.[BaseKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [AffiliatedBaseKey]=0 THEN 1 ELSE 0 END
AS bigint)) AS UnknownRows FROM dbo.[Fact_FHV_Trips];
-- FK_HVFHV_DispatchBase
SELECT COUNT_BIG(*) FROM dbo.[Fact_HVFHV_Trips] f LEFT JOIN dbo.[Dim_Base] d ON
f.[DispatchBaseKey]=d.[BaseKey] WHERE d.[BaseKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [DispatchBaseKey]=0 THEN 1 ELSE 0 END
AS bigint)) AS UnknownRows FROM dbo.[Fact_HVFHV_Trips];
-- FK_HVFHV_OriginBase
SELECT COUNT_BIG(*) FROM dbo.[Fact_HVFHV_Trips] f LEFT JOIN dbo.[Dim_Base] d ON
f.[OriginBaseKey]=d.[BaseKey] WHERE d.[BaseKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [OriginBaseKey]=0 THEN 1 ELSE 0 END AS
bigint)) AS UnknownRows FROM dbo.[Fact_HVFHV_Trips];
-- FK_BPS_Base
SELECT COUNT_BIG(*) FROM dbo.[Fact_Base_Performance_Snapshot] f LEFT JOIN dbo.[Dim_Base] d
ON f.[DispatchBaseKey]=d.[BaseKey] WHERE d.[BaseKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [DispatchBaseKey]=0 THEN 1 ELSE 0 END
AS bigint)) AS UnknownRows FROM dbo.[Fact_Base_Performance_Snapshot];
-- FK_FYG_RateCode
SELECT COUNT_BIG(*) FROM dbo.[Fact_YellowGreen_Trips] f LEFT JOIN dbo.[Dim_RateCode] d ON
f.[RateCodeKey]=d.[RateCodeKey] WHERE d.[RateCodeKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [RateCodeKey]=0 THEN 1 ELSE 0 END AS
bigint)) AS UnknownRows FROM dbo.[Fact_YellowGreen_Trips];
-- FK_FYG_PaymentType
SELECT COUNT_BIG(*) FROM dbo.[Fact_YellowGreen_Trips] f LEFT JOIN dbo.[Dim_PaymentType] d
ON f.[PaymentTypeKey]=d.[PaymentTypeKey] WHERE d.[PaymentTypeKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [PaymentTypeKey]=0 THEN 1 ELSE 0 END AS
bigint)) AS UnknownRows FROM dbo.[Fact_YellowGreen_Trips];
-- FK_FYG_TripType
SELECT COUNT_BIG(*) FROM dbo.[Fact_YellowGreen_Trips] f LEFT JOIN dbo.[Dim_TripType] d ON
f.[TripTypeKey]=d.[TripTypeKey] WHERE d.[TripTypeKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [TripTypeKey]=0 THEN 1 ELSE 0 END AS
bigint)) AS UnknownRows FROM dbo.[Fact_YellowGreen_Trips];
-- FK_FYG_TaxiFlag
```

```
SELECT COUNT_BIG(*) FROM dbo.[Fact_YellowGreen_Trips] f LEFT JOIN dbo.[Dim_TaxiFlags] d ON
f.[TaxiFlagKey]=d.[TaxiFlagKey] WHERE d.[TaxiFlagKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [TaxiFlagKey]=0 THEN 1 ELSE 0 END AS
bigint)) AS UnknownRows FROM dbo.[Fact_YellowGreen_Trips];
-- FK_FHV_Flag
SELECT COUNT_BIG(*) FROM dbo.[Fact_FHV_Trips] f LEFT JOIN dbo.[Dim_FHVFlags] d ON
f.[FHVFlagKey]=d.[FHVFlagKey] WHERE d.[FHVFlagKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [FHVFlagKey]=0 THEN 1 ELSE 0 END AS
bigint)) AS UnknownRows FROM dbo.[Fact_FHV_Trips];
-- FK_HVFHV_Flag
SELECT COUNT_BIG(*) FROM dbo.[Fact_HVFHV_Trips] f LEFT JOIN dbo.[Dim_HVFHVFlags] d ON
f.[HVFHVFlagKey]=d.[HVFHVFlagKey] WHERE d.[HVFHVFlagKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [HVFHVFlagKey]=0 THEN 1 ELSE 0 END AS
bigint)) AS UnknownRows FROM dbo.[Fact_HVFHV_Trips];
-- FK_FYG_PassengerGroup
SELECT COUNT_BIG(*) FROM dbo.[Fact_YellowGreen_Trips] f LEFT JOIN dbo.[Dim_PassengerGroup]
d ON f.[PassengerGroupKey]=d.[PassengerGroupKey] WHERE d.[PassengerGroupKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [PassengerGroupKey]=0 THEN 1 ELSE 0 END
AS bigint)) AS UnknownRows FROM dbo.[Fact_YellowGreen_Trips];
-- FK_FHV_LoadBatch
SELECT COUNT_BIG(*) FROM dbo.[Fact_FHV_Trips] f LEFT JOIN dbo.[Dim_LoadBatch] d ON
f.[LoadBatchKey]=d.[LoadBatchKey] WHERE d.[LoadBatchKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [LoadBatchKey]=0 THEN 1 ELSE 0 END AS
bigint)) AS UnknownRows FROM dbo.[Fact_FHV_Trips];
-- FK_HVFHV_LoadBatch
SELECT COUNT_BIG(*) FROM dbo.[Fact_HVFHV_Trips] f LEFT JOIN dbo.[Dim_LoadBatch] d ON
f.[LoadBatchKey]=d.[LoadBatchKey] WHERE d.[LoadBatchKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [LoadBatchKey]=0 THEN 1 ELSE 0 END AS
bigint)) AS UnknownRows FROM dbo.[Fact_HVFHV_Trips];
-- FK_FYG_LoadBatch
SELECT COUNT_BIG(*) FROM dbo.[Fact_YellowGreen_Trips] f LEFT JOIN dbo.[Dim_LoadBatch] d ON
f.[LoadBatchKey]=d.[LoadBatchKey] WHERE d.[LoadBatchKey] IS NULL;
SELECT COUNT_BIG(*) AS FactRows, SUM(CAST(CASE WHEN [LoadBatchKey]=0 THEN 1 ELSE 0 END AS
bigint)) AS UnknownRows FROM dbo.[Fact_YellowGreen_Trips];
```

Truy vấn reconcile-Full.sql

```
-- FHV trips
SELECT SUM(CAST(TripCount AS bigint)) FROM dbo.Fact_FHV_Trips WHERE 1=1;
-- FHV date 20250101
SELECT SUM(CAST(TripCount AS bigint)) FROM dbo.Fact_FHV_Trips WHERE PickupDateKey=20250101;
-- FHV date 20250115
SELECT SUM(CAST(TripCount AS bigint)) FROM dbo.Fact_FHV_Trips WHERE PickupDateKey=20250115;
-- FHV pickup location 0
SELECT SUM(CAST(TripCount AS bigint)) FROM dbo.Fact_FHV_Trips WHERE PickupLocationKey=0;
-- FHV pickup location 1
SELECT SUM(CAST(TripCount AS bigint)) FROM dbo.Fact_FHV_Trips WHERE PickupLocationKey=1;
-- FHV pickup location 132
SELECT SUM(CAST(TripCount AS bigint)) FROM dbo.Fact_FHV_Trips WHERE PickupLocationKey=132;
-- FHV pickup location 138
SELECT SUM(CAST(TripCount AS bigint)) FROM dbo.Fact_FHV_Trips WHERE PickupLocationKey=138;
-- FHV borough Manhattan
SELECT SUM(CAST(f.TripCount AS bigint)) FROM dbo.Fact_FHV_Trips f JOIN dbo.Dim_Location d
ON f.PickupLocationKey=d.LocationKey WHERE d.Borough=N'Manhattan';
-- FHV borough Queens
SELECT SUM(CAST(f.TripCount AS bigint)) FROM dbo.Fact_FHV_Trips f JOIN dbo.Dim_Location d
ON f.PickupLocationKey=d.LocationKey WHERE d.Borough=N'Queens';
-- FHV avg duration
SELECT SUM(CAST(TripDurationSeconds AS float))/NULLIF(SUM(CASE WHEN TripDurationSeconds IS
NOT NULL THEN CAST(TripCount AS bigint) ELSE 0 END),0)/60.0 FROM dbo.Fact_FHV_Trips WHERE
1=1;
-- HVFHV trips
SELECT SUM(CAST(TripCount AS bigint)) FROM dbo.Fact_HVFHV_Trips WHERE 1=1;
-- HVFHV date 20250101
SELECT SUM(CAST(TripCount AS bigint)) FROM dbo.Fact_HVFHV_Trips WHERE
PickupDateKey=20250101;
-- HVFHV date 20250115
SELECT SUM(CAST(TripCount AS bigint)) FROM dbo.Fact_HVFHV_Trips WHERE
PickupDateKey=20250115;
```

```
-- HVFHV pickup location 0
SELECT SUM(CAST(TripCount AS bigint)) FROM dbo.Fact_HVFHV_Trips WHERE PickupLocationKey=0;
-- HVFHV pickup location 1
SELECT SUM(CAST(TripCount AS bigint)) FROM dbo.Fact_HVFHV_Trips WHERE PickupLocationKey=1;
-- HVFHV pickup location 132
SELECT SUM(CAST(TripCount AS bigint)) FROM dbo.Fact_HVFHV_Trips WHERE
PickupLocationKey=132;
-- HVFHV pickup location 138
SELECT SUM(CAST(TripCount AS bigint)) FROM dbo.Fact_HVFHV_Trips WHERE
PickupLocationKey=138;
-- HVFHV borough Manhattan
SELECT SUM(CAST(f.TripCount AS bigint)) FROM dbo.Fact_HVFHV_Trips f JOIN dbo.Dim_Location d
ON f.PickupLocationKey=d.LocationKey WHERE d.Borough=N'Manhattan';
-- HVFHV borough Queens
SELECT SUM(CAST(f.TripCount AS bigint)) FROM dbo.Fact_HVFHV_Trips f JOIN dbo.Dim_Location d
ON f.PickupLocationKey=d.LocationKey WHERE d.Borough=N'Queens';
-- HVFHV avg duration
SELECT SUM(CAST(TripDurationSeconds AS float))/NULLIF(SUM(CASE WHEN TripDurationSeconds IS
NOT NULL THEN CAST(TripCount AS bigint) ELSE 0 END),0)/60.0 FROM dbo.Fact_HVFHV_Trips WHERE
1=1;
-- HVFHV avg distance
SELECT SUM(TripDistanceMiles)/NULLIF(SUM(CASE WHEN TripDistanceMiles IS NOT NULL THEN
CAST(TripCount AS bigint) ELSE 0 END),0) FROM dbo.Fact_HVFHV_Trips WHERE 1=1;
-- HVFHV speed
SELECT SUM(CASE WHEN TripDurationSeconds>0 AND TripDistanceMiles>=0 THEN
CAST(TripDistanceMiles AS float) END)/NULLIF(SUM(CASE WHEN TripDurationSeconds>0 AND
TripDistanceMiles>=0 THEN CAST(TripDurationSeconds AS float) END),0)*3600 FROM
dbo.Fact_HVFHV_Trips WHERE 1=1;
-- HVFHV avg fare
SELECT SUM(BasePassengerFare)/NULLIF(SUM(CASE WHEN BasePassengerFare IS NOT NULL THEN
CAST(TripCount AS bigint) ELSE 0 END),0) FROM dbo.Fact_HVFHV_Trips WHERE 1=1;
-- HVFHV avg tip
SELECT SUM(TipAmount)/NULLIF(SUM(CASE WHEN TipAmount IS NOT NULL THEN CAST(TripCount AS
bigint) ELSE 0 END),0) FROM dbo.Fact_HVFHV_Trips WHERE 1=1;
-- HVFHV tip percentage
SELECT SUM(CASE WHEN TipAmount IS NOT NULL AND BasePassengerFare IS NOT NULL THEN TipAmount
END)/NULLIF(SUM(CASE WHEN TipAmount IS NOT NULL AND BasePassengerFare IS NOT NULL THEN
BasePassengerFare END),0) FROM dbo.Fact_HVFHV_Trips WHERE 1=1;
-- YellowGreen trips
SELECT SUM(CAST(TripCount AS bigint)) FROM dbo.Fact_YellowGreen_Trips WHERE 1=1;
-- YellowGreen date 20250101
SELECT SUM(CAST(TripCount AS bigint)) FROM dbo.Fact_YellowGreen_Trips WHERE
PickupDateKey=20250101;
-- YellowGreen date 20250115
SELECT SUM(CAST(TripCount AS bigint)) FROM dbo.Fact_YellowGreen_Trips WHERE
PickupDateKey=20250115;
-- YellowGreen pickup location 0
SELECT SUM(CAST(TripCount AS bigint)) FROM dbo.Fact_YellowGreen_Trips WHERE
PickupLocationKey=0;
-- YellowGreen pickup location 1
SELECT SUM(CAST(TripCount AS bigint)) FROM dbo.Fact_YellowGreen_Trips WHERE
PickupLocationKey=1;
-- YellowGreen pickup location 132
SELECT SUM(CAST(TripCount AS bigint)) FROM dbo.Fact_YellowGreen_Trips WHERE
PickupLocationKey=132;
-- YellowGreen pickup location 138
SELECT SUM(CAST(TripCount AS bigint)) FROM dbo.Fact_YellowGreen_Trips WHERE
PickupLocationKey=138;
-- YellowGreen borough Manhattan
SELECT SUM(CAST(f.TripCount AS bigint)) FROM dbo.Fact_YellowGreen_Trips f JOIN
dbo.Dim_Location d ON f.PickupLocationKey=d.LocationKey WHERE d.Borough=N'Manhattan';
-- YellowGreen borough Queens
SELECT SUM(CAST(f.TripCount AS bigint)) FROM dbo.Fact_YellowGreen_Trips f JOIN
dbo.Dim_Location d ON f.PickupLocationKey=d.LocationKey WHERE d.Borough=N'Queens';
-- YellowGreen avg duration
SELECT SUM(CAST(TripDurationSeconds AS float))/NULLIF(SUM(CASE WHEN TripDurationSeconds IS
NOT NULL THEN CAST(TripCount AS bigint) ELSE 0 END),0)/60.0 FROM dbo.Fact_YellowGreen_Trips
WHERE 1=1;
```

```
-- YellowGreen avg distance
SELECT SUM(TripDistanceMiles)/NULLIF(SUM(CASE WHEN TripDistanceMiles IS NOT NULL THEN
CAST(TripCount AS bigint) ELSE 0 END),0) FROM dbo.Fact_YellowGreen_Trips WHERE 1=1;
-- YellowGreen speed
SELECT SUM(CASE WHEN TripDurationSeconds>0 AND TripDistanceMiles>=0 THEN
CAST(TripDistanceMiles AS float) END)/NULLIF(SUM(CASE WHEN TripDurationSeconds>0 AND
TripDistanceMiles>=0 THEN CAST(TripDurationSeconds AS float) END),0)*3600 FROM
dbo.Fact_YellowGreen_Trips WHERE 1=1;
-- YellowGreen avg fare
SELECT SUM(FareAmount)/NULLIF(SUM(CASE WHEN FareAmount IS NOT NULL THEN CAST(TripCount AS
bigint) ELSE 0 END),0) FROM dbo.Fact_YellowGreen_Trips WHERE 1=1;
-- YellowGreen avg tip
SELECT SUM(TipAmount)/NULLIF(SUM(CASE WHEN TipAmount IS NOT NULL THEN CAST(TripCount AS
bigint) ELSE 0 END),0) FROM dbo.Fact_YellowGreen_Trips WHERE 1=1;
-- YellowGreen tip percentage
SELECT SUM(CASE WHEN TipAmount IS NOT NULL AND FareAmount IS NOT NULL THEN TipAmount
END)/NULLIF(SUM(CASE WHEN TipAmount IS NOT NULL AND FareAmount IS NOT NULL THEN FareAmount
END),0) FROM dbo.Fact_YellowGreen_Trips WHERE 1=1;
-- Service YELLOW
SELECT SUM(CAST(TripCount AS bigint)) FROM dbo.Fact_YellowGreen_Trips WHERE
ServiceTypeKey=1;
-- Service GREEN
SELECT SUM(CAST(TripCount AS bigint)) FROM dbo.Fact_YellowGreen_Trips WHERE
ServiceTypeKey=2;
-- Service FHV
SELECT SUM(CAST(TripCount AS bigint)) FROM dbo.Fact_FHV_Trips WHERE ServiceTypeKey=3;
-- Service FHVHV
SELECT SUM(CAST(TripCount AS bigint)) FROM dbo.Fact_HVFHV_Trips WHERE ServiceTypeKey=4;
-- YellowGreen revenue
SELECT SUM(TotalAmount) FROM dbo.Fact_YellowGreen_Trips WHERE 1=1;
-- HVFHV driver pay
SELECT SUM(DriverPayAmount) FROM dbo.Fact_HVFHV_Trips WHERE 1=1;
-- HVFHV avg waiting
SELECT SUM(CAST(WaitingTimeSeconds AS float))/NULLIF(SUM(CASE WHEN WaitingTimeSeconds IS
NOT NULL THEN CAST(TripCount AS bigint) ELSE 0 END),0)/60.0 FROM dbo.Fact_HVFHV_Trips WHERE
1=1;
-- Daily Financial Snapshot TotalTrips
SELECT SUM(TotalTrips) FROM dbo.Fact_Daily_Financial_Snapshot;
-- Daily Financial Snapshot TotalDistanceMiles
SELECT SUM(TotalDistanceMiles) FROM dbo.Fact_Daily_Financial_Snapshot;
-- Daily Financial Snapshot TotalPassengerCharges
SELECT SUM(TotalPassengerCharges) FROM dbo.Fact_Daily_Financial_Snapshot;
-- Daily Financial Snapshot TotalTips
SELECT SUM(TotalTips) FROM dbo.Fact_Daily_Financial_Snapshot;
-- Daily Financial Snapshot TotalTaxesAndFees
SELECT SUM(TotalTaxesAndFees) FROM dbo.Fact_Daily_Financial_Snapshot;
-- Daily Financial Snapshot TotalDriverPay
SELECT SUM(TotalDriverPay) FROM dbo.Fact_Daily_Financial_Snapshot;
-- Zone Traffic Snapshot TotalPickups
SELECT SUM(TotalPickups) FROM dbo.Fact_Zone_Traffic_Snapshot;
-- Zone Traffic Snapshot TotalDropoffs
SELECT SUM(TotalDropoffs) FROM dbo.Fact_Zone_Traffic_Snapshot;
-- Base Performance Snapshot TotalTrips
SELECT SUM(TotalTrips) FROM dbo.Fact_Base_Performance_Snapshot;
-- Base Performance Snapshot TotalTripDurationSeconds
SELECT SUM(TotalTripDurationSeconds) FROM dbo.Fact_Base_Performance_Snapshot;
-- Base Performance Snapshot DurationTripCount
SELECT SUM(DurationTripCount) FROM dbo.Fact_Base_Performance_Snapshot;
-- Base Performance Snapshot TotalWaitingTimeSeconds
SELECT SUM(TotalWaitingTimeSeconds) FROM dbo.Fact_Base_Performance_Snapshot;
-- Base Performance Snapshot WaitingTripCount
SELECT SUM(WaitingTripCount) FROM dbo.Fact_Base_Performance_Snapshot;
-- Base Performance Snapshot TotalSharedRequests
SELECT SUM(TotalSharedRequests) FROM dbo.Fact_Base_Performance_Snapshot;
-- Base Performance Snapshot TotalSharedMatches
SELECT SUM(TotalSharedMatches) FROM dbo.Fact_Base_Performance_Snapshot;
-- Base Performance Snapshot TotalWAVRequests
SELECT SUM(TotalWAVRequests) FROM dbo.Fact_Base_Performance_Snapshot;
```

```
-- Base Performance Snapshot TotalWAVMatches
SELECT SUM(TotalWAVMatches) FROM dbo.Fact_Base_Performance_Snapshot;
-- Base Performance Snapshot TotalDriverPay
SELECT SUM(TotalDriverPay) FROM dbo.Fact_Base_Performance_Snapshot;
-- Shared request rate
SELECT SUM(CAST(TotalSharedRequests AS float))/NULLIF(SUM(TotalTrips),0) FROM
dbo.Fact_Base_Performance_Snapshot WHERE ServiceTypeKey=4;
-- WAV match rate
SELECT SUM(CAST(TotalWAVMatches AS float))/NULLIF(SUM(TotalWAVRequests),0) FROM
dbo.Fact_Base_Performance_Snapshot;
-- Dispatch base 0
SELECT SUM(TotalTrips) FROM dbo.Fact_Base_Performance_Snapshot WHERE DispatchBaseKey=0;
-- Dispatch base 1
SELECT SUM(TotalTrips) FROM dbo.Fact_Base_Performance_Snapshot WHERE DispatchBaseKey=1;
-- Dispatch base 2
SELECT SUM(TotalTrips) FROM dbo.Fact_Base_Performance_Snapshot WHERE DispatchBaseKey=2;
-- Dispatch base 3
SELECT SUM(TotalTrips) FROM dbo.Fact_Base_Performance_Snapshot WHERE DispatchBaseKey=3;
-- Base avg duration
SELECT SUM(CAST(TotalTripDurationSeconds AS float))/NULLIF(SUM(DurationTripCount),0)/60.0
FROM dbo.Fact_Base_Performance_Snapshot;
-- Base avg waiting
SELECT SUM(CAST(TotalWaitingTimeSeconds AS float))/NULLIF(SUM(WaitingTripCount),0)/60.0
FROM dbo.Fact_Base_Performance_Snapshot;
-- Snapshot pickups date 20250101
SELECT SUM(TotalPickups) FROM dbo.Fact_Zone_Traffic_Snapshot WHERE DateKey=20250101;
-- Snapshot pickups date 20250115
SELECT SUM(TotalPickups) FROM dbo.Fact_Zone_Traffic_Snapshot WHERE DateKey=20250115;
-- Snapshot dropoffs location 0
SELECT SUM(TotalDropoffs) FROM dbo.Fact_Zone_Traffic_Snapshot WHERE LocationKey=0;
-- Snapshot dropoffs location 1
SELECT SUM(TotalDropoffs) FROM dbo.Fact_Zone_Traffic_Snapshot WHERE LocationKey=1;
-- Snapshot dropoffs location 132
SELECT SUM(TotalDropoffs) FROM dbo.Fact_Zone_Traffic_Snapshot WHERE LocationKey=132;
-- Fact_FHV_Trips Unknown batch
SELECT SUM(CAST(TripCount AS bigint)) FROM dbo.Fact_FHV_Trips WHERE LoadBatchKey=0;
-- Fact_FHV_Trips Unknown time
SELECT SUM(CAST(TripCount AS bigint)) FROM dbo.Fact_FHV_Trips WHERE PickupTimeKey=0;
-- Fact_FHV_Trips Unknown date
SELECT SUM(CAST(TripCount AS bigint)) FROM dbo.Fact_FHV_Trips WHERE PickupDateKey=0;
-- Fact_FHV_Trips hour 0
SELECT SUM(CAST(f.TripCount AS bigint)) FROM dbo.Fact_FHV_Trips f JOIN dbo.Dim_Time t ON
f.PickupTimeKey=t.TimeKey WHERE t.HourNumber=0 AND t.TimeKey>0;
-- Fact_FHV_Trips hour 8
SELECT SUM(CAST(f.TripCount AS bigint)) FROM dbo.Fact_FHV_Trips f JOIN dbo.Dim_Time t ON
f.PickupTimeKey=t.TimeKey WHERE t.HourNumber=8 AND t.TimeKey>0;
-- Fact_FHV_Trips hour 17
SELECT SUM(CAST(f.TripCount AS bigint)) FROM dbo.Fact_FHV_Trips f JOIN dbo.Dim_Time t ON
f.PickupTimeKey=t.TimeKey WHERE t.HourNumber=17 AND t.TimeKey>0;
-- Fact_FHV_Trips calendar Jan2025
SELECT SUM(CAST(TripCount AS bigint)) FROM dbo.Fact_FHV_Trips WHERE PickupDateKey>=20250101
AND PickupDateKey<20250201;
-- Fact_FHV_Trips calendar Q12025
SELECT SUM(CAST(TripCount AS bigint)) FROM dbo.Fact_FHV_Trips WHERE PickupDateKey>=20250101
AND PickupDateKey<20250401;
-- Fact_HVFHV_Trips Unknown batch
SELECT SUM(CAST(TripCount AS bigint)) FROM dbo.Fact_HVFHV_Trips WHERE LoadBatchKey=0;
-- Fact_HVFHV_Trips Unknown time
SELECT SUM(CAST(TripCount AS bigint)) FROM dbo.Fact_HVFHV_Trips WHERE PickupTimeKey=0;
-- Fact_HVFHV_Trips Unknown date
SELECT SUM(CAST(TripCount AS bigint)) FROM dbo.Fact_HVFHV_Trips WHERE PickupDateKey=0;
-- Fact_HVFHV_Trips hour 0
SELECT SUM(CAST(f.TripCount AS bigint)) FROM dbo.Fact_HVFHV_Trips f JOIN dbo.Dim_Time t ON
f.PickupTimeKey=t.TimeKey WHERE t.HourNumber=0 AND t.TimeKey>0;
-- Fact_HVFHV_Trips hour 8
SELECT SUM(CAST(f.TripCount AS bigint)) FROM dbo.Fact_HVFHV_Trips f JOIN dbo.Dim_Time t ON
f.PickupTimeKey=t.TimeKey WHERE t.HourNumber=8 AND t.TimeKey>0;
-- Fact_HVFHV_Trips hour 17
```

```
SELECT SUM(CAST(f.TripCount AS bigint)) FROM dbo.Fact_HVFHV_Trips f JOIN dbo.Dim_Time t ON
f.PickupTimeKey=t.TimeKey WHERE t.HourNumber=17 AND t.TimeKey>0;
-- Fact_HVFHV_Trips calendar Jan2025
SELECT SUM(CAST(TripCount AS bigint)) FROM dbo.Fact_HVFHV_Trips WHERE
PickupDateKey>=20250101 AND PickupDateKey<20250201;
-- Fact_HVFHV_Trips calendar Q12025
SELECT SUM(CAST(TripCount AS bigint)) FROM dbo.Fact_HVFHV_Trips WHERE
PickupDateKey>=20250101 AND PickupDateKey<20250401;
-- Fact_YellowGreen_Trips Unknown batch
SELECT SUM(CAST(TripCount AS bigint)) FROM dbo.Fact_YellowGreen_Trips WHERE LoadBatchKey=0;
-- Fact_YellowGreen_Trips Unknown time
SELECT SUM(CAST(TripCount AS bigint)) FROM dbo.Fact_YellowGreen_Trips WHERE
PickupTimeKey=0;
-- Fact_YellowGreen_Trips Unknown date
SELECT SUM(CAST(TripCount AS bigint)) FROM dbo.Fact_YellowGreen_Trips WHERE
PickupDateKey=0;
-- Fact_YellowGreen_Trips hour 0
SELECT SUM(CAST(f.TripCount AS bigint)) FROM dbo.Fact_YellowGreen_Trips f JOIN dbo.Dim_Time
t ON f.PickupTimeKey=t.TimeKey WHERE t.HourNumber=0 AND t.TimeKey>0;
-- Fact_YellowGreen_Trips hour 8
SELECT SUM(CAST(f.TripCount AS bigint)) FROM dbo.Fact_YellowGreen_Trips f JOIN dbo.Dim_Time
t ON f.PickupTimeKey=t.TimeKey WHERE t.HourNumber=8 AND t.TimeKey>0;
-- Fact_YellowGreen_Trips hour 17
SELECT SUM(CAST(f.TripCount AS bigint)) FROM dbo.Fact_YellowGreen_Trips f JOIN dbo.Dim_Time
t ON f.PickupTimeKey=t.TimeKey WHERE t.HourNumber=17 AND t.TimeKey>0;
-- Fact_YellowGreen_Trips calendar Jan2025
SELECT SUM(CAST(TripCount AS bigint)) FROM dbo.Fact_YellowGreen_Trips WHERE
PickupDateKey>=20250101 AND PickupDateKey<20250201;
-- Fact_YellowGreen_Trips calendar Q12025
SELECT SUM(CAST(TripCount AS bigint)) FROM dbo.Fact_YellowGreen_Trips WHERE
PickupDateKey>=20250101 AND PickupDateKey<20250401;
```

PHỤ LỤC C HƯỚNG DẪN TRIỂN KHAI VÀ VẬN HÀNH

C.1 Chuẩn bị cấu hình môi trường

Kiểm tra SQL Server, SSAS Multidimensional, thư viện AMO và ADOMD, Python cùng pyarrow và lxml, Power BI Desktop hỗ trợ PBIP, các đường dẫn package và dữ liệu. Đặt đúng SqlServer, SsasServer, PackageRoot, ScriptsRoot, DataRoot và PythonExe. Tài khoản chạy cần quyền tương ứng; không lưu mật khẩu trong source hoặc tự tăng quyền để vượt lỗi cấu hình.
Thực hiện Preflight để kiểm tra kết nối và thư viện. Đối chiếu package thực sự được runner đọc với bản source cần triển khai; khi Visual Studio đang mở file, reload từ đĩa trước khi sửa để tránh lưu bản cũ đè bản mới. Backup hoặc cài đặt ban đầu là tác vụ riêng, không chạy installer cho mỗi đợt ETL.

C.2 Chạy mới và theo dõi

Trong Master đặt khoảng tháng, giới hạn dòng, pResumeRunId trống, pStartStage Extract và retry phù hợp. Khởi chạy một điểm vào thống nhất, theo dõi RunId, stage, file và lỗi trong monitor cùng log. MaxRows 0 nghĩa đọc đầy đủ file được chọn, không có nghĩa xóa và tái tạo mọi lịch sử DWH.
Chờ đủ Extract, Transform, Dwh, Aggregates và Ssas; kiểm tra PipelineRun SUCCEEDED và đối soát trước khi khai thác dashboard. Không coi số dòng wrk hoặc card cũ của Power BI là bằng chứng run mới hoàn tất. Không chạy thêm package con trong lúc Master đang giữ khóa.

C.3 Chạy lại sau lỗi

Đọc failure JSON, PipelineStep và stderr theo đúng RunId và InvocationId. Sửa nguyên nhân, xác nhận tiến trình cũ đã dừng và điều kiện khóa; dùng lại RunId cũ cùng cấu hình để resume từ stage được phép. Nếu stage trước chưa StageComplete SUCCEEDED, chạy từ điểm sớm hơn thay vì sửa log thủ công. Nếu run bị bỏ dở còn RUNNING, dùng quy trình Recover-Run chủ động có kiểm tra applock; không tự coi thiếu heartbeat là tiến trình đã chết. Khi SSAS

hoặc đối soát cuối lỗi, SQL có thể đã commit; cần giữ đầu vào ổn định và chạy lại stage cuối theo checkpoint. Một run không phải transaction phân tán xuyên file, SQL và cube.

C.4 Mở báo cáo và thực hiện demo

Kiểm tra SSAS Processed và probe truy vấn thành công. Mở project PBIP cùng đầy đủ thư mục Report và SemanticModel; chọn năm tháng có dữ liệu và giữ bộ lọc tương đương khi chuyển project. Refresh Power BI chỉ truy vấn lại cube. Thực hiện lần lượt các câu hỏi YellowGreen, tip, HVFHV, FHV, tài chính, địa lý và base; nêu rõ phạm vi sample trước khi nhận xét.

C.5 Bảo trì đối soát và khôi phục

Theo dõi log, file alert, checkpoint FAILED và thay đổi schema nguồn. Khi cần, chạy full reconciliation lúc ETL không hoạt động; nếu phát hiện sửa fact ngoài luồng, đánh ForceReload đúng file sau khi xác minh nguyên nhân. Sửa rule hoặc mapping phải có phiên bản và fixture tương ứng.

Backup database, source và metadata mô hình cần có phạm vi, retention và ca restore kiểm chứng riêng. Hồ sơ có backup không đủ để xác nhận phục hồi thảm họa đã hoàn thiện. Trước khi thay thế database hoặc khôi phục bản cũ phải đánh giá dữ liệu phát sinh sau backup; các thao tác đó không thuộc lần biên soạn báo cáo.